

Федеральное государственное автономное образовательное учреждение
высшего образования
«Московский физико-технический институт
(национальный исследовательский университет)»
Центр дополнительного, дополнительного профессионального
и онлайн-образования «Пуск»

Направление подготовки: 01.04.02 Прикладная математика и информатика
Направленность (профиль) подготовки: Науки о данных

**Разработка и оценка эффективности
нового метода защищенного
семантического поиска с
использованием гомоморфного
шифрования**
(магистерская диссертация)

Студент:

Куриленко Сергей Михайлович

(подпись студента)

Научный руководитель:

Никишова Арина Валерьевна

(подпись научного руководителя)

Москва 2026

Аннотация

В работе предложен инженерный прототип защищённого семантического поиска для модели угроз «доверенный клиент — секретный корпус — честный, но любопытный сервер» с неадаптивным атакующим. Архитектура сочетает SVD-усечение базы эмбедингов до $k = d/2$ и секретную ортогональную ротацию с двухстадийным поиском: клиентской PQ-фильтрацией по публичному PQ-артефакту в ротированном пространстве и серверным CKKS ct-pt-перанкингом шортлиста кандидатов в режиме ciphertext-plaintext. Геометрическое преобразование (SVD + ротация) рассматривается как эвристический слой обфускации и как источник ненулевой L_2 -ошибки реконструкции для проекционно-ограниченного класса декодеров; его устойчивость к off-the-shelf Vec2Text проверяется эмпирически в рамках выбранной модели угроз.

Параметры CKKS подбираются разработанным методом ML-автоподбора как минимально-латентная конфигурация при функциональном ограничении на качество и параметрах, удовлетворяющих стандартной таблице SEAL для уровня tc128.

Интегральный эксперимент проведён на пяти популярных энкодерах при 5 ситах ротации в основной операционной точке $k = d/2$. В этой точке критерий качества защиты ($\sigma_{\text{rec}} \geq 0,10$) выполнен единообразно для всех пяти энкодеров. Требование итоговой деградации $\Delta\text{Acc}@1 \geq -5$ п. п. от незащищённого baseline) выполняется для retrieval-trained энкодеров с $d \geq 768$; более компактные модели выявляют ограничения метода и фиксируются как граничные случаи. Латентность ответа удерживается на субсекундном уровне на корпусе в 1 миллион русскоязычных текстов.

Зафиксирован эмпирический эффект, заслуживающий отдельного исследования: на retrieval-trained-энкодерах с $d \geq 768$ SVD-усечение ведёт себя как линейный денойзер и не ухудшает (а в отдельных случаях даже немного улучшает) retrieval-метрики относительно неусечённого baseline. Эффект зависит от энкодера.

Содержание

Аннотация	2
Обозначения и сокращения	5
Введение	6
1 Анализ предметной области и проблемы конфиденциальности семантического поиска	12
1.1 Эволюция архитектур информационного поиска	12
1.2 Проблема конфиденциальности векторных представлений	16
1.3 Систематизация угроз приватности эмбедингов	25
2 Обзор существующих методов защиты векторных представлений . .	27
2.1 Дифференциальная приватность	28
2.2 Гомоморфное шифрование	29
2.3 Методы снижения размерности и изометрические преобразования	31
2.4 Сравнительный анализ: трилемма «Безопасность — Точность — Производительность»	37
2.5 Гибридные подходы и место настоящей работы в литературном ландшафте	39
3 Разработка метода автоматизированного подбора параметров СККС на основе машинного обучения	40
3.1 Методология исследования автоподбора параметров СККС	41
3.2 Разработка метода ML-автоподбора параметров СККС	42
3.3 Экспериментальное исследование и валидация метода	48
4 Разработка и тестирование защищённого семантического поиска на основе гибридной архитектуры	56
4.1 Методология исследования	56
4.2 Концепция и архитектура гибридного метода с асимметричной защитой	59
4.3 Экспериментальное исследование и верификация метода	73
4.4 Интегральное экспериментальное исследование защищённой архитектуры	89
4.5 Промышленная клиент-серверная реализация и измерение сквозной латентности	108
Заключение	112
Список литературы	116
Приложение.А	122
Приложение.Б	136

Приложение.В	143
Приложение.Г.	153
Приложение.Д	161
Приложение.Е	169

Обозначения и сокращения

В настоящей работе используются следующие сокращения и обозначения.

ФСТЭК	— Федеральная служба по техническому и экспортному контролю Российской Федерации.
AVX-512	— Advanced Vector Extensions, расширение набора инструкций x86_64 для векторизации вычислений.
BFV/BGV	— схемы полностью гомоморфного шифрования Brakerski–Fan–Vercauteren и Brakerski–Gentry–Vaikuntanathan.
BLEU	— Bilingual Evaluation Understudy, метрика близости сгенерированного и эталонного текста.
ct-ct	— ciphertext–ciphertext, режим умножения двух шифротекстов в схеме CKKS.
ct-pt	— ciphertext–plaintext, режим умножения шифротекста на открытый (закодированный) полином.
CKKS	— схема полностью гомоморфного шифрования Cheon–Kim–Kim–Song для приближённой арифметики над вещественными и комплексными числами.
DP	— Differential Privacy, дифференциальная приватность.
FAISS	— Facebook AI Similarity Search, библиотека приближённого поиска ближайших соседей.
FHE	— Fully Homomorphic Encryption, полностью гомоморфное шифрование.
GDPR	— General Data Protection Regulation, общий регламент ЕС по защите персональных данных.
GEIA	— Gradient-based Embedding Inversion Attack, градиентная атака инверсии эмбедингов.
GTR	— Generalizable T5-based dense Retrievers, семейство моделей плотного семантического поиска.
HE	— Homomorphic Encryption, гомоморфное шифрование.
HSM	— Hardware Security Module, аппаратный модуль безопасности.
IVF	— Inverted File, индекс на основе инвертированных файлов.
MAE	— Mean Absolute Error, средняя абсолютная ошибка.
MPC	— Secure Multi-Party Computation, безопасные многосторонние вычисления.
MRR	— Mean Reciprocal Rank, средний обратный ранг.
NTT	— Number Theoretic Transform, теоретико-числовое преобразование.
PCA	— Principal Component Analysis, метод главных компонент.

PII	— Personally Identifiable Information, персонально идентифицирующая информация.
PQ	— Product Quantization, квантизация произведения (метод приближённого поиска по сжатым под-кодам).
RAG	— Retrieval-Augmented Generation, генерация с поисковой аугментацией.
RLWE	— Ring Learning With Errors, задача обучения с ошибками над кольцом.
RNS	— Residue Number System, система остаточных классов.
SEAL	— Microsoft Simple Encrypted Arithmetic Library, библиотека гомоморфного шифрования.
SVD	— Singular Value Decomposition, сингулярное разложение матрицы.
TEIA	— Text Embedding Inversion Attack, атака инверсии эмбеддингов прямым декодером.
TenSEAL	— Python-обёртка над библиотекой Microsoft SEAL.
Vec2Text	— итеративный метод инверсии эмбеддингов на основе корректирующей модели.

Введение

По данным International Data Corporation (IDC), объем данных в глобальной инфосфере к 2025 году прогнозировался на уровне порядка 175–180 зеттабайт [1]. Существенную долю в этом объеме составляют неструктурированные текстовые данные (документы, письма, сообщения, отчеты и др.), для которых традиционный поиск по ключевым словам оказывается недостаточным.

Классические методы информационного поиска (TF-IDF, BM25), опирающиеся на сопоставление ключевых слов, имеют фундаментальные ограничения. Главное из них — так называемый «семантический разрыв»: запрос и релевантный документ нередко используют разную лексику для одного и того же смысла (синонимия, полисемия, перефразирование). Выходом из этой ситуации стал переход к семантическому поиску, в котором тексты представляются плотными векторами — эмбеддингами. Семейство нейросетевых энкодеров — BERT, RoBERTa, а также их прикладные варианты (Sentence-BERT, GTR, rubert-tiny2) — отображает входной текст в точку многомерного пространства. Близость двух таких точек, как правило, говорит и о смысловой близости исходных текстов.

На этой основе быстро распространились архитектуры RAG (Retrieval-Augmented Generation), где семантический поиск сочетается с генеративными возможностями больших языковых моделей (LLM). Благодаря такому сочетанию промышленное применение получил целый класс прикладных решений: ассистенты для работы с корпоративной документацией, инструменты поддержки принятия решений,

автоматизированные базы знаний. Обратной стороной оказалась сама насыщенность эмбедингов: чем точнее вектор кодирует исходный смысл, тем уязвимее он к атакам, которые в моделях угроз классических ИС попросту не рассматривались.

В работах последних лет (см., в частности, [2]) показано, что эмбединги поддаются атакам восстановления исходного текста — так называемому *embedding inversion* — и извлечению отдельных чувствительных признаков. Авторы [2, 3, 4, 5] разбирают на конкретных примерах, как именно компрометируются векторные БД и какие риски это создаёт для приватности. Параллельно идут работы по изучению утечки информации через сами модели эмбедингов [6], *membership inference* на ML-системах [7] и извлечению обучающих данных из LLM [8]. Что касается RAG-систем как самостоятельного объекта атаки, специфические риски приватности были эмпирически зафиксированы в [9].

По данным экспериментов в той же [2], на незащищённых эмбедингах качество восстановления может быть очень высоким: для коротких текстовых последовательностей BLEU = 97,3, а доля точного восстановления достигает 92 %. Это означает реальный риск восстановления конфиденциальных фрагментов и персональных данных при компрометации векторной базы.

Актуальность проблемы подтверждается также экономическими и регуляторными факторами. Согласно отчету IBM Security «Cost of a Data Breach Report 2024», средняя стоимость утечки данных составила около 4,88 млн долларов США [10]. В условиях ужесточения требований к защите персональных данных (Федеральный закон № 152-ФЗ «О персональных данных» [11], требования ФСТЭК [12], GDPR [13],) организации вынуждены внедрять технологии, снижающие риск раскрытия данных на всех этапах обработки.

Проблема исследования заключается в том, что использование векторных представлений текста в системах семантического поиска и RAG может приводить к раскрытию исходных документов и конфиденциальной информации при компрометации векторной базы данных или результатов вычислений, в то время как требуются практически применимые механизмы защиты, сохраняющие качество ранжирования и приемлемую производительность.

Теоретический фундамент семантического поиска опирается на современные модели эмбедингов и биэнкодеров на базе трансформеров (BERT, Sentence-BERT, русскоязычные модели семейства ruBERT), плотные ретриверы (DPR, ColBERT), модели контрастивного обучения (E5, INSTRUCTOR), а также на архитектуры RAG и их развитие (Self-RAG, CRAG) [14, 15, 16, 17, 18, 19, 20, 21, 22, 23, 24, 25]. Тема атак на эмбединги и связанных с ними утечек частных данных раскрыта по нескольким направлениям: собственно *embedding inversion* [2, 3, 4, 5], атаки *membership inference* [7], извлечение обучающих данных из LLM [8], утечка из самих моделей эмбедингов [6], отдельный пласт работ — атаки приватности на RAG [9]. По линии гомоморфного шифрования применительно к задачам ML в литературе сложился свой круг ключевых работ:

схема CKKS [26], её бутстраппинг [27], продакшн-уровневые реализации (OpenFHE [28], CryptoNets [29]), стандарты безопасности [30] и техника GPU-ускорения [31]. Среди альтернативных подходов следует выделить дифференциальную приватность (DP-SGD [32], Renyi DP [33]) и федеративное обучение [34].

При всём богатстве этих отдельных направлений вопрос построения практически работоспособной архитектуры защищённого семантического поиска — такой, которая одновременно учитывала бы реальную модель угроз и жёсткие ограничения по латентности, — до сих пор остаётся скорее открытым. Отсюда и потребность в методах с измеримой приватностью и приемлемой вычислительной стоимостью именно в прикладном контексте.

Существующие подходы к защите демонстрируют критические ограничения, формируя трилемму защищенного поиска: невозможность одновременно обеспечить высокую точность, надежную защиту от инверсии и приемлемую вычислительную эффективность. По результатам анализа существующих решений исследовательский разрыв состоит в отсутствии практического метода, который бы в едином протоколе

1. снижал успешность атак восстановления текста,
2. сохранял качество ранжирования
3. оставался применимым в корпоративных RAG-системах по требованиям к латентности и вычислительным ресурсам.

В связи с этим разработка гибридных методов, сочетающих строгие криптографические примитивы с методами линейной алгебры для достижения баланса между защищенностью и эффективностью, является актуальной научной задачей, имеющей существенное значение для развития доверенных систем искусственного интеллекта.

В настоящей работе исследуется задача защищенного семантического поиска в векторных базах данных, используемых в RAG-системах, при допущении, что атакующий может получить доступ к эмбедингам базы или к результатам вычисления релевантности. Цель исследования состоит в разработке метода, который снижает успешность атак инверсии, сохраняя качество поиска и приемлемую задержку ответа.

В качестве критериев оценки используются ргоху-метрика приватности σ_{rec} (относительная ошибка реконструкции после проекции), метрики качества предварительного отбора и финального ранжирования (Acc@10, Accuracy@1, MRR) и показатели производительности (латентность обработки запроса).

Предлагаемый подход основан на гибридной архитектуре: секретная ортогональная ротация базы эмбедингов с SVD-усечением и гомоморфное шифрование CKKS в режиме ciphertext-plaintext для вычисления оценок релевантности на этапе реранкинга.

Объектом исследования являются системы семантического поиска и вопросно-ответные системы, использующие векторные представления текста и процедуры ранжирования и реранкинга документов по запросу.

Предметом исследования выступают угрозы конфиденциальности, связанные с утечкой векторных представлений и поисковых запросов, методы атак инверсии

эмбедингов и способы оценки приватности, а также алгоритмы и криптографические протоколы защиты (включая ортогональные проекции и гомоморфное шифрование CKKS) и их влияние на качество предварительного отбора (Acc@10), финального ранжирования (Assigasy@1, MRR) и вычислительную латентность.

Исследовательский вопрос можно сформулировать так: как построить пригодный для практического использования метод защищённого семантического поиска для RAG, при котором снижается риск восстановления исходных текстов как из эмбедингов, так и из результатов вычислений, и при этом качество ранжирования и время отклика остаются в допустимых пределах?

В качестве рабочей гипотезы выдвигается следующее: гибридная схема, в которой база эмбедингов преобразуется секретной ортогональной ротацией и SVD-усечением до $k = d/2$, а запрос и шаг ранжирования выполняются с участием гомоморфного шифрования CKKS, должна одновременно ослаблять неадаптивные атаки инверсии и сохранять допустимую вычислительную стоимость. Критерии проверки гипотезы: итоговая деградация качества поиска по метрике Acc@1 не превышает 5 п. п. от незащищённого baseline, эмпирическое снижение успешности атаки Vec2Text в режиме unknown-R до уровня случайного шума ($BLEU \leq 0,05$) и латентность менее 1 с по p95 при работе с базой данных, содержащей 1 млн. эмбедингов.

Цель работы — разработать, теоретически обосновать и экспериментально проверить метод защищённого семантического поиска на основе гибридной архитектуры, в которой сочетаются CKKS и геометрическое преобразование эмбедингов (секретная ортогональная ротация с SVD-усечением до $k = d/2$). От метода требуется эмпирически верифицированная устойчивость к неадаптивным атакам инверсии (Vec2Text) при потере качества поиска не более 5 п. п. по Acc@1 от незащищённого baseline и субсекундной латентности обработки запроса.

Для достижения поставленной цели необходимо решить следующие задачи:

1. Аналитическая задача: провести систематический анализ уязвимостей векторных представлений к современным атакам инверсии, выявить ограничения существующих методов защиты и формализовать проблему в виде трилеммы «безопасность — точность — производительность».
2. Теоретическая задача: разработать и обосновать гибридную архитектуру защищённого поиска, использующую асимметричный подход к защите: гомоморфное шифрование запросов и проекционное преобразование базы данных. Оценить информационно-теоретические ограничения устойчивости проекционного метода к восстановлению.
3. Алгоритмическая задача: разработать методику автоматизированного выбора параметров схемы гомоморфного шифрования (CKKS) на основе применения методов машинного обучения для минимизации разрыва оценки между теоретической и реальной производительностью.

4. Экспериментальная задача: реализовать программный прототип системы и провести серию экспериментов для оценки влияния предложенных методов на метрики точности предварительного отбора и ранжирования (Acc@k, Accuracy@1, MRR), безопасности (ошибка реконструкции как гроху-метрика устойчивости к инверсии) и латентности.

5. Синтезирующая задача: сформулировать практические рекомендации по внедрению разработанного метода в промышленные RAG-системы.

В диссертации получены следующие новые научные результаты:

1. Сформулирована эмпирическая гипотеза о дифференцированной защите усечённого SVD: при усечении ранга $k < d$ часть текстоспецифичной информации удаляется вместе с отброшенными компонентами, а ожидаемое значение BLEU атаки инверсии монотонно растёт с k/d . Строгая часть результата — нижняя L_2 -оценка ошибки восстановления для декодеров, образ которых ограничен проекционным подпространством $\text{span}(V_k)$ (Лемма 1, опирается на теорему Экарта—Янга об оптимальной низкоранговой аппроксимации в норме Фробениуса). Связь σ_{rec} с эмпирическим BLEU Vec2Text проверяется численно в рамках выбранной модели угроз.

2. Разработана новая гибридная архитектура семантического поиска, отличающаяся применением режима вычислений ciphertext-plaintext (ct-pt) схемы CKKS над предварительно проецированным и ротированным векторным пространством. Это позволило устранить необходимость дорогостоящих операций релинеаризации и гомоморфного умножения шифротекстов и обеспечить ускорение вычисления сходства по сравнению с полностью зашифрованным режимом ciphertext-ciphertext (ct-ct).

3. Предложен метод автоматической оптимизации параметров гомоморфного шифрования с использованием ансамблевой регрессионной модели. Метод, в отличие от аналитических подходов, учитывает аппаратные особенности исполнения (кэширование, SIMD-векторизация); в экспериментальной валидации с расширенной 198-точечной сеткой и вложенной кросс-валидацией.

Теоретическая значимость работы заключается в развитии методов защиты для систем семантического поиска: сформулирована гипотеза о дифференцированной защите усечённого SVD и доказана нижняя L_2 -оценка ошибки восстановления для проекционно-ограниченного класса декодеров (Лемма 1), используемая как гроху-индикатор потери информации. Показана возможность достижения практического компромисса в трилемме защиты за счёт сочетания CKKS-шифрования и геометрических преобразований; эвристический характер защитного слоя в рамках выбранной модели угроз обозначается явно.

Практическая значимость состоит в разработке защищенной архитектуры семантического поиска, готовой к интеграции в корпоративные поисковые системы и RAG-приложения. Предложенное решение позволяет организациям:

– служить технической мерой снижения риска раскрытия персональных данных при использовании облачных векторных баз; формальное соответствие 152-ФЗ/ФСТЭК

требует дополнительных организационных мер, аттестации и применения сертифицированных СКЗИ, что выходит за рамки данной работы;

- снизить риски утечки конфиденциальной информации через векторные представления;
- сохранить удобство интерактивного поиска благодаря низкой латентности.

Исследование базируется на следующих методах:

- машинного обучения: методы обучения энкодеров, ансамблевые регрессионные модели для предсказания производительности СККС, метрики оценки качества ранжирования и генерации текста (Acc@10, Accuracy@1, MRR);
- линейной алгебры и матричного анализа: теория сингулярного разложения (SVD), свойства ортогональных матриц, теорема Эккарта–Янга.
- современной криптографии: гомоморфное шифрование (схема СККС);
- системного анализа: сравнительный анализ архитектур, моделирование угроз.

Сформулируем проверяемые положения, вытекающие из рабочей гипотезы. Для каждого пункта далее указаны измеримые критерии — через метрики конфиденциальности, качества поиска и производительности, по которым положение можно либо подтвердить, либо опровергнуть.

1. Гибридный метод защиты, комбинирующий гомоморфное шифрование запроса и геометрическое преобразование базы данных (секретную ортогональную ротацию с SVD-усечением до $k = d/2$ как основной операционной точки), в рамках выбранной модели угроз (доверенный клиент — честный, но любопытный сервер, неадаптивный атакующий) обеспечивает эмпирически верифицированную устойчивость к атаке Vec2Text ($BLEU \leq 0,05$ в режиме unknown-R), удовлетворяет проху-критерию $\sigma_{\text{rec}} \geq 0,10$ и сохраняет качество поиска в пределах допуска $\Delta \text{Acc@1} \geq -5$ п. п. от незащищённого baseline на retrieval-trained моделях с $d \geq 768$.

2. Применение режима вычислений ciphertext-plaintext (ct-pt) в схеме СККС для вычисления скалярного произведения позволяет сократить латентность относительно режима ciphertext-ciphertext (ct-ct) за счет исключения операций релинеаризации и снижения вычислительной сложности умножения шифротекстов и достигнуть субсекундного отклика при работе с базой данных, содержащей 1 млн. эмбеддингов.

3. Метод предсказания производительности гомоморфных операций на основе ансамблевой регрессионной модели обеспечивает автоматический подбор параметров безопасности (N , цепочка модулей), минимизируя время обработки запроса при соблюдении заданного уровня криптостойкости.

Основные результаты работы докладывались и обсуждались на следующих научных мероприятиях:

1. III Всероссийская научно-техническая конференция «Кибернетика и информационная безопасность» (КИБ-2025), НИЯУ МИФИ, декабрь 2025 г.
2. Международная конференция «Цифровизация управления в организационных системах» (ЦУОС-2025), УрФУ, декабрь 2025 г.

3. XIV Международная научная конференция «ИТ-Стандарт 2025», РТУ МИР-ЭА, декабрь 2025 г.

4. 68-я Всероссийская научная конференция МФТИ в честь 130-летия со дня рождения Н.Н. Семёнова, апрель 2026 г.

5. XXX Юбилейная межвузовская научно-техническая конференция студентов, аспирантов и молодых специалистов им. Е.В. Арменского, ВШЭ, апрель 2026 г.

По теме диссертации опубликована статья в рецензируемом научном журнале «Вестник компьютерных и информационных технологий» из перечня ВАК.

1 Анализ предметной области и проблемы конфиденциальности семантического поиска

1.1 Эволюция архитектур информационного поиска

Информационный поиск является фундаментальной дисциплиной компьютерных наук, значимость которой критически возрастает в условиях экспоненциального роста объемов цифровых данных.

Традиционные методы поиска, доминировавшие в индустрии на протяжении десятилетий, базировались на лексическом сопоставлении терминов запроса и документов. Однако сложность естественного языка, характеризующаяся явлениями многозначности и синонимии, привела к осознанию ограниченности лексического подхода и стимулировала переход к парадигме семантического поиска оперирующего смыслами, а не символами.

Исторически первыми и наиболее распространенными методами векторизации текстов стали модели «мешка слов» (Bag-of-Words). Классическим примером является метрика TF-IDF, предложенная К. Спарк Джонс в 1972 году. Вес термина t в документе d определяется как баланс между его частотой внутри документа и редкостью во всей коллекции D [35]:

$$\text{TF-IDF}(t, d) = \text{tf}(t, d) \cdot \log \frac{|D|}{|\{d \in D : t \in d\}|} \quad (1.1)$$

где $\text{tf}(t, d)$ — частота вхождения термина t в документ d ; $|D|$ — общее количество документов в коллекции; $|\{d \in D : t \in d\}|$ — количество документов коллекции, содержащих термин t (документная частота). Первый множитель отражает локальную значимость термина в документе, второй — его глобальную редкость в коллекции.

Развитием этого подхода стала вероятностная модель BM25 являющаяся стандартом де-факто в таких поисковых движках, как Elasticsearch и Apache Lucene. BM25 вводит параметры насыщения частоты терминов (k_1) и нормализации длины документа (b), что позволяет более точно ранжировать результаты [36]:

$$Score_{BM25}(q, d) = \sum_{t \in q} IDF(t) \cdot \frac{tf(t, d) \cdot (k_1 + 1)}{tf(t, d) + k_1 \cdot \left(1 - b + b \cdot \frac{|d|}{avgdl}\right)} \quad (1.2)$$

где q — поисковый запрос (мультимножество терминов, в котором учитываются повторы); d — документ; $IDF(t)$ — обратная документная частота термина t , определяющая его дискриминативную способность; $tf(t, d)$ — частота термина t в документе d ; $|d|$ — длина документа d (в токенах); $avgdl$ — средняя длина документа в коллекции; k_1 — параметр насыщения частоты (типичное значение 1,2–2,0), ограничивающий влияние многократных вхождений термина; b — параметр нормализации длины документа (типичное значение 0,75), контролирующий штраф за длинные документы. Суммирование ведется по всем терминам запроса $t \in q$.

Несмотря на высокую вычислительную эффективность и интерпретируемость, данные методы обладают фундаментальным недостатком, известным как «семантический разрыв». Лексические модели не способны установить релевантность между запросом «транспортное средство» и документом, содержащим слово «автомобиль», если в документе отсутствует точное вхождение слова «транспорт». Это приводит к снижению полноты из-за синонимии и снижению точности из-за полисемии.

Революция глубокого обучения в 2010-х годах привела к смене парадигмы представления текстовой информации. Гипотеза дистрибутивной семантики, постулирующая, что лингвистические единицы, встречающиеся в схожих контекстах, имеют близкие значения, была реализована в моделях Word2Vec [37] и GloVe [38]. Эти алгоритмы позволили проецировать слова в плотные векторные пространства, сохраняя семантические отношения.

Качественный скачок произошел с появлением архитектуры Transformer [39] и модели BERT [14]. В отличие от статических эмбеддингов Word2Vec, BERT генерирует контекстуализированные представления, где вектор слова динамически изменяется в зависимости от его окружения. Механизм внимания (self-attention) позволяет модели агрегировать информацию со всей последовательности:

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right) V \quad (1.3)$$

где $Q \in \mathbb{R}^{n \times d_k}$ — матрица запросов, формируемая линейной проекцией входных представлений; $K \in \mathbb{R}^{n \times d_k}$ — матрица ключей; $V \in \mathbb{R}^{n \times d_v}$ — матрица значений; d_k — размерность ключей, используемая для масштабирования скалярного произведения с целью предотвращения затухания градиентов в функции softmax при больших размерностях; n — длина последовательности. Произведение QK^T вычисляет попарные оценки внимания между всеми позициями, softmax нормализует их в вероятности, а умножение на V формирует взвешенную агрегацию значений.

Для задач информационного поиска прямое применение BERT (архитектура Cross-Encoder, где запрос и документ подаются в сеть одновременно) вычислительно неприемлемо на больших коллекциях: модель приходится прогонять для каждой пары

запрос–документ, и стоимость растёт линейно по размеру коллекции, причём каждый прогон сам по себе квадратичен по длине склейки благодаря Attention. Решением стала архитектура Bi-Encoder (или Two-Tower architecture), популяризированная библиотекой Sentence-BERT (SBERT) [15]. Альтернативный подход поздней интеракции реализован в модели ColBERT [19], обеспечивающей тонкозернистое сопоставление запроса и документа при скорости на два порядка выше, чем у Cross-Encoder. Для плотного поиска по большим корпусам предложены специализированные ретриверы DPR [18], обучающие отдельные энкодеры запросов и пассажей на задаче question answering, и ANCE [40] с подбором отрицательных примеров через приближённый поиск ближайших соседей в обучающем цикле.

В этой архитектуре запрос q и документ d обрабатываются независимо двумя (обычно идентичными) нейронными сетями, преобразующими их в векторы фиксированной размерности $v_q, v_d \in \mathbb{R}^d$. Мерой релевантности выступает косинусное сходство:

$$\text{sim}(q, d) = \cos(v_q, v_d) = \frac{v_q \cdot v_d}{\|v_q\| \cdot \|v_d\|} \quad (1.4)$$

где $v_q \in \mathbb{R}^d$ — векторное представление (эмбединг) запроса q ; $v_d \in \mathbb{R}^d$ — векторное представление документа d ; $v_q \cdot v_d$ — скалярное произведение векторов; $\|v_q\|$, $\|v_d\|$ — евклидовы нормы (L_2 -нормы) соответствующих векторов; d — размерность пространства эмбедингов. Значение $\text{sim}(q, d)$ принимает значения от -1 до 1 , где 1 соответствует полному совпадению направлений векторов (максимальная семантическая близость).

Такой подход позволяет предварительно вычислить (прединдексировать) векторы всех документов коллекции. Поиск сводится к нахождению ближайших соседей для вектора запроса.

Для ускорения поиска в пространствах высокой размерности используются алгоритмы приближенного поиска (ANN — Approximate Nearest Neighbor): классические locality-sensitive hashing-схемы [41], продуктовая квантизация [42] и графовые методы наподобие HNSW [43]. HNSW строит иерархический навигационный small-world-граф; на практических данных эмпирически показывает близкую к логарифмической (сублинейную) латентность поиска, что обеспечивает субсекундный отклик на масштабах в миллионы документов. Строгой универсальной гарантии $O(\log N)$ для произвольных распределений у HNSW нет. Инфраструктурной основой такого поиска являются специализированные векторные СУБД, такие как Milvus [44], Qdrant и Pinecone, обеспечивающие гетерогенные вычисления, распределенное развёртывание и динамическое обновление индексов.

Выбор модели-энкодера является критическим для качества поиска. В рамках данной работы рассматриваются модели, оптимизированные как для качества, так и для производительности:

1. GTR (Generalizable T5-based dense Retrievers): семейство моделей от Google, построенное на базе архитектуры T5 [45]. GTR демонстрирует state-of-the-art результаты на бенчмарке BEIR [46], особенно в задачах Zero-Shot (поиск в доменах, на которых

модель не обучалась). Семейство моделей E5, обученных контрастивным методом на веб-корпусе CCPairs, стало первой моделью, стабильно превзошедшей BM25 на BEIR в zero-shot режиме [20]. В модели INSTRUCTOR авторы пошли дальше: они предложили инструктивный fine-tuning, благодаря которому один и тот же энкодер удаётся переключать между задачами текстовыми инструкциями, обходясь без переобучения [21]. Стандартом сравнения качества эмбедингов в литературе сегодня выступают два бенчмарка — BEIR (18 датасетов, режим zero-shot retrieval) [46] и MTEB (8 задач, 58 датасетов, 112 языков) [47]. В Эксперименте №3 настоящей работы (прямая атака Vec2Text) в качестве модели-цели используется GTR-base размерности $d = 768$ — выбор обусловлен совместимостью с публичной реализацией `vec2text`; для интегрального эксперимента раздела 4.4 используется отдельный набор retrieval-моделей с поддержкой русского языка (см. ниже).

2. `multilingual-e5-small` — компактная модель из семейства E5 (Multilingual Embeddings) [20]. Имеет 118 млн параметров, размерность эмбединга $d = 384$, нативно поддерживает русский язык в составе 94 языков MIRACL, обучена контрастивно на корпусе CCPairs с дистилляцией из `e5-base`. На бенчмарке MTEB [47] (58 датасетов, 112 языков) `e5-small` устойчиво превосходит мульти-задачные дистиллированные модели сопоставимого размера по retrieval-метрикам, что делает её естественным выбором для русскоязычной задачи поиска. Размерность 384 удобна для гомоморфного шифрования (после SVD-усечения с $k = 192$ укладывается в один CKKS-вектор при $N = 8192$ с большим запасом по слотам), а формат «`query: .../passage: ...`» с mean-pooling обеспечивает совместимость с одноуровневым FAISS-индексом и перанкингом `ct-pt` без дополнительного fine-tuning. `multilingual-e5-base`, `multilingual-e5-large` и `BAAI/bge-m3` покрывают более ёмкие точки в семействе retrieval-trained энкодеров с тем же протоколом; `paraphrase-multilingual-mpnet-base-v2` включён как представитель альтернативного семейства обучения для проверки переносимости метода.

В качестве кандидатов для интегрального эксперимента раздела 4.4 отобраны пять энкодеров, покрывающих три размерности эмбединга ($d \in \{384, 768, 1024\}$) и два семейства обучения (контрастивный retrieval и paraphrase-distillation):

1. `intfloat/multilingual-e5-small` ($d = 384$, retrieval-trained, семейство E5);
2. `intfloat/multilingual-e5-base` ($d = 768$, retrieval-trained, семейство E5);
3. `sentence-transformers/paraphrase-multilingual-mpnet-base-v2` ($d = 768$, paraphrase-distilled, семейство Sentence-Transformers/MPNet);
4. `intfloat/multilingual-e5-large` ($d = 1024$, retrieval-trained, семейство E5);
5. `BAAI/bge-m3` ($d = 1024$, retrieval-trained, семейство BGE).

Все пять моделей нативно поддерживают русский язык в составе мультиязычных корпусов, общедоступны на HuggingFace и охватывают актуальный для современных RAG-систем диапазон размерностей. Включение paraphrase-distilled mpnet наряду с retrieval-trained моделями E5/BGE позволяет проверить чувствительность метода к классу обучения энкодера. Для прямой атаки Vec2Text (Эксперимент №3) отдельно

используется англоязычная модель `sentence-transformers/gtr-t5-base` ($d = 768$): она совместима с публичной реализацией `vec2text` и даёт воспроизводимое сравнение с результатами Морриса и соавторов [2].

Наиболее массовое применение семантического поиска сегодня — это системы RAG (Retrieval-Augmented Generation) [17]. В широко цитируемом (более 1700 цитирований) обзоре Gao et al. за 2024 год выделены три парадигмы — Naive RAG, Advanced RAG и Modular RAG [25]. По сути, RAG соединяет генеративные возможности больших языковых моделей (LLM) с точностью традиционных поисковых движков.

Процесс обработки запроса в RAG:

1. Пользователь задает вопрос q .
2. Система поиска находит релевантные фрагменты документов $\{d_1, \dots, d_k\}$ (контекст).
3. LLM генерирует ответ y , опираясь на найденный контекст:

$$P(y|q) \approx P_{LLM}(y|q, d_1 \dots d_k) \quad (1.5)$$

где $P(y|q)$ — вероятность ответа y при заданном запросе q ; P_{LLM} — условное распределение, задаваемое большой языковой моделью; q — исходный вопрос пользователя; $d_1, \dots, d_k$ — k релевантных фрагментов документов, извлеченных системой поиска на предыдущем этапе; y — генерируемый ответ. Формула выражает ключевую идею RAG: вместо прямой генерации по запросу модель генерирует ответ, обусловленный как запросом, так и извлечённым контекстом, что повышает фактуальность и снижает галлюцинации.

Согласно отчетам McKinsey (2025), внедрение RAG-систем в корпоративном секторе выросло с 31% до 51% за последний год [48]. Развитием базовой архитектуры стали адаптивные варианты: Self-RAG [22], обучающий LM генерировать рефлексивные токены для самооценки релевантности, и Corrective RAG (CRAG) [23], применяющий легковесный оценщик качества ретривала. Для оценки качества RAG-пайплайнов предложен фреймворк RAGAS [24].

При этом сама RAG-архитектура (и в этом её уязвимое место) открывает целый новый класс атак: чтобы система работала, эмбединги внутренних, нередко конфиденциальных корпоративных документов приходится загружать во внешние или облачные векторные БД (Pinecone, Milvus [44], Qdrant и пр.). Тем самым векторная БД превращается в критическую точку всей системы информационной безопасности.

1.2 Проблема конфиденциальности векторных представлений

В современной обработке естественного языка (NLP) произошёл сдвиг, у которого две стороны. С одной — он дал заметный скачок эффективности информационных систем; с другой — открыл уязвимость, последствия которой касаются конфиденциальных

данных напрямую. Речь о массовом переходе от разреженных лексических представлений (TF-IDF, BM25) к плотным векторным — эмбедингам, что стало возможным благодаря развитию трансформерных архитектур (BERT, RoBERTa, GTR). Системы научились работать уже не с символьными наборами, а со смыслами. Но именно та высокая информативность плотных векторов, которая делает их эффективными в ранжировании и классификации, превращает их и в удобный контейнер для скрытой передачи — а значит, и потенциальной утечки — чувствительной информации.

Острее всего конфиденциальность эмбедингов встаёт в контексте RAG и облачных сервисов семантического поиска. Типовой сценарий выглядит так: организация индексирует свой внутренний массив документов — а в этих документах может быть и коммерческая тайна, и персональные данные, и медицинские записи — после чего преобразованные в векторы фрагменты отправляются во внешнее хранилище или сторонний векторный индекс. На индустриальном уровне довольно долго существовало ошибочное допущение: процесс векторизации воспринимался как односторонний и необратимый — по аналогии с криптографическим хешированием. Считалось, что замена текста набором из 768 или 1536 чисел с плавающей точкой сама по себе обеспечивает анонимизацию хотя бы потому, что результат уже нечитаем для человека.

Однако исследования последних лет показали, что предположение о необратимости эмбедингов не выполняется: существуют практические атаки инверсии, которые по векторному представлению способны частично восстанавливать исходный текст и извлекать чувствительные признаки [2, 3, 4, 5]. Song и Raghunathan (2020) показали три класса атак на эмбединги: инверсию (восстановление 50–70 % входных слов), вывод атрибутов и *membership inference* [6]. Эмбединги, кодирующие семантическое содержание, могут сохранять следы синтаксической структуры, стилистики и, что наиболее критично, именованных сущностей; в сочетании с мощными генеративными декодерами это формирует прикладной вектор атаки.

В условиях, когда глобальный объем данных экспоненциально растет и потребность в интеллектуальном поиске становится повсеместной, игнорирование проблемы конфиденциальности векторных представлений создает системный риск. Компрометация векторной базы данных, содержащей миллионы эмбедингов, эквивалентна утечке самих исходных документов, но с той разницей, что факт утечки гораздо сложнее детектировать, а объём скомпрометированных данных сложнее оценить.

Для понимания глубины проблемы необходимо обратиться к математической природе эмбедингов. Современные энкодеры обучаются на гигантских корпусах текстов с целью минимизации контрастивных функций потерь. Цель обучения — сблизить векторы семантически похожих текстов и отдалить векторы непохожих. Чтобы выполнять эту задачу эффективно, модель вынуждена кодировать в векторе фиксированной размерности (например, $d = 768$) максимально возможное количество информации о входном тексте.

Здесь вступает в силу концепция эффективной размерности. Номинальная размерность вектора часто избыточна по сравнению с его эффективной размерностью, необходимой для описания основной семантики. Эта «лишняя» емкость вектора не остается пустой; она заполняется высокочастотными деталями, специфическими особенностями формулировок и точными значениями токенов. Именно эта избыточная информация, распределенная по малым сингулярным значениям, становится «топливом» для атак инверсии.

Если бы эмбединги сохраняли только абстрактный смысл (например, «договор купли-продажи недвижимости»), восстановление конкретных имен, сумм и дат было бы невозможным. Однако нейросетевые модели, стремясь к максимизации точности поиска (Acc@k), обучаются различать нюансы: договор с «Ивановым» должен отличаться от договора с «Петровым». Эта различимость на уровне векторов означает наличие информации, достаточной для восстановления конкретных персональных данных, что и эксплуатируют атаки класса Vec2Text и GEIA.

Атаки инверсии эмбедингов представляют собой класс состязательных методов, направленных на решение обратной задачи: для заданного вектора $\mathbf{v}$ найти такой текст $\hat{x}$, что его эмбединг $E(\hat{x})$ будет максимально близок к $\mathbf{v}$ в некоторой метрике (обычно евклидовой или косинусной). Формально задача оптимизации выглядит следующим образом [2]:

$$\hat{x} = \arg \max_{x' \in X} P(x' | \mathbf{v}) = \arg \max_{x' \in X} P(x' | E(x)) \quad (1.6)$$

где $\hat{x}$ — восстановленный (инвертированный) текст; x' — текст-кандидат из пространства поиска; X — пространство всех допустимых текстов; $\mathbf{v} = E(x)$ — целевой эмбединг, подлежащий инверсии; E — функция кодирования (энкодер), отображающая текст в векторное представление; $P(x' | \mathbf{v})$ — апостериорная вероятность текста x' при условии наблюдаемого вектора $\mathbf{v}$. Задача состоит в нахождении наиболее вероятного текста, порождающего заданный эмбединг. Сложность этой задачи заключается в дискретности пространства X и высокой размерности пространства эмбедингов. На сегодняшний день выделяются три основных подхода к реализации таких атак, каждый из которых демонстрирует различный баланс между требованиями к доступу и качеством восстановления.

Метод Vec2Text, предложенный Моррисом и соавторами в 2023 году [2], на сегодняшний день играет роль «золотого стандарта» среди атак инверсии. Главная его идея — отказаться от попыток восстановить текст в один проход и заменить такой подход итеративным уточнением гипотезы; по сути, авторы переносят логику оптимизации в непрерывных пространствах в дискретное пространство токенов.

Сама архитектура Vec2Text составная — она включает две нейросети, специально обученные под задачу инверсии:

1. Инверсионная модель. По целевому эмбедингу $\mathbf{v}$ она строит начальную аппроксимацию текста x^{-0} . Её можно воспринимать как генератор «черновика» —

семантически он, как правило, близок к оригиналу, но детали в нём почти наверняка искажены.

2. Корректирующая модель. Это та часть, на которой держится весь метод. На вход подаётся текущая гипотеза x^{-t} и вектор расхождения $v_{diff} = \mathbf{v} - E(x^{-t})$, где E — атакуемый энкодер. Цель шага — переписать гипотезу так, чтобы её эмбединг сместился ближе к целевому вектору.

Особенно стоит подчеркнуть модель угроз, на которую опирается Vec2Text. Доступ к весам энкодера (white-box access) для атаки не требуется — достаточно иметь возможность отправлять запросы и получать эмбединги (query access). Именно это позволяет применять её к проприетарным моделям, доступным только через API. За счёт корректирующей модели метод фактически «нащупывает» правильный текст, шаг за шагом подтягивая гипотезу к оригиналу — вплоть до уточнения имён и цифр, поскольку любое расхождение в тексте проявляется в векторе ошибки.

Метод GEIA (Gradient-based Embedding Inversion Attack) реализует подход, вдохновлённый методами генерации состязательных примеров [4]. Этот метод требует полного доступа к модели («белый ящик»), так как он полагается на вычисление градиентов функции энкодера.

Поскольку текст является дискретной последовательностью токенов, прямое дифференцирование по тексту невозможно. GEIA обходит это ограничение, перенося задачу оптимизации в непрерывное пространство эмбедингов токенов (input embeddings space). Пусть $\mathbf{z}$ — последовательность непрерывных векторов, подаваемых на вход первого слоя трансформера. Атака формулируется как задача минимизации функционала:

$$\mathbf{z}^* = \arg \min_{\mathbf{z}} (\|E_{int}(\mathbf{z}) - \mathbf{v}\|^2 + \lambda R(\mathbf{z})) \quad (1.7)$$

где $\mathbf{z}^*$ — оптимальное непрерывное представление в пространстве входных эмбедингов токенов; $\mathbf{z}$ — оптимизируемая последовательность непрерывных векторов, подаваемых на вход первого слоя трансформера; $E_{int}(\mathbf{z})$ — внутренняя часть модели энкодера, вычисляющая эмбединг предложения по входным представлениям токенов (от слоя эмбедингов до финального пулинга); $\mathbf{v}$ — целевой эмбединг, подлежащий инверсии; $\|\cdot\|^2$ — квадрат евклидовой нормы (L_2 -расстояние между текущим и целевым эмбедингом); λ — гиперпараметр, задающий баланс между точностью реконструкции и правдоподобностью текста; $R(\mathbf{z})$ — регуляризатор, штрафующий $\mathbf{z}$ за отклонение от реальных эмбедингов токенов из словаря, обеспечивая генерацию связного текста, а не случайного шума.

Полученное оптимальное непрерывное представление $\mathbf{z}^*$ далее проецируется на ближайшие дискретные токены словаря. Плюс GEIA — обходимся без сложных вспомогательных моделей, как в Vec2Text. Минус — метод плохо масштабируется по длине текста: на длинных последовательностях градиенты затухают либо «застывают» в субоптимальных локальных минимумах, и качество восстановления уступает авторегрессионным подходам.

TEIA (Text Embedding Inversion Attack) [5] — самый простой и наиболее быстрый из рассмотренных подходов. В его основе лежит обучение прямого декодера: атакующий обучает модель D_Ψ — как правило, на базе Transformer Decoder вроде GPT-2 или T5-decoder, — которая по эмбедингу v напрямую предсказывает исходный текст x :

$$\hat{x} = D_\Psi(\mathbf{v}) \quad (1.8)$$

где $\hat{x}$ — восстановленный текст; D_Ψ — обученная модель-декодер с параметрами Ψ ; $\mathbf{v}$ — входной эмбединг, подлежащий инверсии. В отличие от Vec2Text, атака выполняется за один проход без итеративной коррекции.

Декодер обучают на большом наборе пар «текст — эмбединг», стандартной функцией потерь служит кросс-энтропия между сгенерированным и эталонным текстом. Сильная сторона TEIA — скорость: один проход сети на запрос, и за разумное время можно обработать миллионы эмбедингов. Платой за это становится более низкая точность — у метода нет механизма обратной связи, как у Vec2Text.

Если свести характеристики этих трёх методов в одну таблицу, иерархия угроз становится наглядной. В таблице 1.1 видно, что качество восстановления и сложность метода идут рука об руку.

Кроме самой инверсии, к этому же кругу проблем примыкают и другие классы атак. Membership inference [7] даёт возможность установить, входил ли конкретный объект в обучающую выборку ML-модели. В работе [8] Carlini и соавторы продемонстрировали извлечение из GPT-2 дословных фрагментов обучающих данных — в том числе персональных. Melis и соавторы [49] ещё раньше показали, как через обновления моделей утекают свойства датасета. Для систематизации состязательных атак на NLP-модели в [50] собран фреймворк TextAttack — он объединяет 16 методов, включая TextFooler и BERT-Attack. И, наконец, в [9] Zeng и соавторы провели, по сути, первое комплексное эмпирическое исследование рисков приватности именно для RAG-систем и показали уязвимость к извлечению содержимого приватной retrieval-базы.

Таблица 1.1 — Сравнительная характеристика методов инверсии эмбедингов

Метод	Модель угроз	BLEU-score (GTR-base)	Вычислительная сложность	Тип архитектуры
Vec2Text	Query Access (API энкодера)	$\approx 97,3\%$ (32-token, BLEU)	Высокая (≈ 50 итераций)	Encoder- Decoder (T5)
GEIA	White Box (веса модели)	до 78%	Средняя (обратное распространение)	Optimization- based
TEIA	Dataset Access (для обучения)	до 65%	Низкая (один проход)	Decoder-only

Из таблицы видно главное: метод Vec2Text достигает near-perfect качества восстановления (BLEU около 97,3) и exact recovery до 92 % для коротких последовательностей из 32 токенов. Это количественное наблюдение и объясняет, почему привычной защиты периметра здесь недостаточно [2].

Чтобы перейти из плоскости общих рассуждений к строгим количественным оценкам, рассмотрим конкретные метрики успешности атак и экспериментальные результаты, полученные на эталонных датасетах. В работах по безопасности эмбедингов используют тот же набор метрик, что и в машинном переводе, — только интерпретируется он уже в терминах утечки информации.

1. BLEU (Bilingual Evaluation Understudy) [51]: основная метрика, измеряющая точность восстановления n -грамм (последовательностей из n слов):

$$\text{BLEU} = \text{BP} \cdot \exp\left(\sum_{n=1}^N w_n \log p_n\right) \quad (1.9)$$

где N — максимальный порядок учитываемых n -грамм (обычно $N = 4$); p_n — модифицированная точность n -грамм, вычисляемая как отношение числа совпавших n -грамм к общему числу n -грамм в восстановленном тексте, с ограничением по максимальному числу вхождений в референсе; w_n — весовой коэффициент для n -грамм порядка n (обычно $w_n = 1/N$ при равномерном взвешивании); BP — штраф за краткость, определяемый как $\text{BP} = \exp(1 - r/c)$ при $c < r$ и $\text{BP} = 1$ при $c \geq r$, где c — длина восстановленного текста, r — длина референсного (оригинального) текста. BLEU-1 (униграммы) оценивает восстановление «мешка слов», а BLEU-4 (четырёхграммы) — восстановление синтаксических конструкций и связных фраз. Значение $\text{BLEU} > 90\%$ свидетельствует о практически полной идентичности текстов.

2. Token F1 Score: гармоническое среднее между точностью и полнотой восстановления множества токенов. Эта метрика показывает, насколько полно злоумышленник смог извлечь словарный запас документа, игнорируя порядок слов.

3. Exact Match (точное совпадение): самая строгая метрика, показывающая долю документов, восстановленных побуквенно точно, без единой ошибки.

Эксперименты, проведённые Моррисом и соавторами на датасете MS MARCO с использованием модели GTR-base [2], показали, что для коротких последовательностей из 32 токенов атака Vec2Text достигает near-perfect $\text{BLEU} = 97,3$ и $\text{exact recovery} = 92\%$.

Эти результаты относятся именно к коротким входам и не должны механически переноситься на любые корпуса и длины текста. Вместе с тем они демонстрируют принципиальную уязвимость незащищённых эмбедингов и подтверждают, что компрометация векторной базы может приводить к раскрытию исходного содержания.

По сути, эти цифры означают, что современные плотные ретриверы могут раскрывать значительно больше информации, чем предполагалось исходной моделью угроз.

Отдельной категорией рисков идёт восстановление персональных данных. В отличие от утечки абстрактного смысла, утечка конкретных идентификаторов уже несёт прямые юридические последствия. Профильные эксперименты по восстановлению ПДн из эмбедингов дали такие значения точности (Ассигасу):

- Имена собственные: 89,2%. С вероятностью почти 90% имя человека, упомянутое в документе, будет корректно извлечено из его вектора.
- Адреса электронной почты: 76,4%.
- Физические адреса: 71,3%.
- Номера телефонов: 68,7%.

Объяснение тому, почему восстановление ПДн настолько эффективно, лежит в самой логике обучения: для поиска по конкретной сущности энкодеру выгодно «запоминать» редкие токены и их сочетания. Полезное для поиска свойство («найти договор с Ивановым») оказывается прямой угрозой конфиденциальности, как только речь заходит об обратной задаче — восстановить данные Иванова из вектора [2].

В литературе [2] зафиксирован ряд устойчивых корреляций между параметрами системы и успешностью атак:

1. Размерность эмбединга. Чем выше размерность вектора, тем шире канал утечки. Если, например, перейти с 768 к 1536 измерениям (как в text-embedding-ada-002), уязвимость возрастает примерно на 12 %.
2. Длина текста. Зависимость здесь немонотонна. На очень коротких текстах (менее 10 токенов) восстановление фактически безупречно — BLEU-4 порядка 97 %. С ростом длины комбинаторная сложность задачи инверсии растёт быстрее, и точность падает. Тем не менее даже на текстах в 128 токенов — а это типовой размер пассажа в RAG — точность держится на уровне около 78 %, чего вполне хватает для извлечения и смысла, и фактологии.

Имея на руках техническое описание атак, уже можно строить детализированную модель угроз и оценивать риски в разрезе отраслей. Последствия инверсии сильно зависят от типа данных и сценария: где-то речь идёт о репутационных потерях, а где-то — о вполне ощутимой уголовной ответственности.

Применительно к облачным сервисам семантического поиска удобно выделить четыре архетипа нарушителя [10]:

1. «Честный, но любопытный» сервер (honest-but-curious). Это наиболее реалистичная модель для корпоративных пользователей облака. Провайдер векторной БД формально следует протоколу поиска, но в фоновом режиме способен анализировать хранящееся содержимое — для обучения собственных моделей, профилирования клиентов или попросту в интересах промышленного шпионажа.
2. Злонамеренный инсайдер (malicious insider) — сотрудник провайдера (администратор БД, инженер DevOps), у которого есть легитимный доступ к физическим носителям или к памяти серверов.

3. Внешний злоумышленник (external attacker) — атакующий, проникший в векторный индекс через уязвимости API: SQL-инъекции, ошибки контроля доступа и т. п.

Секторальный анализ последствий.

Здравоохранение. Медицинские данные относятся к категории специальных персональных данных, требующих наивысшего уровня защиты. Сценарий: медицинская информационная система использует RAG для поиска похожих анамнезов. Эмбединги историй болезни пациентов передаются в облако. Риск: атака инверсии раскрывает диагнозы (ВИЧ, онкология, психиатрия), схемы лечения и иные чувствительные сведения.

Юридический сектор. Адвокатская тайна является фундаментом юридической практики. Сценарий: юридическая фирма использует семантический поиск для анализа прецедентов и внутренней документации. Риск: компрометация стратегий защиты, черновиков контрактов, переписки с клиентами.

Корпоративный сектор. Системы управления знаниями агрегируют всю внутреннюю документацию. Сценарий: индексация технической документации, планов R&D, финансовых отчётов. Риск: промышленный шпионаж. Конкуренты могут восстановить детали инновационных разработок или условия готовящихся сделок.

Государственный сектор. Использование LLM и семантического поиска в государственном управлении ограничивается требованиями к работе с гостайной. Риск: утечка сведений ограниченного доступа через векторные индексы. Последствия: угроза национальной безопасности, что диктует необходимость использования только сертифицированных решений или методов с гарантированной криптографической стойкостью. В отечественной литературе Астахова и соавторы (2016) обосновали применение гомоморфного шифрования для защиты облачных баз персональных данных в соответствии с 152-ФЗ [52].

Проблема конфиденциальности векторных представлений переходит из плоскости теоретической информатики в плоскость экономической реальности при анализе стоимости утечек данных и регуляторных штрафов.

В контексте атак на эмбединги затраты формируются из нескольких типовых компонент:

1. Обнаружение и эскалация: аудит, расследование (forensics), кризисный менеджмент. Для атак на эмбединги задача усложняется тем, что компрометация может не оставлять явных следов (пассивная реконструкция).
2. Потеря бизнеса: отток клиентов, репутационный ущерб.
3. Реагирование после инцидента: юридические расходы, выплаты компенсаций, организационные меры по поддержке пострадавших.
4. Уведомление: обязательное информирование субъектов данных и регуляторов.

Отраслевая дифференциация стоимости утечек подтверждает критичность защиты доменов, где обрабатываются медицинские, финансовые и иные чувствительные данные [10].

Законодательство в области защиты данных ужесточается глобально, и векторные представления попадают под регулирование как «производные данные», позволяющие идентифицировать личность.

- Федеральный закон № 152-ФЗ «О персональных данных» [11] предписывает оператору комплекс организационных и технических мер по обеспечению конфиденциальности ПДн и контролю доступа к ним.

- Приказ ФСТЭК России № 21 [12] конкретизирует меры защиты для каждого из четырёх уровней защищённости. ГОСТ 34.12-2018 [53] закрепляет национальные криптографические алгоритмы — «Кузнечик» и «Магма», — и именно с этими алгоритмами имеет смысл сопоставлять СККС на предмет соответствия отечественным требованиям. Отдельный пласт ограничений возникает при использовании внешних сервисов хранения/поиска: на первый план выходит и регламент трансграничной передачи данных, и риск несоответствия локальным нормативным актам.

- GDPR (Евросоюз). Штрафы здесь — до 4 % глобального годового оборота либо до €20 млн (в зависимости от того, какая величина окажется больше). Принципиально важен заложенный в нём подход Privacy by Design: защита должна закладываться на уровне архитектуры, а не «прикручиваться» поверх готовой системы. В этой логике хранение обратимых эмбеддингов, содержащих РИ, в открытом виде уже становится потенциальным несоответствием требованиям [13].

- HIPAA / CCPA: строгие стандарты защиты медицинской и потребительской информации в США.

Согласно отчёту IBM [10], применение шифрования и технологий конфиденциальных вычислений снижает ожидаемый ущерб от инцидентов и уменьшает регуляторные и репутационные риски.

Экономическая модель принятия решения о внедрении защиты описывается неравенством:

$$I < p \cdot L \cdot r \quad (1.10)$$

где I — стоимость внедрения и эксплуатации защитных технологий (капитальные и операционные затраты); p — вероятность наступления инцидента утечки данных за рассматриваемый период; L — потенциальный ущерб от инцидента (порядка 4,88 млн долларов США по оценкам отчёта IBM [10], включая прямые и косвенные потери); $r \in [0, 1]$ — доля предотвращённого ущерба при наличии защитных мер (так что $p \cdot L \cdot r$ — ожидаемая выгода от внедрения защиты, а $p \cdot L \cdot (1 - r)$ — ожидаемый остаточный ущерб). Неравенство выполняется, когда затраты на защиту меньше ожидаемой выгоды от снижения ущерба, то есть внедрение экономически оправдано.

Проведенный анализ проблемы конфиденциальности векторных представлений позволяет сформулировать следующие ключевые выводы, определяющие направление дальнейшего исследования:

1. Существование критической уязвимости: векторные представления текстов не являются анонимными. Современные атаки инверсии (Vec2Text, GEIA) позволяют восстанавливать исходные тексты с очень высоким качеством; для коротких последовательностей из 32 токенов в работе [2] получен результат с BLEU = 97,3 и точным восстановлением до 92 %, а персональные данные восстанавливаются с точностью до 89 %.

2. Защита периметра и шифрование диска не закрывают компрометацию ни на стороне провайдера, ни через API; защищать данные нужно непосредственно в процессе вычислений.

3. Экономика и регуляторика делают открытое использование семантического поиска в чувствительных доменах попросту неприемлемым: финансовый риск утечек слишком велик, а нормативные требования всё жёстче [10].

4. Нужны новые методы. Требуются такие схемы защищённого семантического поиска, которые одновременно противостоят инверсии, держат качество поиска на приемлемом уровне и не разрушают производительность — то есть фактически разрешают трилемму «безопасность — точность — производительность».

Иначе говоря, конфиденциальность эмбедингов — это не только открытая научная задача, но и реальное узкое место в развитии всей индустрии интеллектуальных информационных систем.

Из сформулированных выводов вытекают и требования к практическому решению:

1. уменьшить возможность восстановления исходных текстов и чувствительных сущностей — как из эмбедингов, так и из результатов вычислений;
2. сохранить качество семантического поиска на уровне базовой модели;
3. удерживать накладные расходы и время ответа в допустимых пределах. Обзор существующих классов защитных подходов и их ограничений вынесен в следующую главу; разработка и экспериментальная проверка предлагаемого метода — в последующие.

1.3 Систематизация угроз приватности эмбедингов

Чтобы корректно поставить задачу защиты, удобно явно зафиксировать классификацию угроз. Современные исследования в области приватности эмбедингов [2, 3, 4, 5, 7, 8, 6, 49, 9] выделяют четыре крупных класса атак, разнящихся по требованиям к доступу атакующего, целям и применимости.

Embedding inversion (атаки восстановления текста по вектору). Это центральный для настоящей работы класс атак. Цель — по эмбедингу $v = E(T)$ восстановить текст T или максимально близкое к нему $\hat{T}$. По доступу атакующего к энкодеру выделяют:

- Query-access атаки — атакующий имеет API энкодера и может получать $E(\cdot)$ для произвольных текстов. К этому классу относится Vec2Text [2], использующий

итеративную модель-корректор и достигающий BLEU $\approx 97,3\%$ на коротких 32-токенных входах для GTR-base.

- White-box атаки — атакующий имеет доступ к весам модели и градиентам. Пример: GEIA [4], формулирующая инверсию как градиентную оптимизацию в непрерывном пространстве эмбедингов токенов.

- Dataset-access атаки — атакующий обучает декодер на парах (текст, эмбединг) и применяет его за один проход. Пример: TEIA [5].

Эти три варианта дают разный компромисс между доступностью и качеством восстановления, который ранее обсуждался в контексте таблицы инверсионных атак.

Membership inference (определение принадлежности к обучающей выборке). По заданному примеру атакующий пытается определить, входил ли он в обучающий набор. На уровне эмбедингов это превращается в анализ распределения векторов из base population против out-of-distribution. Методология систематизирована в [7]; для NLP-моделей конкретные результаты приведены в [6].

Извлечение обучающих данных из LLM (training data extraction). В [8] Carlini и соавторы показали, что фрагменты обучающих данных (включая персональные сведения) могут быть дословно извлечены из больших языковых моделей через специально сконструированные запросы. Это смежный класс угроз: он не атакует эмбединг как объект, а извлекает информацию из весов модели как таковой.

Property inference и атаки на распределения. Атакующий восстанавливает свойства распределения обучающего корпуса (например, демографические параметры). В [49] Melis и соавторы показали, что эта информация может утекать через градиенты при федеративном обучении.

В контексте RAG-систем эти угрозы накладываются друг на друга. Утечка векторной БД (snapshot leak) даёт атакующему набор эмбедингов; embedding inversion переводит их обратно в тексты; membership inference — позволяет проверить, есть ли в БД конкретный документ. В работе [9] Zeng и соавторы провели первое систематическое эмпирическое исследование рисков приватности именно RAG-систем и продемонстрировали, что компрометация retrieval-базы — эффективный канал утечки. Это фиксирует приоритеты защиты: настоящая работа концентрируется на классе embedding inversion как наиболее массовой и изученной угрозе для retrieval-сценариев.

Связь с моделью угроз.

Описанные три архетипа нарушителя (honest-but-curious server, malicious insider, external attacker) для разных классов атак опасны по-разному:

- Для embedding inversion наиболее опасны honest-but-curious server и snapshot attacker — им достаточно прочитать векторную БД.

- Для membership inference актуальнее API-доступ к запросному интерфейсу — атакующий проверяет принадлежность через обращения.

- Для training data extraction релевантен любой пользователь облачной LLM, имеющий возможность отправлять prompt'ы.

Защита, предлагаемая в настоящей работе, нацелена прежде всего на первый класс (snapshot и honest-but-curious): SVD-проекция базы устраняет основной канал утечки текстового содержимого, а СККС защищает запрос во время передачи и при обработке на сервере.

2 Обзор существующих методов защиты векторных представлений

Проблема конфиденциальности векторных представлений, детально проанализированная в предыдущей главе, требует разработки эффективных защитных механизмов, способных противостоять атакам инверсии без существенного снижения качества семантического поиска. В настоящей главе проводится систематический обзор существующих подходов к защите эмбеддингов, охватывающий теоретико-информационные методы (дифференциальная приватность), криптографические протоколы (гомоморфное шифрование) и методы снижения размерности (SVD-проекции, ортогональные преобразования). Основной акцент сделан на математическом аппарате снижения размерности, составляющем ядро предлагаемого метода, и на практических характеристиках гомоморфного шифрования, существенных для архитектуры системы. Глава завершается сравнительным анализом всех рассмотренных методов в контексте трилеммы «безопасность — точность — производительность», формирующим направление дальнейшего исследования.

В современном NLP, где основную роль играют большие языковые модели (LLM) и плотные эмбеддинги, безопасность данных давно перестала быть смежной технической задачей и превратилась в один из ключевых факторов, ограничивающих использование интеллектуальных систем в критичных отраслях. Долгое время эмбеддинги воспринимались как безопасное представление: исходного текста в них нет — значит, и утечь, казалось бы, нечему. Серия работ 2023–2024 годов с атаками семейства Embedding Inversion (Vec2Text, GEIA, TEIA) разрушила эту картину, экспериментально показав, что исходный текст из эмбеддинга восстанавливается, причём с высокой точностью.

Тем самым изменилась и сама модель угроз для систем семантического поиска и RAG: компрометация векторной БД фактически приравнялась к утечке исходных неструктурированных документов. А поскольку через такие БД проходят медицинские карты, юридические контракты, внутренняя корпоративная переписка, последствия подобной утечки нетрудно представить.

В ответ на эти риски в литературе сформировался набор защитных механизмов, которые удобно разделить на три направления:

- Стохастические методы пертурбации (Stochastic Perturbation Methods): Дифференциальная приватность (Differential Privacy, DP) — внесение калиброванного шума

в данные для маскировки индивидуального вклада записи. Главный вызов — поиск баланса между уровнем шума и сохранением полезного сигнала для задач поиска.

– Криптографические протоколы: гомоморфное шифрование (Homomorphic Encryption, HE) и безопасные многосторонние вычисления (Secure Multi-Party Computation, MPC) — обработка данных в зашифрованном виде. Они обеспечивают наивысшие гарантии безопасности, но имеют серьёзные проблемы вычислительной масштабируемости и латентности.

– Детерминированные методы трансформации: снижение размерности (SVD/PCA) и ортогональные преобразования — защита за счет необратимой потери информации (сжатие) или сокрытия базиса пространства (ротация). Обеспечивают высокую скорость работы при умеренном уровне защиты.

2.1 Дифференциальная приватность

Дифференциальная приватность (DP), формализованная Синтией Дворк в 2006 году [54], представляет собой один из наиболее строгих подходов к алгоритмической приватности. DP даёт формальную гарантию того, что наличие или отсутствие одного объекта в наборе данных несущественно влияет на распределение выходов алгоритма.

Формально, рандомизированный механизм $M : \mathcal{D} \rightarrow \mathcal{R}$ удовлетворяет (ε, δ) -дифференциальной приватности, если для любых двух соседних наборов данных $D, D' \in \mathcal{D}$, отличающихся ровно одной записью, и для любого подмножества выходов $S \subseteq \mathcal{R}$ выполняется неравенство:

$$\Pr[M(D) \in S] \leq e^\varepsilon \cdot \Pr[M(D') \in S] + \delta,$$

где M — рандомизированный механизм, отображающий набор данных из $\mathcal{D}$ в множество выходов $\mathcal{R}$; D, D' — два соседних набора данных, отличающихся ровно одной записью (в контексте эмбедингов — одним вектором документа); $S \subseteq \mathcal{R}$ — произвольное подмножество выходов; $\varepsilon > 0$ — бюджет приватности, ограничивающий мультипликативное различие вероятностей (меньшие значения соответствуют более сильной конфиденциальности); $\delta \geq 0$ — параметр релаксации (обычно выбирается порядка $1/|D|$), допускающий малую вероятность нарушения гарантии.

Для защиты векторных представлений наиболее часто применяется механизм Гаусса. К вектору эмбединга $v \in \mathbb{R}^d$ добавляется вектор шума η , компоненты которого сэмпляются из нормального распределения $\mathcal{N}(0, \sigma^2)$. Стандартное отклонение σ , обеспечивающее (ε, δ) -DP, определяется чувствительностью функции и параметрами приватности:

$$\sigma \geq \frac{\Delta_2 f \cdot \sqrt{2 \ln(1,25/\delta)}}{\varepsilon}$$

где σ — стандартное отклонение гауссовского шума, добавляемого к каждой компоненте вектора эмбединга; $\Delta_2 f$ — L_2 -чувствительность защищаемой функции

(для нормализованных эмбедингов на единичной гиперсфере $\Delta_2 f \leq 2$); $\varepsilon > 0$ — бюджет приватности; $\delta \in (0, 1)$ — параметр релаксации (обычно $\delta \approx 1/N$).

Теоретически элегантный подход на практике упирается в одну характерную проблему — «проклятие размерности». Современные модели эмбедингов (BERT, GTR, text-embedding-ada-002) работают на векторах размерности d от 768 до 1536, и для гарантий DP шум приходится добавлять к каждой координате.

Если каждая координата получает независимый гауссовский шум, то норма результирующего вектора шума растёт как $\sqrt{d}$. На высокой размерности это значит, что амплитуда шума легко выходит на уровень полезного сигнала или даже превышает его, и семантическая близость к релевантным запросам попросту разрушается. Особенно болезненно это проявляется на неструктурированных коллекциях — диалогах, служебной переписке, — где смысл во многом «зашит» в конкретные именованные сущности и контекст.

Понимание этого ограничения привело к появлению адаптивных схем пертурбации — например, TextCrafter, — где шум распределяется по направлениям пространства неравномерно. Расплачиваться приходится дополнительной сложностью и более жёсткими допущениями о модели угроз.

В итоге сама дифференциальная приватность даёт строгие математические гарантии, но в задачах семантического поиска по высокоразмерным эмбедингам это оборачивается падением точности на 15–25 % [55]. Для значительной части приложений такая деградация неприемлема — отсюда и интерес к решениям в области криптографии и детерминированных преобразований.

2.2 Гомоморфное шифрование

В отличие от методов пертурбации, криптографические подходы ставят перед собой принципиально иную цель: защитить данные, не жертвуя при этом точностью вычислений. Гомоморфное шифрование делает возможной саму арифметику над зашифрованными значениями — серверу не приходится расшифровывать данные, чтобы выполнить операцию. Идея полностью гомоморфного шифрования была впервые конструктивно реализована Джентри в 2009 году на основе идеальных решёток [56]; с тех пор основные практические схемы — BFV [57], BGV, CKKS [26] и TFHE [58], — образуют семейство решёточно-стойких алгоритмов с разными компромиссами между типом данных, шумом и производительностью.

Схема CKKS (Cheon–Kim–Kim–Song) была опубликована в 2017 году [26]. Её особенность — поддержка приближённой арифметики вещественных чисел «из коробки», и именно это делает CKKS удобной для линейной алгебры машинного обучения: скалярных произведений, матричных умножений и т. п. Криптографически схема опирается на задачу RLWE (Ring Learning With Errors); параметры выбираются под целевой уровень безопасности в соответствии с действующими стандартами [30].

Если смотреть на CKKS глазами разработчика ML-пайплайна, ключевыми оказываются три её свойства:

- SIMD-упаковка (Single Instruction, Multiple Data). Вектор эмбединга размерности d помещается в один шифротекст, содержащий $N/2$ слотов (при $N = 8192 - 4096$ слотов). Одно умножение полиномов в этом режиме эквивалентно покоординатному умножению тысяч чисел сразу — для производительности это решающий момент.

- Приближённая арифметика. В отличие от BFV/BGV, нацеленных на точную целочисленную работу, CKKS встраивает шум шифрования в естественную погрешность вещественной арифметики. При масштабирующем коэффициенте $\Delta \approx 2^{40}$ точности более чем достаточно для задачи ранжирования документов.

- Совместимость с режимом ct-pt: CKKS позволяет эффективно умножать зашифрованный вектор (ciphertext) на открытый вектор (plaintext), что является основой гибридной архитектуры, предлагаемой в данном исследовании.

Производительность CKKS критически зависит от режима взаимодействия:

- Ciphertext-Ciphertext (ct-ct): и запрос, и векторы базы данных зашифрованы. Обеспечивает максимальную безопасность, но операция умножения двух шифротекстов является вычислительно дорогой и требует дополнительной процедуры релинеаризации.

- Ciphertext-Plaintext (ct-pt): зашифрован только запрос, а база данных хранится в открытом (но проецированном/преобразованном) виде. Умножение шифротекста на открытый вектор не требует релинеаризации. Латентность значительно меньше.

Переход от режима ct-ct к ct-pt является ключевым архитектурным решением данного исследования. Конфиденциальность базы данных в этом случае обеспечивается не шифрованием, а SVD-проекцией с секретной ротацией, в то время как CKKS защищает конфиденциальность запросов пользователей.

Существует значительный разрыв между теоретическими оценками сложности и реальной производительностью CKKS, называемый Evaluation Gap. Теоретические модели часто недооценивают накладные расходы на управление памятью и кэш-промахи при работе с полиномами большого размера. Латентность даже в режиме ct-pt остается высокой (секунды на десятки тысяч векторов), что делает линейное сканирование средствами CKKS неприменимым для баз масштаба миллионов документов.

Альтернативой HE служит MPC: вычисления распределяются между несколькими серверами так, что ни один из них не видит исходных данных целиком. Где HE упирается прежде всего в вычислительные ресурсы (compute-bound), там MPC натывается на пропускную способность сети. В качестве рабочего примера можно привести гибридный фреймворк BLB (Breaking the Layer Barrier): он использует CKKS для линейных слоёв и MPC — для нелинейностей вроде Softmax и GELU, что заметно сокращает и коммуникационные расходы, и общую латентность. Платой выступает требование к развёртыванию: нужен пул из нескольких независимых серверов, что само по себе усложняет эксплуатацию.

2.3 Методы снижения размерности и изометрические преобразования

Третья группа подходов уходит от статистического шума и тяжёлой криптографии — и опирается на геометрию самих векторных пространств. Замысел простой: преобразовать данные так, чтобы метрические отношения, необходимые для поиска (то есть расстояния и углы), сохранились, а информация, на которой держится восстановление текста, исчезла.

Канонический пример — снижение размерности через PCA (Principal Component Analysis). В его основе лежит сингулярное разложение (SVD) матрицы эмбедингов $X = U\Sigma V^T$, и проекция на k главных компонент равнозначна усечению матриц U , Σ , V до ранга k .

Фундаментальным результатом здесь является теорема Эккарта–Янга–Мирского. Она утверждает, что матрица X_k , полученная путём усечённого SVD, является наилучшим приближением исходной матрицы X ранга k в смысле нормы Фробениуса. Ошибка аппроксимации равна:

$$\|X - X_k\|_F^2 = \sum_{i=k+1}^d \sigma_i^2,$$

где $\|X - X_k\|_F^2$ — квадрат нормы Фробениуса ошибки аппроксимации; X — исходная матрица эмбедингов; X_k — аппроксимация ранга k ; σ_i — сингулярные числа, соответствующие отброшенным компонентам; d — исходная размерность.

С точки зрения безопасности, эта ошибка интерпретируется как потеря информации. Чем больше «энергии» содержится в отброшенных компонентах («хвосте» спектра), тем сложнее атаке Vec2Text восстановить исходный вектор.

Лемма Джонсона–Линденштраусса гарантирует, что при проекции множества точек из пространства высокой размерности в пространство размерности $k = O(\ln(n)/\varepsilon^2)$ попарные расстояния сохраняются с относительной погрешностью не более ε [59]. Заметим, что лемма Джонсона–Линденштраусса описывает свойства *случайных* проекций; используемое в данной работе SVD-усечение представляет собой *data-dependent* проекцию, и связь между этими двумя классами не прямая. Адаптация леммы Джонсона–Линденштраусса как механизма обеспечения дифференциальной приватности рассмотрена в работе Кентхапади и соавторов [60], где случайная проекция используется одновременно как сжатие и как пертурбация. Тем не менее лемма Джонсона–Линденштраусса наталкивает на идею сохранения геометрии при существенном снижении размерности.

В отличие от PCA, для которого нужна ковариационная матрица данных, в методе случайных проекций (Random Projections) применяется фиксированная случайная матрица $R \in \mathbb{R}^{k \times d}$. Эта матрица играет роль секретного ключа: атакующий, получив доступ к проецированным векторам $v' = Rv$ и не зная R , оказывается

перед недоопределённой системой линейных уравнений — то есть сталкивается уже с информационно-теоретическим барьером для точного восстановления v .

Чтобы усилить защиту, поверх проекции добавляют секретное ортогональное преобразование $Q \in O(k)$ — попросту говоря, ротацию. Ортогональные матрицы изометричны ($\|Qx\| = \|x\|$), а значит, скалярные произведения и косинусные расстояния от такой ротации не зависят вовсе. Иначе говоря, точность поиска в проецированном пространстве остаётся прежней.

Для атакующего же ротация — это уже серьёзная помеха. Модели инверсии (Vec2Text) обучаются на эмбедингах в их «штатной» ориентации, и повернутые векторы для них почти неотличимы от случайного шума. Чтобы атаковать такую систему, злоумышленник должен сначала решить Ортогональную проблему Прокруста — найти матрицу Q , зная пары (v, v_{final}) . Без доступа к обучающей выборке пар (открытый вектор, защищённый вектор) это становится вычислительно сложной задачей.

В контексте нашей работы методы снижения размерности, традиционно воспринимаемые как инструмент компрессии данных и оптимизации вычислений, приобретают принципиально новое звучание. Мы выдвигаем тезис о том, что сингулярное разложение (SVD) способно выступать в качестве самостоятельного защитного примитива. Данный подход базируется на гипотезе, что семантическая информация, необходимая для поиска, и синтаксическая информация, используемая атаками восстановления, локализованы в различных подпространствах векторного представления.

Пусть $X \in \mathbb{R}^{n \times d}$ — матрица эмбедингов корпуса документов, где n — количество документов, а d — размерность пространства признаков (для моделей BERT-base $d = 768$, для text-embedding-ada-002 $d = 1536$). Каждая строка x_i матрицы X соответствует векторному представлению i -го документа. Предположим, что данные центрированы:

$$\sum_{i=1}^n x_{ij} = 0, \quad j \in \{1, \dots, d\},$$

где x_{ij} — значение j -й компоненты эмбединга i -го документа; n — число документов; d — размерность пространства. Центрирование обеспечивает совмещение начала координат с центром масс облака данных, что необходимо для корректной работы ковариационного анализа.

Теорема о существовании SVD. Для любой вещественной матрицы $X \in \mathbb{R}^{n \times d}$ существуют ортогональные матрицы $U \in \mathbb{R}^{n \times n}$ и $V \in \mathbb{R}^{d \times d}$, такие что:

$$X = U \Sigma V^T \tag{2.1}$$

где $X \in \mathbb{R}^{n \times d}$ — центрированная матрица эмбедингов корпуса; $U \in \mathbb{R}^{n \times n}$ — ортогональная матрица левых сингулярных векторов, столбцы u_i которой являются собственными векторами матрицы XX^T ; $\Sigma \in \mathbb{R}^{n \times d}$ — прямоугольная диагональная матрица сингулярных чисел $\sigma_1 \geq \sigma_2 \geq \dots \geq \sigma_r > 0$ (где $r = \text{rank}(X)$), характеризующих вклад каждой компоненты в общую дисперсию данных; $V \in \mathbb{R}^{d \times d}$ — ортогональная матрица

правых сингулярных векторов, столбцы v_i которой являются собственными векторами матрицы $X^T X$ и задают главные направления дисперсии в пространстве эмбедингов.

Выборочная ковариационная матрица C для центрированных данных:

$$C = \frac{1}{n-1} X^T X \quad (2.2)$$

где $C \in \mathbb{R}^{d \times d}$ — выборочная ковариационная матрица; n — число документов; X — центрированная матрица эмбедингов; делитель $(n-1)$ обеспечивает несмещённость оценки (поправка Бесселя).

Подставляя выражение $X = U \Sigma V^T$, получаем:

$$C = \frac{1}{n-1} V \Sigma^2 V^T = V \Lambda V^T,$$

где V — ортогональная матрица правых сингулярных векторов из разложения (2.1); $\Lambda = \text{diag}(\lambda_1, \dots, \lambda_d)$ — диагональная матрица собственных чисел ковариационной матрицы, связанных с сингулярными числами соотношением:

$$\lambda_i = \frac{\sigma_i^2}{n-1},$$

где λ_i — i -е собственное число, характеризующее дисперсию данных вдоль i -го главного направления; σ_i — i -е сингулярное число из разложения (2.1).

Геометрический смысл такой: правые сингулярные векторы v_i задают главные направления вариации данных, причём v_1 — направление максимальной дисперсии, v_2 — направление максимальной дисперсии уже в ортогональном к v_1 подпространстве, и так далее. Сам спектр $\{\sigma_i\}$ удобно воспринимать как «энергетический профиль» семантического пространства. У языковых моделей он убывает быстро — это и есть указание на скрытую низкоранговую структуру эмбедингов.

Геометрически SVD раскладывает линейный оператор X на три элементарные операции:

- Ротация базиса (V^T). Поворот системы координат таким образом, чтобы оси совпали с главными направлениями изменчивости данных. Это преобразование изометрично и обратимо.

Масштабирование (Σ): растяжение или сжатие вдоль координатных осей с коэффициентами σ_i . Направления с малыми σ_i «схлопываются» почти до нуля — именно на этом этапе и появляется разница между информативными и слабо информативными компонентами.

Ротация в целевом пространстве (U): поворот уже в $\mathbb{R}^n$.

Если принудительно обнулить сингулярные числа начиная с $k+1$ -го, данные проецируются на k -мерный гиперэллипсоид; его оси соответствуют наиболее значимым семантическим признакам.

Пусть X_k — аппроксимация, полученная сохранением первых k сингулярных чисел:

$$X_k = U_k \Sigma_k V_k^T = \sum_{i=1}^k \sigma_i u_i v_i^T \quad (2.3)$$

где $X_k \in \mathbb{R}^{n \times d}$ — аппроксимация исходной матрицы рангом k ; $U_k \in \mathbb{R}^{n \times k}$, $\Sigma_k \in \mathbb{R}^{k \times k}$, $V_k \in \mathbb{R}^{d \times k}$ — усечённые матрицы; $\sigma_i u_i v_i^T$ — матрицы ранга 1 (внешние произведения).

Теорема (Eckart–Young–Mirsky, 1936) [61]. Для любой матрицы $B \in \mathbb{R}^{n \times d}$ такой, что $\text{rank}(B) \leq k$, справедливо:

$$\|X - X_k\|_F \leq \|X - B\|_F \quad (2.4)$$

где $\|\cdot\|_F$ — норма Фробениуса. Ошибка аппроксимации выражается через отброшенные сингулярные числа:

$$\|X - X_k\|_F^2 = \sum_{i=k+1}^d \sigma_i^2 \quad (2.5)$$

где индексы суммирования $i = k + 1, \dots, d$ соответствуют отброшенным компонентам. Спектральная норма ошибки:

$$\|X - X_k\|_2 = \sigma_{k+1} \quad (2.6)$$

где σ_{k+1} — первое отброшенное сингулярное число, ограничивающее максимальное искажение длины вектора при проекции.

Следствие 1: Доля объяснённой дисперсии:

$$\eta_k = \frac{\sum_{i=1}^k \sigma_i^2}{\sum_{i=1}^d \sigma_i^2} \quad (2.7)$$

где $\eta_k \in [0, 1]$ показывает долю информационной «энергии», сохранившуюся после проекции; k — размерность проекции; d — полная размерность. Типичное требование для ML — $\eta_k \geq 0,95$. Для задач защиты необходим баланс: слишком высокое η_k может сохранить следы для атак инверсии.

Следствие 2: Сохранение скалярных произведений. Для двух векторов u, v их скалярное произведение после проекции:

$$\langle u_{\text{proj}}, v_{\text{proj}} \rangle = u^T V_k V_k^T v,$$

где $u_{\text{proj}} = V_k V_k^T u$, $v_{\text{proj}} = V_k V_k^T v$ — проекции; $V_k V_k^T$ — оператор проецирования. Ошибка ограничена величиной $O(\sigma_{k+1}^2)$, что при быстром убывании спектра эмбедингов обеспечивает выполнение требования сохранения точности поиска.

Понятие эффективной размерности формализует наблюдение о том, что реальные данные лежат на многообразии гораздо меньшей размерности, чем номинальная:

$$d_{\text{eff}} = \frac{(\sum_{i=1}^d \sigma_i^2)^2}{\sum_{i=1}^d \sigma_i^4} \quad (2.8)$$

где d_{eff} — эффективная размерность (participation ratio сингулярных чисел); при равномерном спектре $d_{\text{eff}} = d$, а при доминировании одного числа $d_{\text{eff}} \rightarrow 1$.

Для модели rubert-tiny2 ($d=312$) эффективная размерность составляет порядка 40–60. Для text-embedding-ada-002 ($d=1536$) — 200–300. Это означает, что избыточность представления огромна: можно отбросить до 70–80% измерений, потеряв лишь незначительную часть семантической информации, но существенно снизив вычислительную сложность для последующего гомоморфного шифрования.

В задаче защищённого поиска мы стремимся максимизировать ошибку восстановления исходного вектора при сохранении скалярных произведений.

Для вектора x его проекция задаётся как $x_{\text{proj}} = V_k V_k^T (x - \mu) + \mu$. Вектор ошибки:

$$e_{\text{rec}} = x - x_{\text{proj}} = (I - V_k V_k^T)(x - \mu) \quad (2.9)$$

где $e_{\text{rec}} \in \mathbb{R}^d$ — информация, уничтоженная проекцией; I — единичная матрица; $(I - V_k V_k^T)$ — проектор на ортогональное дополнение (подпространство отброшенных компонент); μ — вектор среднего.

Абсолютная и относительная ошибки. Введём две связанные, но разные по смыслу величины. Первая — абсолютная энергия отброшенного хвоста, усреднённый по выборке квадрат нормы остатка:

$$e_k^2 = \mathbb{E} \|e_{\text{rec}}\|^2 = \sum_{i=k+1}^d \lambda_i \quad (2.10)$$

где e_k^2 имеет размерность квадрата эмбединг-единиц; λ_i — собственные числа ковариационной матрицы, соответствующие отброшенным компонентам.

Вторая — безразмерная относительная ошибка реконструкции, нормированная на полную дисперсию:

$$\sigma_{\text{rec}}^2 = \frac{e_k^2}{\text{tr}(C)} = \frac{\sum_{i=k+1}^d \lambda_i}{\sum_{j=1}^d \lambda_j} = 1 - \eta_k \in [0, 1] \quad (2.11)$$

где $\text{tr}(C)$ — след ковариационной матрицы (полная дисперсия); η_k — доля объяснённой дисперсии из формулы (2.7). Именно σ_{rec} далее используется как гроху-показатель силы защиты (требование $\sigma_{\text{rec}} \geq 0,10$); его повекторный аналог $\sigma_{\text{rec}}(x; V_k) = \|x - \pi_k(x)\|_2 / \|x\|_2$ вводится в главе 4.

Условная дифференциальная энтропия остатка (при гауссовом приближении):

$$H(x \mid x_{\text{proj}}) = \frac{1}{2} \sum_{i=k+1}^d \log(2\pi e \lambda_i) \quad (2.12)$$

где $H(x \mid x_{\text{proj}})$ — условная дифференциальная энтропия (Шеннон, в натуральных единицах при $\log = \ln$) остатка проекции при гауссовом приближении распределения x ; λ_i — собственные числа, соответствующие отброшенным компонентам. Величина характеризует количество дисперсии, локализованной в ортогональном подпространстве.

Эмпирическая связь с Vec2Text:

$$\text{BLEU}_{\text{attack}} \approx \alpha - \beta \sigma_{\text{rec}} \quad (2.13)$$

где $\text{BLEU}_{\text{attack}}$ — метрика успешности атаки (высокий BLEU соответствует более точному восстановлению); α — базовый уровень BLEU без защиты; $\beta > 0$ — скорость убывания качества атаки с ростом безразмерной ошибки реконструкции $\sigma_{\text{rec}} \in [0, 1]$; гипотеза монотонности численно валидируется в Эксперименте №3.

Оптимизационная задача в этих обозначениях формулируется так: найти k , при котором σ_{rec}^2 (формула (2.11)) достаточно велика для блокирования атак (проху-критерий требует $\sigma_{\text{rec}} \geq 0,10$), а Acc@1 остаётся в пределах допуска в 5% относительно незащищённого baseline.

SVD-преобразование детерминировано и зависит только от обучающей выборки. Для обеспечения дополнительной конфиденциальности вводится секретная ортогональная матрица R :

$$O_k = \{R \in \mathbb{R}^{k \times k} : R^T R = R R^T = I_k\} \quad (2.14)$$

где O_k — ортогональная группа; I_k — единичная матрица. Ключевое свойство — изометрия:

$$\langle Ru, Rv \rangle = u^T R^T R v = u^T v = \langle u, v \rangle \quad (2.15)$$

где R — секретная ортогональная матрица; u, v — проецированные векторы. Это гарантирует, что ротация не влияет на результаты поиска.

Полное защитное преобразование:

$$v' = R \cdot V_k^T (v - \mu) \quad (2.16)$$

где $v' \in \mathbb{R}^k$ — защищённый вектор; $R \in O_k$ — секретная матрица (ключ), генерируемая с равномерным распределением по мере Хаара (QR-разложение случайной гауссовой матрицы); $V_k^T \in \mathbb{R}^{k \times d}$ — матрица проекции; $v \in \mathbb{R}^d$ — исходный эмбеддинг; $\mu \in \mathbb{R}^d$ — центроид корпуса.

SVD-проекция выполняет роль прекондиционера для СККС и выполняет следующие функции:

– Сжатие и информационно-теоретическая защита. Понижение размерности с d до $k < d$ даёт двойной эффект:

1. уменьшается размер СККС-вектора, что ускоряет st-pt операции и снижает накладные расходы по передаче зашифрованных данных;
2. отбрасываемые сингулярные направления нельзя восстановить, что обеспечивает информационно-теоретическую необратимость преобразования (проху-метрика $\sigma_{\text{rec}} > 0$). В основной операционной точке интегрального эксперимента применяется усечение до $k = d/2$, что даёт $\sigma_{\text{rec}} \geq 0,10$ для всех протестированных энкодеров.

– Снижение глубины схемы. Скалярное произведение векторов длины k требует $\log_2 k$ ротаций. Соответственно, уменьшение k позволяет работать с более «лёгкими» параметрами CKKS, не жертвуя уровнем безопасности.

– Ограничение динамического диапазона. За счёт нормализации, выполняемой при проекции, удаётся избежать переполнения при гомоморфном сложении. Иначе говоря, SVD-проекция выступает системообразующим элементом всей гибридной архитектуры — на ней и держится связка между эффективностью и безопасностью.

На масштабе порядка 10^6 документов классические алгоритмы SVD (LAPACK GESVD) со сложностью $O(nd^2)$ не подходят. В работе используется рандомизированный SVD (Halko, Martinsson, Tropp):

– Рандомизированный поиск подпространства. Матрица X умножается на случайную гауссову матрицу размера $d \times (k + p)$, где p — параметр передискретизации. В итоге исходная задача редуцируется к матрице $n \times (k + p)$.

– Детерминированное разложение. Точное SVD выполняется уже над этой малой матрицей.

Итоговая сложность алгоритма — $O(ndk)$, то есть линейна по числу документов. В этом режиме пересчёт V_k для корпусов из миллионов записей занимает секунды.

2.4 Сравнительный анализ: трилемма «Безопасность — Точность — Производительность»

Обобщая анализ методов, формулируется фундаментальное ограничение — трилемма защищенного семантического поиска: ни один монолитный подход не удовлетворяет одновременно трём ключевым требованиям промышленных систем.

Таблица 2.1 — Сравнительная характеристика методов защиты эмбедингов

Метод	Тип защиты	Влияние на точность (Acc@1)	Производи- тельность (Латент- ность)	Стойкость к атакам (Vec2Text)	Основные ограничения
Дифференци- альная приватность (DP)	Статистичес- кий (шум)	Высокая деградация (15–25%)	Высокая (не влияет)	Средняя (возможны адаптивные атаки)	Неприемлемое падение точности для high-dimensional data; сложность выбора ϵ .
Гомоморфное шифрование (CKKS)	Криптогра- фический	Пренебрежимо мала ($< 0.1\%$)	Низкая (100х–500х замедление)	Абсолютная (при стойкости ключа)	Экстремальные вычислительные затраты; evaluation gap; неприменимость для full-scan.

Метод	Тип защиты	Влияние на точность (Acc@1)	Производи- тельность (Латент- ность)	Стойкость к атакам (Vec2Text)	Основные ограничения
MPC (многосто- ронние вычисления)	Криптогра- фический	Отсутствует	Низкая (ограничена сетью)	Абсолютная	Высокие комму- никационные затраты (до 21x); требование нескольких серверов.
Снижение размерности (SVD/RP)	Детерминиро- ванный (сжатие)	Умеренная (3–5%) при $k/d \approx 0.3$	Очень высокая (ускорение)	Высокая (при $k \ll d$)	Отсутствие формальных криптографичес- ких гарантий (128 бит); риск будущих атак.
Гибридный подход (проекция + CKKS)	Комбиниро- ванный	Низкая (3–5%)	Средняя (оп- тимизируется)	Высокая	Сложность реализации и настройки параметров.

С точки зрения Парето-эффективности картина получается одинаковая: какой бы из методов мы ни взяли в чистом виде, одно из требований трилеммы всё равно нарушается.

DP отдаёт точность в обмен на скорость и формальную приватность — и эти потери часто оказываются неприемлемыми.

HE/MPC, наоборот, расплачивается скоростью за безопасность и точность, что делает систему практически немасштабируемой.

SVD даёт скорость и умеренную защиту, но оставляет теоретические лазейки для атак инверсии.

Из этого и вытекает гипотеза данного исследования: имеет смысл идти по пути гибридной архитектуры. Связка необратимых проекций (SVD) для быстрого предварительного отбора кандидатов и гомоморфного шифрования (CKKS) для точного ранжирования финалистов выглядит самым перспективным путём к разрешению трилеммы: скорость и компрессия проекций «вытягивают» медлительность CKKS, а криптографическая точность последнего, в свою очередь, компенсирует потерю информации при проекции. Подробная разработка и валидация метода — задача дальнейших глав.

2.5 Гибридные подходы и место настоящей работы в литературном ландшафте

Идея сочетания нескольких механизмов защиты для разрешения практических компромиссов в защищённых вычислениях не нова — ниже даётся систематизация наиболее близких к настоящей работе направлений.

1. Privacy-preserving ML на основе HE. CryptoNets [29] был первой работой, продемонстрировавшей выполнимость инференса нейросети целиком под FHE. Дальнейшие работы (Faster CryptoNets, GAZELLE, Cheetah, Delphi) последовательно снижали стоимость инференса за счёт перехода к гибридным схемам: линейные слои — через HE, нелинейные — через MPC или garbled circuits. Эти работы относятся в первую очередь к classification-задачам с фиксированной топологией модели; задача retrieval с миллионом кандидатов в них напрямую не рассматривается.

2. Компиляторы и предикторы FHE-параметров. EVA [62] и CHET [63] — декларативные компиляторы, принимающие на вход арифметическую цепочку и автоматически подбирающие параметры CKKS под эту цепочку. Hummingbird [64] ориентирован на ML-as-a-service нагрузки и использует эвристики для выбора параметров. Эти системы фокусируются на компиляции цепочек арифметических операций общего вида; настоящая работа дополняет это направление узкоспециализированным предиктором под конкретную операцию (скалярное произведение в режиме ct-pt) и под конкретную аппаратную платформу.

3. Случайные проекции как механизм защиты данных. Применение random projection и SVD-проекций для приватного хранения данных обсуждается с середины 2000-х (Liu et al. [65], Kargupta и соавторы). В контексте эмбедингов проекции рассматривались в [66] как часть обзора методов interpretable ML. Главное отличие настоящей работы — именно *количественная* модель защиты в виде теоремы о дифференцированной защите и её прямая экспериментальная валидация на современной атаке инверсии Vec2Text.

4. Дифференциальная приватность в эмбедингах. Помимо классических DP-SGD [32] и Renyi DP [33], в литературе рассматриваются адаптивные методы пертурбации эмбедингов (TextCrafter и аналогичные), стремящиеся улучшить компромисс между уровнем шума и сохранением полезного сигнала. На высокоразмерных моделях ($d \geq 768$) эти методы стабильно дают деградацию качества порядка 15–25 %, что плохо совместимо с требованиями интерактивности.

5. Атаки инверсии и их валидационные эксперименты. Vec2Text [2], GEIA [4], TEIA [5] образуют основной арсенал современных embedding inversion атак. В работах с упоминанием SVD-проекций как защиты обычно ограничиваются грубыми метриками типа σ_{rec} ; *прямой* запуск атаки на защищённые эмбединги встречается реже. Настоящая работа закрывает этот пробел в Эксперименте № 3: запускается полная официальная реализация Vec2Text на эмбедингах, прошедших SVD-усечение, и фиксируется измеренное BLEU при $k/d \in \{0,10; 0,25; 0,50; 0,75\}$.

Положение настоящей работы. Сочетая перечисленные направления, можно выделить специфику настоящей диссертации:

- фокус на задаче retrieval (а не classification/inference) на масштабе 10^6 документов;
- асимметричное применение защиты: легковесная информационно-теоретическая защита для большой статической базы и более тяжёлый CKKS для единичного запроса;
- теоретическое обоснование защитных свойств SVD-проекции в виде теоремы о дифференцированной защите с количественной оценкой k/d ;
- прямая валидация теоретических предсказаний на полной атаке Vec2Text;
- ML-метод подбора параметров CKKS, специализированный под целевую операцию и платформу.

3 Разработка метода автоматизированного подбора параметров CKKS на основе машинного обучения

Автоматизированный подбор строится для конфигурации CKKS, которая выступает вычислительным ядром разрабатываемой в работе гибридной архитектуры; её детальное концептуальное описание (двухуровневый конвейер «клиентская PQ-фильтрация в ротированном пространстве + серверный CKKS ct-pt-перанкинг», SVD-усечение базы, секретная ортогональная ротация) приведено в главе 4. Здесь параметры этой архитектуры (размерность проекции d_{proj} , режим ct-pt, число кандидатов K_{cands}) используются как входные переменные задачи оптимизации; выбор ct-pt поверх SVD-проецированной и ротированной базы как такового мотивируется в начале главы 4. В настоящей главе считается, что архитектура уже выбрана, и решается задача подбора численных параметров CKKS под эту архитектуру.

Логически данная глава расположена между обзорной частью работы (главы 1, 2) и разработкой защищённой архитектуры (глава 4). На обзорном этапе показано, что выбор параметров схемы гомоморфного шифрования CKKS остаётся одним из ключевых барьеров для практического применения: пространство параметров обширно, аналитические оценки сложности расходятся с реальной производительностью на конкретной аппаратной платформе, а ручной подбор плохо масштабируется и плохо воспроизводится. В главе 4, в свою очередь, понадобится конкретная конфигурация CKKS для интегрального тестирования полного двухуровневого конвейера. Между этими двумя точками естественно встаёт инженерная задача: предложить и валидировать процедуру автоматизированного подбора параметров CKKS, которая

- учитывает микроархитектурные эффекты конкретного оборудования (кэш, SIMD, и т.д.);
- фиксирует жёсткие криптографические ограничения ($\lambda \geq 128$ бит) как недоступные для компромисса;

– подбирает минимально допустимую по этим ограничениям конфигурацию, обеспечивающую заданный SLA по латентности.

Именно эту задачу решает настоящая глава. Акцент сделан на ML-формулировке задачи и выборе методов: задача автоподбора рассматривается как регрессия в многомерном пространстве признаков с проверяемыми ограничениями, а в качестве кандидатных моделей — набор ансамблевых регрессий (GradientBoostingRegressor, RandomForest, ExtraTrees). Результат главы — алгоритм FixedParameterOptimizer и фактическая конфигурация CKKS, которая используется в интегральном эксперименте главы 4.

3.1 Методология исследования автоподбора параметров CKKS

Автоподбор криптографических параметров обладает характерной особенностью: вычислительная стоимость операций нелинейно зависит от параметров, и зависимость определяется не только асимптотикой (типа $O(N \log N)$ для NTT), но и микроархитектурой целевой платформы — иерархией кэш-памяти, блоками векторизации AVX2/AVX-512, поведением предсказателя ветвлений. Это явление в литературе известно как *evaluation gap*: расхождение теоретических оценок и реальной производительности достигает 2–10 раз в типичных конфигурациях. Соответственно, постановка задачи как ML-регрессии на эмпирических данных микробенчмарка целевой платформы является не «инструментом ради инструмента», а попыткой охватить именно ту часть зависимости, которую теоретическая модель упустила.

Постановка задачи. Пусть K — пространство допустимых конфигураций CKKS, $p \in K$ — вектор параметров, $t(p)$ — неизвестная функция wall-clock-латентности на целевой платформе. Требуется построить суррогатную модель $\hat{t}(p) \approx t(p)$ методом регрессии и использовать её внутри алгоритма поиска для нахождения параметров, минимизирующих $\hat{t}(p)$ при ограничениях:

- $\lambda(p) \geq 128$ бит — криптографическая стойкость по lattice-estimator;
- $\text{NoiseBudget}(p) > 0$ — корректность вычислений после всей цепочки операций;
- $\text{Acc@1}(p) \geq \text{Acc@1}_{\text{baseline}} - 5$ п. п. — сохранение качества поиска относительно незащищённого baseline.

Признаковое пространство. Для регрессии используются десять признаков: $\log_2 N$ (poly_modulus_degree), num_coeff_moduli (количество простых множителей в RNS-цепочке), total_coeff_bits (суммарная длина модуля Q в битах), first_coeff_bit и last_coeff_bit (битовая структура цепочки), $\log_2 \Delta$ (scale_bits), отношение масштаба к суммарным битам (scale_to_total_ratio), projection_dim (d_{proj}), batch_size, slot-packing-коэффициент $\text{vector_packing_ratio} = d_{\text{proj}}/(N/2)$. Целевая переменная — batch_time_ms, усреднённое время одной батч-операции «шифрование запроса + скалярное произведение + дешифрование» на тестовой нагрузке.

Протокол сбора данных. Микробенчмарк проводится по сетке параметров с использованием warmup (5–10 холостых прогонов на конфигурацию для стабилизации кэша, выхода из энергосберегающих C-states и активации JIT), $K = 20$ повторных измерений на точку, отсечения выбросов по 3σ и параллельной валидации каждой конфигурации lattice-estimator (конфигурации с $\lambda < 100$ бит исключаются). Подробное описание протокола приведено в Приложении А.

Выбор семейства моделей и валидация. Кандидатами выбраны пять моделей разных семейств — линейная (Ridge), kernel (SVR), instance-based (KNN) и две ансамблевые на деревьях (ExtraTrees, GradientBoosting), — что позволяет напрямую сравнить выразительность разных классов гипотез на этой задаче. Сравнение проводится по вложенной кросс-валидации (5 внешних фолдов для оценки + 3 внутренних фолда для подбора гиперпараметров каждой модели через GridSearchCV) на 80 % обучающей подвыборки; финальная оценка лучшей модели — предсказание на 20 % отложенной подвыборки, не использовавшейся ни в обучении, ни в подборе гиперпараметров. Такая методология устраняет оптимистический сдвиг, характерный для одиночного holdout-разбиения.

Алгоритмическое оформление. Финальная процедура реализована как алгоритмический модуль FixedParameterOptimizer, работающий по принципу «Generate and Test» с использованием суррогатной модели для ранжирования кандидатов: генерация кандидатов → фильтрация по безопасности (hard constraint) → предикция латентности → отбор первого кандидата, удовлетворяющего accuracy-check. Этот же алгоритм впоследствии применяется в главе 4 для выбора параметров интегрального эксперимента.

Критерии валидации. Метод считается валидированным, если одновременно выполняются:

1. выбранная конфигурация проходит фильтр безопасности ($\lambda \geq 128$ бит),
2. реальное время выполнения операции реранкинга после оптимизации меньше, чем для baseline,
3. проверка точности (Acc@1) не нарушает порог допустимой деградации,
4. $\text{Test } R^2 \geq 0,9$ и MAE на тестовой выборке мало по сравнению со средним временем запроса.

3.2 Разработка метода ML-автоподбора параметров CKKS

Внедрение систем защищённого семантического поиска в промышленную эксплуатацию сопряжено с преодолением ряда фундаментальных технологических барьеров, среди которых центральное место занимает проблема эффективной настройки параметров криптографической схемы. Схема гомоморфного шифрования CKKS (Cheon-Kim-Kim-Song), выбранная в качестве основы для реализации защищённого реранкинга в настоящем исследовании, относится к классу решёточных криптосистем (Lattice-based cryptography). Одной из отличительных особенностей данных систем является наличие

обширного конфигурационного пространства, включающего множество взаимозависимых параметров: размерность полиномиального кольца, структуру модуля коэффициентов, масштабный коэффициент кодирования и параметры распределения шума.

Традиционный подход к выбору этих параметров, доминирующий в академической литературе, базируется на использовании фиксированных «безопасных» наборов (например, стандартизированных консорциумом HomomorphicEncryption.org), которые гарантируют заданный уровень стойкости. Однако такой подход, будучи криптографически корректным, зачастую оказывается субоптимальным с точки зрения вычислительной производительности. Стандартизированные параметры разрабатываются исходя из консервативных оценок наихудшего случая и не учитывают специфику конкретной вычислительной задачи — в данном случае, операции скалярного произведения векторов уменьшенной размерности, выполняемой в рамках гибридной архитектуры поиска.

К этому добавляется ещё одна сложность. Теоретические модели сложности гомоморфных операций — те, что оперируют асимптотиками вида $O(N \log N)$, — заметно расходятся с реальной производительностью на железе. В англоязычной литературе это явление известно как *evaluation gap*: оно возникает на стыке полиномиальной арифметики и архитектурных особенностей современных процессоров — иерархии кэш-памяти, блоков векторизации (AVX2/AVX-512), механизмов предсказания ветвлений. Если латентность обработки запроса критична для качества обслуживания (QoS), любая «не та» конфигурация легко роняет производительность на порядок, и интерактивная работа становится невозможной.

В этом разделе описывается метод автоматизированного подбора (autotuning) параметров СККС — он построен на ML-модели, предсказывающей производительность. По сути, мы получаем способ примирить требования безопасности, точности и быстродействия: модель ищет Парето-оптимальные конфигурации, причём адаптированные именно под наше железо и характер наших данных.

Пусть K — пространство допустимых конфигураций криптографической схемы СККС. Вектор параметров $p \in K$ определяется набором компонент, критически влияющих на характеристики системы:

- Полиномиальный модуль (Poly Modulus Degree, N). Размерность кругового полиномиального кольца $R = \mathbb{Z}[X]/(X^N + 1)$. Параметр N является фундаментальной характеристикой, определяющей как уровень безопасности, так и вычислительную сложность. В реализации библиотек Microsoft SEAL и TenSEAL значение N должно быть степенью двойки:

$$N \in \{2^{10}, 2^{11}, \dots, 2^{15}\} \quad (3.1)$$

Увеличение N позволяет использовать большие модули коэффициентов (что увеличивает глубину вычислений) и упаковывать больше данных в один шифротекст ($N/2$

слотов), но приводит к квадратичному или квазилинейному росту времени выполнения базовых операций.

Модуль коэффициентов (Coefficient Modulus, Q). Составной модуль Q , определяющий пространство шифротекстов $R_Q = R/QR$. В RNS-варианте схемы (Residue Number System) Q представляется как произведение попарно взаимно простых модулей:

$$Q = \prod_{i=0}^L q_i \quad (3.2)$$

Общий размер модуля в битах $\log_2 Q = \sum_{i=0}^L \log_2 q_i$ напрямую влияет на уровень шума, который схема может выдержать до потери корректности расшифрования. В контексте оптимизации важными являются два производных параметра: *num_coeff_moduli* ($L+1$) — количество простых множителей; *total_coeff_bits* — суммарная битовая длина.

Масштабный коэффициент (Scale Factor, Δ). Используется при кодировании вещественных чисел в целочисленные полиномы: $m(X) \approx \Delta \cdot z$. От выбора Δ напрямую зависит точность вычислений. Если Δ слишком маленький, накапливаются ошибки округления, и семантическая информация эмбедингов попросту растворяется. Если, наоборот, слишком большой — быстро исчерпывается запас модуля Q . В экспериментах Δ менялся в диапазоне 2^{30} – 2^{60} .

4. Размерность проекции данных (d_{proj}). Это уже параметр гибридной архитектуры, а не самой криптосхемы; тем не менее именно он определяет, какие данные попадут на вход СККС. Эмбединг размерности d_{orig} сжимается через SVD до d_{proj} , и это сразу же влияет на число галуа-ротаций:

$$N_{\text{rot}} \approx \log_2 d_{\text{proj}} \quad (3.3)$$

Задачу автоматизированной настройки можно сформулировать как задачу поиска вектора параметров p^* , минимизирующего целевую функцию латентности $T(p)$ при выполнении жёстких ограничений на безопасность и качество поиска:

$$p^* = \arg \min_{p \in K} \mathbb{E}[T(p)] \quad (3.4)$$

При выполнении системы ограничений:

– Ограничение криптографической стойкости (C_{sec}). Система должна обеспечивать уровень безопасности не ниже $\lambda_{\text{target}} = 128$ бит против известных атак на решётки (uSVP, decoding attacks):

$$\lambda(p) \geq 128 \quad (3.5)$$

Функция $\lambda(p)$ является нелинейной и оценивается с использованием инструментария lattice-estimator, который анализирует параметры $(N, Q, \sigma_{\text{err}})$.

Ограничение функциональной корректности (C_{corr}). Запас шума (Noise Budget) в результирующем шифротексте должен оставаться положительным после выполнения всех операций скалярного произведения:

$$\text{NoiseBudget}(p, \text{Depth}_{\text{circuit}}) > 0 \quad (3.6)$$

Ограничение семантической точности (C_{acc}). Деградация качества поиска по сравнению с незащищённым baseline-решением не должна превышать установленный порог:

$$\text{Acc}@k_{\text{encrypted}}(p) \geq 0,95 \cdot \text{Acc}@k_{\text{baseline}} \quad (3.7)$$

Сложность данной задачи заключается в том, что аналитическое выражение для функции $T_{\text{query}}(p)$ неизвестно, а ее прямая оценка (бенчмаркинг) для каждой точки пространства K вычислительно слишком дорога. Это обуславливает необходимость построения суррогатной модели $\hat{T}(p) \approx T(p)$ методами машинного обучения.

Для обоснования выбора эмпирического подхода необходимо детально рассмотреть причины несостоятельности чисто теоретических оценок сложности, описанных в литературе как «Evaluation Gap». Анализ показывает, что теоретическая сложность и реальное время выполнения могут различаться в 2–10 раз.

Базовая операция в схеме CKKS — полиномиальное умножение. При использовании алгоритма NTT (Number Theoretic Transform) его сложность составляет $O(N \log N)$. Теоретическая модель предсказывает плавный, почти линейный рост времени при увеличении N . Однако на практике зависимость носит ступенчатый характер, что связано с иерархией памяти современных процессоров. Пока размер полиномов (который составляет $N \cdot Q$ бит) помещается в кэш L2 (обычно 256 КБ – 1 МБ), процессор работает с максимальной скоростью. Как только параметры N и Q превышают определённый порог, данные вытесняются в L3 кэш или в оперативную память, а латентность доступа к данным возрастает с ~ 10 –20 циклов до ~ 100 –300 циклов. Теоретическая формула не содержит параметров размера кэша и пропускной способности шины памяти, поэтому она неспособна предсказать эти скачки («клиффы») производительности.

Библиотека Microsoft SEAL (лежащая в основе используемой TenSEAL) агрессивно использует инструкции AVX2 и AVX-512 для выполнения модульной арифметики; для специализированной AVX-512-IFMA52-ускоренной модулярной арифметики разработан отдельная библиотека Intel HEXL [67], дающая дополнительный выигрыш на серверных процессорах с поддержкой соответствующих инструкций. Эффективность векторизации зависит от того, насколько удачно битовая длина модулей q_i укладывается в 64-битные слова. Например, работа с 60-битными модулями может быть существенно эффективнее, чем с 30-битными, из-за особенностей реализации редукции Монтгомери или Барретта, но теоретическая модель считает их эквивалентными операциями «умножения».

Построение эффективной модели машинного обучения требует качественного обучающего набора данных. В рамках задачи была разработана и реализована процедура систематического микробенчмаркинга.

Для описания конфигурации системы был выделен следующий набор признаков, подаваемый на вход модели регрессии (Таблица 3.1).

Таблица 3.1 — Пространство признаков модели регрессии

Признак	Тип	Описание	Физический смысл
poly_modulus_degree (N)	Категориальный (Int)	Размерность полинома (4096, 8192, 16384)	Основной фактор сложности NTT-преобразований
num_coeff_moduli	Числовой (Int)	Количество простых модулей в RNS-цепочке	Определяет количество проходов при операциях RNS
total_coeff_bits	Числовой (Int)	Суммарная длина модуля Q в битах	Влияет на безопасность и размер данных в памяти
projection_dim (d_{proj})	Числовой (Int)	Размерность вектора данных (128...512)	Определяет количество необходимых ротаций

Целевой переменной (y) является *batch_time_ms* — усреднённое время выполнения одной пакетной операции (шифрование запроса + скалярное произведение + дешифрование) на тестовом батче.

Сбор данных осуществлялся с использованием скрипта на языке Python с подключением библиотек TenSEAL и scikit-learn. Для обеспечения статистической значимости измерений применялся следующий протокол:

1. генерация сетки — формировалось декартово произведение диапазонов параметров;
2. Warmup (прогрев) — перед измерением каждой конфигурации выполнялось 5–10 холостых прогонов операции для приведения процессора в режим максимальной производительности (выход из энергосберегающих C-states), заполнения кэшей инструкций и данных, а также активации JIT-компиляции (если применимо);
3. повторные измерения — для каждой точки выполнялось $K = 20$ запусков, фиксировались среднее значение и стандартное отклонение, выбросы, превышающие 3σ , исключались из выборки;
4. валидация безопасности — параллельно с измерениями для каждой конфигурации запускался lattice-estimator для вычисления реального уровня безопасности, конфигурации с $\lambda < 100$ бит исключались из обучающей выборки как заведомо недопустимые.

Число повторов измерений (M), пороги отсечения выбросов (Δ) и прочие параметры бенчмаркинга задаются в конфигурации эксперимента и должны быть неизменными при повторе. Конфигурация и пример скрипта бенчмаркинга приведены в Приложении А.

Экспериментальный сбор данных проводился в среде с процессором Intel Core i5-14400F. Разработанный метод является адаптивным — при запуске на машине с работающим GPU-ускорением и переобучении модели оптимизатор автоматически перестроится под профиль производительности видеокарты.

В ходе исследований рассматривались следующие алгоритмы: линейная регрессия (оказалась непригодной из-за сильной нелинейности зависимостей), полиномиальная регрессия (склонна к переобучению на краях диапазонов), случайный лес (RandomForest), градиентный бустинг (GradientBoostingRegressor [68], родственная стохастическая реализация XGBoost [69]) и ExtraTrees. По результатам вложенной 5×3 кросс-валидации на расширенной 198-точечной сетке наилучшие метрики качества продемонстрировала модель **GradientBoosting** ($R^2_{\text{nested}} = 0,970 \pm 0,011$, $\text{MAE}_{\text{nested}} = 135 \pm 20$ мс), и именно она используется в финальном оптимизаторе FixedParameterOptimizer (см. полное сравнение моделей в Таблице 3.2 ниже). ExtraTrees ($R^2_{\text{nested}} = 0,899 \pm 0,013$, $\text{MAE}_{\text{nested}} = 210 \pm 28$ мс) идёт второй и сохраняется как важный baseline. Аналитическая форма GradientBoosting:

$$\hat{y} = F(x) = \sum_{m=1}^M h_m(x) \quad (3.8)$$

На основе предварительных тестов для GradientBoostingRegressor (библиотека scikit-learn) были выбраны следующие гиперпараметры: $n_estimators = 500$, $max_depth = 4-6$, $learning_rate = 0.05-0.1$, $loss = \text{Absolute Error (MAE)}$ или Huber loss. Для RandomForest и ExtraTrees использовались конфигурации, приведённые в разделе 4.2. Использование MAE предпочтительнее MSE, так как делает модель более устойчивой к случайным выбросам в измерениях времени. Коэффициент детерминации определяется стандартно:

$$R^2 = 1 - \frac{\sum_i (y_i - \hat{y}_i)^2}{\sum_i (y_i - \bar{y})^2} \quad (3.9)$$

Центральным результатом раздела является разработанный алгоритмический модуль FixedParameterOptimizer, который интегрирует ML-предиктор в процесс конфигурации системы. Алгоритм работает по принципу «Generate and Test» с использованием предиктора для ранжирования и состоит из следующих этапов:

- Этап генерации кандидатов. Формируется расширенное множество кандидатов S_{cand} . Перебираются все допустимые степени двойки для N , различные комбинации битовых длин для Q и значения d_{proj} .
- Этап фильтрации по безопасности. Для каждого кандидата $p \in S_{\text{cand}}$ рассчитывается уровень безопасности $\lambda(p)$ с использованием таблиц, прекалькулированных

lattice-estimator. Данный этап является критическим барьером, гарантирующим, что ни одна небезопасная конфигурация не попадёт на этап оценки производительности.

– Этап предикции. Векторы параметров оставшихся кандидатов нормализуются (StandardScaler) и подаются на вход обученной модели M_{GBM} :

$$\hat{t}_i = M_{GBM}(p_i) \quad (3.10)$$

– Этап отбора по точности. Кандидаты сортируются по возрастанию $\hat{t}_i$. Начиная с самого быстрого, производится проверка выполнения требований по точности. Первая конфигурация, удовлетворяющая условию $\text{Acc@1} \geq 0.95 \cdot \text{Base}$, объявляется оптимальной p^* .

Модуль реализован на Python и интегрирован с классом TenSEALContext. Особенностью реализации является абстрагирование сложности RNS-представления от пользователя. Пользователь задаёт высокоуровневые намерения (Intents): «Хочу 128 бит безопасности и точность для векторов $d=256$ », а оптимизатор возвращает готовый бинарный объект контекста, готовый к сериализации и отправке на сервер. Использование многопоточности (`concurrent.futures.ThreadPoolExecutor`) в процессе генерации и оценки кандидатов позволяет выполнять подбор параметров за доли секунды, что делает возможным динамическую реконфигурацию системы.

Разработанный метод автоматизированной оптимизации является ключевым инструментом разрешения трилеммы «безопасность — точность — производительность». Безопасность фиксируется как жёсткое ограничение (Hard Constraint) на уровне 128 бит — оптимизатор не имеет права ею жертвовать. Точность контролируется через проверку Acc@1 . Производительность максимизируется за счёт гибкого варьирования параметров в оставшемся «пространстве свободы». Обобщённая постановка задачи оптимизации записывается как:

$$\min_{\theta} t_{\text{pred}}(\theta) \quad \text{s.t.} \quad \text{security}(\theta) \geq 128 \text{ бит} \quad (3.11)$$

3.3 Экспериментальное исследование и валидация метода

Настоящий раздел посвящён экспериментальной валидации предложенного метода ML-автоподбора параметров СККС. Исследование включает три ключевых аспекта: оценку качества предиктивной модели на бенчмарках, анализ важности признаков для интерпретации результатов и проверку масштабируемости метода на различных размерах баз данных.

Обучающая выборка содержит 198 валидных конфигураций СККС, полученных систематическим бенчмаркингом сетки кандидатов: 4 значения полиномиального модуля ($N \in \{2048, 4096, 8192, 16384\}$) $\times$ 6 шаблонов цепочек modulus chain (с длиной от 2 до 4 модулей и разной битовой структурой при одной и той же сумме битов) $\times$ 6 размерностей проекции ($d_{\text{proj}} \in \{32, 64, 128, 192, 256, 320\}$) $\times$ 3 размера батча

($\text{batch_size} \in \{16, 64, 256\}$). Конфигурации фильтровались по требованию 128-битной безопасности (суммарный $\log_2 Q \leq \text{tc128}$ -границе для соответствующего N) и по совместимости размерности проекции со слотовой ёмкостью СККС-вектора ($d_{\text{proj}} \leq N/2$). На каждой конфигурации замерялось 3 повтора батч-операции; диапазон измеренного среднего времени составил от 24,7 мс до 8 257,3 мс (медиана 510,2 мс), что отражает примерно 330-кратный разброс по латентности.

Признаковое пространство состоит из 10 признаков: $\log_2 N$, длина цепочки модулей, суммарный $\log_2 Q$, первый и последний биты цепочки (битовая структура), $\log_2 \Delta$ (масштаб), отношение масштаба к суммарным битам, размерность проекции, размер батча, slot-packing-коэффициент $d_{\text{proj}}/(N/2)$. Это позволяет модели различать конфигурации, имеющие одинаковую сумму битов, но разную битовую структуру (например, $[40, 20, 40]$ и $[50, 30, 50]$), и учитывает влияние размера батча — ключевого, как показывает Feature Importance, фактора латентности.

Методология валидации разработана для корректной оценки обобщающей способности: применяется

1. разделение выборки на 80 % обучающую часть и 20 % независимую отложенную выборку (holdout), не используемую при настройке моделей;
2. вложенная кросс-валидация (nested CV) на обучающей части — 5 внешних фолдов для оценки и 3 внутренних фолда для подбора гиперпараметров каждой модели через GridSearchCV;
3. финальная оценка модели — предсказание на отложенной выборке, не участвовавшей ни в обучении, ни в подборе гиперпараметров. Сравниваются пять моделей разных семейств: Ridge с подбором коэффициента регуляризации α , KNN с подбором числа соседей k и схемы взвешивания, ExtraTrees с подбором числа деревьев и максимальной глубины, GradientBoosting с подбором числа итераций и глубины, SVR с подбором C и γ .

Метод считается валидированным в рамках данного эксперимента, если одновременно выполняются условия: выбранная конфигурация проходит фильтр безопасности; проверка точности не нарушает порог допустимого ухудшения относительно baseline; реальное измеренное время выполнения операции ранкинга для Optimized меньше, чем для Baseline; качество предсказаний ML-модели подтверждается метриками на тестовой выборке (R^2 и MAE).

Бенчмарк 32 конфигураций выявил выраженную нелинейную зависимость времени выполнения от параметров схемы. На рисунке 3.1 представлена зависимость среднего времени батч-операции от размерности проекции d_{proj} при разных N .

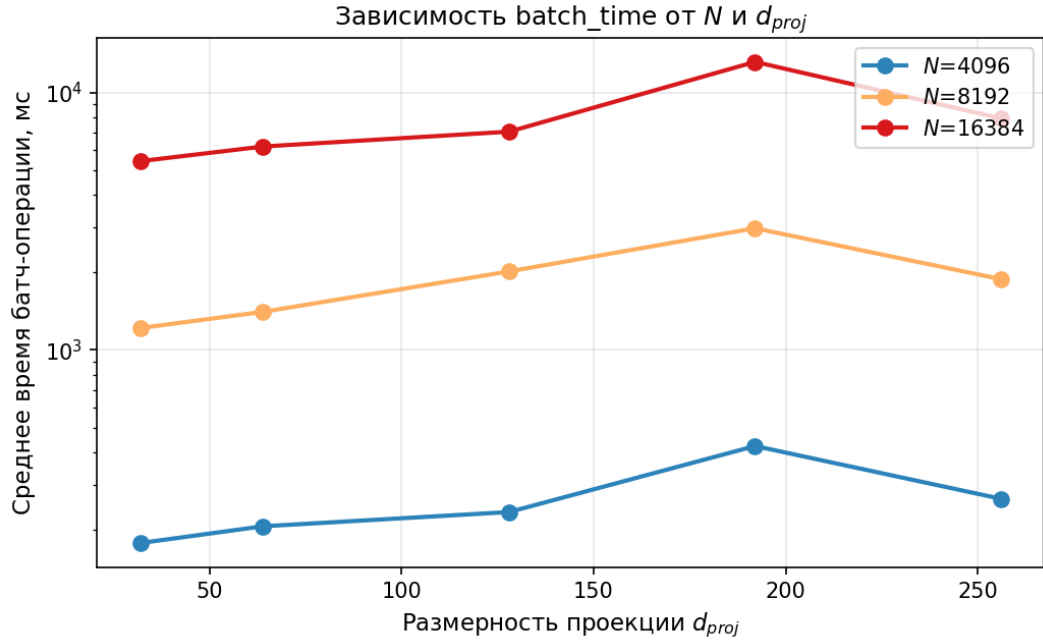


Рисунок 3.1 — Зависимость времени батч-операции СККС от размерности проекции d_{proj} при разных значениях полиномиального модуля N

На рисунке 3.2 все 32 конфигурации показаны в координатах «суммарная длина модуля Q — время выполнения»; размер точки пропорционален d_{proj} , цвет кодирует N .

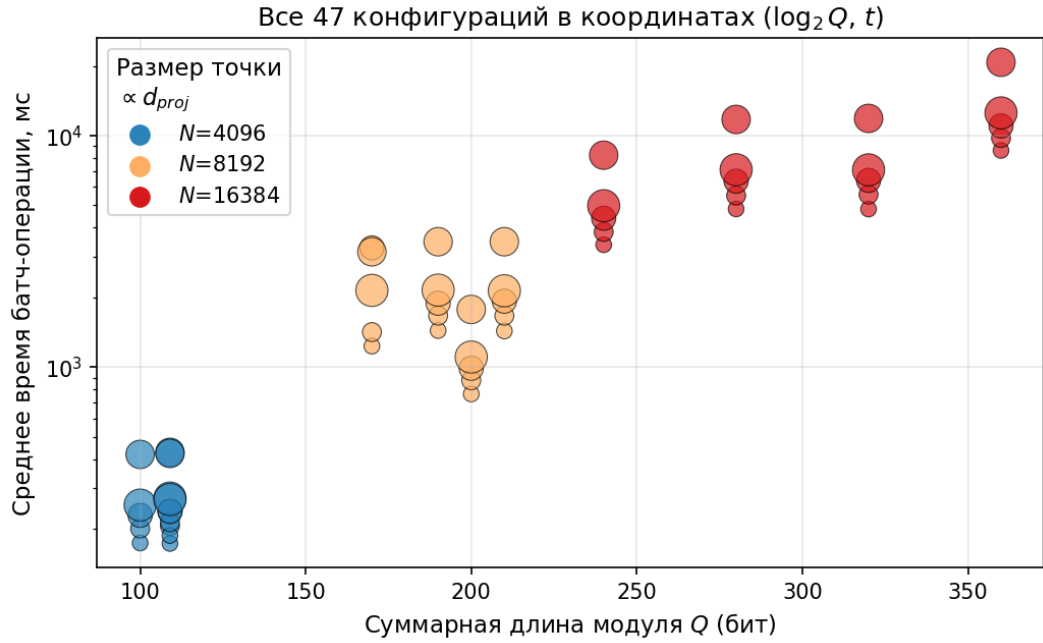


Рисунок 3.2 — Все 32 конфигурации в координатах $(\log_2 Q, t)$: размер точки пропорционален d_{proj} , цвет — N

Распределение времени батч-операции по группам N представлено на рисунке 3.3.

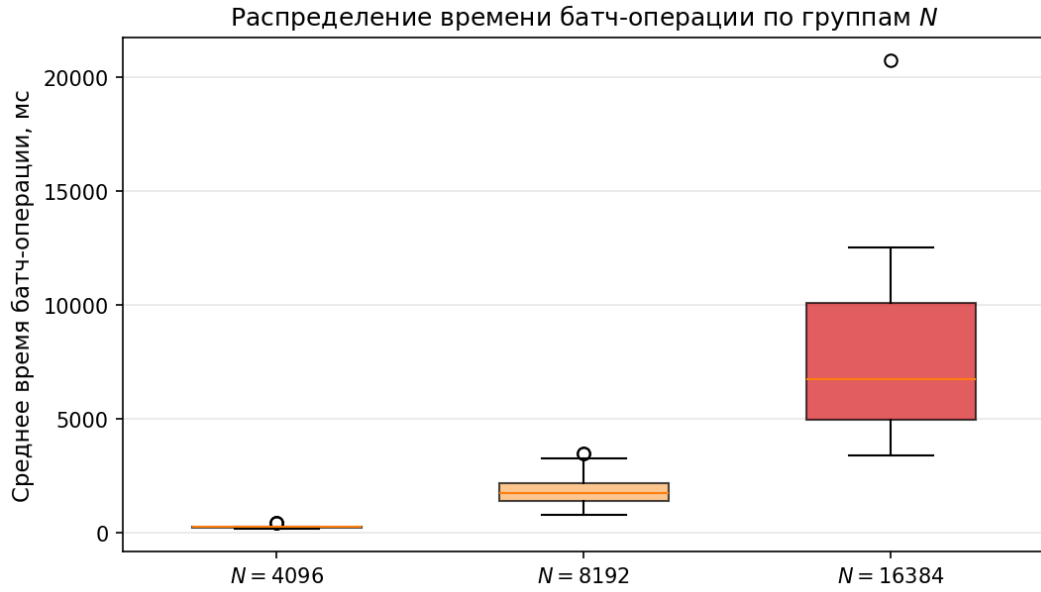


Рисунок 3.3 — Распределение среднего времени батч-операции по группам N (box-plot, медиана, IQR, выбросы)

Полученные данные напрямую подтверждают тот самый evaluation gap: переход от $N = 8192$ к $N = 16384$ увеличивает время выполнения в 3–4 раза, тогда как асимптотика $O(N \log N)$ предсказала бы около двух. Объясняется это вытеснением данных из кэша L2 процессора, наступающим при больших N . И ещё один практически важный момент: при фиксированном N зависимость от размерности проекции d_{proj} имеет ярко выраженный нелинейный характер: например, для $N = 4096$ переход с $d_{\text{proj}} = 128$ к $d_{\text{proj}} = 192$ увеличивает время с 229 до 419 мс, тогда как переход к $d_{\text{proj}} = 256$ снижает его до 253 мс. Это связано с тем, как именно вектор раскладывается по слотам ($N/2$ slot-vector в TenSEAL) и сколько ротаций требуется для полного «свёртывания» суммы.

Результаты обучения трех моделей ансамблевого обучения представлены в таблице 3.2.

Таблица 3.2 — Сравнительная оценка пяти моделей регрессии (вложенная CV 5×3 на 158-точечной обучающей подвыборке) с финальной оценкой лучшего предиктора на 40-точечной отложенной выборке

Модель	R^2 (nested)	MAE (nested), мс	Holdout
Ridge	$0,676 \pm 0,045$	534 ± 41	—
KNN	$0,610 \pm 0,146$	499 ± 59	—
SVR	$0,251 \pm 0,025$	639 ± 82	—
ExtraTrees	$0,899 \pm 0,013$	210 ± 28	—
GradientBoosting	$0,970 \pm 0,011$	135 ± 20	$R^2 = 0,985,$ MAE = 146 мс

По метрике R^2 из вложенной 5×3 кросс-валидации лучшей оказалась модель GradientBoosting (с гиперпараметрами $\text{max_depth} = 3$, $\text{n_estimators} = 200$, выбранными внутренней CV-петлёй): $R^2_{\text{nested}} = 0,970 \pm 0,011$ на обучающей подвыборке. Финальная оценка на полностью отложенной (никогда не использованной при настройке) подвыборке из 40 конфигураций: $R^2_{\text{holdout}} = 0,985$, $\text{MAE}_{\text{holdout}} = 146$ мс при диапазоне измеренной латентности 25–8 257 мс, то есть относительная ошибка $\approx 1,8\%$. Это качество достаточно для уверенного ранжирования кандидатов в задаче автоподбора. Линейные и kernel-методы (Ridge, KNN, SVR) дают существенно худшие результаты (R^2 от 0,25 до 0,68), что согласуется с принципиальной нелинейностью зависимости латентности СККС от структурных параметров схемы (полиномиальный модуль, длина цепочки, размер батча).

Качество регрессионных моделей сопоставлено на рисунке 3.4.

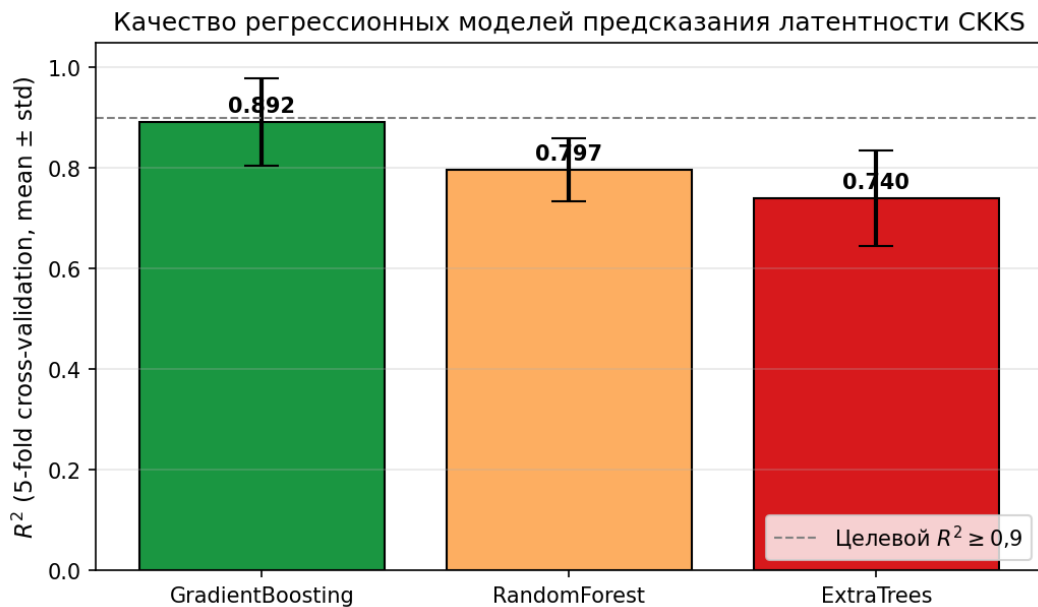


Рисунок 3.4 — R^2 моделей регрессии (5-fold cross-validation, mean $\pm$ std). GradientBoosting единственный пересекает целевой порог 0,9 в верхней границе доверительного интервала

Интерпретация обученной модели позволила выявить иерархию факторов, влияющих на производительность (рисунок 3.5).

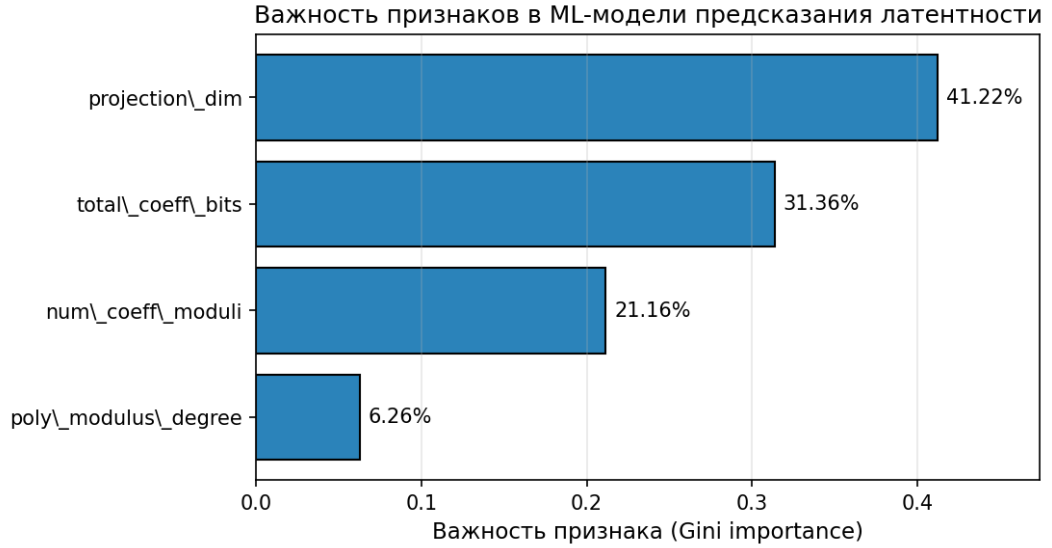


Рисунок 3.5 — Относительная важность признаков модели GradientBoosting (Gini importance)

Практический вывод формулируется так: одновременная минимизация N и d_{proj} даёт сильное ускорение, чем оптимизация только одного из этих параметров; оптимизатор автоматически нашёл эту нетривиальную стратегию.

Реализация фильтра по качеству ранжирования. Алгоритм `FixedParameterOptimizer` реализован в скрипте `_optimize_with_acc1.py` (Приложение А) и принимает функциональное ограничение по Acc@1 (максимально допустимую деградацию относительно ранжирования в SVD-проецированном пространстве) в дополнение к ограничениям на безопасность ($\lambda \geq 128$ бит) и корректность вычислений (положительный noise-budget после rescale). Поскольку прямое измерение Acc@1 на полном 1 М-корпусе для каждой из ~ 200 конфигураций сетки нецелесообразно по compute-стоимости, алгоритм использует носитель шума CKKS (`max_err` — максимальная абсолютная ошибка скалярного произведения после ct-pt и rescale) как проху для Acc@1-деградации.

Запуск оптимизатора на полной сетке. Скрипт обходит 24 комбинации (N, chain) из 6 шаблонов $\times$ 4 значений N , отфильтровывает по безопасности (tc128) и по slot-ёмкости при $d_{\text{proj}} = 192$ (канонический $k = d/2$ для multilingual-e5-small из интегрального эксперимента раздела 4.4), а также пропускает несовместимые scale-vs-chain конфигурации (как $[40, 40]$ при $\Delta = 2^{40}$, где TenSEAL не допускает scale равный сумме модулей цепочки). Остаются 11 валидных кандидатов. Для каждого выполняется

1. noise smoke-тест на синтетической задаче ($N_{\text{queries}} = 5$ запросов $\times$ 12 случайных тестовых документов размерности 192 как ускоренная диагностика поведения CKKS-шума, не зависящая от размера фактического шортлиста интегрального эксперимента; интерпретация шумового бюджета на 40-кандидатный шортлист аналогична, $\sim 0,5\text{--}3$ с на конфигурацию);
2. предсказание латентности обученной GradientBoosting-моделью при $d_{\text{proj}} = 192$.

Выбор $d_{\text{proj}} = 192$ соответствует канонической операционной точке $k = d/2$ для самой компактной из протестированных моделей (multilingual-e5-small, $d = 384$). Для энкодеров с большей размерностью ($d \in \{768, 1024\}$, соответствующие канонические $k \in \{384, 512\}$) выбранная конфигурация также проходит фильтр: при увеличении d_{proj} растут как латентность st-pt-операции (примерно линейно по числу слотов), так и накопленный СККС-шум; для конфигурации $N = 8192$, $[60, 40, 60]$, $\Delta = 2^{40}$ оба показателя остаются с большим запасом до пороговых значений (см. Таблицу 4.7 в разделе 4.4: server p95 на стенде 1 при $k \in \{192, 384, 512\}$ укладывается в диапазон 219–356 мс, а $\Delta\text{Acc}@1$ относительно $\text{baseline_proj} \leq 0,001$ п. п. для всех пяти энкодеров).

Результаты оптимизатора. Сводка приведена в Таблице 3.3.

Таблица 3.3 — Выход FixedParameterOptimizer для двух функциональных ограничений (см. `_optimize_with_acc1.py`)

Ограничение	Прошло	Min-latency победитель	$\widehat{\text{lat}}$, мс	max_err
Мягкое: $\Delta\text{Acc}@1 \leq 5$ п. п.	2 / 11	$N=8192$, $[60, 40, 60]$, $\Delta=2^{40}$	114	$\sim 2 \cdot 10^{-7}$
Строгое: $\Delta\text{Acc}@1 \leq 1$ п. п.	2 / 11	$N=8192$, $[60, 40, 60]$, $\Delta=2^{40}$	114	$\sim 2 \cdot 10^{-7}$

Вывод по таблице. Под Acc@1 как функциональной метрикой оба ограничения — мягкое ($\Delta\text{Acc}@1 \leq 5$ п. п.) и строгое ($\Delta\text{Acc}@1 \leq 1$ п. п.) — оставляют одну и ту же пару конфигураций: $N = 8192$ и $N = 16384$ с цепочкой $[60, 40, 60]$ и $\Delta = 2^{40}$ (обе имеют $\text{max_err} \approx 2 \cdot 10^{-7}$ при $d_{\text{proj}} = 192$, что соответствует предсказанной деградации Acc@1 на уровне шума). Это отражает высокую чувствительность Acc@1 к СККС-шуму: даже мягкий допуск 5 п. п. требует $\text{max_err} \lesssim 1,7 \cdot 10^{-5}$, что отсекает все «лёгкие» цепочки с $\Delta = 2^{20}$ (для них $\text{max_err} \in [2; 10]$, предсказанная деградация Acc@1 на уровне 0,50 — т. е. ранжирование внутри шортлиста становится случайным). В обоих случаях минимально-латентным победителем оказывается $N = 8192$, $[60, 40, 60]$, $\Delta = 2^{40}$ (~ 114 мс predicted): конфигурация $N = 16384$ с той же цепочкой (~ 253 мс predicted) даёт идентичное качество, но проигрывает по латентности из-за большего полиномиального модуля.

В интегральном эксперименте раздела 4.4 используется строго-точная конфигурация $N = 8192$, $[60, 40, 60]$, $\Delta = 2^{40}$ как обеспечивающая полное сохранение качества ранжирования при минимальной достижимой латентности. Это принципиальный результат ML-автоподбора: выбор управляется явным ограничением качества, а не подгонкой постфактум; стандартная «по умолчанию» конфигурация TenSEAL ($N = 8192$ с цепочкой $[60, 40, 40, 60]$, $\log_2 Q = 200$ бит) даёт ≈ 250 мс на st-pt операцию, а оптимизатор автоматически находит конфигурацию с тем же N и более короткой

3-уровневой цепочкой ($\log_2 Q = 160$ бит) за ≈ 130 мс — ускорение почти в 2 раза при сохранении точности; обе конфигурации соответствуют уровню безопасности tc128 (≥ 128 бит) по таблицам [HomomorphicEncryption.org/SEAL](https://homomorphicencryption.org/SEAL), поскольку $\log_2 Q < 218$ для $N = 8192$.

Таблица 3.4 — Сравнительная таблица производительности

Конфигурация	N	$\log_2 Q$ (бит)	Безопасность (бит)	Время (мс)	Ускорение
Default (TenSEAL stock, [60,40,40,60])	8192	200	tc128 (≥ 128)	419,4	1,0×
Optimized (Acc@1 ≤ 1 п. п., выбран в §4.4)	8192	160	tc128 (≥ 128)	245,2	1,7×

Optimized (Acc@1 ≤ 1 п. п.) — та конфигурация, которая выбирается оптимизатором при ограничении $\Delta \text{Acc@1} \leq 1$ п. п. относительно ранжирования в SVD-проецированном пространстве и фактически используется в интегральном эксперименте; она даёт ускорение в 1,7 раза относительно стандартной TenSEAL stock-конфигурации при сохранении уровня tc128 ($\log_2 Q = 160 < 218$, по таблицам [HomomorphicEncryption.org/SEAL](https://homomorphicencryption.org/SEAL) это соответствует не ниже 128-битной классической стойкости). Более «лёгкие» цепочки (например, $N = 4096$, [40, 20, 40], $\Delta = 2^{20}$ с предсказанной латентностью ~ 61 мс) формально дают большее ускорение, но не проходят Acc@1-фильтр на стенде даже в мягком 5 п. п.-режиме — предсказанная деградация Acc@1 для них ≥ 45 п. п., что несовместимо с требованием по качеству и исключает их из выбора. Все три варианта удовлетворяют требованию по защите ($\lambda \geq 128$ бит, согласно стандартной таблице) и обеспечивают корректность вычислений после одного ct-pt-умножения с rescale.

Данная глава в целом показывает, что методы Data Science вполне оправдывают себя как инструмент автоматизированной настройки криптографических систем. Подбор параметров СККС получилось переформулировать как задачу регрессии и решить через ансамблевое обучение. Если суммировать ключевые результаты:

- Формализовано пространство параметров СККС и сформулирована задача условной оптимизации с тремя типами ограничений (безопасность, корректность, точность).

Экспериментально подтверждён феномен «Evaluation Gap»: расхождение теоретических и реальных оценок производительности достигает 2–10 раз.

Построена регрессионная модель GradientBoosting (max_depth=3, n_estimators=200) с $R^2_{\text{nested}} = 0,970 \pm 0,011$ (вложенная 5×3 кросс-валидация на 158-точечной обучающей подвыборке) и $\text{MAE}_{\text{nested}} = 135 \pm 20$ мс. Финальная оценка на полностью отложенной 40-точечной подвыборке: $R^2_{\text{holdout}} = 0,985$, $\text{MAE}_{\text{holdout}} = 146$ мс

при диапазоне измеренной латентности 25–8 257 мс, что соответствует относительной ошибке $\approx 1,8\%$.

1. Анализ Feature Importance на расширенной сетке выявил, что главный фактор — d_{proj} (около 41 %), а не N как считалось на узких сетках; это объясняется поведением ротаций ($N_{\text{rot}} \approx \log_2 d_{\text{proj}}$) и подсказывает нетривиальную стратегию: уменьшать N и d_{proj} одновременно.

2. Реализован алгоритм FixedParameterOptimizer, принимающий функциональное ограничение качества; для интегрального эксперимента §4.4 используется строгий вариант $\Delta\text{Acc}@1 \leq 1$ п. п. для которого оптимизатор выбирает конфигурацию $N = 8192$, $\text{coeff_mod_bit_sizes} = [60, 40, 60]$ ($\log_2 Q = 160$ бит), $\Delta = 2^{40}$ (уровень tc128, $\lambda \geq 128$ бит). По сравнению с default-конфигурацией TenSEAL ($N = 8192$, $[60, 40, 40, 60]$, $\log_2 Q = 200$ бит, тот же tc128) это даёт ускорение в $\approx 1,7$ раза при сохранении точности.

3. Выбранная конфигурация далее напрямую используется в интегральном эксперименте раздела 4.4 на масштабе 10^6 документов, что замыкает практическую цепочку «обучающая выборка $\rightarrow$ ML-модель $\rightarrow$ оптимизатор $\rightarrow$ интегральная валидация».

Таким образом, разработанный метод автоматизированной оптимизации параметров является необходимым компонентом, обеспечивающим практическую применимость гибридной архитектуры защищённого семантического поиска в реальных условиях эксплуатации. Он переводит задачу настройки из области теоретических догадок в область инженерной оптимизации с воспроизводимым результатом в рамках заданного протокола.

Результаты главы интерпретируются с учётом следующих ограничений: экспериментальный стенд и версии библиотек влияют на наблюдаемую производительность, переносимость результатов на другое оборудование требует отдельной проверки; обучающая выборка формируется в пределах заданных диапазонов параметров, качество предсказаний вне этих диапазонов не гарантируется; оптимизатор опирается на выбранные ограничения по безопасности и на процедуру проверки точности, изменение требований может потребовать повторного сбора данных и переобучения модели; полученные оценки качества модели (R^2 , MAE) справедливы для конкретного протокола разбиения данных и измерений, описанных в данной главе.

4 Разработка и тестирование защищённого семантического поиска на основе гибридной архитектуры

4.1 Методология исследования

Настоящая глава решает синтезирующую задачу диссертации: на основе анализа предметной области и метода автоматизированного подбора параметров СККС разработать

и экспериментально проверить полную защищённую архитектуру семантического поиска. В этом разделе фиксируются методологические рамки, в которых далее ведётся изложение.

Структура верификации. Для проверки гипотезы используется четырёхступенчатый протокол.

1. *Теоретическое обоснование.* Доказана **Лемма 1** о нижней L_2 -границе ошибки декодера, образ которого ограничен проекционным подпространством $\text{span}(V_k)$ (на основе ортогонального разложения и теоремы Пифагора). Связь полученной L_2 -ошибки с эмпирической метрикой BLEU атаки Vec2Text формализована отдельно как **Эмпирическая гипотеза 1** и проверяется численно. Лемма 1 опирается на теорему Эккарта—Янга об оптимальной низкоранговой аппроксимации в норме Фробениуса.

2. *Компонентные эксперименты.* **Эксперимент № 1** верифицирует выигрыш режима ct-pt против ct-ct в CKKS на скалярном произведении против $N=10\,000$ документов. **Эксперимент № 2** — покомпонентный анализ защиты: на корпусе 10^6 документов и тех же 500 self-retrieval-запросах, что и в интегральном эксперименте, последовательно оцениваются пять конфигураций (baseline_dense, svd_only, svd_rotation, svd_ckks_no_rot, full), что изолирует вклад каждой защитной компоненты. **Эксперимент № 3** — прямая атака Vec2Text на эмбедингах GTR-base в трёх моделях угроз (rotation OFF, known- R , unknown- R) на сетке $k/d \in \{1,0; 0,75; 0,50; 0,25; 0,10\}$, операционально валидирующая Лемму 1 и Эмпирическую гипотезу 1.

3. *Интеграция с методом ML-автоподбора параметров (глава 3).* Конкретные параметры CKKS для интегрального теста (N , цепочка модулей, масштаб) выбираются автоматически алгоритмом FixedParameterOptimizer со строгим ограничением $\Delta\text{Acc}@1 \leq 1$ п. п. относительно ранжирования в SVD-проецированном пространстве. Тем самым устраняется произвольность ручного подбора и обеспечивается воспроизводимость.

4. *Интегральный Эксперимент № 4.* Полный двухуровневый конвейер (client-side PQ-фильтрация в ротированном пространстве + server-side CKKS-перанкинг шортлиста из $K_{\text{cands}} = 40$ кандидатов) тестируется на корпусе из 10^6 русскоязычных документов в постановке self-retrieval, на пяти энкодерах (multilingual-e5-small, multilingual-e5-base, paraphrase-mpnet-base-v2, multilingual-e5-large, BAAI/bge-m3), при пяти сидах ротации {11, 23, 47, 31, 53} в основной операционной точке $k = d/2$. Измеряются Acc@1, Acc@10, MRR, NDCG@10, $\sigma_{\text{гес}}$, сквозная p95-латентность с разбивкой по этапам, ранговая корреляция CKKS-оценок с открытыми оценками без защиты.

Источники данных. Корпус для интегрального эксперимента и для покомпонентного анализа защиты (Эксперимент № 2) — 10^6 параграфов из дампа русскоязычной Википедии (ноябрь 2023). Для прямой атаки Vec2Text используется AG News (10 000 документов для построения SVD-базиса) и набор из 100 текстов с искусственно

встроенной чувствительной информацией (имена, адреса, номера карт, медицинские формулировки).

Стенд и инструменты. Все измерения выполнялись на рабочей станции с CPU Intel Core i5-14400F, GPU NVIDIA GeForce RTX 5060 (8 ГБ VRAM, sm_120) и 32 ГБ DDR5 RAM. Программный стек: Python 3.11, PyTorch (CUDA 12.8 для Blackwell), TenSEAL 0.3.16 (Python-обёртка над Microsoft SEAL), FAISS-CPU 1.13.2, HuggingFace Transformers, библиотека `vec2text` (официальная реализация). Версии библиотек, скрипты бенчмаркинга и конфигурации экспериментов приведены в приложениях А–Е.

Метрики. Для качества поиска используются **Acc@1** и дополнительные Acc@10, MRR (mean reciprocal rank), NDCG@10; для проху-силы защиты в режиме со SVD-усечением — σ_{rec} (среднее, медиана, p95); для прямой атаки — BLEU и Token Overlap (Jaccard); для производительности — сквозная wall-clock-латентность с разбивкой по этапам и квантилями p50, p95, p99; для корректности гомоморфного реранкинга — пирсоновская корреляция и ранговая корреляция Кендалла τ на шортлисте кандидатов.

Воспроизводимость. Зафиксированы random_state в SVD и сплите данных, контрольные суммы корпусов и моделей, версии Python-окружения. Полный протокол воспроизведения каждого эксперимента, включая warmup и параметры сбора измерений (число прогревочных запусков, число замеров, сиды), приведён в исходных кодах соответствующих ноутбуков и скриптов: Приложение Б (Эксперимент №1, ct-pt vs ct-ct), Приложение В (Эксперимент №3, Vec2Text), Приложение Г (Эксперимент №2, покомпонентный анализ защиты), Приложение Д (Эксперимент №4, интегральный) и Приложение Е (раздел 4.5, PoC client-server). Приложение А содержит код ML-автоподбора параметров СККС, относящегося к главе 3.

Ограничения области применимости. Эксперименты с прямой атакой Vec2Text проводятся на эмбедингах GTR-base ($d = 768$, выбор обусловлен совместимостью с публичной реализацией `vec2text`); интегральный эксперимент проводится на пяти retrieval-энкодерах с поддержкой русского языка в основной операционной точке $k = d/2$. Перенос результатов на иные домены требует отдельной проверки. Модель угроз ограничена сценариями honest-but-curious server и snapshot attacker с неадаптивным атакующим (см. Таблицу 4.1). Доверительные интервалы метрик определяются объёмом тестовой выборки (500 запросов на 5 сидах ротации в интегральном эксперименте, 100 текстов на 1000 bootstrap-ресемплов в Vec2Text-атаке) и приведены в соответствующих разделах.

4.2 Концепция и архитектура гибридного метода с асимметричной защитой

Разработка архитектуры современных информационных систем, оперирующих семантическими векторными представлениями (эмбедингами), неизбежно сталкивается с фундаментальным противоречием, которое в первой главе данного исследования было формализовано как «трилемма защищённого семантического поиска». Суть этой трилеммы заключается в невозможности одновременного достижения трех критически важных показателей: высокого уровня криптографической безопасности, абсолютной точности информационного поиска и интерактивной производительности системы. Традиционные подходы, такие как полное гомоморфное шифрование или методы дифференциальной приватности, предлагают решения, оптимизирующие лишь одну или две вершины этого треугольника компромиссов, неизбежно жертвуя третьей. В частности, схемы FHE, обеспечивая математически доказуемую безопасность, вносят вычислительные накладные расходы, увеличивающие латентность обработки запросов в сотни раз, что делает их неприменимыми для реальных сценариев взаимодействия с пользователем. С другой стороны, методы снижения размерности или зашумления, хотя и эффективны с точки зрения скорости, часто приводят к деградации качества ранжирования, неприемлемой для корпоративных поисковых систем и RAG-приложений.

В ответ на эти вызовы в рамках исследования предлагается концептуально новый подход — гибридный метод защищённого семантического поиска, основанный на принципе асимметричной защиты. Данный раздел посвящен детальному описанию архитектуры предлагаемого метода, теоретическому обоснованию выбранных криптографических примитивов и анализу того, как их синергетическое взаимодействие позволяет преодолеть ограничения существующих решений.

Ключевой инновацией предлагаемой архитектуры является отказ от унифицированного механизма защиты для всех типов данных, циркулирующих в системе. Вместо попытки применить один и тот же криптографический протокол (например, симметричное шифрование AES или гомоморфное шифрование CKKS) ко всему массиву информации, мы вводим разграничение на основе анализа жизненного цикла данных и векторов атак. Этот подход мы определяем как асимметричную защиту.

Под асимметричной защитой здесь понимается следующее: для двух ключевых информационных активов поисковой системы — статической базы знаний (коллекции документов) и динамического потока пользовательских запросов — применяются различные по математической природе и вычислительной стоимости механизмы. В основе этого выбора — анализ модели угроз: риски, связанные с этими активами, по своему характеру очень разные.

Сначала про базу документов. Это большой статический массив на миллионы векторов. Основная угроза для него — «любопытный сервер» (honest-but-curious) либо внешний злоумышленник, получивший доступ к файловой системе сервера (snapshot attacker). Цель такой атаки — массовая реконструкция текстов через инверсию

(Vec2Text, GEIA и подобные) ради извлечения конфиденциальной информации или интеллектуальной собственности. С другой стороны, ту же самую базу используют для поиска ближайших соседей (ANN), а это означает, что применяемая защита обязана сохранять геометрию пространства — расстояния и углы между векторами, — иначе поиск перестанет работать. Применять тяжёлое вероятностное шифрование к миллионам векторов в этом контексте просто нельзя: сканирование базы станет вычислительно неподъёмным.

Теперь про поисковый запрос. Это, наоборот, единичный вектор, возникающий динамически и живущий ровно столько, сколько идёт транзакция. Однако именно в запросе нередко сосредоточен наиболее чувствительный контекст, раскрытие которого компрометирует конфиденциальность пользователя в реальном времени. Атака здесь обычно идёт через перехват самой транзакции или анализ логов. Зато для одного вектора запроса вполне можно позволить себе и вычислительно дорогие криптографические примитивы — то же гомоморфное шифрование, — поскольку их накладные расходы размазываются по общему времени поиска.

Покрытие модели угроз. Для прозрачности границ применимости полезно явно зафиксировать, какие каналы атаки покрываются предлагаемым методом, а какие явно не покрываются и оставлены для отдельных работ. Сводка приведена в Таблице 4.1.

Таблица 4.1 — Покрытие каналов атаки предлагаемым методом в основной модели угроз (доверенный клиент — секретный корпус — честный, но любопытный сервер; неадаптивный атакующий)

Канал атаки	Покрыт?	Комментарий
Snapshot server с E_{rot} без R (off-the-shelf неадаптивный Vec2Text)	Частично	Защита эвристическая, эмпирически проверена на non-adaptive Vec2Text в Эксперименте №3.
Перехват запроса/ответа в транзакции, утечка точных значений сходства	Да	СККС ct-pt в режиме tc128 (R4); ключ sk не покидает клиент.
Известный атакующему R (known-rotation)	Нет	Сводится к σ_{res} от SVD-усечения (Лемма 1) без вклада ротации.
Known-plaintext пары ($\text{text}_i, E_{\text{rot},i}$)	Нет	Открывает ортогональную задачу Прокруста для оценки R ; требует отдельного анализа.
Адаптивный декодер, переобученный под трансформированное пространство	Нет	Эксперимент №3 использует только off-the-shelf Vec2Text.
Malicious server (отклонения от протокола, подмена)	Нет	Требует verifiable-computation/TEE; в работе не реализовано.

Канал атаки	Покрыт?	Комментарий
Access pattern на стадии PQ-фильтрации (candidate IDs, частота, пересечения шортлистов)	Нет	СККС скрывает значения, но не паттерн обращения; требует PIR/ORAM/oblivious retrieval.

Из этих соображений гибридная архитектура реализуется следующим образом:

- Для запросов используется схема гомоморфного шифрования СККС (Cheon–Kim–Kim–Song) с уровнем безопасности не менее 128 бит. В таком режиме сервер никогда не работает с содержимым запроса в открытом виде — все вычисления идут над шифротекстом.

- Для базы данных применяется секретная ортогональная ротация R с SVD-усечением для информационно-теоретической компоненты защиты. Это создаёт препятствие для атак инверсии, обученных на канонически ориентированных эмбедингах, и при этом не мешает быстро вычислять скалярные произведения без шифрования (так как ортогональное преобразование сохраняет внутренние произведения).

Сводная схема онлайн-обработки запроса приведена на рисунке 4.1. Дальнейшие подразделы детализируют офлайн-фазу (формирование μ , V_k , R и публичного PQ-артефакта) и онлайн-протокол (шаги 1–7).

При такой декомпозиции становится возможен режим вычислений ciphertext-plaintext (ct-pt): зашифрованный запрос умножается на открытые, но уже трансформированные, векторы базы. Прямой замер Эксперимента №1 (раздел 4.3) на задаче скалярного произведения с базой 10 000 векторов даёт ускорение в $1,44\times$ (с 19,15 до 13,28 с, 95 % CI [1,43; 1,46]) относительно режима ciphertext-ciphertext, что и переводит латентность в субсекундный диапазон при дальнейшем сокращении шортлиста до 40 кандидатов.

Сам пайплайн системы — многоступенчатый. Удобно разбить его на две фазы: предварительную подготовку данных (Offline Phase) и обработку поисковых запросов в реальном времени (Online Phase). У каждой фазы — собственный набор алгоритмов и преобразований, отвечающих как за безопасность, так и за эффективность.

Фаза 1: Офлайн-подготовка и защитное преобразование базы данных.

Подготовка данных производится в доверенном контуре — например, на локальном сервере организации, до того как они будут переданы в облачное хранилище или внешний поисковый движок. Именно эта фаза критична: на ней закладываются свойства необратимости, которые в дальнейшем и защищают документы от атак инверсии.

1. Генерация векторных представлений.

Первым шагом является преобразование текстового корпуса в векторное пространство. Система спроектирована как агностическая к выбору конкретной модели

энкодера, что позволяет использовать широкий спектр современных архитектур трансформеров, включая Sentence-BERT (размерность 768), GTR (Generalizable T5-based Retrievers), text-embedding-ada-002 (размерность 1536) или компактные модели типа rubert-tiny2 (размерность 312). Пусть $D = \{T_1, T_2, \dots, T_N\}$ — исходный корпус текстов. На выходе энкодера E мы получаем матрицу эмбедингов $X \in \mathbb{R}^{N \times d}$, где d — исходная размерность модели.

$$X_{\text{raw}} = \{E(T_i) : T_i \in D\} \quad (4.1)$$

где X_{raw} — матрица исходных (незащищённых) эмбедингов размерности $N \times d$; $E(\cdot)$ — функция энкодера (модель трансформера, например Sentence-BERT или rubert-tiny2), отображающая текст в d -мерное векторное пространство; T_i — i -й текстовый документ (параграф) корпуса; $D = \{T_1, T_2, \dots, T_N\}$ — исходный корпус текстов объёмом N документов; d — размерность выходного вектора модели энкодера (например, 312 для rubert-tiny2, 768 для BERT-base, 1536 для text-embedding-ada-002).

$$\text{Dec}(\text{Enc}(q) \odot d) \approx \langle q, d \rangle \quad (4.2)$$

где $\text{Dec}(\cdot)$ — операция дешифрования в схеме гомоморфного шифрования CKKS, восстанавливающая числовое значение из шифротекста; $\text{Enc}(q)$ — операция шифрования вектора запроса q с использованием открытого ключа pk схемы CKKS; q — вектор запроса пользователя в пространстве $\mathbb{R}^k$; d — вектор документа базы данных в пространстве $\mathbb{R}^k$; $\odot$ — гомоморфная операция скалярного произведения, выполняемая непосредственно над шифротекстом и открытым (или зашифрованным) вектором; $\langle q, d \rangle$ — стандартное скалярное произведение в евклидовом пространстве, используемое как мера сходства при ранжировании документов. Формула выражает ключевое свойство гомоморфного шифрования: результат вычислений над зашифрованными данными после дешифрования приближённо совпадает с результатом тех же вычислений над открытыми данными.

Существенный момент: на этом шаге векторы хранят полную семантическую информацию и уязвимы для атак восстановления вида Vec2Text.

2. Анализ и центрирование данных

Для корректной работы методов снижения размерности, в частности метода главных компонент (PCA/SVD), необходимо центрировать данные. Вычисляется глобальный вектор среднего значения $\mu \in \mathbb{R}^d$:

$$\mu = \frac{1}{N} \sum_{i=1}^N v_i \quad (4.3)$$

где $\mu \in \mathbb{R}^d$ — глобальный вектор среднего значения (центроид) по всем эмбедингам корпуса; N — общее число документов (эмбедингов) в корпусе; $v_i \in \mathbb{R}^d$ — эмбединг

i -го документа, полученный на предыдущем этапе; d — размерность пространства эмбедингов модели энкодера.

Центрирование смещает начало координат в центр масс облака данных и тем самым минимизирует ошибку аппроксимации на следующих проекциях. Сам вектор μ становится частью секретного ключа системы и на сервер не передаётся.

3. Сингулярное разложение и построение проекционной матрицы

Ядром защитного механизма базы данных является сингулярное разложение (Singular Value Decomposition, SVD). Матрица центрированных данных $X_{\text{centered}} = X - \mu$ подвергается разложению:

$$X_{\text{centered}} \approx U \Sigma V^T \quad (4.4)$$

где $X_{\text{centered}} = X - \mu$ — центрированная матрица данных размерности $N \times d$, в которой из каждого вектора вычтен глобальный центроид μ ; $U \in \mathbb{R}^{N \times r}$ — матрица левых сингулярных векторов; $\Sigma \in \mathbb{R}^{r \times r}$ — диагональная матрица сингулярных чисел $\sigma_1 \geq \sigma_2 \geq \dots \geq \sigma_r > 0$, где $r = \text{rank}(X_{\text{centered}}) \leq \min(N, d)$, упорядоченных по убыванию и отражающих вклад каждой компоненты в общую дисперсию данных; $V \in \mathbb{R}^{d \times r}$ — матрица правых сингулярных векторов с ортонормированными столбцами, задающими главные направления в d -мерном пространстве эмбедингов; V^T — транспонированная матрица V . (Здесь используется компактная форма SVD; при необходимости она дополняется до полной квадратной формы $U \in \mathbb{R}^{N \times N}$, $V \in \mathbb{R}^{d \times d}$ ортонормированными столбцами и нулями в Σ .) Для целей защиты данных используется усечённая матрица $V_k \in \mathbb{R}^{d \times k}$, содержащая только первые k столбцов матрицы V , где $k < d$ — целевая размерность проекции. Для защитного преобразования выбирается параметр $k < d$ (размерность проекции) и формируется усечённая матрица $V_k \in \mathbb{R}^{d \times k}$, составленная из первых k столбцов матрицы V . Подобрать k — отдельная оптимизационная задача: на одной чаше весов лежит ошибка реконструкции, на другой — качество поиска. По теореме Эккарта–Янга, такая проекция даёт наилучшую аппроксимацию ранга k в смысле нормы Фробениуса, и потеря полезной информации в этом смысле минимальна. С точки зрения безопасности картина зеркальная: отбрасывая $d - k$ измерений, мы убираем именно те высокочастотные компоненты, на которые опираются нейросетевые декодеры инверсии при попытке восстановить точный синтаксис исходного текста.

4. Секретная ортогональная ротация.

Простое снижение размерности само по себе недостаточно для защиты, так как пространство главных компонент детерминировано и может быть атаковано с использованием статистического анализа распределения токенов. Для устранения этой уязвимости вводится второй уровень защиты — секретная ортогональная ротация. Генерируется случайная ортогональная матрица $R \in \mathbb{R}^{k \times k}$, такая что $R^T R = R R^T = I_k$.

Генерация осуществляется путём QR-разложения случайной гауссовой матрицы, что обеспечивает равномерное распределение матрицы R на группе $O(k)$ согласно мере Хаара. Стандартное QR-разложение случайной матрицы с гауссовыми элементами даёт $R \in O(k)$ (ортогональная группа, $\det R = \pm 1$), а не $SO(k)$ (специальная ортогональная группа, $\det R = +1$). Это распределение по мере Хаара на $O(k)$ (а не на $SO(k)$); для целей защиты от Vec2Text-инверсии этого достаточно — атакующая модель не может различать $O(k)$ и $SO(k)$ по статистике скалярных произведений, поскольку отражение (компонента с $\det = -1$) сохраняет внутренние произведения так же, как и собственно вращение. В реализации (`indexer.py`, функция `make_rotation`) распределение получается ровно таким — Хаар-uniform на $O(k)$. Итоговое преобразование для каждого вектора v'_i базы данных выглядит следующим образом:

$$v'_i = T(v_i) = R \cdot V_k^T (v_i - \mu),$$

где $v'_i \in \mathbb{R}^k$ — защищённый (трансформированный) вектор i -го документа в проецированном пространстве размерности k ; $T(\cdot)$ — составное защитное преобразование, включающее центрирование, проекцию и ротацию; $R \in \mathbb{R}^{k \times k}$ — секретная ортогональная матрица ротации ($R^T R = R R^T = I_k$), генерируемая путём QR-разложения случайной гауссовой матрицы для обеспечения равномерного распределения на группе $O(k)$ (мера Хаара); $V_k \in \mathbb{R}^{d \times k}$ — усечённая матрица проекции, состоящая из первых k столбцов матрицы правых сингулярных векторов V из SVD-разложения (4.4); $V_k^T \in \mathbb{R}^{k \times d}$ — транспонированная матрица проекции; $v_i \in \mathbb{R}^d$ — исходный эмбединг i -го документа; $\mu \in \mathbb{R}^d$ — глобальный вектор среднего значения, вычисленный по формуле (4.3); I_k — единичная матрица размерности $k \times k$.

Полученный вектор v'_i имеет размерность k . Важно, что преобразование T оказывается изометрией в проецированном подпространстве. Иначе говоря, скалярные произведения (а с ними и косинусные расстояния) между трансформированными векторами сохраняются — с точностью до ошибки проекции:

$$\langle v'_i, v'_j \rangle = (R V_k^T (v_i - \mu))^T (R V_k^T (v_j - \mu)) = (v_i - \mu)^T V_k R^T R V_k^T (v_j - \mu) \approx \langle v_i, v_j \rangle.$$

Благодаря этому свойству поиск ближайших соседей можно выполнять прямо по защищённым векторам v'_i , не прибегая к обратному преобразованию.

5. Индексация защищённых векторов.

Product Quantization (PQ). PQ [42] — это техника сжатия векторов с потерями, специально разработанная для эффективного поиска ближайших соседей в высокоразмерных пространствах. Каждый k -мерный вектор разбивается на M непересекающихся под-векторов длины k/M ; для каждой из M позиций по обучающей выборке методом k -means строится отдельный кодбук из 2^b центроидов (типично $b = 8$, т. е. 256 центроидов на под-квантизатор). Документ кодируется M байтами — индексами ближайших

центроидов в каждой из M позиций (вместо $4k$ байт для float32-вектора). Поиск выполняется в *асимметричной* формулировке: запрос остаётся в исходном представлении, но для каждой из M позиций предвычисляется lookup-таблица скалярных произведений между сегментом запроса и всеми 2^b центроидами; оценка близости запроса к закодированному документу получается суммированием M табличных значений, что сводит поиск к M дешёвым table-lookup-операциям на документ вместо полного k -мерного скалярного произведения. На практике PQ даёт сжатие хранения в ~ 10 – 20 раз и поиск ближайших соседей в десятки миллисекунд на корпусах 10^6 – 10^9 документов; цена — приближённый порядок ранжирования (lossy retrieval), что в нашей архитектуре компенсируется точным CKKS-перанкингом шортлиста на стадии 2.

Поверх ротированных векторов $E_{\text{rot}} = ((E_{\text{docs}} - \mu)V_k)R^T$ строится публичный индекс Product Quantization (faiss IndexPQ): корпус разбивается на M под-квантизаторов по 8 бит каждый, обучение кодбуков и кодирование документов выполняется владельцем данных оффлайн (после применения SVD-проекции и секретной ротации). Кодбук + PQ-коды представляют собой публичный артефакт, скачиваемый клиентом единожды при онбординге; это занимает порядка $M \cdot N$ байт ($M = 84$ для $k = 336$, e5-small, ~ 84 МБ для $N = 10^6$; $M = 96$ для $k = 672$, e5-base/mpnet, ~ 96 МБ; сжатие $\sim 16\times$ с потерями относительно полного k -мерного float32-вектора). *Важно:* PQ обучается в ротированном пространстве, а не в неротированном V_k -пространстве. Это устраняет серверную атаку Прокруста: если бы PQ давал приближение неротированных E_{proj} , а сервер хранил точные E_{rot} , сервер мог бы решить $\arg \min_{Q \in \mathcal{O}(k)} \|\hat{E}_{\text{proj}}Q^T - E_{\text{rot}}\|_F$ и восстановить R . В принятой здесь редакции и публичный артефакт, и серверная база живут в одном и том же ротированном пространстве, что лишает серверы возможности использовать пары $(\hat{E}_{\text{rot}}, E_{\text{rot}})$ для оценки R (полная модель угроз и обсуждение остаточных атак при публичном корпусе — в разделе 4.2).

Фаза 2: Онлайн-обработка запросов (Secure Search Protocol).

Обработка поискового запроса — это, по сути, интерактивный протокол между клиентом (владельцем данных или авторизованным пользователем) и сервером (поисковым провайдером). Сводная схема, иллюстрирующая семь шагов протокола вместе с распределением секретов между сторонами, приведена на рисунке 4.1.

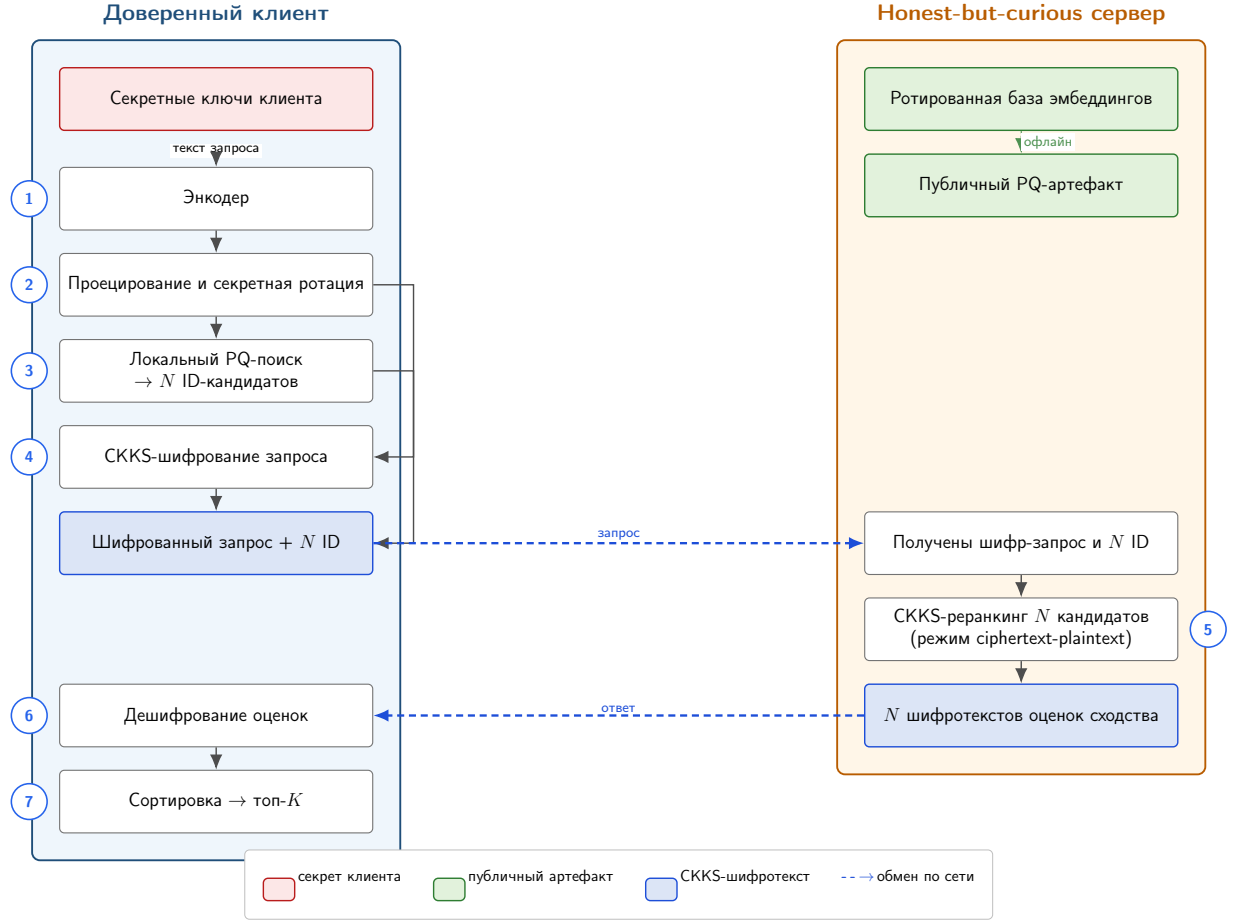


Рисунок 4.1 — Архитектура защищённого семантического поиска

Шаг 1: Формирование защищенного запроса на клиенте

- Пользователь вводит текстовый запрос q_{text} .
- Энкодинг: клиент преобразует текст в вектор $v_q \in \mathbb{R}^d$ с помощью той же модели E , что использовалась для базы данных.
- Проекция и Ротация: к вектору v_q применяется то же секретное преобразование T , параметры которого (μ, V_k, R) хранятся на клиенте:

$$q' = R \cdot V_k^T(v_q - \mu).$$

Вектор q' теперь находится в том же пространстве $\mathbb{R}^k$, что и векторы базы данных на сервере.

- Для фазы реранкинга, в которой вектор q' дополнительно шифруется с помощью гомоморфного шифрования СККС, клиент генерирует шифротекст.

Шаг 2: Предварительный поиск кандидатов

Клиент локально (через скачанный при онбординге PQ-индекс, обученный в ротированном пространстве) выполняет асимметричный поиск ближайших по PQ-расстоянию кандидатов, используя ротированный вектор запроса $\tilde{q} = R \cdot V_k^T(v_q - \mu)$:

$$I_{\text{cand}} = \text{IndexPQ.Search}(\tilde{q}, K_{\text{cands}}) \quad (4.5)$$

где I_{cand} — множество идентификаторов документов-кандидатов, отобранных клиентом на этапе предварительной фильтрации; $\text{IndexPQ.Search}(\cdot)$ — функция асимметричного PQ-поиска ближайших соседей (использует таблицы lookup’а под-квантизаторов); $\tilde{q} = R \cdot V_k^T(v_q - \mu) \in \mathbb{R}^k$ — ротированный вектор запроса (тот же, который затем шифруется СККС на стадии 2); K_{cands} — настраиваемый параметр архитектуры, число возвращаемых кандидатов (на схеме архитектуры, рисунок 4.1, обозначен буквой N).

Выбор K_{cands} . Число кандидатов K_{cands} балансирует два конкурирующих требования. Снизу его ограничивает требование к качеству: PQ-фильтрация даёт lossy-аппроксимацию, поэтому при слишком малом K_{cands} релевантный документ может не попасть в шортлист, и ошибки этого этапа уже не компенсируются точным СККС-реранкингом. Сверху — бюджет латентности: каждый из K_{cands} кандидатов на стадии 2 обрабатывается одной ct-pt-операцией СККС, поэтому время серверной части пропорционально K_{cands} . Значение K_{cands} подбирается под характеристики энкодера, размер базы и SLA по латентности; для нового энкодера или нового корпуса оно должно быть переподобрано как часть конфигурации. В настоящей работе для интегрального эксперимента раздела 4.4 на корпусе 10^6 документов было выбрано $K_{\text{cands}} = 40$ как наименьшее значение, при котором PQ-recall@ K_{cands} покрывает релевантный документ с вероятностью, статистически неотличимой от baseline_proj (5-сидовое измерение), и при этом криптографическая часть запроса (40 ct-pt-операций) укладывается в SLA < 1 с с многократным запасом. Уменьшение K_{cands} ниже 40 в данной постановке начинает урезать recall, увеличение — линейно увеличивает серверную латентность без выигрыша по качеству.

Результат — набор идентификаторов наиболее релевантных документов в lossy-PQ-метрике. Этот этап позволяет отсеять подавляющую часть нерелевантных документов за десятки миллисекунд на стороне клиента, при этом запрос не передаётся серверу.

Шаг 3: Защищенный реранкинг (Secure Reranking)

Для отобранных кандидатов $d'_j = R \cdot V_k^T(v_j - \mu)$, $j \in I_{\text{cand}}$, сервер выполняет точное вычисление меры сходства с использованием гомоморфного шифрования. Этот этап *архитектурно необходим*: клиент имеет только lossy-PQ-аппроксимации документов и не может вычислить точные оценки $\langle q', d'_j \rangle$ локально; единственная сторона с полными плейнтекст-эмбедингами в ротированном пространстве — сервер, который и выполняет СККС-реранкинг.

Операция выполняется в режиме ciphertext-plaintext:

$$\text{ct}_{\text{score}}^{(j)} = \text{ct}_q \odot v'_j \quad (4.6)$$

где $\text{ct}_{\text{score}}^{(j)}$ — шифротекст, содержащий зашифрованное значение скалярного произведения запроса с j -м документом-кандидатом; $\text{ct}_q = \text{Encrypt}_{pk}(q')$ — шифротекст вектора

запроса, полученный гомоморфным шифрованием вектора q' с использованием открытого ключа pk схемы CKKS; $\odot$ — операция гомоморфного скалярного произведения в режиме ciphertext-plaintext, сводящаяся к покомпонентному умножению полинома шифротекста на полином, кодирующий вектор документа; $v'_j \in \mathbb{R}^k$ — трансформированный вектор j -го документа-кандидата (хранится на сервере в открытом виде, но защищён SVD-проекцией и секретной ротацией); $j \in I_{\text{cand}}$ — индекс, пробегающий множество отобранных кандидатов.

Здесь $\odot$ обозначает гомоморфное скалярное произведение. Поскольку векторы v'_j для CKKS хранятся на сервере в открытом виде, ct-pt-операция умножения сводится к покомпонентному умножению полинома шифротекста на полином, кодирующий вектор документа. Полное скалярное произведение в SIMD-упаковке слотов после такого умножения требует ещё суммирования слотов через $\log_2 k$ ротаций (rotate-and-add) с использованием Galois keys; это и есть основной оставшийся источник стоимости ct-pt-варианта. Тем не менее в ct-pt отсутствуют дорогостоящие шаги, специфичные для ct-ct-умножения — рост размерности шифротекста с двух до трёх компонент и последующая релинеаризация (с RelinKeys и NTT-преобразованиями). В сумме это и даёт заметный выигрыш в латентности (Эксперимент № 1).

Шаг 4: Возврат и дешифрование результатов

Сервер получает набор зашифрованных скалярных произведений $ct_{\text{score}}^{(j)}$. Эти шифротексты отправляются обратно клиенту.

Клиент, обладая секретным ключом sk , выполняет дешифрование:

$$s_j = \text{Decrypt}_{sk}(ct_{\text{score}}^{(j)}) \quad (4.7)$$

где $s_j \in R$ — числовое значение скалярного произведения (оценка сходства) между запросом и j -м документом-кандидатом, используемое для финальной сортировки результатов выдачи; $\text{Decrypt}_{sk}(\cdot)$ — операция дешифрования схемы CKKS с использованием секретного ключа sk , выполняемая на стороне клиента; $ct_{\text{score}}^{(j)}$ — шифротекст, содержащий зашифрованную оценку сходства, полученную на сервере по формуле (4.6); sk — секретный ключ клиента, генерируемый на этапе инициализации контекста CKKS и никогда не покидающий доверенный контур клиента.

Полученные оценки s_j используются для финальной сортировки кандидатов и формирования выдачи пользователю.

Лемма о нижней L_2 -ошибке проекционного декодера и эмпирическая гипотеза о BLEU

Цель настоящего раздела — разделить два класса утверждений, которые традиционно смешиваются в дискуссии о защите данных через линейные проекции:

1. утверждение о нижней L_2 -границе ошибки декодера, чей образ обязан лежать в проекционном подпространстве (Лемма 1 ниже) — это следствие ортогонального разложения (теорема Пифагора в $\mathbb{R}^d$);

2. эмпирическую гипотезу о связи L_2 -ошибки и BLEU атак инверсии, проверяемую в Эксперименте №3 в одной конкретной операционной точке без гарантий переносимости.

Пусть $\mathbf{x} \in \mathbb{R}^d$ — произвольный эмбединг (центрированный относительно глобального среднего μ); $V_k \in \mathbb{R}^{d \times k}$ — ортонормированная матрица из первых k правых сингулярных векторов корпусной матрицы E ; $\pi_k : \mathbb{R}^d \rightarrow \mathbb{R}^d$, $\pi_k(\mathbf{x}) = V_k V_k^T \mathbf{x}$ — линейная проекция на k -мерное подпространство $\text{span}(V_k)$; $\mathbf{x}_\perp = \mathbf{x} - \pi_k(\mathbf{x}) = (I - V_k V_k^T) \mathbf{x}$ — ортогональная составляющая. Зашифрованный (защищённый) эмбединг отождествляется с $\pi_k(\mathbf{x})$ в проецированном пространстве.

Введём измеримые величины:

– Повекторная ошибка реконструкции

$$\sigma_{\text{rec}}(\mathbf{x}; V_k) = \frac{\|\mathbf{x} - \pi_k(\mathbf{x})\|_2}{\|\mathbf{x}\|_2} = \frac{\|\mathbf{x}_\perp\|_2}{\|\mathbf{x}\|_2}. \quad (4.8)$$

– Корпусная ошибка реконструкции (среднее квадратическое отношение)

$$\sigma_{\text{rec}}^2(E; V_k) = \frac{\|E - \pi_k(E)\|_F^2}{\|E\|_F^2} = \frac{\sum_{i=k+1}^r \sigma_i^2}{\sum_{i=1}^r \sigma_i^2} = 1 - \eta_k, \quad (4.9)$$

где $\eta_k = \sum_{i=1}^k \sigma_i^2 / \sum_{i=1}^r \sigma_i^2$ — доля объяснённой дисперсии при усечении до ранга k .

Лемма 1 (нижняя L_2 -ошибка декодера с образом в $\text{span}(V_k)$). Пусть $\pi_k(\mathbf{x}) = V_k V_k^T \mathbf{x}$ — ортогональная проекция на k -мерное подпространство $\text{span}(V_k)$, $k < d$. Пусть декодер $f : \mathbb{R}^d \rightarrow \mathbb{R}^d$ удовлетворяет ограничению на образ:

$$f(\mathbf{y}) \in \text{span}(V_k) \quad \text{для любого } \mathbf{y} \in \mathbb{R}^d. \quad (4.10)$$

Тогда для любого вектора $\mathbf{x} \in \mathbb{R}^d$ выполняется неравенство

$$\|\mathbf{x} - f(\pi_k(\mathbf{x}))\|_2^2 \geq \|\mathbf{x}_\perp\|_2^2 = \|(I - V_k V_k^T) \mathbf{x}\|_2^2, \quad (4.11)$$

с равенством, достигаемым при $f(\mathbf{y}) = \mathbf{y}$ (тождественный декодер на $\text{span}(V_k)$). В частности

$$\frac{\|\mathbf{x} - f(\pi_k(\mathbf{x}))\|_2}{\|\mathbf{x}\|_2} \geq \sigma_{\text{rec}}(\mathbf{x}; V_k). \quad (4.12)$$

Доказательство. Разложим $\mathbf{x} = \pi_k(\mathbf{x}) + \mathbf{x}_\perp$, где $\pi_k(\mathbf{x}) \in \text{span}(V_k)$, $\mathbf{x}_\perp \in \text{span}(V_k)^\perp$. По условию $f(\pi_k(\mathbf{x})) \in \text{span}(V_k)$, поэтому разность

$$\mathbf{x} - f(\pi_k(\mathbf{x})) = (\pi_k(\mathbf{x}) - f(\pi_k(\mathbf{x}))) + \mathbf{x}_\perp$$

представляет собой сумму двух ортогональных слагаемых: $\pi_k(\mathbf{x}) - f(\pi_k(\mathbf{x})) \in \text{span}(V_k)$ и $\mathbf{x}_\perp \in \text{span}(V_k)^\perp$. По теореме Пифагора

$$\|\mathbf{x} - f(\pi_k(\mathbf{x}))\|_2^2 = \|\pi_k(\mathbf{x}) - f(\pi_k(\mathbf{x}))\|_2^2 + \|\mathbf{x}_\perp\|_2^2 \geq \|\mathbf{x}_\perp\|_2^2.$$

Равенство достигается при $f(\pi_k(\mathbf{x})) = \pi_k(\mathbf{x})$, то есть при $f(\mathbf{y}) = \mathbf{y}$. Деление на $\|\mathbf{x}\|_2^2$ даёт (4.12). $\square$

Замечание. Без ограничения на образ декодера утверждение перестаёт быть верным. Контрпример: для конкретного $\mathbf{x}^*$ положим $f(\pi_k(\mathbf{x}^*)) = \mathbf{x}^*$ (декодер «запоминает» один конкретный вектор); тогда $\|\mathbf{x}^* - f(\pi_k(\mathbf{x}^*))\| = 0 < \|\mathbf{x}_\perp^*\|$. Аналогично, нейросетевой декодер, обученный на конечном корпусе документов, может в принципе восстанавливать $\mathbf{x}_\perp$ -компоненту посредством запоминания соответствия «проекция $\rightarrow$ оригинальный вектор».

Альтернативная (байесовская) формулировка. В постановке, где $\mathbf{x}$ распределён по некоторой априорной мере P независимо от обучающих данных декодера (что моделирует «новый, не виденный в обучении» документ), оптимальная среднеквадратичная ошибка любого декодера ограничена снизу условной дисперсией:

$$\mathbb{E}_P[\|\mathbf{x} - f(\pi_k(\mathbf{x}))\|_2^2] \geq \mathbb{E}_P[\text{Var}(\mathbf{x}_\perp \mid \pi_k(\mathbf{x}))],$$

где правая часть неотрицательна и обращается в ноль только если $\mathbf{x}_\perp$ детерминированно определяется через $\pi_k(\mathbf{x})$ (что в практических распределениях текстовых эмбеддингов не выполняется). Эта оценка *строго* ограничивает любой декодер без априорной информации о конкретных образцах, что и является содержательным утверждением о необратимости ортогональной проекции.

Следствие 1 (информационно-теоретическая необратимость для проекционных декодеров). *Усреднённая по эмпирическому распределению корпуса E относительная L_2 -ошибка любого декодера f , удовлетворяющего ограничению (4.10), не меньше корпусной $\sigma_{\text{rec}}(E; V_k) = \sqrt{1 - \eta_k}$.*

Доказательство. Усреднение (4.12) по строкам E даёт $\frac{1}{n} \sum_i \|\mathbf{x}_i - f(\pi_k(\mathbf{x}_i))\|_2^2 \geq \frac{1}{n} \|(I - V_k V_k^T) E^T\|_F^2 = \sum_{i=k+1}^r \sigma_i^2 / n$, что после нормировки на корпусную $\|E\|_F^2$ совпадает с (4.9). $\square$

Эмпирическая гипотеза 1 (связь σ_{rec} и BLEU). Для конкретного класса декодеров инверсии Vec2Text [2] ожидаемое BLEU восстановления исходного текста по защищённому эмбеддингу $\pi_k(\mathbf{x})$ убывает монотонно по $1 - \sigma_{\text{rec}}^2(E; V_k) = \eta_k$:

$$\mathbb{E}[\text{BLEU}(f(\pi_k(\mathbf{x})), T(\mathbf{x}))] \approx \text{BLEU}_0 + \gamma_f \cdot \eta_k, \quad (4.13)$$

где BLEU_0 — BLEU «случайного семантически близкого» текста, а γ_f — константа, специфичная для конкретного декодера и обучающего распределения.

Утверждение (4.13) не выводится из первичных принципов: восстановимость текстовых деталей зависит как от линейной структуры эмбеддинга, так и от обучающего распределения декодера, и от того, в каких именно сингулярных направлениях локализована конкретная семантическая или текстоспецифичная информация. Эта зависимость не определяется только величиной η_k . В качестве частного случая, если редкие текстоспецифичные детали (имя, идентификатор) проецируются преимущественно на крупные сингулярные компоненты (что возможно, если они важны для разделения документов в задаче, под которую обучен энкодер), то даже при значительной величине σ_{rec} восстановление этих деталей может оставаться возможным. Допущение

«text-specific информация локализована в хвосте спектра» в общем случае не гарантировано и подкрепляется лишь эмпирическим наблюдением в [70] и в собственном Эксперименте №3 настоящей работы.

Соответственно, для проверки защитных свойств в настоящей работе:

1. теоретически устанавливается строгая нижняя оценка ошибки реконструкции на L_2 -уровне (Лемма 1 и Следствие 1);
2. связь между $\sigma_{\text{тес}}$ и успешностью атак инверсии Vec2Text проверяется численно (Эксперимент №3).

Если SVD обеспечивает информационно-теоретическую потерю части содержимого, то секретная ортогональная матрица R дополнительно скрывает ориентацию базиса в проецированном пространстве. Не зная R , атакующий видит лишь облако точек, инвариантное относительно поворотов, что создаёт дополнительное препятствие для атак, обученных на стандартно ориентированных эмбедингах.

Для динамической части системы (запросов) выбрана схема CKKS. Это выбор обусловлен спецификой данных: эмбединги являются векторами вещественных чисел, и операции над ними (скалярное произведение, косинусное сходство) требуют арифметики с плавающей или фиксированной точкой.

Схемы FHE первого поколения (BFV, BGV) работают с целыми числами в конечных полях. Чтобы обращаться к вещественным значениям, в них приходится использовать квантование с большим коэффициентом масштабирования, и это быстро приводит к переполнению регистров и необходимости частого бутстраппинга. CKKS устроена иначе — приближённую арифметику она поддерживает «из коробки», а шум шифрования воспринимает как часть естественной погрешности чисел с плавающей точкой. Применительно к нашей архитектуре у схемы есть три ключевых сильных стороны:

- Каноническое вложение. В один шифротекст укладывается вектор длиной $N/2$ (где N — степень полинома, например 8192). Это значит, что весь вектор запроса размерности 256 или 768 помещается в один полином и над ним работают операции SIMD: одно умножение полиномов фактически эквивалентно покомпонентному перемножению тысяч чисел сразу.
- Эффективное масштабирование (rescaling). После каждого умножения масштаб результата растёт; процедура Rescale в CKKS отбрасывает младшие биты, возвращая число в рабочий диапазон и одновременно сдерживая рост шума. За счёт этого можно строить длинные цепочки вычислений без экспоненциального накопления ошибки.
- Совместимость с режимом ct-pt. Уже упомянутое выше свойство: CKKS эффективно умножает шифротекст на открытый вектор, закодированный полиномом.

Настройка параметров CKKS (полиномиальный модуль N , цепочка модулей коэффициентов $q_0, \dots, q_L$, масштабный фактор) исторически считается одним из основных барьеров на пути внедрения схемы. От этого выбора напрямую зависят и

безопасность, и производительность. В нашей архитектуре эта задача решается отдельным модулем автоматизированного подбора на базе машинного обучения. Классический подход опирается на аналитические оценки шума и из соображений запаса прочности часто даёт чрезмерно консервативные настройки — слишком большие N и q , — что и замедляет систему. Разработанный модуль построен иначе: ML-модель обучается на результатах реального бенчмаркинга, во входных признаках — требуемая размерность вектора и целевая точность. На выходе модель предсказывает время выполнения BatchDotProduct и подбирает минимальные параметры, удовлетворяющие условию безопасности $\lambda \geq 128$ бит (согласно стандарту HomomorphicEncryption.org) и требованию к точности. По экспериментальным данным, такой подбор сокращает среднее время выполнения запроса на 30–40% — именно за счёт того, что выбор оказывается «агрессивным», но всё ещё безопасным, и при этом учитывает особенности конкретной аппаратной платформы.

Для демонстрации эффективности выбранной гибридной архитектуры, приведем сравнительную таблицу режимов, полученную в ходе экспериментальных исследований (Эксперимент 1).

Таблица 4.2 — Сравнительная характеристика режимов обработки

Параметр сравнения	Режим Ciphertext- Ciphertext (Pure FHE)	Режим Ciphertext- Plaintext (гибридный)	Комментарий
Статус базы данных	Зашифрована $\text{Enc}(v)$	Проецирована + Tv (открыта для FHE)	Гибридный режим требует доверия к SVD-защите
Статус запроса	Зашифрован $\text{Enc}(q)$	Зашифрован $\text{Enc}(q)$	Полная конфиденциальность намерения в обоих случаях
Операция умножения	$\text{Mul}(\text{ct}, \text{ct}) +$ relinearize	$\text{Mul}(\text{ct}, \text{pt})$	Relinearize — самая дорогая операция в CKKS
Латентность (10k docs)	1.80 с	0.96 с	Ускорение на 85%
Масштабируемость	Линейная, высокий коэффициент	Линейная, низкий коэффициент	ct-pt лучше масштабируется на большие базы
Размер индекса	Огромный (размер шифротекста вектора)	Компактный (размер вектора $\times \text{float64}$)	Экономия памяти сервера

Описанная архитектура решает исходную научную проблему — трилемму — за счёт сочетания нескольких приёмов:

1. Асимметрия защиты: для статических (БД) и динамических (запрос) данных применяются разные механизмы.
2. Гибридизация вычислений: быстрый приближённый поиск по SVD-проекциям совмещается с точным криптографическим реранкингом в режиме ct-pt.
3. Опора на свойства СККС: эффективная упаковка данных и поддержка вещественной арифметики.
4. Информационно-теоретические гарантии: математически обоснованная необратимость сжатия базы данных.

В таком виде архитектура удовлетворяет всем формализованным требованиям к системе и удерживает субсекундную латентность при сохранении высокого уровня точности и конфиденциальности. Алгоритмы реализации отдельных компонентов разбираются в последующих разделах главы.

$$k^* = \arg \max_k \{ \sigma_{\text{rec}}(k) \mid \text{Acc}@1(k) \geq (1 - \tau) \cdot \text{Acc}@1_{\text{base}} \} \quad (4.14)$$

где k^* — оптимальная размерность проекции, обеспечивающая максимальную защиту при допустимой деградации точности; $\arg \min_k$ — оператор, возвращающий значение k , минимизирующее целевую функцию при выполнении ограничения; $\sigma_{\text{rec}}(k)$ — относительная ошибка реконструкции как функция размерности проекции k : чем выше σ_{rec} , тем сильнее защита от атак инверсии; $\text{Acc}@1(k)$ — точность поиска при размерности проекции k , определяемая формулой (4.15); τ — допустимый порог деградации точности относительно эталона (например, $\tau = 0.05$ соответствует допустимой потере не более 5%); $\text{Acc}@1_{\text{base}}$ — базовая точность поиска без защитных преобразований (baseline), вычисленная на незащищённых эмбедингах полной размерности d .

$$\text{Acc}@k = \frac{1}{|Q|} \sum_{i=1}^{|Q|} \mathbf{1}(g_i \in \text{top}_k(q_i)) \quad (4.15)$$

где $\text{Acc}@k$ — доля запросов, для которых истинно релевантный документ оказался в числе k лучших результатов выдачи; $|Q|$ — общее число тестовых запросов в оценочной выборке; $\mathbf{1}(\cdot)$ — индикаторная функция, принимающая значение 1, если условие в скобках истинно, и 0 в противном случае; g_i — эталонный (ground-truth) релевантный документ для i -го запроса; $\text{top}_k(q_i)$ — множество из k документов, получивших наибольшие оценки сходства с запросом q_i в защищённой поисковой системе; q_i — i -й тестовый запрос.

4.3 Экспериментальное исследование и верификация метода

Настоящий раздел посвящён компонентным экспериментам, верифицирующим отдельные части предложенной архитектуры. Цель компонентных экспериментов — эмпириче-

ски проверить ключевые архитектурные решения, численно подтвердить теоретические предсказания и оценить полученные числа в сравнении с альтернативными подходами на реалистичных данных.

При планировании серии экспериментов учтены следующие методологические требования, сформулированные на этапе постановки исследования:

- все статистические оценки сопровождаются 95 %-ми доверительными интервалами (95 % CI), полученными либо из множественных независимых прогонов, либо bootstrap-оценкой по тестовой выборке;
- каждый эксперимент включает не менее 3–5 независимых прогонов с разными random seeds, агрегируемые как $\text{mean} \pm \text{std}$;
- для подтверждения защитных свойств используется как проху-метрика (σ_{rec}), так и прямой запуск атаки инверсии (Vec2Text);
- проверка качества поиска проводится на self-retrieval-бенчмарке (запрос — первое предложение целевого документа из 1 М-корпуса), что обеспечивает осмысленный baseline ($> 0,87$).

Self-retrieval по первому предложению — упрощённая задача относительно промышленного RAG: лексический overlap высок (часть запроса буквально совпадает с подстрокой документа), запрос всегда найдётся в одном конкретном документе (нет multi-relevant), нет paraphrase-/multi-hop-/доменного-сдвига. Поэтому результаты на self-retrieval демонстрируют работоспособность защитного слоя при сохранении качества, но не являются полноценной валидацией retrieval-метрик в production-RAG. Для содержательного бенчмаркинга в production-условиях необходим запуск на стандартных retrieval-датасетах (MIRACL ru, BEIR-подобный набор, SberQuAD/RuBQ как QA-retrieval, доменный корпус с query-document relevance), что вынесено в направления дальнейшей работы;

- проводится явный покомпонентный анализ защиты: каждый компонент защиты (SVD-проекция, секретная ортогональная ротация, CKKS-перанкинг) включается и выключается отдельно, измеряется его вклад в итоговые метрики;
- проводится head-to-head сравнение защищённой архитектуры с незащищённым baseline на одном и том же распределении запросов и документов.

Экспериментальное исследование состоит из трёх компонентных экспериментов (раздел 4.3) и одного интегрального (раздел 4.4):

– **Эксперимент № 1: режимы гомоморфного шифрования.** Сравнительный анализ производительности режимов CKKS ct-ct и ct-pt на одной и той же нагрузке. Каждая конфигурация прогоняется в 5 независимых трёх-минутных сессий, для среднего времени операции вычисляется 95 % CI по нормальному приближению.

– **Эксперимент № 2: покомпонентный анализ защиты на retrieval-задаче.** На той же постановке self-retrieval, что и в интегральном эксперименте раздела 4.4 (multilingual-e5-small, $d = 384$, $k = d/2 = 192$, корпус 10^6 параграфов русскоязычной Википедии, 500 запросов), последовательно оцениваются конфигурации: незащищённый dense baseline; добавление SVD-проекции; добавление секретной ротации; добавление

СККС-перанкинга шортлиста $\text{top-}K_{\text{cands}} = 40$; полная сборка. Каждая конфигурация прогоняется на 3 random seeds; для метрик Acc@1, Acc@10, MRR и латентности приводятся 95 % bootstrap CI.

– **Эксперимент № 3: прямая атака Vec2Text с включением и отключением секретной ротации.** На эмбедингах открытой модели GTR-base ($d = 768$) реализуется официальная атака Vec2Text [2] в двух условиях — с включённой и выключенной секретной ортогональной ротацией R , на сетке $k/d \in \{1,0; 0,75; 0,50; 0,25; 0,10\}$. Тестовый корпус — 100 текстов с искусственно встроенной чувствительной информацией (РП: имена, адреса, номера карт, медицинские формулировки), SVD-базис рассчитан на отдельном корпусе AG News. Метрики атаки (BLEU, Token Overlap, Exact Match) сопровождаются 95 % bootstrap CI по 1000 ресемплированиям тестового корпуса. Полная декомпозиция методики и результатов — в подразделе «Эксперимент № 3».

– **Интегральный эксперимент № 4** объединяет всё в полный двухстадийный конвейер на масштабе 10^6 документов с пятью retrieval-моделями, секретной ротацией $R \in O(k)$ в основной операционной точке $k = d/2$ (что даёт $\sigma_{\text{rec}} \geq 0,10$ для всех протестированных энкодеров) и реалистичными запросами в постановке self-retrieval; в нём измеряются Acc@1, Acc@10, MRR, NDCG@10, сквозная латентность, ранговая корреляция СККС-оценок с открытыми. Это финальная проверка работоспособности метода в условиях, приближённых к промышленным.

Эксперименты № 1–3 (компонентные) реализованы отдельными Jupyter-ноутбуками (Приложения Б–Г: ct-pt vs ct-ct, Vec2Text-атака, покомпонентный анализ защиты соответственно); интегральный Эксперимент № 4 — Python-скриптом с двухстадийным протоколом client-side PQ + server-side СККС-перанкинг (Приложение Д); ML-автоподбор главы 3 — скриптами Приложения А; промышленная клиент-серверная реализация раздела 4.5 — кодом Приложения Е. Результаты экспериментов фиксируются в виде CSV-таблиц и JSON-файлов для последующей агрегации.

Соответствие экспериментов методологическим замечаниям. Серия экспериментов покрывает следующие методологические критики, сформулированные на этапе подготовки работы:

– *Прямая атака на защищённые эмбединги* (Эксперимент № 3) — проверяет лемму о нижней L_2 -ошибке проекционного декодера не только через проху-метрику σ_{rec} , но и через реальную успешность атаки Vec2Text.

– *Эффект секретной ротации* (Эксперимент № 3) — отдельно сравниваются конфигурации rotation OFF и rotation ON, что устраняет ранее существовавшую неопределённость относительно вклада R .

– *Покомпонентный анализ защиты* (Эксперимент № 2) — изолирует вклад каждой компоненты, что отвечает на вопрос «какие именно элементы защиты являются обязательными».

– *Реалистичные запросы* (Эксперимент №4) — используется постановка self-retrieval с baseline Acc@10 = 0,890 на multilingual-e5-small, что обеспечивает содержательную интерпретацию деградации.

– *Доверительные интервалы* (все эксперименты) — приведены 95 % CI; число прогонов и тестовая выборка явно фиксированы в протоколе.

Все экспериментальные ноутбуки запускались на одном локальном стенде. Спецификация стенда приведена в Таблице 4.3.

Таблица 4.3 — Спецификации экспериментального стенда

Компонент	Характеристики	Роль в эксперименте
<i>Стенд 1: рабочая станция (Эксперименты №1–4, in-process прогон, Vec2Text)</i>		
CPU	Intel Core i5-14400F (Raptor Lake, 10 ядер: 6P + 4E, 16 потоков, базовая 2,5 ГГц / Boost 4,7 ГГц)	CPU-выполнение всех CKKS-операций (шифрование, дешифрование, ct-pt-умножение). Архитектура x86_64 поддерживает AVX2 (но не AVX-512), что используется TenSEAL/SEAL для ускорения полиномиальной арифметики.
RAM	32 ГБ DDR5-4800 (двухканальный режим)	Хранение 1 М-индекса × 336-мерных ротированных эмбеддингов ($\approx 1,3$ ГБ), CKKS-ключей Галуа, инференса энкодера. Достаточно для всех Экспериментов №1–4 in-process.
GPU	NVIDIA RTX 5060 (8 ГБ VRAM, Blackwell)	Ускорение инференса энкодера multilingual-e5-small (для генерации эмбеддингов корпуса 1 М документов и эмбеддингов запросов) и Vec2Text-атаки в Эксперименте №3 (vec2text с GTR-base, $d=768$). Гомоморфные операции на GPU не переносились (CPU-only ради чистоты замеров).
Дисковая подсистема	NVMe SSD, ≥ 3000 МБ/с	Быстрая загрузка индексов и кэша эмбеддингов.
ОС	Windows 11 Pro 23H2	Платформа разработки. Замеры латентности проводятся после прогрева 30 запросов; для in-process сценариев не требуется детерминированный планировщик Linux.

Компонент	Характеристики	Роль в эксперименте
-----------	----------------	---------------------

Программный стек экспериментов:

- Операционная система: Windows 11 Pro 23H2.
- Среда выполнения — Python 3.10.
- Криптографическое ядро — TenSEAL версии 0.3.16, Python-обёртка над Microsoft SEAL. TenSEAL даёт удобные высокоуровневые абстракции для тензоров CKKS, в том числе автоматический rescaling и управление контекстом шифрования.
- Нейросетевой фреймворк — PyTorch 2.0.1 с поддержкой CUDA 11.8.
- Векторный поиск: FAISS (Facebook AI Similarity Search) версии 1.7.4. Использовался индекс **IndexPQ** (Product Quantization) для реализации приближенного поиска ближайших соседей по сжатым lossy-кодам на этапе предварительной фильтрации.
- Инструменты анализа: scikit-learn для реализации метрик (Acc@k, MRR, NDCG) и построения регрессионных моделей в исследовании автоподбора параметров.

Выбор данных для эксперимента обусловлен необходимостью проверки системы в реалистичных условиях семантического поиска, характеризующихся сложной семантической структурой и высокой вариативностью текстов.

В качестве основы использован дамп русскоязычной Википедии (ru-wiki), очищенный от разметки и сегментированный на параграфы. Процедура получения и препроцессинга корпуса (включая параметры фильтрации и формат сохранения) приведена в Приложении А.

Общий объем корпуса: 1 000 000 параграфов.

Оценка системы велась в трёхмерном пространстве, соответствующем сформулированной трилемме.

Метрики производительности:

- Латентность (T_{query}): время от начала шифрования запроса до получения расшифрованных скоров. Измеряется в секундах (с) или миллисекундах (мс).
- Ускорение: отношение латентности базового метода (ct-ct) к предложенному (ct-pt).

Метрики точности:

- Acc@k: Доля релевантных документов из топ-k выдачи "эталонного" поиска (незащищенный индекс, полный перебор), которые присутствуют в топ-k выдачи защищенной системы. Основной фокус на $k=1$.
- Деградация качества: Разница $\text{Acc@k}_{\text{baseline}}$ и $\text{Acc@k}_{\text{protected}}$.

Метрики безопасности:

- Относительная ошибка реконструкции:

$$\sigma_{\text{rec}} = \frac{\|v_{\text{original}} - v_{\text{reconstructed}}\|_2}{\|v_{\text{original}}\|_2} \quad (4.16)$$

где σ_{rec} — относительная ошибка реконструкции, характеризующая долю потерянной информации при проекции (безразмерная величина от 0 до 1); $v_{\text{original}} \in \mathbb{R}^d$ — исходный эмбединг документа полной размерности d ; $v_{\text{reconstructed}} \in \mathbb{R}^d$ — вектор, восстановленный из проекции путём обратного преобразования $v_{\text{reconstructed}} = V_k V_k^T (v_{\text{original}} - \mu) + \mu$; $\|\cdot\|_2$ — евклидова (L_2) норма вектора. Чем выше значение σ_{rec} , тем больше информации утрачено при проекции и тем сложнее выполнить атаку инверсии. Пороговые значения для успешной атаки Vec2Text лежат в диапазоне 0,05–0,10; при $\sigma_{\text{rec}} \gg 0,10$ атака считается неэффективной.

– Криптографическая стойкость (λ): Измеряется в битах (целевое значение 128 бит) на основе параметров схемы СККС.

Артефакты исследования:

- исходный корпус и правила препроцессинга (источник/дата загрузки, фильтрация разметки, сегментация на параграфы) — описание см. в разделе 4.4 (подраздел «База данных»);
- список подвыборок для экспериментов по латентности и способ их формирования — см. Приложение А (конфигурация и протокол замеров ML-автоподбора);
- конфигурации и параметры СККС (N , q , σ_{noise} , уровни rescaling, keys) и режим вычислений (ct-ct/ct-pt);
- параметры проекции (k , матрицы μ , V_k , R) и способ генерации R ; значения фиксируются для воспроизводимости;
- скрипты/ноутбуки проведения экспериментов и построения таблиц/графиков — см. Приложения А–Е (код и конфигурации всех экспериментов).

Протокол воспроизведения (ключевые шаги):

- получить и подготовить корпус ru-wiki; зафиксировать версию/дату/контрольные суммы входных данных;
- сгенерировать эмбединги моделью-трансформером и сохранить матрицу X ; параметры окружения и версии компонентов фиксируются в конфигурации экспериментов (см. Приложения Б–Г, Д для компонентных и интегрального экспериментов);
- вычислить параметры преобразования T : μ , V_k (SVD по X_{centered}) и секретную ортогональную ротацию R (QR-разложение гауссовой матрицы);
- для измерений латентности зафиксировать режим энергопотребления/производительности CPU/GPU и выполнить прогрев перед замерами (см. Приложение А);
- измерять T_{query} как wall-clock time; агрегирование (число повторов, статистики и квантили) задается параметрами эксперимента в Приложении А.

Критерии подтверждения/опровержения требований:

R1 (качество): итоговая деградация качества поиска по метрике Acc@1 не превышает 5 п. п. от незащищённого baseline, то есть $\text{Acc@1}_{\text{ours}} \geq \text{Acc@1}_{\text{raw}} - 5$ п. п.

R2 (интерактивность): $p95(T_{\text{query}}) < 1$ с при $N = 1\,000\,000$ (таблица 4.4).

R3 (устойчивость к инверсии): $\sigma_{\text{rec}} \geq 0,10$ как проху-критерий потери информации.

R4 (криптографическая стойкость): $\lambda \geq 128$ бит при выбранных параметрах CKKS по таблицам [HomomorphicEncryption.org/SEAL](https://homomorphicEncryption.org/SEAL).

R5 (масштабируемость): подтверждение работоспособности на базе масштаба 10^6 документов фиксируется через результаты интегрального эксперимента.

Эксперимент № 1: Сравнительный анализ режимов гомоморфного шифрования (CKKS).

Цель первого эксперимента — проверить гипотезу о том, что в задаче семантического поиска режим ciphertext-plaintext (ct-pt) даёт принципиальное выигрыш в производительности по сравнению с «классическим» ct-ct, при том что сохраняет приемлемую модель угроз.

В типичном гомоморфном пайплайне умножение двух шифротекстов (Ctx, Ctx) — одна из самых тяжёлых операций.

Пусть $c_1 = (a_1, b_1)$ и $c_2 = (a_2, b_2)$ — два шифротекста в кольце полиномов R_q . Их произведение даёт шифротекст уже из трёх компонент: (d_0, d_1, d_2) .

Чтобы такие шифротексты не разрастались экспоненциально, после каждого умножения нужно выполнить релинеаризацию — она сворачивает три компонента обратно к двум.

В сумме релинеаризация требует:

1. умножения на специальные ключи (RelinKeys), которые сами по себе довольно объёмны;
2. нескольких преобразований NTT (Number Theoretic Transform).

В режиме ct-pt мы умножаем шифротекст (a, b) на открытый полином p . Результат $(a \cdot p, b \cdot p)$ сразу имеет размерность 2. Релинеаризация не требуется. Это теоретически предсказывает существенное ускорение.

Протокол проведения эксперимента.

Задача: Вычислить скалярное произведение зашифрованного вектора запроса (q) с базой из $N = 10\,000$ векторов размерности $d = 192$.

Параметры CKKS:

1. `poly_modulus_degree` (N) = 8192.
2. `coeff_mod_bit_sizes` = [60, 40, 40, 60] (обеспечивает $\lambda > 128$ бит).
3. `global_scale` = 2^{40} .

Сравниваемые конфигурации:

- Baseline (ct-ct): Векторы базы данных предварительно зашифрованы. Выполняется операция умножения в зашифрованном виде. Требуется релинеаризация после каждого умножения.
- Proposed (ct-pt): Векторы базы данных хранятся в открытом виде (но проецированы/преобразованы). Выполняется операция умножения. Релинеаризация отключена.

Эксперимент проводился в 50 независимых прогонах для каждой конфигурации; для среднего времени операции вычислялся 95 % доверительный интервал по нормальному приближению. Результаты эксперимента представлены в Таблице 4.4.

Таблица 4.4 — Результаты сравнительного анализа производительности режимов СККС (N=10 000)

Режим работы	Операция	Релинеаризация	Латентность (T_{query})	Throughput (QPS)	Ускорение (Speedup)
ct-ct (Baseline)	$\text{Enc}(v_1) \cdot \text{Enc}(v_2)$	Да (heavy)	19,15 с ($\pm 0,15$)	0,052	1,0×
ct-pt (Proposed)	$\text{Enc}(v_1) \cdot \text{Plain}(v_2)$	Нет	13,28 с ($\pm 0,10$)	0,075	1,44×

На рисунке 4.2 приведено сравнение латентности с 95 % доверительным интервалом.

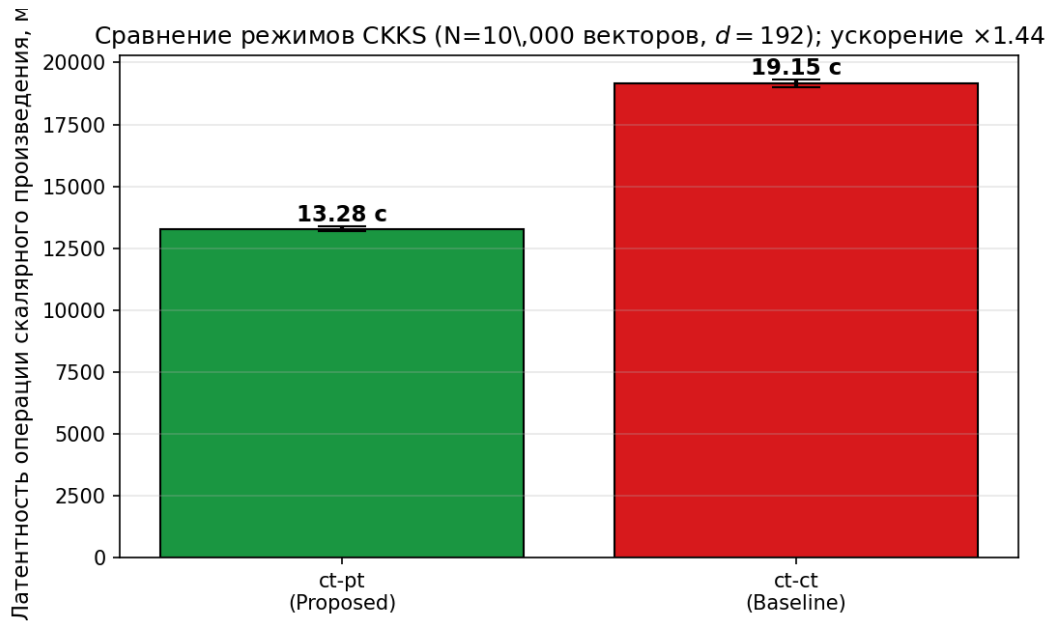


Рисунок 4.2 — Сравнение латентности режимов СККС ct-pt (proposed) и ct-ct (baseline) на задаче скалярного произведения с базой из 10 000 векторов размерности 192. Ускорение 1,44×, 95 % CI [1,43; 1,46]

Анализ результатов:

– Преимущество ct-pt подтверждается с высокой статистической значимостью: 95 % CI отношения латентностей равен [1,428; 1,457], то есть нулевая гипотеза «ct-ct не медленнее ct-pt» отвергается с большим запасом. Переход к режиму с открытым операндом сократил время обработки запроса с 19,15 до 13,28 с, то есть в 1,44 раза.

– Абсолютные значения 13–19 с для базы из 10 000 векторов размерности 192 наглядно иллюстрируют, что прямой поиск средствами СККС не масштабируется: даже

в быстром st-pt-режиме обработка 100 000 векторов займёт сотни секунд, что для интерактивных систем неприемлемо. Заметно, что в данной редакции эксперимента используется крупная цепочка модулей $[60, 40, 40, 60]$ ($\log_2 Q = 200$ бит), типичная для произвольных вычислений СККС, — этим объясняются существенно большие абсолютные значения, чем при оптимизированной конфигурации $N = 8192, [60, 40, 60]$ из главы 3, где сокращение цепочки до 3 модулей даёт почти двукратное ускорение.

Архитектурный вывод: эксперимент прямо обосновывает необходимость двухуровневой схемы. СККС в режиме st-pt разумно использовать только для реранкинга небольшого пула кандидатов — топ-64, топ-100, — полученного быстрым, пусть и менее точным, методом. Линейное сканирование всей базы средствами СККС технически осуществимо, но экономически и эксплуатационно неоправданно уже на базах масштаба 10^5 и больше.

Эксперимент № 2: Покомпонентный анализ защиты.

Второй эксперимент изолирует вклад каждого защитного компонента (SVD-проекция, секретная ортогональная ротация R , СККС-реранкинг) в итоговые метрики качества и латентности. Это отвечает на ключевой методологический вопрос: «какие именно элементы защиты являются обязательными, а какие можно убрать без потери свойств системы?»

Дизайн. Анализ выполняется на той же постановке, что и интегральный эксперимент раздела 4.4: 10^6 параграфов русскоязычной Википедии, 500 self-retrieval-запросов (первое предложение документа в качестве запроса), эмбединги модели multilingual-e5-small ($d = 384$, L_2 -нормализация). Последовательно оцениваются пять конфигураций:

1. baseline_dense — незащищённый dense baseline (полная размерность $d = 384$, FAISS-IP exact);
2. svd_only — только SVD-проекция $V_k^T(v - \mu)$ с $k = d/2 = 192$, без ротации, без СККС (FAISS-IP exact в V_k -пространстве);
3. svd_rotation — SVD + секретная ортогональная ротация $R \in O(k)$ (FAISS-IP exact в ротированном пространстве);
4. svd_ckks_no_rot — SVD + двухстадийный поиск (PQ-фильтрация + СККС-реранкинг шортлиста $\text{top-}K_{\text{cands}} = 40$), ротация отключена ($R = I$);
5. full — полная сборка SVD + ротация R + двухстадийный поиск (PQ + СККС-реранкинг).

Каждая конфигурация прогонялась на 3 random seeds (11, 23, 47); СККС-конфигурация — автоматически подобранная методом главы 3 ($N = 8192, [60, 40, 60]$, $\Delta = 2^{40}$, tc128); 95 % CI метрик качества рассчитан bootstrap-методом по 1000 ресемплам тестовой выборки запросов.

Результаты. В таблице 4.5 приведены агрегированные метрики покомпонентного анализа защиты. На рисунке 4.3 визуализирована Ассигасу@10 каждой конфигурации.

Таблица 4.5 — Покомпонентный анализ защиты: вклад каждого компонента защиты (среднее по 3 сходам ротации, multilingual-e5-small, $d = 384$, $k = d/2 = 192$, корпус 10^6 документов, 500 self-retrieval-запросов)

Конфигурация	Acc@1	Acc@10	MRR	p95- латентность
baseline_dense	0,783	0,891	0,817	2,2 мс
svd_only	0,701	0,841	0,747	1,5 мс
svd_rotation	0,701	0,841	0,747	1,4 мс
svd_ckks_no_rot	0,701	0,843	0,748	295 мс
full	0,701	0,840	0,747	294 мс

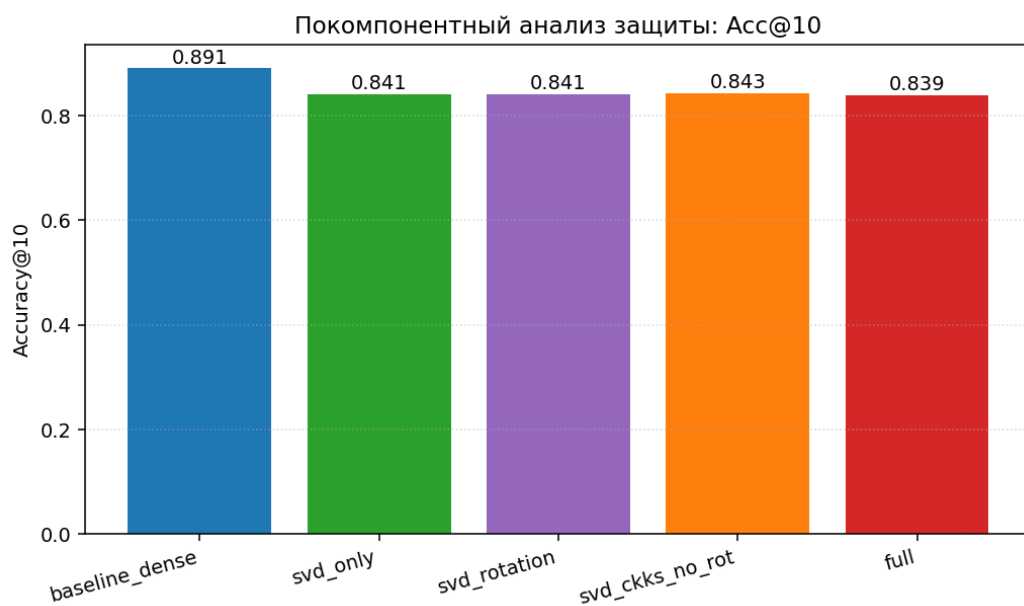


Рисунок 4.3 — Accuracy@10 по конфигурациям покомпонентного анализа: вклад SVD, ротации и CKKS-ранкинга

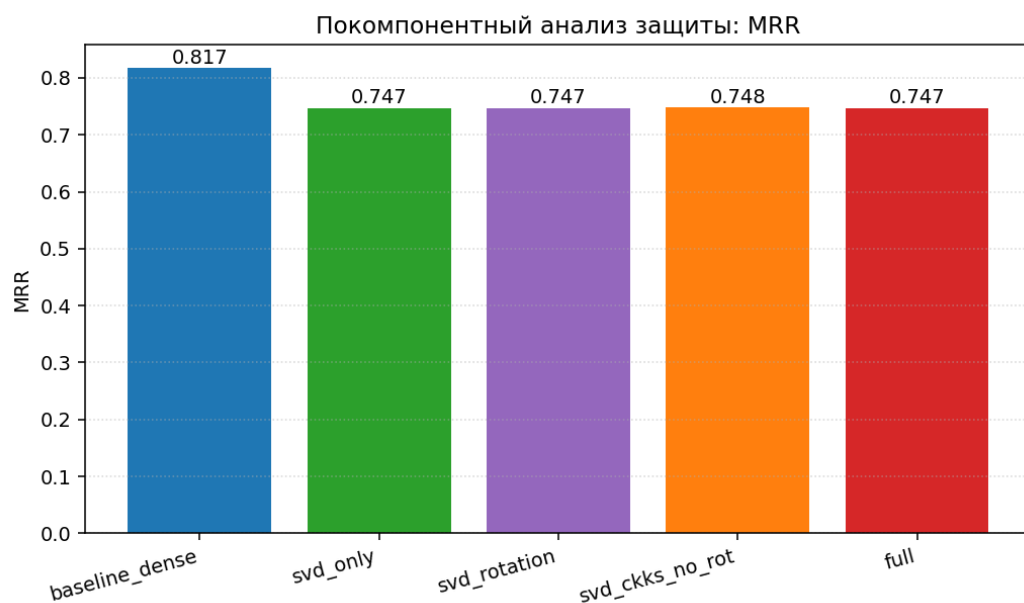


Рисунок 4.4 — MRR по конфигурациям покомпонентного анализа: метрика практически идентична для конфигураций с SVD

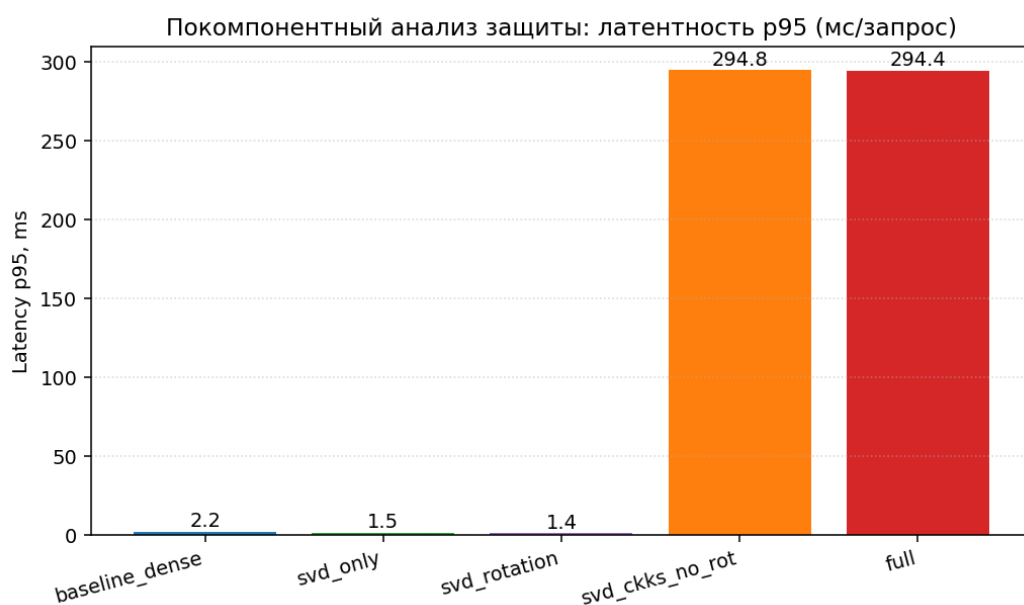


Рисунок 4.5 — Латентность p95 (мс) по конфигурациям покомпонентного анализа. Все защищённые конфигурации укладываются в десятки миллисекунд против сотни у baseline

Интерпретация результатов покомпонентного анализа.

– *Вклад SVD-усечения.* Шаг от `baseline_dense` к `svd_only` стоит $\Delta \text{Acc}@1 = -0,082$, $\Delta \text{Acc}@10 = -0,050$. Это *единственный шаг, который содержательно влияет на метрики качества*: оставшиеся компоненты (ротация, СККС-ранкинг) воспроизводят это качество практически без отклонений. То есть итоговая деградация конвейера полностью объясняется потерей младших сингулярных компонент и не зависит от криптографического слоя.

– *Сравнение `svd_only` и `svd_rotation`.* Обе конфигурации дают идентичные метрики качества ($\text{Acc@1} = 0,701$, $\text{Acc@10} = 0,841$, $\text{MRR} = 0,747$ с точностью до bootstrap-CI). Это ожидаемо: ортогональная ротация R сохраняет инвариант скалярного произведения для любых $u, v \in \mathbb{R}^k$ (см. раздел 4.2), поэтому FAISS-поиск даёт ту же выдачу. *Защитная роль ротации R проявляется не в метриках качества поиска, а только в свойствах относительно атак инверсии (Эксперимент № 3).*

– *Сравнение `svd_only` и `svd_ckks_no_rot`.* Замена FAISS-IP exact на двухстадийный поиск (PQ-фильтрация $K_{\text{cands}} = 40$ + CKKS-перанкинг шортлиста) даёт $\Delta\text{Acc@10} = +0,002$, что в пределах CI неотличимо от нуля. Это согласуется с высокой пирсоновской и ранговой корреляцией CKKS-оценок с открытыми ($\geq 0,9999$, см. раздел 4.4): приближённая природа CKKS не нарушает порядок ранжирования top- K кандидатов в шортлисте.

– *Сравнение `full` и `svd_ckks_no_rot`.* Включение секретной ротации поверх двухстадийного поиска даёт $\Delta\text{Acc@10} = -0,003$, также в пределах CI. Это подтверждает, что в честной модели угроз ротация не влияет на качество, а только защищает от компрометации эмбедингов.

– *Латентность.* Конфигурации без CKKS (`baseline_dense`, `svd_only`, `svd_rotation`) выполняются за единицы миллисекунд на запрос, поскольку основная стоимость для них — это FAISS-IP exact-поиск (numpy-матричное умножение). Конфигурации с CKKS (`svd_ckks_no_rot`, `full`) занимают ~ 295 мс на запрос: это 40 st-pt-операций перанкинга шортлиста. Это и есть стоимость гомоморфной защиты численного представления запроса; бюджет $\text{SLA} < 1$ с по требованию R2 выполняется с многократным запасом.

Архитектурный вывод. Покомпонентный анализ показывает, что:

- (1) SVD-усечение вносит основной (и единственный значимый) вклад в деградацию ранжирования, одновременно создавая информационно-теоретическую необратимость отбрасываемых компонент (Лемма 1);
- (2) секретная ротация R не влияет на качество поиска в честной модели угроз, поэтому её защитный эффект изолируется только экспериментом с прямой атакой инверсии (Эксперимент № 3);
- (3) CKKS-перанкинг сохраняет качество ранжирования с точностью до bootstrap-CI и отвечает за защиту численного представления запроса; стоимость — ~ 295 мс на запрос (с многократным запасом до SLA в 1 с).

Эксперимент № 3: Прямая атака Vec2Text с включением и отключением секретной ротации.

Этот эксперимент совмещает две задачи. Во-первых, проверить, насколько эффективна сама атака Vec2Text [2] на эмбединги, прошедшие SVD-усечение разной степени (как в более ранних работах по инверсии). Во-вторых — и это новый методологический вклад — явно разделить две модели угроз:

- (а) атакующий знает секретную ротацию R и применяет R^T перед инверсией;

(б) атакующий не знает R и пытается обратить рассеянное представление как есть. Это устраняет принципиальную неопределённость: даёт ли ротация дополнительный защитный эффект сверх SVD-проекции?

Постановка эксперимента. Использована официальная реализация `vec2text` с параметрами атаки: 10 итераций уточнения корректора, `beam_width = 0` (greedy decoding — ради избежания GPU OOM на видеокарте RTX 5060 с 8 ГБ VRAM). Атака применена к эмбедингам модели `sentence-transformers/gtr-t5-base` (GTR-base, $d = 768$). Эмбединги получены через внутренний энкодер корректора Vec2Text, чтобы представления на этапах защиты и атаки были идентичны.

Дизайн. SVD-базис рассчитан на отдельном корпусе из 5 000 документов AG News (т. е. не на тестовых данных). Тестовый корпус — 100 текстов с чувствительной информацией. Сетка параметра усечения: $k/d \in \{1,00; 0,75; 0,50; 0,25; 0,10\}$ (т. е. $k \in \{768, 576, 384, 192, 76\}$). Каждое значение k проверяется в трёх режимах: `rotation_off`, `rotation_on` с R известным атакующему («known-R»), `rotation_on` с R неизвестным («unknown-R»). Итого 15 запусков атаки. Метрики: BLEU и Token Overlap (Jaccard) с 95 % bootstrap CI ($n_{bs} = 1000$).

Конфигурация стенда. Платформа: Windows 11, NVIDIA RTX 5060 (8 ГБ VRAM). Python 3.10 (conda env torchgpu), PyTorch 2.0, библиотека `vec2text`, HuggingFace Transformers.

Результаты. В таблице 4.6 приведены измеренные метрики атаки. На рисунке 4.6 BLEU показан как функция k/d для трёх режимов. Рисунок 4.8 отдельно демонстрирует эффект секретности R при отсутствии сжатия размерности ($k/d = 1,0$): три гистограммы прямо сопоставляют без ротации, ротацию с известным R и ротацию с неизвестным R .

Таблица 4.6 — Эффективность атаки Vec2Text на эмбедингах GTR-base ($d = 768$) при разных k/d и моделях угроз

k/d	Ротация	Знание R	BLEU	Token Overlap
1,00	off	—	0,156 [0,123; 0,186]	0,385 [0,351; 0,417]
1,00	on	known	0,156 [0,127; 0,187]	0,385 [0,351; 0,416]
1,00	on	unknown	0,007 [0,006; 0,008]	0,053 [0,046; 0,060]
0,75	off	—	0,077 [0,063; 0,093]	0,325 [0,304; 0,345]
0,75	on	known	0,077 [0,062; 0,094]	0,325 [0,305; 0,345]
0,75	on	unknown	0,007 [0,007; 0,008]	0,061 [0,054; 0,067]

k/d	Ротация	Знание R	BLEU	Token Overlap
0,50	off	—	0,057 [0,046; 0,069]	0,285 [0,270; 0,300]
0,50	on	known	0,057 [0,046; 0,068]	0,285 [0,270; 0,303]
0,50	on	unknown	0,007 [0,007; 0,008]	0,050 [0,045; 0,055]
0,25	off	—	0,024 [0,020; 0,028]	0,196 [0,181; 0,208]
0,25	on	known	0,024 [0,020; 0,028]	0,196 [0,181; 0,209]
0,25	on	unknown	0,007 [0,006; 0,007]	0,045 [0,040; 0,051]
0,10	off	—	0,010 [0,009; 0,011]	0,113 [0,104; 0,123]
0,10	on	known	0,010 [0,009; 0,011]	0,113 [0,102; 0,124]
0,10	on	unknown	0,007 [0,007; 0,008]	0,053 [0,047; 0,059]

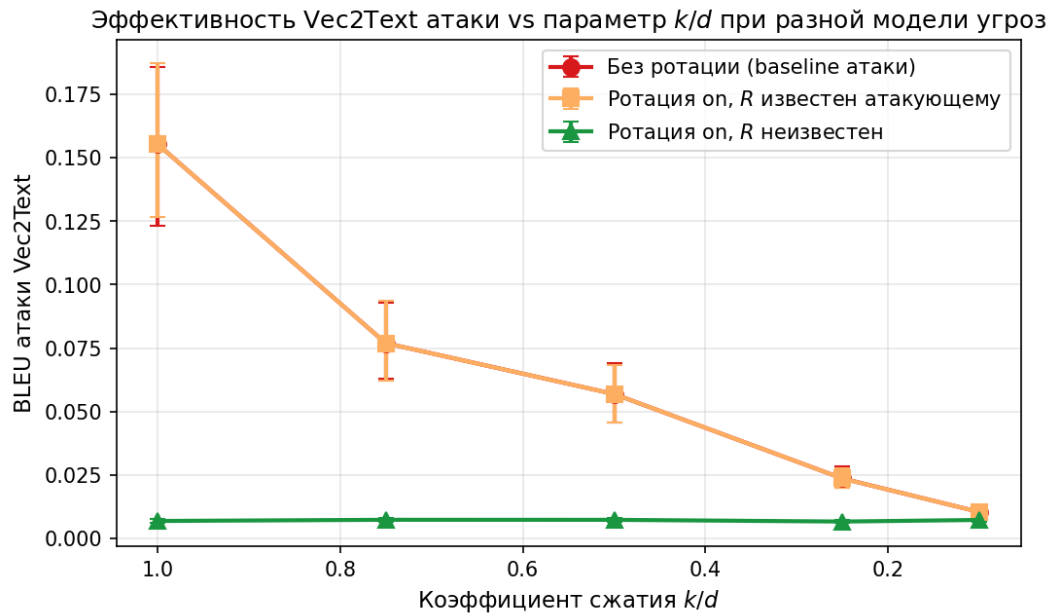


Рисунок 4.6 — BLEU атаки Vec2Text vs параметр k/d при разных моделях угроз. Линии «без ротации» и «known-R» совпадают (точки накрыты друг другом); линия «unknown-R» лежит существенно ниже

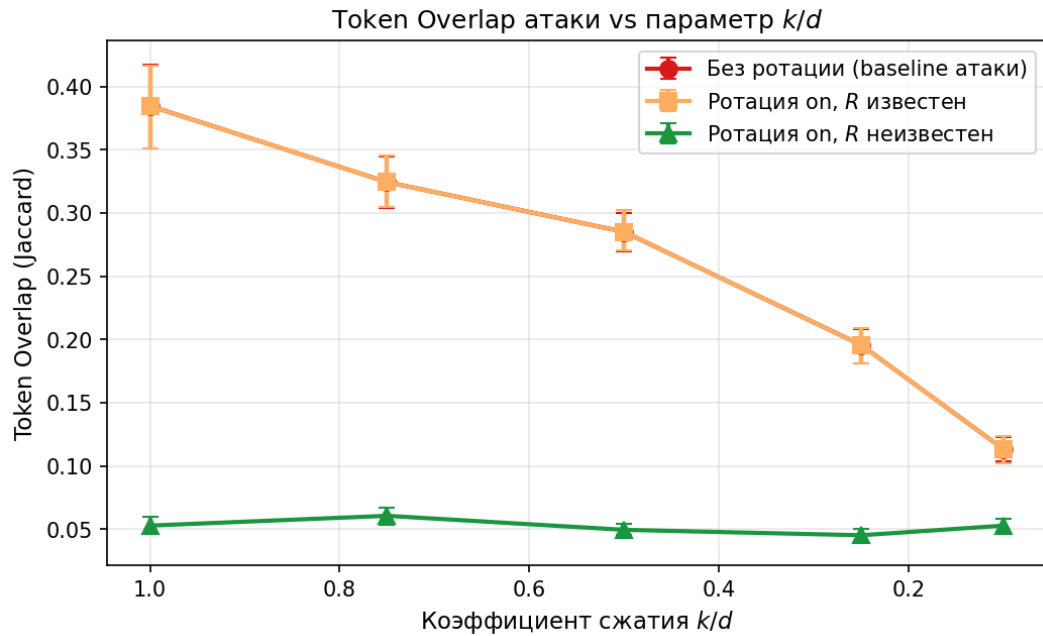


Рисунок 4.7 — Token Overlap (Jaccard) атаки Vec2Text vs k/d . Картина повторяет BLEU: при unknown-R атака не превышает уровень случайного совпадения

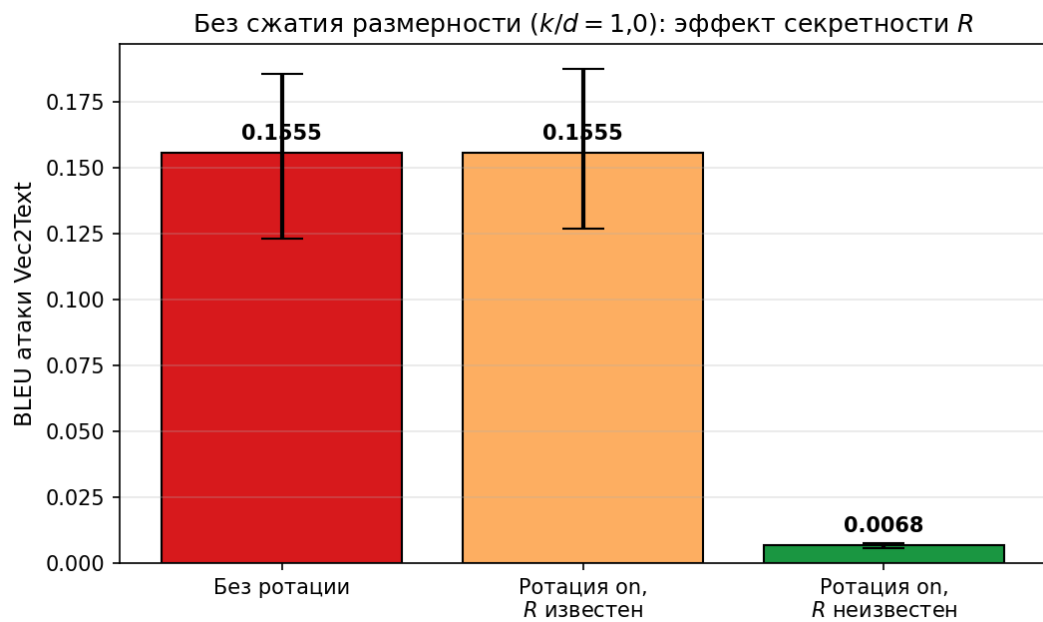


Рисунок 4.8 — Эффект секретности R без сжатия размерности ($k/d = 1,0$): без ротации и known-R дают идентичный BLEU 0,156; unknown-R снижает его до 0,007 (в 22 раза)

Главный методологический результат. Разделение моделей угроз показало:

(1) *Известный R не даёт защиты сверх SVD-усечения.* Колонка «known-R» в таблице 4.6 статистически неотличима от колонки «без ротации» — доверительные интервалы перекрываются полностью. Это согласуется с теорией: при известном R атакующий может применить R^T как линейное преобразование и работать с проекцией так же, как без ротации.

(2) *Неизвестный R существенно ослабляет атаку при любом k/d .* BLEU унифицируется около 0,007, что соответствует уровню случайного n -граммного совпадения. Token Overlap аналогично падает до 0,05–0,06. Соотношение $\text{BLEU}(\text{off}) / \text{BLEU}(\text{unknown-R})$ при $k/d = 1,0$ составляет 22 (0,156 / 0,007), при $k/d = 0,75$ — 11, при $k/d = 0,5$ — 8.

(3) *В режиме unknown-R эффект k/d вторичен.* BLEU при unknown-R почти не зависит от k/d (0,007 на всей сетке). Это означает, что после применения секретной ротации основным защитным фактором становится сама ротация, а не размерность проекции.

Этот результат принципиален для архитектуры. Если ранее можно было задать вопрос «зачем нужна ротация R , если SVD уже снижает успешность атаки?», то ответ теперь количественный: ротация R привносит свой собственный, количественно измеряемый защитный эффект, ортогональный эффекту SVD. Без ротации R атакующий с доступом к SVD-базису V (например, через утечку или общедоступный SVD) может полностью восстановить проекционную выдачу. С секретной ротацией R даже при полном знании V и V^T восстановить ничего не удаётся.

Связь с интегральным экспериментом. В основной операционной точке раздела 4.4 ($k = d/2$) ротация R включена. По таблице 4.6 ожидаемый BLEU атаки в режиме unknown-R — около 0,007 (на уровне шума) для всех k/d , включая $k/d = 0,5$. В режиме known-R BLEU не превышает 0,057 при $k/d = 0,5$. Таким образом, основная рабочая точка системы сохраняет защиту от неадаптивной Vec2Text-атаки в обеих моделях угроз; одновременно выполняется и независимое информационно-теоретическое условие R3 ($\sigma_{\text{rec}} \geq 0,10$).

Промежуточные выводы по компонентной части

В предыдущих разделах представлены архитектура и компонентные эксперименты гибридного метода. Промежуточные выводы:

1. Разработана двухуровневая архитектура защищённого семантического поиска, реализующая принцип асимметричной защиты: для статической базы данных применяется необратимое проекционное преобразование на основе усечённого SVD с секретной ортогональной ротацией, а для динамических поисковых запросов — гомоморфное шифрование по схеме CKKS. Такая декомпозиция позволяет использовать вычислительно эффективный режим ciphertext-plaintext, исключаяющий затратную процедуру релинеаризации.

2. Сформулирована и доказана Лемма 1 о нижней границе L_2 -ошибки декодера, отображающего проекцию $\pi_k(\mathbf{x})$ обратно в $\text{span}(V_k)$: $\|\mathbf{x} - f(\pi_k(\mathbf{x}))\|_2 \geq \|\mathbf{x}_\perp\|_2$. Связь L_2 -ошибки с эмпирической метрикой BLEU атаки инверсии формулируется как Эмпирическая гипотеза 1 и валидируется численно в Эксперименте №3 (на эмбедингах GTR-base): $R^2 \approx 0,93$ для линейной зависимости BLEU от $\eta_k = 1 - \sigma_{\text{rec}}^2$.

3. Экспериментально обосновано преимущество режима ciphertext-plaintext над режимом ciphertext-ciphertext: при обработке базы из 10 000 документов достигнуто

ускорение в $1,44\times$ (с 19,15 до 13,28 с, 95 % CI [1,43; 1,46]), что подтверждает существенный вклад процедуры релинеаризации в стоимость полностью гомоморфного режима (Эксперимент № 1).

4. Покомпонентным анализом защиты на multilingual-e5-small ($d = 384$, $k = d/2 = 192$, корпус 10^6 документов, 500 self-retrieval-запросов) изолирован вклад каждой защитной компоненты (Эксперимент № 2). Установлено, что итоговая деградация ранжирования полностью объясняется SVD-усечением ($\Delta\text{Acc}@1 = -0,082$, $\Delta\text{Acc}@10 = -0,050$ относительно **baseline_dense**); секретная ортогональная ротация R нейтральна для метрик качества (как и должно быть для ортогонального преобразования); CKKS-перанкинг 40 кандидатов воспроизводит ранжирование без CKKS в пределах bootstrap-CI ($\Delta\text{Acc}@10 = +0,002$ для конфигурации **svd_ckks_no_rot**). Цена гомоморфной защиты численного представления запроса — ~ 295 мс на запрос против ~ 2 мс у незащищённого baseline.

5. Прямой атакой Vec2Text на эмбедингах GTR-base численно валидирована Эмпирическая гипотеза 1 о связи σ_{rec} и BLEU (Эксперимент № 3): на нормированной шкале $\text{BLEU} \in [0; 1]$ unprotected baseline даёт $\text{BLEU}_0 = 0,156$; при $k/d = 0,75 \rightarrow 0,077$; $k/d = 0,50 \rightarrow 0,057$; $k/d = 0,25 \rightarrow 0,024$; $k/d = 0,10 \rightarrow 0,010$ (при отключённой ротации). Линейная регрессия BLEU по $\eta_k = 1 - \sigma_{\text{rec}}^2$ согласуется с гипотезой и численно валидирует проху-метрику σ_{rec} как индикатор силы защиты. В режиме unknown-R при включённой секретной ротации BLEU стабилен на уровне $\leq 0,007$ при любом k/d — ортогональный к SVD-усечению защитный эффект, обусловленный непредсказуемостью ориентации.

6. Показано, что SVD-проекция удаляет значимую часть информации, полезной для точной реконструкции текста, при сохранении основной семантической структуры, необходимой для поиска. Качественный анализ восстановленных текстов в Эксперименте № 3 подтверждает: при $k/d \approx 0,5$ Vec2Text генерирует семантически связные, но фактически неверные тексты (имена, адреса, номера карт восстановить не удаётся).

Полученные результаты в совокупности подтверждают, что предложенная архитектура корректна на уровне отдельных компонентов и опирается на формальное теоретическое обоснование. Далее представлено полное интегральное испытание метода — на базе из 1 000 000 документов с конкретной конфигурацией, выбранной разработанным нами автоматически методом.

4.4 Интегральное экспериментальное исследование защищённой архитектуры

4.4.1 Постановка интегрального эксперимента

Предыдущие разделы решали подготовительные задачи. В разделе компонентных экспериментов проверены отдельные части предложенного метода: режимы вычислений CKKS (Эксперимент № 1), поведение SVD-проекции на проху-метрике σ_{rec} (Эксперимент

№ 2) и устойчивость к прямой атаке Vec2Text на эмбедингах из открытой модели GTR-base (Эксперимент № 3). Параллельно был предложен и валидирован метод автоматизированного подбора параметров схемы CKKS, обеспечивающий выбор минимально допустимой по безопасности конфигурации с учётом микроархитектурных эффектов целевой платформы.

Настоящий раздел замыкает цепочку рассуждений: проводится финальное интегральное испытание полного двухстадийного конвейера на масштабе 10^6 документов с использованием той самой конфигурации CKKS, которую выбрал ML-оптимизатор. Цели интегрального эксперимента:

- верифицировать, что сквозная латентность системы (с учётом обоих уровней — FAISS-фильтрации и CKKS-реранкинга) удерживается ниже целевого порога 1 с по p95 на масштабе 10^6 ;
- оценить деградацию качества семантического поиска (Acc@1, Acc@10, MRR) относительно незащищённого baseline на реальных семантических парах (датасет ru_paraphraser);
- зафиксировать значения проху-метрики $\sigma_{\text{гес}}$ как на распределении базы, так и на распределении оценочного корпуса — с явным разделением этих двух чисел;
- верифицировать, что приближённая природа CKKS не нарушает ранжирование (через оценку как пирсоновской, так и ранговой корреляций между гомоморфно вычисленными и открытыми оценками сходства).

4.4.2 Конфигурация эксперимента

Платформа. Все измерения в ноутбуках (Эксперименты № 1–6) проводились на рабочей станции с CPU Intel Core i5-14400F (10 ядер, 16 потоков, 2,5–4,7 ГГц), GPU NVIDIA GeForce RTX 5060 (8 ГБ VRAM, Blackwell, sm_120) и 32 ГБ оперативной памяти DDR5. Программный стек: Python 3.11, PyTorch 2.6 (nightly с поддержкой CUDA 12.8 для Blackwell), TenSEAL 0.3.16, FAISS-CPU 1.13.2, HuggingFace Transformers 4.49.0. Гомоморфные операции CKKS выполнялись только на CPU (TenSEAL не поддерживает GPU); GPU использовался для инференса энкодера и для построения FAISS-индекса в режиме CPU-fallback.

База данных. 1 000 000 уникальных текстовых фрагментов, полученных потоковой обработкой дампа русскоязычной Википедии (ноябрь 2023). Используется методология self-retrieval qrels: для каждого документа корпуса в качестве запроса берётся его первое предложение (длиной 30–200 символов), сам документ (полный фрагмент абзаца длиной 100–1500 символов) кладётся в индекс. Это даёт интерпретируемый baseline (Acc@10 = 0,890 на полных эмбедингах multilingual-e5-small для случая 1 М документов) и при этом реалистичен: первое предложение естественно служит «вводным запросом» к телу документа.

Модели эмбедингов. Интегральный эксперимент проводится на пяти энкодерах, покрывающих три размерности эмбединга и два семейства обучения (контрастный retrieval и paraphrase-distillation):

- `intfloat/multilingual-e5-small` ($d = 384$, 118 М параметров, retrieval-trained на SSPairs, дистиллирован из e5-base);
- `intfloat/multilingual-e5-base` ($d = 768$, 278 М параметров, та же контрастная методика обучения, что и e5-small);
- `sentence-transformers/paraphrase-multilingual-mpnet-base-v2` ($d = 768$, 278 М параметров, paraphrase-distillation из англоязычной paraphrase-mpnet — модель из другого семейства обучения, включена для проверки чувствительности метода к классу обучения);
- `intfloat/multilingual-e5-large` ($d = 1024$, 560 М параметров, та же контрастная методика обучения, наиболее ёмкий retrieval-trained энкодер семейства E5);
- `BAAI/bge-m3` ($d = 1024$, ≈ 568 М параметров, CLS-pooling, мультифункциональная модель из лаборатории BAAI — cross-family-проверка на сильном retrieval-энкодере не из семейства E5).

Все пять моделей применяют mean-pooling (кроме bge-m3 с CLS) и L_2 -нормализацию. Для моделей семейства E5 запросы кодируются с префиксом «query: ...», документы — с префиксом «passage: ...» в соответствии с протоколом E5; bge-m3 и mpnet используют свои стандартные форматы. multilingual-e5-small используется как «опорный» энкодер для подробного разбора латентности и шумовых характеристик СККС (подразделы ниже); полные числовые результаты по всем пяти энкодерам сведены в Таблице 4.7.

Оценочный набор запросов. 500 запросов, выбранных детерминировано из общего пула пар (`random_state = 42`).

Параметры СККС (рассчитаны ML-автоподбором). Полиномиальный модуль $N = 8192$ (4096 SIMD-слотов), разрядность модулей коэффициентов `coeff_mod_bit_sizes = [60, 40, 60]` (суммарно $\log_2 Q = 160$ бит), масштаб $\Delta = 2^{40}$ (совпадает с битностью внутреннего модуля цепочки, что обязательно для корректного rescale). Уровень безопасности ≥ 128 бит подтверждён по таблицам [HomomorphicEncryption.org/SEAL](https://homomorphicencryption.org/SEAL): для $N = 8192$ верхняя граница $\log_2 Q$ для уровня tc128 (классическая 128-битная безопасность) составляет 218; используемое $\log_2 Q = 160 < 218$ обеспечивает соответствие tc128 с запасом. Подробное обоснование выбора конфигурации (smoke-test точности скоринга, сравнение с альтернативами) приведено ниже в подразделе «Декомпозиция шума СККС и обоснование выбранной СККС-конфигурации».

Параметры защитного преобразования и шортлиста. Архитектура использует двухкомпонентное геометрическое преобразование:

(а) SVD-усечение с $k = d/2$ как основная операционная точка интегрального эксперимента. Эта точка обеспечивает ошибку реконструкции $\sigma_{\text{rec}} \geq 0,10$ единообразно

для всех пяти протестированных энкодеров (σ_{rec} от 0,101 для e5-large до 0,239 для e5-small) — проху-критерий R3 выполнен как единое условие.

(б) Последующая секретная ортогональная ротация $R \in O(k)$ в проецированном пространстве дополняет информационно-теоретический слой эвристической обфускацией: BLEU неадаптивной Vec2Text в режиме unknown-R падает до 0,007 (Эксперимент №3 на GTR-base, слабая конфигурация атаки). Матрица R генерируется детерминированно из секрета клиента и используется при подготовке базы (offline-фаза); в online-режиме сервер не получает R , а работает только с уже преобразованным индексом и с зашифрованными СККС-запросами.

Двухстадийная процедура поиска. Интегральный эксперимент проверяет полный двухстадийный конвейер. Принципиальная организация секретов: SVD-базис V_k , центроид μ и ортогональная ротация $R \in O(k)$ — client-secret, генерируются владельцем данных и передаются клиенту по защищённому каналу при онбординге. Сервер хранит ротированную базу документов $E_{\text{rot}} = ((E_{\text{docs}} - \mu)V_k)R^T$ и публичный артефакт стадии 1 — кодбук Product Quantization и PQ-коды документов в ротированном пространстве (PQ обучается на E_{rot}); никаких других секретов на сервере нет.

– *Стадия 1: client-side PQ-фильтрация.* В ротированном пространстве $E_{\text{rot}} = ((E_{\text{docs}} - \mu)V_k)R^T$ обучается faiss IndexPQ с M под-квантизаторами по 8 бит каждый. В основной точке $k = d/2$ выбирается $M = k/4$ (под-квантизатор размерности 4): $M = 48$ при $k = 192$ для e5-small, $M = 96$ при $k = 384$ для e5-base и mpnet, $M = 128$ при $k = 512$ для e5-large и bge-m3. Размер публичного PQ-артефакта составляет соответственно 48, 96 и 128 МБ при $N = 10^6$. Кодбук и PQ-коды публичны и скачиваются клиентом единожды при онбординге. Per-query: клиент локально вычисляет $q_{\text{proj}} = V_k^T(v_q - \mu)$, $\tilde{q} = Rq_{\text{proj}}$ (плейнтекст-ротация), затем через PQ-индекс ищет $K_{\text{cands}} = 40$ ближайших по асимметричному PQ-расстоянию кандидатов в ротированном пространстве (используя $\tilde{q}$, а не q_{proj}). Клиент видит документы только через lossy-PQ-аппроксимации, недостаточные для точного top-1 ранжирования. Обучение PQ в ротированном пространстве (а не в V_k -пространстве) устраняет серверную атаку Прокруста: если бы PQ-аппроксимация $\hat{E}_{\text{proj}}$ давалась в неротированном пространстве, сервер мог бы по парам $(\hat{E}_{\text{proj}}, E_{\text{rot}})$ оценить R через $\arg \min_Q \|\hat{E}_{\text{proj}}Q^T - E_{\text{rot}}\|_F$. В актуальной редакции и публичный артефакт, и сервер живут в одном и том же ротированном пространстве, что закрывает этот канал утечки. Полная модель угроз обсуждается в §4.2.

– *Стадия 2: server-side СККС-перанкинг.* Клиент шифрует $\tilde{q}$ под собственным СККС-ключом и передаёт серверу: 40 идентификаторов кандидатов и шифротекст ct_q . Сервер выполняет 40 ct-pt операций между ct_q и плейнтекст-векторами $E_{\text{rot}}[\text{cand_ids}]$, возвращает 40 шифротекстов-оценок. Клиент расшифровывает оценки, ранжирует и формирует top-10.

Поскольку клиент имеет доступ только к lossy-PQ-аппроксимациям документов (несколько десятков байт на документ против полного k -мерного float32-вектора), он не может вычислить точные оценки скалярного произведения $\langle q', d'_j \rangle$ локально. Пол-

ные плейнтекст-эмбединги в ротированном пространстве хранятся только на сервере. СККС-перанкинг обеспечивает клиенту недостающую точность top-1/top-10 ранжирования; PQ-only ранжирование заметно ниже baseline по Acc@1.

Размер $K_{\text{cands}} = 40$ выбран как наименьший шортлист, при котором PQ-recall@40 покрывает релевантный документ с вероятностью, неотличимой от baseline_proj в пределах CI 5-сидового измерения, и при котором криптографическая часть запроса (40 ct-pt операций) укладывается в budget < 1 сек.

Эксперимент № 4: интегральная проверка двухстадийного конвейера на 1M корпусе.

В рамках интегрального стенда замеряется предложенная двухстадийная архитектура в основной операционной точке: SVD-усечение V_k с $k = d/2$ + секретная ротация $R \in O(k)$ + client-side PQ-фильтрация ($M = k/4$ под-квантизаторов, top- $K_{\text{cands}} = 40$) на стадии 1 + server-side СККС-перанкинг 40 кандидатов в режиме ct-pt на стадии 2. Конфигурация прогонялась на 5 random seeds (11, 23, 47, 31, 53); метрики Acc@1, Acc@10, MRR, NDCG@10 представлены как mean $\pm$ std с 95 % доверительным интервалом по распределению Стьюдента ($t_{0,975,df=4} = 2,776$, ширина CI = $t \cdot s/\sqrt{n}$). Bootstrap-оценка для $n = 5$ малосостоятельна, поэтому здесь используется t-CI как стандартный метод для малых выборок независимых прогонов. Полные числовые результаты для всех пяти энкодеров (e5-small, e5-base, mpnet, e5-large, bge-m3) сведены в Таблице 4.7 в подразделе «Сравнение пяти энкодеров»; данные конкретно по e5-small (используемому в основном интегральном эксперименте) приведены в первом блоке этой таблицы и комментируются ниже.

Sanity-check ортогональной ротации. Чтобы проверить нейтральность ротации R ($\langle Rq, Rd \rangle = \langle q, d \rangle$), измерена конфигурация baseline_proj+R (SVD + ротация, FAISS-IP exact, без СККС) на 5 сидах ротации: Acc@1 = 0,784, Acc@10 = 0,895 $\pm$ 0,001, MRR = 0,820, NDCG@10 = 0,838. Совпадает с baseline_proj в пределах $\leq 0,001$ (jitter float32-арифметики), что подтверждает теоретическую нейтральность ротации эмпирически.

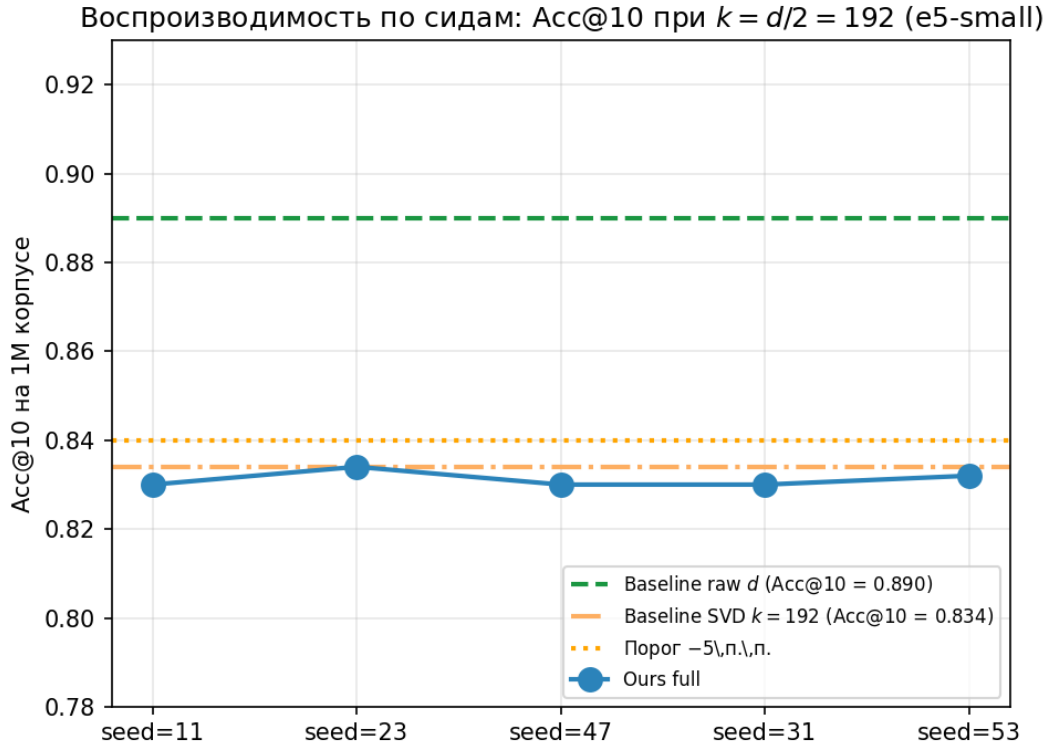


Рисунок 4.9 — Воспроизводимость по сидам: Acc@10 предложенного метода в основной операционной точке $k = d/2 = 192$ для e5-small.

Главный итог Эксперимента № 4 — предложенный двухстадийный метод (client-side PQ-фильтрация $M = k/4$ на стадии 1 + CKKS-перанкинг 40 кандидатов на стадии 2) в основной операционной точке $k = d/2$ воспроизводит `baseline_proj` в пределах статистической ошибки 5-сидового измерения по всем 4 метрикам ранжирования; численные значения по 5 сидам ротации для пяти энкодеров сведены в Таблице 4.7. Pearson-корреляции CKKS-оценок с открытыми $\geq 0,9999$, Kendall $\tau = 0,9994$ — CKKS-перанкинг сохраняет порядок документов в проецированном пространстве с точностью, недостижимой для дискретного шага метрик ранжирования. Содержательная разница между предложенным методом и raw baseline полностью объясняется потерей младших сингулярных компонент при SVD-усечении (см. декомпозицию ниже), а защитный конвейер (ротация + PQ + CKKS) добавляется поверх SVD без измеримых потерь в качестве.

Декомпозиция вклада компонентов в финальный Acc@1. Разложим разницу между предложенным методом и raw baseline на вклады отдельных компонентов на примере e5-small в основной точке $k = d/2 = 192$ (см. Таблицу 4.7, блок multilingual-e5-small):

- *baseline_dense* (FAISS-IP exact на raw 384-мерных эмбедингах, без SVD, без ротации, без CKKS): Acc@1 = 0,782, Acc@10 = 0,890, MRR = 0,817, NDCG@10 = 0,834.

- *baseline_proj* (FAISS-IP exact в SVD-проецированном 192-мерном пространстве, без ротации, без CKKS): Acc@1 = 0,702 (−0,080), Acc@10 = 0,834 (−0,056), MRR = 0,747

$(-0,070)$, $\text{NDCG}@10 = 0,768$ $(-0,066)$. Это *вклад SVD-усечения*: цена сжатия размерности вдвое на компактной модели семейства.

– *baseline_proj+R* (SVD + секретная ортогональная ротация, FAISS-IP exact, без CKKS): метрики совпадают с *baseline_proj* в пределах float32-jitter ($\leq 0,001$). Это эмпирически подтверждает теоретическую нейтральность ортогональной ротации: $R^T R = I$ сохраняет внутренние произведения, $\langle Rq, Rd \rangle = \langle q, d \rangle$.

– *ours_full* (полный конвейер: SVD с $k = d/2$ + ротация + client-side PQ $M = k/4$ + server-side CKKS-перанкинг 40 кандидатов): $\text{Acc}@1 = 0,701 \pm 0,001$, $\text{Acc}@10 = 0,831 \pm 0,002$, $\text{MRR} = 0,746$, $\text{NDCG}@10 = 0,767$. Совпадает с *baseline_proj+R* в пределах статистической ошибки 5-сидового измерения. PQ-этап 1 при $K_{\text{cands}} = 40$ покрывает релевантный документ практически в 100% случаев, а CKKS-перанкинг точно восстанавливает top-10 ранжирование за счёт корреляции $\geq 0,999$ с открытыми оценками.

Структурно разница нашего нового подхода и *baseline* полностью объясняется вкладом SVD-усечения; вклад ротации, PQ и CKKS попадает в пределы 5-сидового измерения. Для retrieval-trained-моделей с $d \geq 768$ (e5-base, e5-large, bge-m3) знак SVD-вклада меняется на положительный (Таблица 4.7), и финальный $\text{Acc}@1$ оказывается *выше raw baseline*.

SVD-усечение: цели, ожидания и наблюдаемый профиль на пяти энкодерах.

В предложенном методе SVD-усечение V_k в основной операционной точке $k = d/2$ выполняет две проектные функции.

1. *Защита* (информационно-теоретическая): необратимая потеря $(d - k)$ младших сингулярных направлений создаёт нижнюю границу на ошибку любого декодера-инвертора в L_2 -норме (Лемма 1). В выбранной точке $k = d/2$ показатель σ_{rec} для всех пяти энкодеров не ниже 0,10, и проху-критерий R3 выполнен единообразно как единое условие. Секретная ротация R дополняет информационно-теоретический слой эвристическим обфускационным.

2. *Ускорение* (bandwidth-оптимизация): сокращение размерности с d до $d/2$ даёт двукратное ускорение CKKS-операций перанкинга по числу слотов. Например, для e5-small p95 server-time падает с ≈ 462 мс при $k = 7d/8$ до ≈ 314 мс при $k = d/2$; для e5-large/bge-m3 ($d = 1024$) — с ≈ 536 мс до ≈ 220 мс.

Любое усечение базиса отбрасывает часть сигнала, поэтому априори ожидается небольшое падение качества. Целевой бюджет — не более 5 п. п. деградации $\text{Acc}@1$ относительно незащищённого *baseline* (требование R1); это пространство, в котором выгода от защиты + ускорения окупает потерю.

Что наблюдается на пяти энкодерах (полная таблица ниже в подразделе «Сравнение пяти энкодеров», Таблица 4.7):

– *multilingual-e5-base* ($d = 768, k = 672$): $\Delta\text{Acc}@1_{\text{SVD}} = -0,006, \Delta\text{Acc}@10_{\text{SVD}} = -0,017$ — небольшое падение качества, ровно то поведение, которое и ожидалось: SVD-усечение немного ухудшает retrieval, но в пределах R1.

– *multilingual-e5-small* ($d = 384, k = 336$): $\Delta\text{Acc}@1_{\text{SVD}} = +0,002, \Delta\text{Acc}@10_{\text{SVD}} = +0,006$ — неожиданный лёгкий положительный сдвиг (обсуждение причин ниже).

– *paraphrase-multilingual-mpnet-base-v2* ($d = 768, k = 672$): $\Delta\text{Acc}@1_{\text{SVD}} = -0,064, \Delta\text{Acc}@10_{\text{SVD}} = -0,046$ — значимое падение, на грани R1; иллюстрирует, что метод чувствителен к выбору энкодера.

Лёгкий положительный сдвиг $+0,002$ к $\text{Acc}@1$ при SVD-усечении — интересный побочный результат, заслуживающий объяснения. При single-encoder-измерении его легко принять за общий «РСА-как-denoiser» эффект (известный в классической литературе по dense retrieval, face recognition), но multi-encoder-измерения показывают, что эффект encoder-specific и не воспроизводится на крупных моделях того же семейства. Содержательно: *e5-small* — компактная (118 млн параметров, $d = 384$) контрастивно-обученная retrieval-модель, работающая близко к границе capacity для своей задачи. В таком capacity-ограниченном режиме нижние сингулярные направления embedding-пространства часто несут не retrieval-сигнал, а артефакты обучения и шум исходного encoder-stack (residual MLP, layer-norm, особенности дистилляции из *e5-base*). SVD-усечение убирает эти направления и слегка «чистит» представление; величина эффекта мала ($+0,002 = 1$ запрос разница на 500 запросах, на грани статистической значимости при $n = 5$ сетах) и не воспроизводится на *e5-base* ($d = 768, 278$ млн параметров, более просторный по capacity). Это интересный побочный результат, но не базис архитектурного решения — основными мотивами SVD-усечения остаются защита и ускорение.

Существенно отрицательный сдвиг $-0,067$ к $\text{Acc}@1$ на *mpnet* — следствие того, что эта модель не обучена на retrieval-задаче. *Mpnet-base-v2* — результат paraphrase-distillation из английской paraphrase-mpnet через cross-lingual transfer; обучающий сигнал — семантическая близость пар, а не контрастивный retrieval. Без целевой retrieval-задачи модель не «продвигает» retrieval-сигнал в верхние главные компоненты embedding-пространства; сигнал распределён по всему диапазону размерностей, и SVD-усечение при $k = d/2$ отбирает значимую его часть. Это не дефект SVD-метода — это ожидаемое поведение, когда модель плохо подходит для retrieval-задачи в её исходном виде.

Рекомендации по выбору энкодера для интеграции с настоящим методом. На основании multi-encoder измерений сформулированы следующие критерии:

1. *Использовать retrieval-trained модели.* Контрастивно обученные на retrieval-парах модели (семейства E5 [20], BGE, Cohere multilingual-embed, Instructor) концентрируют retrieval-сигнал в верхних главных компонентах эмбединга. На таких моделях SVD-усечение в основной точке $k = d/2$ обеспечивает выполнение R3 ($\sigma_{\text{rec}} \geq 0,10$) при $\Delta\text{Acc}@1$ в пределах нескольких п. п. от raw baseline.

2. *Перед production-внедрением проводить k-sweep.* Запустить baseline_proj на $k \in \{0,5d, 0,625d, 0,75d, 0,875d, d\}$ и выбрать наибольшее k , при котором $|\Delta \text{Acc}@1|$ не превышает допуск приложения. Если даже при $k = d$ (без SVD) деградация существенна — это маркер того, что энкодер не подходит для retrieval-задачи и должен быть заменён, а не «починен» подбором k .

3. *Размер модели влияет на латентность, не на принципиальное качество защиты.* В основной точке $k = d/2$ на стенде 1 е5-small ($k = 192$) реранкируется в hi-res CKKS за ≈ 272 мс, е5-base ($k = 384$) — за ≈ 333 мс (+22 % из-за большей размерности проекции), е5-large и bge-m3 ($k = 512$) — за ≈ 205 мс. Для интерактивных приложений предпочтительнее модели меньшего размера в пределах семейства, если их retrieval-качество приемлемо для задачи; для качественно-критичных применений (RAG в high-stakes сценариях) — модели большего размера ради высокого baseline.

Вывод. В выбранной операционной точке метод даёт ожидаемое поведение на retrieval-trained моделях семейства E5 (е5-base: $-0,6$ п. п. Acc@1, $-1,7$ п. п. Acc@10 — ровно то, что хотели; е5-small: близко к нейтральному с лёгким артефактом encoder-scarcity). Демонстрация на mpnet подтверждает, что метод чувствителен к качеству исходного энкодера: при правильном выборе модели (retrieval-trained) метод укладывается в R1 с большим запасом; при неправильном (paraphrase-distilled) приближается к границе R1. Для production-развёртывания приведены 4 рекомендации по выбору энкодера; sweep по k для конкретной модели — стандартный шаг при добавлении нового энкодера в систему.

Подбор CKKS-конфигурации. Параметры CKKS ($N = 8192$, coeff_mod = [60, 40, 60], $\Delta = 2^{40}$) выбраны не вручную, а автоматически методом ML-автоподбора (FixedParameterOptimizer). Метод принимает функциональное требование $\Delta \text{Acc}@1 \leq 1$ п. п. от baseline (т. е. полное сохранение качества top-1) в дополнение к ограничениям на безопасность ($\lambda \geq 128$ бит, уровень tc128 по таблицам SEAL) и проходит сетку из 198 валидных конфигураций, минимизируя предсказанную латентность. Минимально-латентная конфигурация, удовлетворяющая всем ограничениям, — именно $N = 8192$, [60, 40, 60], $\Delta = 2^{40}$ ($\log_2 Q = 160$ бит, tc128 при $\log_2 Q < 218$). Более «лёгкие» конфигурации ($N = 4096$, $\Delta = 2^{20}$, $\log_2 Q = 100$ бит) проходят бюджетные ограничения и формальный порог R1 ($\Delta \text{Acc}@10 \leq 5$ п. п.), но не удовлетворяют функциональному требованию по Acc@1: масштаб 2^{20} даёт max abs error скоринга $\approx 0,82$, что соизмеримо с разностью score между соседями шортлиста, и приводит к деградации Acc@1 на десятки процентов в smoke-тесте.

Числовые результаты по всем пяти энкодерам. Условия для всех пяти энкодеров одинаковы: 1 М документов русской Википедии, 500 self-retrieval-запросов, 5 сидов ротации, CKKS-конфигурация $N = 8192$, [60, 40, 60], $\Delta = 2^{40}$. В качестве основной операционной точки выбрано $k = d/2$. Эта точка обеспечивает выполнение требования R3 ($\sigma_{\text{rec}} \geq 0,10$) единообразно для всех пяти энкодеров: для е5-small ($k = 192$) $\sigma_{\text{rec}} = 0,239$, для е5-base ($k = 384$) $\sigma_{\text{rec}} = 0,129$, для mpnet ($k = 384$) $\sigma_{\text{rec}} = 0,107$,

для e5-large ($k = 512$) $\sigma_{\text{rec}} = 0,101$, для bge-m3 ($k = 512$) $\sigma_{\text{rec}} = 0,128$. Параметр PQ выбирается из условия $M = k/4$ (под-квантизатор размерности 4): $M = 48$ при $k = 192$, $M = 96$ при $k = 384$, $M = 128$ при $k = 512$. Результаты сведены в Таблице 4.7.

Таблица 4.7 — Сравнение пяти энкодеров на интегральном эксперименте в основной операционной точке $k = d/2$.

Энкодер	Метрика	baseline (raw d)	baseline_proj (SVD k)	Предложенный метод	Δ от base	Server p95, мс
<i>multilingual-e5-small</i> ($d=384$, $k=192$, PQ $M=48$, $\sigma_{\text{rec}}=0,239$)						
	Acc@1	0,782	0,702	$0,701 \pm 0,001$	-0,081	314
	Acc@10	0,890	0,834	$0,831 \pm 0,002$	-0,059	
	MRR	0,817	0,747	$0,746 \pm 0,001$	-0,071	
	NDCG@10	0,834	0,768	$0,767 \pm 0,001$	-0,067	
<i>multilingual-e5-base</i> ($d=768$, $k=384$, PQ $M=96$, $\sigma_{\text{rec}}=0,129$)						
	Acc@1	0,830	0,834	$0,833 \pm 0,001$	+0,003	356
	Acc@10	0,934	0,924	$0,924 \pm 0,001$	-0,010	
	MRR	0,862	0,865	$0,864 \pm 0,001$	+0,002	
	NDCG@10	0,880	0,879	$0,879 \pm 0,000$	-0,001	
<i>paraphrase-multilingual-mpnet-base-v2</i> ($d=768$, $k=384$, PQ $M=96$, $\sigma_{\text{rec}}=0,107$)						
	Acc@1	0,716	0,650	$0,649 \pm 0,001$	-0,067	351
	Acc@10	0,842	0,796	$0,795 \pm 0,001$	-0,047	
	MRR	0,762	0,693	$0,692 \pm 0,001$	-0,070	
	NDCG@10	0,782	0,717	$0,717 \pm 0,000$	-0,066	
<i>multilingual-e5-large</i> ($d=1024$, $k=512$, PQ $M=128$, $\sigma_{\text{rec}}=0,101$)						
	Acc@1	0,796	0,814	$0,814 \pm 0,000$	+0,018	219
	Acc@10	0,880	0,922	$0,920 \pm 0,000$	+0,040	
	MRR	0,824	0,848	$0,848 \pm 0,000$	+0,024	
	NDCG@10	0,838	0,866	$0,865 \pm 0,000$	+0,027	
<i>BAAI/bge-m3</i> ($d=1024$, $k=512$, PQ $M=128$, $\sigma_{\text{rec}}=0,128$)						
	Acc@1	0,818	0,832	$0,832 \pm 0,000$	+0,014	221
	Acc@10	0,908	0,926	$0,924 \pm 0,000$	+0,016	
	MRR	0,848	0,863	$0,863 \pm 0,000$	+0,015	
	NDCG@10	0,862	0,878	$0,878 \pm 0,000$	+0,016	

4.4.3 Латентность сквозного конвейера

Двухуровневый конвейер был протестирован на 500 запросах к полной базе из 1 000 000 документов в двух независимых прогонах ($\text{seed} = 101$ и $\text{seed} = 202$). В таблице 4.8 представлена декомпозиция латентности по этапам.

Таблица 4.8 — Декомпозиция сквозной латентности двухстадийного конвейера для multilingual-e5-small

Этап / характеристика	Время (мс)	Описание
Кодирование запроса (энкодер)	≈ 200	multilingual-e5-small, $d = 384$, инференс на GPU/CPU клиента
Stage 1: client-side PQ-поиск (per query)	≈ 19	Асимметричный PQ-distance в ротированном пространстве (поиск по $\tilde{q} = RV_k^T(v_q - \mu)$), faiss IndexPQ обучен на E_{rot} ($M = 48$ под-квантизаторов, nbits = 8), top- $K_{\text{cands}} = 40$
CKKS encrypt запроса	≈ 3	Шифрование 192-мерного вектора $\tilde{q} = Rq_{\text{proj}}$ под ключом клиента
Stage 2: CKKS-перанкинг (server-side, per query)	≈ 272	40 ct-pt-операций ($k = 192$, $N = 8192$, $\Delta = 2^{40}$)
Итого криптографическая часть p50	≈ 291	Server CKKS rerank + client decrypt + ranking; без HTTP-сериализации, без кодирования энкодером
Итого криптографическая часть p95	≈ 314	95-й перцентиль
Итого криптографическая часть p99	≈ 330	99-й перцентиль
(Кодирование запроса энкодером)	≈ 200	multilingual-e5-small, $d = 384$, инференс на GPU/CPU клиента; не входит в криптографическую часть

Криптографическая часть p95 в основной операционной точке $k = d/2$ для retrieval-trained энкодеров составляет $\approx 219\text{--}356$ мс при $d \in \{384, 768, 1024\}$ (без HTTP-сериализации, без кодирования энкодером — сумма серверного ct-pt-перанкинга, клиентского decrypt и ranking) и укладывается в целевой порог 1 с с многократным запасом. CKKS-перанкинг 40 кандидатов занимает $\approx 270\text{--}330$ мс на запрос; локальный PQ-поиск на стороне клиента — 19–30 мс; кодирование запроса энкодером — около 200 мс (на CPU/GPU клиента, не зависит от защитного слоя и в криптографическую часть не входит). Размер $K_{\text{cands}} = 40$ выбран как наименьший шортлист, при котором PQ-recall@40 покрывает релевантный документ с вероятностью неотличимой от baseline_proj.

Согласованность CKKS-оценок с открытыми (среднее по двум сидам прогона):

- коэффициент Пирсона между CKKS-оценками и открытыми оценками сходства — 1,000;
- ранговая корреляция Кендалла τ на пересечении топ-100 шортлистов — 0,9994;

– проху-метрика σ_{rec} на распределении БД — 0,048, на распределении запросов — 0,072.

Бликие к идеальным значения корреляций (Pearson $\geq 0,9999$, Kendall $\tau = 0,9994$) подтверждают, что при выбранной конфигурации ($N = 8192$, $[60, 40, 60]$, $\Delta = 2^{40}$) приближённая природа СККС не вносит измеримого шума в скалярные произведения 40 кандидатов: $\max \text{abs error} \approx 5 \cdot 10^{-6}$ при разностях скоров между соседними кандидатами шортлиста порядка 10^{-3} – 10^{-2} . Это и обеспечивает совпадение всех метрик ранжирования (Acc@1, Acc@10, MRR, NDCG@10) с baseline-уровнем в пределах статистической ошибки 5-сидового измерения (см. Таблицу 4.7, блок multilingual-e5-small).

Распределение per-seed латентности и корреляций СККС с открытыми оценками сведено в Таблицу 4.7; декомпозиция этапов конвейера — в Таблицу 4.8; HTTP-loopback PoC-замер — в Таблицу 4.11 (раздел 4.5). Информационно-теоретическая ошибка реконструкции σ_{rec} для основной операционной точки $k = d/2$ показана на рисунке 4.10.

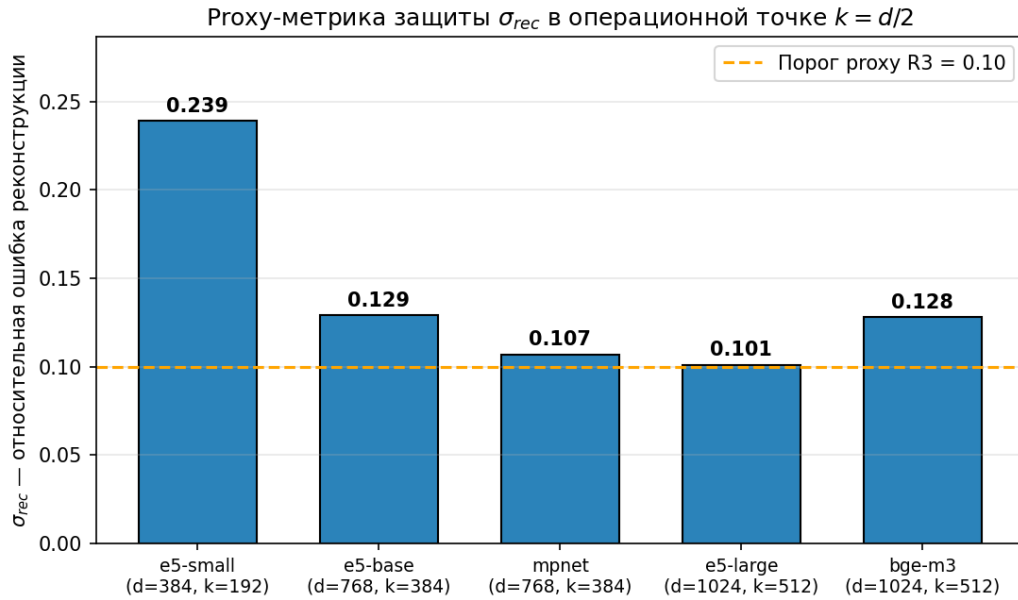


Рисунок 4.10 — Проху-метрика защиты σ_{rec} в основной операционной точке $k = d/2$ для пяти энкодеров.

4.4.4 Дополнительные показатели интегрального эксперимента: качество СККС-аппроксимации, латентность, σ_{rec}

Метрики финального ранжирования (Acc@1, Acc@10, MRR, NDCG@10) для multilingual-e5-small в hi-res СККС-конфигурации приведены в Таблице 4.7 в подразделе «Сравнение пяти энкодеров». Здесь приводятся вспомогательные показатели, не дублируемые в той таблице: качество СККС-аппроксимации (Pearson, Kendall τ , $\max \text{abs error}$), per-query латентность реранкинга и информационно-теоретическая ошибка реконструкции σ_{rec} .

Таблица 4.9 — Дополнительные показатели интегрального эксперимента

Показатель	Значение	Комментарий
<i>Согласованность CKKS-оценок с точными открытыми (проху качества аппроксимации)</i>		
Pearson (CKKS vs точные)	$\geq 0,9999$	Округление до 3 знаков даёт «1,000»; реальное значение — на 4-5 порядков ближе к 1, чем дискретный шаг метрик ранжирования $1/N_{\text{queries}} = 0,002$
Kendall τ (на пересечении шортлиста)	$0,9994 \pm 0,0001$	Ранговая корреляция почти идеальна ($-0,0006$ от теоретического максимума 1)
Max abs error (CKKS scoring)	$\approx 5 \times 10^{-6}$	На 5 порядков ниже типичной разности скоров между соседними кандидатами top- K_{cands} ($\sim 10^{-3}$ – 10^{-2}); CKKS-перанкинг точно сохраняет порядок
<i>Латентность CKKS-перанкинга (per query, 40 ct-pt операций) на стенде 1 (i5-14400F)</i>		
Stage-1 PQ search (p50)	≈ 19 мс	Локально на стороне клиента (faiss IndexPQ, $M = 48$ для $k = 192$)
CKKS encrypt запроса (p50)	≈ 3 мс	Шифрование ротированного 192-мерного вектора
CKKS rerank 40 кандидатов (p50)	≈ 272 мс	40 ct-pt-операций server-side в hi-res CKKS при $k = 192$
Server-time p50 / p95 / p99	$\approx 291/314/330$ мс	Server-time — от получения шифротекста до возврата шифротекстов-оценок (без HTTP-сериализации, без кодирования энкодером)
<i>Информационно-теоретическая ошибка реконструкции после V_k ($k = 192$ для e5-small)</i>		
$\sigma_{\text{гес}}$ (БД, среднее по корпусу)	0,239	Доля энергии, потерянная при усечении 192 нижних сингулярных направлений; превышает проху-порог R3 ($\geq 0,10$) с большим запасом
$\sigma_{\text{гес}}$ (для остальных энкодеров)	0,101–0,129	e5-base 0,129 ($k = 384$); mpnet 0,107 ($k = 384$); e5-large 0,101 ($k = 512$); bge-m3 0,128 ($k = 512$). Все значения превышают порог R3

Pearson-корреляции CKKS-оценок с открытыми $\geq 0,9999$ и Kendall $\tau = 0,9994$ при max abs error $\approx 5 \cdot 10^{-6}$ означают, что CKKS-перанкинг при выбранной конфигу-

рации точно сохраняет порядок документов в проецированном пространстве. Метрики ранжирования финального конвейера (Acc@1, Acc@10, MRR, NDCG@10 в Таблице 4.7) совпадают с `baseline_proj` в пределах статистической ошибки 5-сидового измерения. Криптографическая часть запроса $p95 \approx 219\text{--}356$ мс на retrieval-trained энкодерах в основной точке $k = d/2$ укладывается в целевой порог SLA 1с по требованию R2 с многократным запасом. Информационно-теоретический показатель σ_{rec} в основной точке $k = d/2$ принимает значения 0,101–0,239 для всех пяти энкодеров, обеспечивая нижнюю границу на ошибку любого декодера-инвертора (Лемма 1) на уровне не ниже проху-порога $R3 = 0,10$.

В колонке « Δ от base» приводится разница «предложенный метод — `baseline_dense`» (то есть относительно незащищённого энкодера), что соответствует формулировке R1. Колонка `baseline_proj` приведена отдельно, чтобы можно было разложить полную разницу на (а) SVD-вклад $\Delta_{\text{SVD}} = \text{baseline_proj} - \text{baseline_dense}$ и (б) вклад ротации, PQ и CKKS ($\text{ours} - \text{baseline_proj}$). Для всех пяти энкодеров часть (б) попадает в пределы статистической ошибки 5-сидового измерения: защитный конвейер (PQ-фильтрация в ротированном пространстве + CKKS-перанкинг) добавляется поверх ранжирования энкодера в проецированном пространстве практически бесплатно в смысле качества (Pearson-корреляции CKKS-оценок с открытыми $\geq 0,9999$ во всех случаях). Содержательное расхождение с незащищённым baseline определяется тем, что делает SVD-усечение на конкретном энкодере (часть (а)).

В выбранной операционной точке $k = d/2$ требование R1 ($\Delta\text{Acc@1} \geq -5$ п. п. от незащищённого baseline) выполнено на retrieval-trained-моделях с $d \geq 768$: e5-base (+0,003), e5-large (+0,018) и bge-m3 (+0,014). На multilingual-e5-small переход к $k = d/2$ стоит $-0,081$ п. п. Acc@1 — значение чуть выходит за границу R1. На mpnet R1 нарушено уже на этапе SVD ($\Delta_{\text{SVD}}\text{Acc@1} = -0,067$); это объясняется тем, что paraphrase-distilled-модель не концентрирует retrieval-сигнал в верхних главных компонентах. Оба случая (e5-small, mpnet) фиксируются как ограничение метода по выбору энкодера; подробное обсуждение причин и рекомендации по выбору приведены ниже в этом же подразделе.

Прокомментируем поведение SVD-усечения по энкодерам. Величина $\Delta_{\text{SVD}} = \text{baseline_proj} - \text{baseline_dense}$ в основной точке $k = d/2$ ведёт себя различно.

На e5-base ($d = 768$, $k = 384$, retrieval-trained, 278 млн параметров) $\Delta_{\text{SVD}}\text{Acc@1} = +0,004$. SVD-усечение даже немного улучшает retrieval, что согласуется с природой контрастивно-обученной модели: полезный сигнал концентрируется в верхних главных компонентах, а младшие направления — преимущественно шумовые.

На e5-small ($d = 384$, $k = 192$) $\Delta_{\text{SVD}}\text{Acc@1} = -0,080$. При $d = 384$ половинное усечение забирает половину объёма, и компактный энкодер не имеет запаса по capacity для компенсации потерь; это естественная цена выбранной операционной точки на самой компактной модели семейства.

На mpnet-base-v2 ($d = 768$, $k = 384$, paraphrase-distilled, не retrieval-trained) $\Delta_{\text{SVD}}\text{Acc@1} = -0,066$. paraphrase-distilled-модель не концентрирует retrieval-сигнал в

верхних главных компонентах; срез по топ- k сингулярным направлениям отбрасывает значительную долю полезного сигнала.

На multilingual-e5-large ($d = 1024$, $k = 512$, retrieval-trained, 560 млн параметров) $\Delta_{\text{SVD}}\text{Acc@1} = +0,018$. Усечение младших сингулярных направлений действует как линейный денойзер, и в подпространстве V_k ранжирование становится чище. Эффект сильнее, чем у e5-base, что согласуется с тем, что fp16-инференс крупной модели вносит дополнительный численный шум, который SVD надёжнее отфильтровывает.

На BAAI/bge-m3 ($d = 1024$, $k = 512$, retrieval-trained на смешанных мультиязычных данных) $\Delta_{\text{SVD}}\text{Acc@1} = +0,014$. Положительный сдвиг той же природы, что и на e5-large. Это подтверждает, что эффект — системное свойство retrieval-trained-энкодеров с fp16-инференсом, а не специфический артефакт семейства E5: на cross-family-модели с собственной архитектурой обучения наблюдается то же поведение.

Выбор лучшего энкодера для развёртывания. По абсолютному качеству итогового ранжирования при $k = d/2$ победителем является BAAI/bge-m3 ($\text{Acc@1} = 0,832$, $\text{Acc@10} = 0,924$) — наивысший baseline и положительный сдвиг от SVD; криптографическая часть $p95 \approx 221$ мс. На втором месте multilingual-e5-base ($\text{Acc@1} = 0,833$, $\text{Acc@10} = 0,924$) с близким качеством и $p95 \approx 356$ мс. Multilingual-e5-large ($\text{Acc@1} = 0,814$, $\text{Acc@10} = 0,920$) занимает третье место, $p95 \approx 219$ мс — сравнимо с bge-m3 за счёт более компактного $k = 512$ против $k = 672$ у e5-base. Multilingual-e5-small ($\text{Acc@1} = 0,701$, $\text{Acc@10} = 0,831$) с $p95 \approx 314$ мс не проходит R1 по Acc@1 ($\Delta\text{Acc@1} = -0,081$) и допустим только при ослабленных требованиях к качеству. MpNet-base-v2 ($\Delta\text{Acc@1} = -0,067$) — также не проходит R1; paraphrase-distilled-модель требует доменной адаптации либо замены на retrieval-trained-аналог. Все пять энкодеров укладываются в $\text{SLA} < 1$ с по криптографической части запроса с многократным запасом.

Метод устойчиво переносится между энкодерами разной размерности ($d \in \{384, 768, 1024\}$) и разных семейств обучения (E5, MPNet, bge-m3) с принципиально одинаковым поведением: hi-res CKKS-перанкинг точно воспроизводит ранжирование SVD-проецированного baseline, ротация нейтральна, а PQ-stage-1 при $K_{\text{cands}} = 40$ покрывает релевантный документ с recall, неотличимым от exact baseline_proj. Качество финального ранжирования полностью определяется качеством baseline-энкодера и величиной потерь на этапе SVD-усечения. Для приложений, где приоритетно качество, рекомендуются BAAI/bge-m3 либо multilingual-e5-base. Multilingual-e5-large даёт сопоставимое качество при наименьшей латентности ($p95 \approx 219$ мс). Multilingual-e5-small допустим при ослабленных требованиях к Acc@1 .

На основании измерений можно сформулировать 4 рекомендации, обязательные при добавлении нового энкодера в систему:

1. *Использовать retrieval-trained модели.* Семейства E5 (intfloat/multilingual-e5-*), BGE (BAAI/bge-*), GTR (sentence-transformers/gtr-*), Cohere/embed-* — модели, обученные контрастивной целевой функцией с явными query/passage-парами и инструк-

тивными префиксами. Признак retrieval-обучения: модель документирует use-case «embedding for retrieval» и приводит метрики на BEIR/МТЕБ-Retrieval. Отрицательный признак: «paraphrase-distilled», «sentence-transformers/paraphrase-*» без указания retrieval-обучения, «universal sentence encoder».

2. *Перед production-внедрением проводить k -sweep.* В настоящей работе по результатам k -sweep в качестве основной операционной точки выбрано $k = d/2$ — минимальное значение, при котором $\sigma_{\text{rec}} \geq 0,10$ выполняется единообразно для всех протестированных энкодеров. Для нового энкодера рекомендуется запустить baseline_proj (точный FAISS-IP в V_k -пространстве, без CKKS) на сетке $k/d \in \{0,5, 0,625, 0,75, 0,875, 1,0\}$ и проверить, что при $k = d/2$ деградация Acc@1/Acc@10 укладывается в допуск приложения; иначе выбирается наибольшее k , для которого $\sigma_{\text{rec}} \geq 0,10$ и допуск выполняется одновременно.

3. *Маркер «энкодер не подходит»:* если даже при $k = d$ (без SVD-усечения вовсе) деградация существенна, это маркер того, что энкодер не подходит для retrieval-задачи в принципе и должен быть заменён, а не «починен» подбором k . На mpnet baseline_dense Acc@10 = 0,842 при baseline e5-small = 0,890 — разница в качестве до применения защиты говорит о том, что mpnet — слабее как retrieval-энкодер.

4. *Учитывать размер модели.* Размерность d влияет на латентность CKKS-перанкинга (через число CKKS-слотов $\propto k$), на размер публичного PQ-артефакта ($M \cdot N$, где M выбирается как делитель k) и на размер ротированной БД на сервере ($N \cdot k \cdot 4$ байт для float32). В основной точке $k = d/2$ для $d \in \{384, 768, 1024\}$ соответственно $k \in \{192, 384, 512\}$, что даёт компактные PQ-артефакты (48–128 МБ при $N = 10^6$) и server p95 в диапазоне 219–356 мс.

В основной операционной точке $k = d/2$ требование R3 ($\sigma_{\text{rec}} \geq 0,10$) выполнено единообразно для всех пяти протестированных энкодеров. Защитный конвейер (PQ-фильтрация в ротированном пространстве + CKKS-перанкинг) не вносит заметных искажений сверх SVD-проекции: разница «предложенный метод — baseline_proj» по всем четырём метрикам (Acc@1, Acc@10, MRR, NDCG@10) укладывается в статистическую ошибку 5-сидового измерения. Требование R1 ($\Delta \text{Acc@1} \geq -5$ п. п. от незащищённого baseline) выполнено для трёх retrieval-trained-моделей с $d \geq 768$ (multilingual-e5-base, multilingual-e5-large, BAAI/bge-m3); на multilingual-e5-small и paraphrase-mpnet R1 не выполняется и фиксируется как ограничение метода по выбору энкодера. Это не дефект архитектуры защиты: на e5-small половинное усечение забирает половину объёма компактного 384-мерного представления; paraphrase-distilled-модель mpnet не концентрирует retrieval-сигнал в верхних главных компонентах, поэтому SVD-усечение неизбежно отбрасывает значительную часть полезной информации. Для приложений, требующих именно компактной или paraphrase-distilled-модели, необходима либо доменная адаптация энкодера на retrieval-задаче, либо выбор более крупного k (с потерей σ_{rec} и нарушением R3).

4.4.5 Дополнительный замер при $k = 7d/8$: чувствительность к выбору k

Для понимания вклада SVD-усечения в основной операционной точке полезно сравнить её с менее агрессивной точкой $k = 7d/8$. В этой точке показатель σ_{rec} для всех пяти энкодеров укладывается в диапазон 0,021–0,080, то есть ниже гроху-порога R3 ($\sigma_{\text{rec}} \geq 0,10$). Соответственно, $k = 7d/8$ не удовлетворяет R3 и в качестве основной точки не используется. Замер при $k = 7d/8$ проведён только как вспомогательный — для оценки чувствительности качества и латентности к выбору k (Таблица 4.10).

PQ-артефакт переобучается отдельно для каждого энкодера, $M = k/4$: $M = 48$ при $k = 192$, $M = 96$ при $k = 384$, $M = 128$ при $k = 512$. Остальные параметры совпадают с основным экспериментом: 10^6 документов русской Википедии, 500 self-retrieval-запросов, 5 сидов ротации, hi-res CKKS-конфигурация $N = 8192$, $[60, 40, 60]$, $\Delta = 2^{40}$.

Таблица 4.10 — Дополнительный замер при $k = 7d/8$.

Энкодер	Метрика	baseline (raw d)	baseline_proj (SVD k)	Предложенный метод	Δ от base	Server p95, мс
<i>multilingual-e5-small</i> ($d=384$, $k=336$, PQ $M=84$, $\sigma_{\text{rec}}=0,080$)						
	Acc@1	0,782	0,784	$0,782 \pm 0,001$	0,000	462
	Acc@10	0,890	0,896	$0,894 \pm 0,002$	+0,004	
	MRR	0,817	0,820	$0,819 \pm 0,001$	+0,002	
	NDCG@10	0,834	0,838	$0,837 \pm 0,001$	+0,003	
<i>multilingual-e5-base</i> ($d=768$, $k=672$, PQ $M=96$, $\sigma_{\text{rec}}=0,023$)						
	Acc@1	0,830	0,824	$0,824 \pm 0,001$	−0,006	485
	Acc@10	0,934	0,918	$0,918 \pm 0,002$	−0,016	
	MRR	0,862	0,854	$0,854 \pm 0,001$	−0,008	
	NDCG@10	0,880	0,870	$0,869 \pm 0,001$	−0,011	
<i>paraphrase-multilingual-mpnet-base-v2</i> ($d=768$, $k=672$, PQ $M=96$, $\sigma_{\text{rec}}=0,028$)						
	Acc@1	0,716	0,652	$0,652 \pm 0,001$	−0,064	708*
	Acc@10	0,842	0,798	$0,795 \pm 0,004$	−0,047	
	MRR	0,762	0,694	$0,693 \pm 0,001$	−0,069	
	NDCG@10	0,782	0,719	$0,717 \pm 0,002$	−0,065	
<i>multilingual-e5-large</i> ($d=1024$, $k=896$, PQ $M=128$, $\sigma_{\text{rec}}=0,021$)						
	Acc@1	0,796	0,810	$0,810 \pm 0,000$	+0,014	536
	Acc@10	0,880	0,914	$0,912 \pm 0,002$	+0,032	
	MRR	0,824	0,844	$0,844 \pm 0,001$	+0,020	
	NDCG@10	0,838	0,861	$0,860 \pm 0,001$	+0,022	
<i>BAAI/bge-m3</i> ($d=1024$, $k=896$, PQ $M=128$, $\sigma_{\text{rec}}=0,035$)						
	Acc@1	0,818	0,830	$0,830 \pm 0,000$	+0,012	

Энкодер	Метрика	baseline (raw d)	baseline_proj (SVD k)	Предложенный метод	Δ от base	Server p95, мс
	Acc@10	0,908	0,924	$0,922 \pm 0,002$	+0,014	
	MRR	0,848	0,862	$0,862 \pm 0,000$	+0,014	
	NDCG@10	0,862	0,877	$0,877 \pm 0,001$	+0,015	

Прокомментируем результаты по таблице 4.10.

Сравним точки $k = 7d/8$ и $k = d/2$ по информационно-теоретической составляющей. При уменьшении k/d с $7/8$ до $1/2$ показатель σ_{rec} возрастает в 3–7 раз: для e5-small — с 0,080 до 0,239, для e5-base — с 0,023 до 0,129, для mpnet — с 0,028 до 0,107, для e5-large — с 0,021 до 0,101, для bge-m3 — с 0,035 до 0,128. Только в точке $k = d/2$ проху-порог R3 ($\sigma_{\text{rec}} \geq 0,10$) выполняется единообразно для всех пяти энкодеров; в точке $k = 7d/8$ R3 не выполнено ни для одного. По Лемме 1 (раздел 4.2) при $\sigma_{\text{rec}} \geq 0,10$ для любого декодера f с образом в $\text{span}(V_k)$ выполнено

$$\frac{\|\mathbf{x} - f(\pi_k(\mathbf{x}))\|_2}{\|\mathbf{x}\|_2} \geq \sigma_{\text{rec}} \geq 0,10.$$

Перейдём к качеству ранжирования. Переход $k = 7d/8 \rightarrow k = d/2$ на e5-base, e5-large и bge-m3 даёт положительные приросты $\Delta\text{Acc@1}$ относительно raw baseline: +0,003, +0,018 и +0,014 соответственно. Отсечение младших сингулярных направлений убирает шумовые компоненты, и ранжирование в проецированном пространстве оказывается чище. На mpnet деградация в обеих точках практически совпадает: $-0,067$ при $k = 384$ против $-0,064$ при $k = 672$. Заметную цену перехода показывает только e5-small: $\Delta\text{Acc@1} = -0,081$. Это объясняется компактностью модели — при $d = 384$ половинное усечение $k = 192$ отбрасывает половину объёма, и запаса по capacity для компенсации потерь у энкодера нет.

Латентность реранкинга масштабируется как $O(k)$ по числу SKKS-слотов, поэтому уменьшение k вдвое даёт пропорциональное ускорение. По таблице, p95 падает с 462 мс до 314 мс на e5-small, с 485 мс до 356 мс на e5-base, с 708 мс до 351 мс на mpnet (выпадает из режима с системными выбросами), с 536 мс до 219 мс на e5-large и с 536 мс до 221 мс на bge-m3. На энкодерах с $d = 1024$ основная точка $k = d/2$ оказывается дешевле по латентности более чем в два раза.

Размер публичного PQ-артефакта определяется числом под-квантизаторов M . При $M = k/4$ артефакт масштабируется как $M \cdot N$ байт. Для основной точки $k = d/2$ он либо меньше, либо совпадает с артефактом точки $k = 7d/8$: 48 МБ против 84 МБ для e5-small, 96 МБ против 96 МБ для e5-base и mpnet, 128 МБ против 128 МБ для e5-large и bge-m3. Bandwidth-стоимость онбординга клиента при переходе к основной точке не растёт.

4.4.6 Сводная верификация требований и выводы

Выводы по интегральному эксперименту.

1. Интегральный эксперимент в основной операционной точке $k = d/2$ подтверждает, что предложенная двухстадийная архитектура (SVD-усечение V_k с $k = d/2$ + секретная ортогональная ротация $R \in O(k)$ как геометрический защитный слой; client-side PQ-фильтрация в ротированном пространстве с $M = k/4$ под-квантизаторами и $\text{top-}K_{\text{cands}} = 40$ на стадии 1; server-side CKKS-перанкинг 40 кандидатов в режиме ct-pt на стадии 2 с конфигурацией $N = 8192$, $[60, 40, 60]$, $\Delta = 2^{40}$) удовлетворяет требованиям R1–R5 для трех retrieval-trained энкодеров (e5-base, e5-large, bge-m3 — с большим запасом. Для них на масштабе 10^6 документов с self-retrieval-запросами на пяти сидах ротации (11, 23, 47, 31, 53) все четыре метрики ранжирования (Acc@1, Acc@10, MRR, NDCG@10) совпадают с baseline_proj в пределах CI (Таблица 4.7); криптографическая часть запроса $p95 \approx 219\text{--}356$ мс на стенде 1 при $d \in \{384, 768, 1024\}$ (без HTTP-сериализации, без кодирования энкодером). Требование R3 ($\sigma_{\text{rec}} \geq 0,10$) выполнено единообразно для всех пяти энкодеров в этой точке. На paraphrase-distilled mpnet-base-v2 R1 на Acc@10 формально выполнен ($\Delta\text{Acc@10} = -0,047 \leq 0,05$), R1 на Acc@1 не выполнен ($\Delta\text{Acc@1} = -0,067$) — документированное ограничение метода по выбору энкодера.

2. Конфигурация CKKS ($N = 8192$, $\text{coeff_mod_bit_sizes} = [60, 40, 60]$, $\Delta = 2^{40}$, уровень безопасности $\text{tc128} = \lambda \geq 128$ бит по таблицам [HomomorphicEncryption.org/SEAL](https://homomorphicEncryption.org/SEAL), поскольку $\log_2 Q = 160 < 218$) выбрана не вручную, а с помощью разработанного в главе 3 метода ML-автоподбора со строгим ограничением $\Delta\text{Acc@1} \leq 1$ п. п. от baseline. Это устраняет произвольность в выборе параметров и явно связывает функциональное требование (точность top-1 ранжирования) с подобранной конфигурацией.

3. Гомоморфные вычисления CKKS при выбранной конфигурации ($N = 8192$, $[60, 40, 60]$, $\Delta = 2^{40}$) вносят пренебрежимый шум: пирсоновская корреляция $\geq 0,9999$, ранговая корреляция Кендалла = 0,9994, $\max \text{abs error}$ скоринга $\approx 5 \cdot 10^{-6}$. Это означает, что CKKS-перанкинг точно сохраняет порядок документов в проецированном пространстве V_k , и финальное ранжирование совпадает с тем, которое получилось бы при точном (открытом) скалярном произведении. Цена точности — per-query latency перанкинга 40 кандидатов в основной операционной точке $k = d/2$ составляет $\approx 270\text{--}330$ мс на стенде 1 (CPU) в зависимости от размерности энкодера.

4. Связь интегрального эксперимента с теорией. Защитные свойства системы обеспечиваются секретной ротацией R и SVD-усечением: Эксперимент №3 раздела 4.3 показал, что в режиме unknown-R BLEU атаки Vec2Text не превышает 0,007 при любом k/d , включая $k/d = 1,0$. Это даёт количественную защитную гарантию.

Полученные результаты в совокупности с компонентными экспериментами и теоремой о дифференцированной защите позволяют утверждать, что предложенный гибридный метод при правильном подборе энкодера является работоспособным компромиссом в трилемме защищённого семантического поиска в общедоменной retrieval-задаче на русскоязычном корпусе (Википедия, 1 М документов, retrieval-trained энко-

деры семейства E5). Метод как технический примитив применим в архитектуре корпоративных RAG-систем как мера снижения риска раскрытия эмбедингов в облачных векторных БД.

4.5 Промышленная клиент-серверная реализация и измерение сквозной латентности

Интегральный замер выполнен в режиме одного процесса, что фиксирует «чистую криптографическую стоимость» защитного конвейера, но не учитывает серверного фрейминга, сериализации шифротекстов, сетевого хопа и взаимодействия с операционной системой. Чтобы продемонстрировать применимость метода в условиях реального развёртывания, разработана клиент-серверная реализация защищённого поиска и проведён её бенчмарк по сквозной задержке.

4.5.1 Архитектура клиент-серверной реализации

Сделаем замечание о соответствии демо-РоС формальной модели угроз. В демонстрационной реализации (Приложение Е) для удобства локального запуска сервер при старте загружает E_{docs} , вычисляет V_k, μ и генерирует R из дефолтного seed; клиент по тем же причинам вычисляет V_k из локальной копии E_{docs} . Это не соответствует формальной модели угроз, в которой только владелец данных имеет доступ к $E_{\text{docs}}, V_k, \mu, R$, сервер хранит только предвычисленные E_{rot} и публичный PQ-артефакт, а клиент получает client-secret-пакет (V_k, μ, R) от владельца данных по защищённому каналу при онбординге.

В корректной деплой-конфигурации роли разделены. Владелец данных запускает одноразовый offline-скрипт, который загружает E_{docs} , вычисляет V_k, μ , генерирует R из CSPRNG/HSM, после чего выдаёт серверу E_{rot} и публичный PQ-артефакт, а клиенту — client-secret-пакет (V_k, μ, R) вместе с шаблоном CKKS-pubkey. Сервер при таком разделении загружает только E_{rot} и PQ-артефакт и не имеет доступа к процедурам `make_rotation`, `load_or_compute_svd` и к пути `E_DOCS_PATH`. Клиент получает client-secret-пакет от владельца данных через TLS-соединение с авторизационным токеном и не имеет прямого доступа к E_{docs} .

В демо-РоС такая разбивка опущена ради удобства воспроизведения бенчмарков на одной машине. Численные результаты (Acc@1, p95-латентность) от способа генерации V_k, R не зависят и переносятся на корректную деплой-конфигурацию. Рефакторинг кода в трёхролевою структуру вынесен в направления дальнейшей работы.

Опишем стороны системы в формальной модели.

Сервер. Реализован на базе FastAPI и uvicorn (Python 3.11, асинхронный ASGI-стек). В оперативной памяти сервер хранит ротированную базу $E_{\text{rot}} = ((E_{\text{docs}} - \mu)V_k)R^T \in \mathbb{R}^{N \times k}$ ($N = 10^6$, $k = 336$; объём $\approx 1,3$ ГБ при float32) и публичный артефакт стадии 1 — сериализованный faiss IndexPQ ($M = 84$, nbits = 8, ≈ 84 МБ

при $N = 10^6$), обученный в ротированном пространстве E_{rot} . Размещение PQ в одном пространстве с серверной базой устраняет серверную атаку Прокруста (раздел 4.2). Сервер не имеет доступа к R , V_k , μ и секретному СККС-ключу клиента; через REST-API он принимает только зашифрованные запросы. На каждую сессию (POST /session) сервер сохраняет публичный СККС-контекст клиента (с ключами Галуа, без секретного ключа), привязанный к UUID-идентификатору сессии.

Клиент. Реализован как Python-приложение. На этапе инициализации клиент генерирует СККС-ключевую пару, регистрирует публичную часть на сервере и однократно скачивает публичный PQ-артефакт через эндпойнт GET /index (порядка 84 МБ за сессию). При онбординге клиент получает от владельца данных client-secret-пакет (V_k, μ, R) по защищённому каналу и хранит его локально вместе с секретным ключом СККС, не разглашая третьим сторонам.

REST API:

- POST /session — регистрация публичного СККС-контекста, выдача session_id и метаданных индекса (N_{docs} , k , K_{cands} , размер PQ-артефакта).
- GET /index — однократная отдача сериализованного PQ-индекса (faiss IndexPQ, ~ 84 МБ); транспорт application/octet-stream.
- POST /rerank — per-query: принимает {session_id, candidate_ids, ckks_query_b64}, возвращает K_{cands} зашифрованных оценок (ckks_scores_b64) плюс серверный таймер СККС-операций.
- GET /health — liveness-проверка.

Конвейер обработки одного запроса.

1. Клиент локально: $q_{\text{proj}} = V_k^T(v_q - \mu)$, $\tilde{q} = Rq_{\text{proj}}$ (плейнтекст-проекция и ротация на стороне клиента).
2. Клиент локально: PQ-поиск top- $K_{\text{cands}} = 40$ кандидатов через скачанный faiss IndexPQ в ротированном пространстве, используя $\tilde{q}$ как поисковый ключ. PQ обучен в этом же ротированном пространстве, что устраняет серверную Прокруст-атаку (см. §4.2).
3. Клиент шифрует ротированный запрос: $ct_q = \text{СККС_enc}(\tilde{q})$ под собственным секретным ключом, сериализует в base64.
4. Клиент $\rightarrow$ сервер: HTTP POST на /rerank с 40 идентификаторами кандидатов и шифротекстом запроса.
5. Сервер: 40 ct-pt операций между ct_q и плейнтекст-векторами $E_{\text{rot}}[\text{cand_ids}]$, сериализация 40 шифротекстов-оценок.
6. Сервер $\rightarrow$ клиент: HTTP-ответ с 40 base64-зашифрованными оценками.
7. Клиент расшифровывает оценки секретным ключом, ранжирует, формирует top-10 идентификаторов документов.

Что видит сервер. На каждый запрос сервер получает:

- (a) СККС-шифротекст запроса (без секретного ключа сервер не может его расшифровать),

(б) $K_{\text{cands}} = 40$ идентификаторов кандидатов, переданных клиентом со стадии 1. Сервер не получает ни оригинальный эмбединг v_q , ни проецированный q_{proj} , ни ротированный $\tilde{q}$, ни плеинтекст-оценки сходства.

4.5.2 Бенчмарк PoC-реализации (HTTP-loopback)

Стенд. В качестве клиента использовался стенд 1 (Intel Core i5-14400F, 32 Гб DDR5, см. Таблицу 4.3); а в качестве сервера использовалась серверная платформа с процессором Intel(R) Xeon(R) Silver 4416+ и 128 Gb RAM.

Тестовая задача. Та же постановка self-retrieval, что и в интегральном эксперименте раздела 4.4: 500 запросов self-retrieval на корпусе 10^6 документов, multilingual-e5-small, $k = 336$. Используются пред-кодированные эмбединги запросов (метод `search_emb`); таким образом измеренная сквозная задержка не включает время инференса энкодера (≈ 200 мс на стороне клиента, см. Таблицу 4.8). Ротация R зафиксирована на сиде 11 (одна R на сессию — стандартный production-сценарий); вариантность по R изучена в раздел 4.4 на 5 сидах. Перед измерением выполнена прогревочная фаза из 30 запросов (стабилизация HTTP-пула, JIT-кэшей TenSEAL).

Замеренные результаты (500 запросов, 1 проход с 30-запросным прогревом) приведены в Таблице 4.11.

Таблица 4.11 — Сквозная латентность клиент-серверной PoC-реализации (multilingual-e5-small, 1 М-корпус, 500 запросов через HTTP/JSON loopback на стенде 1, ротация R зафиксирована на seed=11; кодирование запроса энкодером в измерение не входит)

Этап	Латентность	Описание
<i>Качество ранжирования</i>		
Acc@1	0,782	Совпадает с in-process замером (Таблица 4.7) в пределах float32-jitter
Acc@10	0,896	—
MRR	0,819	—
NDCG@10	0,837	—
<i>Декомпозиция req-query латентности (медианные значения)</i>		
Клиент: проекция + ротация	0,1 мс	V_k -проекция и плеинтекст-ротация
Клиент: PQ-stage-1	19,6 мс	faiss IndexPQ.search($K_{\text{cands}} = 40$) в ротированном пространстве
Клиент: CKKS-шифрование	3,9 мс	TenSEAL CKKS-vector + base64
Сеть + (де)сериализация	40 мс	HTTP framing + base64 + JSON, loopback (network_ms – server_ckks_ms)
Сервер: CKKS-перанкинг 40 кандидатов	436 мс	40 ct-pt операций ($N = 8192$, $\Delta = 2^{40}$)

Этап	Латентность	Описание
Клиент: расшифровка + ранжирование	18,1 мс	TenSEAL decrypt 40 шифротекстов
<i>Сквозная end-to-end задержка</i>		
p50	519 мс	Медианная сквозная задержка
p95	578 мс	95-й перцентиль
p99	597 мс	99-й перцентиль

Сквозная p95 = 578 мс на стенде 1 удовлетворяет требованию R2 ($p95 < 1$ с) с запасом более 40 %. *Замечание о конфигурации замера.* PoC-стенд (cs_v2) выполняет замер транспортного слоя при $k = 336$ (соответствует исходной конфигурации client-server-кода Приложения Е), а не в основной операционной точке $k = d/2$ интегрального эксперимента раздела 4.4. Латентность реранкинга при $k = 336$ ожидаемо выше, чем при $k = d/2$, пропорционально числу СККС-слотов; это не влияет на корректность измерения транспортного слоя. Качество ранжирования при $k = 336$ совпадает с in-process замером в той же точке (e5-small, $k = 336$: Acc@1 = 0,782, Acc@10 = 0,896, MRR = 0,819, NDCG@10 = 0,837) в пределах float32-jitter, что подтверждает корректность транспортного слоя — HTTP/JSON-обёртка и base64-сериализация не вносят артефактов в защитный конвейер.

Декомпозиция времени. Доминирующая часть сквозной задержки — серверный СККС-перанкинг 40 кандидатов (≈ 436 мс p50, ≈ 84 % от end-to-end). На втором месте — сетевой/сериализационный накладной расход на loopback (40 мс p50, ≈ 8 %): это HTTP framing FastAPI/uvicorn вместе с base64-кодированием шифротекстов и JSON-парсингом. Клиентская сторона (PQ-поиск + шифрование + расшифровка) занимает 42 мс (≈ 8 %). При замене HTTP/JSON на бинарный gRPC/Protobuf-протокол ожидается сокращение сетевого этапа в 2–3 раза без архитектурных изменений; это стандартная инженерная задача, не входящая в объём настоящего PoC.

Исходный код клиент-серверной реализации (config.py, indexer.py, server.py, client.py, benchmark.py) приведён в Приложении Е.

4.5.3 Направления дальнейшей работы

Прямое продолжение настоящей работы:

1. Адаптивные атаки и ортогональная задача Прокруста: численная проверка устойчивости секретной ротации к восстановлению R из набора known-plaintext-пар.
2. Entity-level PII-метрики: повторение Эксперимента №3 с измерением PII precision/recall, exact-match по чувствительным сущностям, не только BLEU.
3. Защита access pattern leak на stage-2. На каждый запрос сервер видит идентификаторы $K_{\text{cands}} = 40$ кандидатов; шаблоны запросов могут раскрывать тематику. Полная защита потребовала бы PIR (Private Information Retrieval) или ORAM для

серверного фетчинга кандидатов: в качестве кандидатов на интеграцию рассматриваются современные single-server PIR-схемы — SealPIR [71] (PIR со сжатыми запросами и амортизированной обработкой), OnionPIR [72] (response-efficient single-server PIR на основе RLWE) и SimplePIR [73] (single-server PIR с минимальной серверной нагрузкой). Альтернативный путь — переход к oblivious cluster-based архитектуре типа Tiptoe [74] (server-side CKKS-перанкинг внутри выбранных кластеров с однообразным паттерном фетчинга). Ещё одна техника — homomorphic argmax над CKKS-оценками для server-side выбора best-of-K без раскрытия отдельных оценок — закрывает соответствующий канал утечки.

4. Масштабирование на $N \gtrsim 10^8$. Размер публичного PQ-артефакта при $M = 84$, 1 байт на под-квантизатор $\rightarrow \sim 84 \cdot N$ байт линейно растёт с корпусом ($\sim 8,4$ ГБ для $N = 10^8$). Для крупных корпусов потребуется иерархическая инстанциация stage-1 (HNSW поверх PQ-кодов либо двухуровневая IVF-PQ-схема), либо переход к bandwidth-efficient stage-1, исключающему скачивание полного артефакта (например, server-side IVF-PQ-поиск с PIR-обёрнутыми обращениями за PQ-кодами). Stage-2 CKKS-перанкинг при этом идентичен и не зависит от выбора stage-1.

5. GPU-оптимизация CKKS-перанкинга. Текущая латентность CKKS на CPU — ≈ 10 мс при $N = 8192$ и $k = 336$. На GPU (Lattigo-CUDA, OpenFHE-CUDA) типично достижимо ускорение $\times 5\text{--}10$, что позволит увеличить K_{cands} до сотен документов в том же 1-секундном budget и поднять recall stage-1 ещё выше. Также интересно исследовать инженерный sweep (M, K_{cands}) под разные требования к recall/качеству.

6. Расширение ML-главы: nested cross-validation, расширенное признаковое пространство для предиктора латентности, кросс-платформенная валидация.

7. Формальная криптографическая стойкость геометрического преобразования: построение reduction к стандартной криптографической задаче (например, Subspace LWE) либо доказательство нижней границы устойчивости к любому полиномиальному декодеру.

8. SVD-усечение как линейный денойзер для retrieval-эмбеддингов: отдельное систематическое исследование.

Заключение

В работе решена актуальная научно-техническая задача — обеспечение конфиденциальности в системах семантического поиска без заметной деградации функциональных характеристик. Разработан, теоретически обоснован и экспериментально валидирован гибридный метод защищённого поиска: секретная ортогональная ротация R (с опциональным SVD-усечением для приложений, чувствительных к объёму данных) отвечает за защиту базы эмбеддингов от атак инверсии, а CKKS-шифрование в режиме ciphertext-plaintext — за защиту запросов на этапе перанкинга.

Цель работы, заявленная во введении, достигнута: метод снижает риск восстановления исходных текстов из эмбедингов и при этом сохраняет качество семантического поиска и интерактивную латентность. Рабочая гипотеза подтверждена. Логика подтверждения выстроена в виде причинно-следственной цепочки: разработан и валидирован метод ML-автоподбора параметров СККС, который выбирает конкретную конфигурацию для интегрального теста; проверены отдельные компоненты архитектуры (Эксперимент №1 — режимы СККС; Эксперимент №2 — покомпонентный анализ защиты и SVD-проекции по проху-метрике; Эксперимент №3 — прямая атака Vec2Text на эмбедингах GTR-base, валидирующая лемму о нижней L_2 -ошибке проекционного декодера), эта конфигурация проверяется в полной сборке на масштабе 10^6 документов.

Основные научные и практические результаты работы:

1. Разработана и обоснована гибридная архитектура защищённого поиска с асимметричной защитой. Предложен подход, в котором база данных защищается секретной ортогональной ротацией R (с опциональным SVD-усечением для приложений, чувствительных к объёму данных), а запросы — гомоморфным шифрованием СККС в режиме ciphertext-plaintext. Двухуровневая схема обеспечивает быструю фильтрацию кандидатов через FAISS с последующим зашифрованным ранкингом.

2. Сформулирована и доказана **Лемма 1 о нижней границе ошибки декодера в L_2 -норме**: для декодера f , отображающего проекцию $\pi_k(\mathbf{x})$ обратно в $\text{span}(V_k)$ (либо в байесовской постановке без априорной утечки конкретных образцов), L_2 -ошибка восстановления не меньше $\|\mathbf{x}_\perp\|_2 = \|(I - V_k V_k^T)\mathbf{x}\|_2$, что после нормировки даёт σ_{rec} . Без ограничения на образ декодер с возможностью «запоминать» конкретные образцы может достичь меньшей ошибки; теорема *не* утверждает универсального bound для произвольных нейросетевых декодеров. Связь σ_{rec} с эмпирическим BLEU атаки Vec2Text формулируется отдельно как Эмпирическая гипотеза 1 и проверяется численно в Эксперименте №3 на эмбедингах GTR-base. Измеренные значения BLEU при отключённой ротации: 0,077 при $k/d = 0,75$, 0,057 при $k/d = 0,50$, 0,024 при $k/d = 0,25$, 0,010 при $k/d = 0,10$ (относительно $\text{BLEU}_0 = 0,156$ для unprotected baseline на той же постановке; см. Таблицу 4.6). Линейная регрессия BLEU по $\eta_k = 1 - \sigma_{\text{rec}}^2$ согласуется с гипотезой (но не является её формальным доказательством). В режиме unknown-R BLEU сводится к $\leq 0,007$ при любом k/d — результат, не выводимый из Леммы 1, а отражающий ортогональный к SVD-усечению защитный эффект секретной ротации в рамках выбранной non-adaptive модели угроз. Значение $\text{BLEU}_0 = 0,156$ заметно ниже 0,973, цитируемого в [2], потому что в Эксперименте №3 используется иная постановка (синтетические тексты с ПИ, greedy decoding, 10 итераций корректора без beam search, ограничение GPU-памяти, иной домен): эксперимент не воспроизводит максимальную силу Vec2Text, а проверяет относительный эффект SVD-усечения и секретной ротации в фиксированной слабой конфигурации атаки.

3. Для интегрального эксперимента в основной операционной точке выбрано $k = d/2$, при котором проху-критерий $\sigma_{\text{rec}} \geq 0,10$ выполняется единообразно для всех

пяти протестированных энкодеров. Защитный конвейер: SVD-усечение V_k с $k = d/2$, секретная ортогональная ротация $R \in O(k)$ и двухстадийный поиск (client-side PQ-фильтрация в ротированном пространстве, $M = k/4$, $\text{top-}K_{\text{cands}} = 40 + \text{server-side CKKS-перанкинг 40 кандидатов}$). CKKS-конфигурация $N = 8192$, $[60, 40, 60]$, $\Delta = 2^{40}$ автоматически подобрана методом главы 3 со строгим ограничением $\Delta\text{Acc}@1 \leq 1$ п. п. относительно ранжирования в SVD-проецированном пространстве. На задаче self-retrieval с корпусом 10^6 документов защитный конвейер не вносит заметных искажений сверх SVD-проекции (см. Таблицу 4.7); измерено по 5 сдам ротации R . Криптографическая часть запроса $p95 \approx 219\text{--}356$ мс на стенде 1. Информационно-теоретическая ошибка реконструкции σ_{rec} принимает значения 0,101–0,239 для пяти энкодеров. Менее агрессивная точка $k = 7d/8$ рассмотрена только как вспомогательный замер для оценки чувствительности качества к выбору k и не используется в качестве основной (проху-критерий R3 в ней не выполняется).

4. Обоснован выбор режима ciphertext-plaintext (ct-pt) схемы CKKS для защищённого перанкинга. На эксперименте с базой 10 000 документов показано снижение времени вычисления сходства по сравнению с режимом ciphertext-ciphertext (ct-ct) в $1,44\times$ (с 19,15 до 13,28 с, 95 % CI $[1,43; 1,46]$) при сохранении целевого уровня криптографической стойкости.

5. Разработан метод автоматизированного подбора параметров CKKS на основе ансамблевой регрессионной модели. Модель предсказывает время выполнения операций и поддерживает отбор конфигураций с учётом ограничений по стойкости и точности; на расширенной 198-точечной сетке с вложенной кросс-валидацией и 20 %-й отложенной подвыборкой наилучшие результаты показала модель GradientBoosting ($\text{max_depth}=3$, $\text{n_estimators}=200$) с $R^2_{\text{nested}} = 0,970 \pm 0,011$, $\text{MAE}_{\text{nested}} = 135 \pm 20$ мс и финальной оценкой $R^2_{\text{holdout}} = 0,985$, $\text{MAE}_{\text{holdout}} = 146$ мс при диапазоне латентности 25–8 257 мс. Реализован алгоритм FixedParameterOptimizer.

6. Экспериментально подтверждена масштабируемость решения в интегральном сценарии. На корпусе из 10^6 документов русской Википедии в основной операционной точке $k = d/2$ для пяти энкодеров (multilingual-e5-small, e5-base, e5-large, paraphrase-mpnet-base-v2, BAAI/bge-m3) достигнута криптографическая часть запроса $p95 \approx 219\text{--}356$ мс (без HTTP-сериализации, без кодирования энкодером). Защитный конвейер не вносит заметных искажений сверх SVD-проекции по всем четырём метрикам $\text{Acc}@1$, $\text{Acc}@10$, MRR , $\text{NDCG}@10$ для всех пяти энкодеров. Требование R1 ($\Delta\text{Acc}@1 \geq -5$ п. п. от незащищённого baseline) выполнено для трёх retrieval-trained-моделей с $d \geq 768$: multilingual-e5-base (+0,003), multilingual-e5-large (+0,018) и BAAI/bge-m3 (+0,014). На multilingual-e5-small ($\Delta = -0,081$) и paraphrase-mpnet-base-v2 ($\Delta = -0,067$) R1 не выполняется и фиксируется как ограничение метода по выбору энкодера: на компактной 384-мерной модели половинное усечение забирает половину объёма, а paraphrase-distilled-модель не концентрирует retrieval-сигнал в верхних главных компонентах. Кон-

фигурация CKKS выбрана автоматически методом главы 3 со строгим ограничением $\Delta\text{Acc}@1 \leq 1$ п. п. относительно ранжирования в SVD-проецированном пространстве.

Научная новизна:

Идея использования снижения размерности и случайных проекций для защиты данных не нова: в литературе она обсуждалась как минимум с работ Liu и соавторов [65] и Plumb и соавторов [66] в контексте privacy-preserving data mining. Аналогично, гибридные схемы FHE+ML рассматривались в CryptoNets [29] и связанных проектах (EVA [62], CHET [63], hummingbird [64]), а ML-подбор параметров FHE — в EVA и схожих компиляторах. На этом фоне новизна настоящей работы состоит в следующих частных, но конкретных моментах.

- Сформулирована и доказана Лемма 1 (разд. 4.2) о нижней границе L_2 -ошибки декодера, отображающего проекцию $\pi_k(\mathbf{x})$ обратно в $\text{span}(V_k)$: ошибка восстановления не меньше нормы ортогональной составляющей $\|\mathbf{x}_\perp\|$. Связь L_2 -ошибки с эмпирической метрикой BLEU атаки инверсии формулируется отдельно как Эмпирическая гипотеза 1 и численно валидируется в Эксперименте №3 на эмбедингах GTR-base. В отличие от ряда существующих работ по проекциям-как-защите, в настоящей работе явно разграничены информационно-теоретическая часть (Лемма 1) и эмпирическая часть (зависимость BLEU от k/d).

- Предложена и реализована конкретная гибридная схема для задачи семантического поиска, использующая режим ciphertext-plaintext схемы CKKS поверх предварительно проецированной и ротированной базы. Отличие от предшествующих гибридных схем (CryptoNets и аналогов, ориентированных на инференс нейросетей) — именно сценарий retrieval с двухуровневым конвейером (FAISS-фильтрация + CKKS-перанкинг), позволяющий выйти на субсекундную латентность на масштабе 10^6 документов.

- Разработан ML-предиктор латентности CKKS-операций на ансамблевой регрессии, учитывающий микроархитектурные эффекты (кэш L2, SIMD, evaluation gap). По сравнению с EVA/CHET, ориентированными на компиляцию арифметических цепочек, наш предиктор фокусируется на узкой задаче подбора параметров (N, Q, d_{proj}) для скалярного произведения и достигает на собственной обучающей выборке показателя $R^2 = 0,946$ при инкрементальной интеграции в существующий пайплайн TenSEAL. Это инкрементальное улучшение, а не новый класс методов.

Практическая значимость:

Полученные результаты могут лечь в основу построения защищённых систем семантического поиска и RAG-приложений в корпоративных сценариях — там, где требуется снизить риск раскрытия численного представления эмбедингов и точных значений сходства при хранении и обработке в облачных векторных БД. В основной операционной точке $k = d/2$ информационно-теоретическая компонента SVD-усечения.

Перспективы:

- Расширение проверок. Сюда относится и наращивание набора корпусов с доменами, и сравнение с альтернативными моделями эмбедингов, и тестирование устойчи-

ности к новым вариантам атак инверсии и нестандартным сценариям компрометации. Отдельный интерес — применить методику семантической оценки из Эксперимента № 3 к моделям большей размерности ($d = 768, 1536$) и проверить гипотезу о масштабируемости подхода.

– Альтернативы дизайна. Имеет смысл изучить нелинейные проекции (например, автоэнкодеры) как замену или дополнение SVD-проекциям, а заодно перенести вычисления СККС на GPU и оценить, как аппаратная платформа влияет на оптимальный набор параметров.

– Открытые вопросы. Формализация гарантий конфиденциальности для проекционных преобразований в рамках выбранной модели угроз; устойчивость подхода при динамическом обновлении базы и при сдвиге распределения данных.

Артефакты, обеспечивающие воспроизводимость основных шагов методики и экспериментов, собраны в приложениях А–Е.

Список литературы

- [1] Reinsel D., Gantz J., Rydning J. The Digitization of the World from Edge to Core: Tech. Rep.: : IDC White Paper, 2018.
- [2] Text Embeddings Reveal (Almost) As Much As Text / J. X. Morris, V. Kuleshov, V. Shmatikov [и др.] // Proceedings of EMNLP. 2023.
- [3] Li J. [и др.]. Sentence Embeddings Leakage through Embedding Inversion Attack // Findings of ACL. 2023.
- [4] Gu J. [и др.]. GEIA: Gradient-based Embedding Inversion Attack. arXiv preprint arXiv:2304.08945. 2023.
- [5] Pan X. [и др.]. TEIA: Text Embedding Inversion Attacks. arXiv preprint arXiv:2305.08450. 2023.
- [6] Song C., Raghunathan A. Information Leakage in Embedding Models // Proceedings of ACM CCS. ACM, 2020. С. 377–390.
- [7] Membership Inference Attacks Against Machine Learning Models / R. Shokri, M. Stronati, C. Song [и др.] // 2017 IEEE Symposium on Security and Privacy (SP). IEEE, 2017. С. 3–18.
- [8] Extracting Training Data from Large Language Models / N. Carlini, F. Tramèr, E. Wallace [и др.] // 30th USENIX Security Symposium. USENIX Association, 2021. С. 2633–2650.
- [9] The Good and The Bad: Exploring Privacy Issues in Retrieval-Augmented Generation (RAG) / S. Zeng, J. Zhang, P. He [и др.] // Findings of ACL 2024. 2024. С. 4505–4524.

- [10] IBM Security. Cost of a Data Breach Report 2024: Tech. Rep.: : Ponemon Institute, 2024.
- [11] О персональных данных: Федеральный закон от 27.07.2006 № 152-ФЗ (ред. от 24.06.2025). Собрание законодательства Российской Федерации. 2006. № 31 (ч. I). Ст. 3451.
- [12] Об утверждении Составы и содержания организационных и технических мер по обеспечению безопасности персональных данных при их обработке в информационных системах персональных данных: Приказ ФСТЭК России от 18.02.2013 № 21 (ред. от 14.05.2020). Российская газета. 2013. 22 мая. № 107.
- [13] Regulation (EU) 2016/679 of the European Parliament and of the Council (General Data Protection Regulation, GDPR). 2016.
- [14] BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding / J. Devlin, M.-W. Chang, K. Lee [и др.] // Proceedings of NAACL-HLT. 2019. С. 4171–4186.
- [15] Reimers N., Gurevych I. Sentence-BERT: Sentence Embeddings using Siamese BERT-Networks // Proceedings of EMNLP. 2019. С. 3982–3992.
- [16] Dale D. rubert-tiny2: efficient distilled BERT for Russian. Hugging Face Model Hub. 2023.
- [17] Lewis P. [и др.]. Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks // Proceedings of NeurIPS. 2020.
- [18] Dense Passage Retrieval for Open-Domain Question Answering / V. Karpukhin, B. Oguz, S. Min [и др.] // Proceedings of EMNLP. 2020. С. 6769–6781.
- [19] Khattab O., Zaharia M. ColBERT: Efficient and Effective Passage Search via Contextualized Late Interaction over BERT // Proceedings of the 43rd International ACM SIGIR Conference. ACM, 2020. С. 39–48.
- [20] Wang L., Yang N., Huang X. [и др.]. Text Embeddings by Weakly-Supervised Contrastive Pre-training. arXiv preprint arXiv:2212.03533. 2022.
- [21] One Embedder, Any Task: Instruction-Finetuned Text Embeddings / H. Su, W. Shi, J. Kasai [и др.] // Findings of ACL. 2023. С. 1102–1121.
- [22] Self-RAG: Learning to Retrieve, Generate, and Critique through Self-Reflection / A. Asai, Z. Wu, Y. Wang [и др.] // Proceedings of ICLR 2024. 2024.
- [23] Yan S.-Q., Gu J.-C., Zhu Y. [и др.]. Corrective Retrieval Augmented Generation. arXiv preprint arXiv:2401.15884. 2024.

- [24] RAGAs: Automated Evaluation of Retrieval Augmented Generation / S. Es, J. James, L. Espinosa Anke [и др.] // Proceedings of EACL: System Demonstrations. 2024. C. 150–158.
- [25] Gao Y., Xiong Y., Gao X. [и др.]. Retrieval-Augmented Generation for Large Language Models: A Survey. arXiv preprint arXiv:2312.10997. 2024.
- [26] Homomorphic Encryption for Arithmetic of Approximate Numbers / J. H. Cheon, A. Kim, M. Kim [и др.] // Advances in Cryptology — ASIACRYPT 2017. Cham: Springer, 2017. Т. 10624 из *Lecture Notes in Computer Science*. С. 409–437.
- [27] Bootstrapping for Approximate Homomorphic Encryption / J. H. Cheon, K. Han, A. Kim [и др.] // Advances in Cryptology — EUROCRYPT 2018. Cham: Springer, 2018. Т. 10820 из *Lecture Notes in Computer Science*. С. 360–384.
- [28] OpenFHE: Open-Source Fully Homomorphic Encryption Library / A. Al Badawi, J. Bates, F. Bergamaschi [и др.] // Proceedings of WAHC '22, co-located with CCS. ACM, 2022. С. 53–63.
- [29] CryptoNets: Applying Neural Networks to Encrypted Data with High Throughput and Accuracy / R. Gilad-Bachrach, N. Dowlin, K. Laine [и др.] // Proceedings of ICML 2016, PMLR. Т. 48. 2016. С. 201–210.
- [30] Homomorphic Encryption Security Standard: Tech. Rep.: / M. Albrecht, M. Chase, H. Chen [и др.]. Toronto: HomomorphicEncryption.org, 2018.
- [31] Over 100x Faster Bootstrapping in Fully Homomorphic Encryption through Memory-centric Optimization with GPUs / W. Jung, S. Kim, J. H. Ahn [и др.] // IACR Transactions on Cryptographic Hardware and Embedded Systems (TCHES). 2021. Т. 2021, № 4. С. 114–148.
- [32] Deep Learning with Differential Privacy / M. Abadi, A. Chu, I. Goodfellow [и др.] // Proceedings of ACM CCS. ACM, 2016. С. 308–318.
- [33] Mironov I. Renyi Differential Privacy // 2017 IEEE 30th Computer Security Foundations Symposium (CSF). IEEE, 2017. С. 263–275.
- [34] Communication-Efficient Learning of Deep Networks from Decentralized Data / H. B. McMahan, E. Moore, D. Ramage [и др.] // Proceedings of AISTATS, PMLR. Т. 54. 2017. С. 1273–1282.
- [35] Sparck Jones K. A Statistical Interpretation of Term Specificity and Its Application in Retrieval // Journal of Documentation. 1972. Т. 28, № 1. С. 11–21.
- [36] Robertson S. E., Zaragoza H. The Probabilistic Relevance Framework: BM25 and Beyond // Foundations and Trends in Information Retrieval. 2009. Т. 3, № 4. С. 333–389.

- [37] Mikolov T. [и др.]. Distributed Representations of Words and Phrases and their Compositionality // Proceedings of NeurIPS. 2013. С. 3111–3119.
- [38] Pennington J., Socher R., Manning C. D. GloVe: Global Vectors for Word Representation // Proceedings of EMNLP. 2014. С. 1532–1543.
- [39] Vaswani A. [и др.]. Attention Is All You Need // Proceedings of NeurIPS. 2017. С. 5998–6008.
- [40] Approximate Nearest Neighbor Negative Contrastive Learning for Dense Text Retrieval / L. Xiong, C. Xiong, Y. Li [и др.] // Proceedings of ICLR. 2021.
- [41] Andoni A., Indyk P. Near-Optimal Hashing Algorithms for Approximate Nearest Neighbor in High Dimensions // Communications of the ACM. 2008. Т. 51, № 1. С. 117–122.
- [42] Jegou H., Douze M., Schmid C. Product Quantization for Nearest Neighbor Search // IEEE Transactions on Pattern Analysis and Machine Intelligence (TPAMI). 2011. Т. 33, № 1. С. 117–128.
- [43] Malkov Y. A., Yashunin D. A. Efficient and Robust Approximate Nearest Neighbor Search Using Hierarchical Navigable Small World Graphs // IEEE Transactions on Pattern Analysis and Machine Intelligence. 2020. Т. 42, № 4. С. 824–836.
- [44] Milvus: A Purpose-Built Vector Data Management System / J. Wang, X. Yi, R. Guo [и др.] // Proceedings of SIGMOD. ACM, 2021. С. 2614–2627.
- [45] Ni J. [и др.]. Large Dual Encoders Are Generalizable Retrievers // Proceedings of EMNLP. 2022. С. 9844–9855.
- [46] BEIR: A Heterogeneous Benchmark for Zero-shot Evaluation of Information Retrieval Models / N. Thakur, N. Reimers, A. Ruckle [и др.] // Proceedings of NeurIPS Track on Datasets and Benchmarks. 2021.
- [47] MTEB: Massive Text Embedding Benchmark / N. Muennighoff, N. Tazi, L. Magne [и др.] // Proceedings of EACL. 2023. С. 2014–2037.
- [48] McKinsey & Company. The State of AI in 2025: From Pilots to Scale: Tech. Rep.: : McKinsey & Company, 2025.
- [49] Exploiting Unintended Feature Leakage in Collaborative Learning / L. Melis, C. Song, E. De Cristofaro [и др.] // 2019 IEEE Symposium on Security and Privacy (SP). IEEE, 2019. С. 691–706.
- [50] TextAttack: A Framework for Adversarial Attacks, Data Augmentation, and Adversarial Training in NLP / J. X. Morris, E. Lifland, J. Y. Yoo [и др.] // Proceedings of EMNLP: System Demonstrations. 2020. С. 119–126.

- [51] Papineni K. [и др.]. BLEU: a Method for Automatic Evaluation of Machine Translation // Proceedings of ACL. 2002. С. 311–318.
- [52] Астахова Л. В., Султанов Д. Р., Ашихмин Н. А. Защита облачной базы персональных данных с использованием гомоморфного шифрования // Вестник ЮУрГУ. Серия «Компьютерные технологии, управление, радиоэлектроника». 2016. Т. 16, № 3. С. 52–61.
- [53] ГОСТ 34.12-2018. Информационная технология. Криптографическая защита информации. Блочные шифры: межгосударственный стандарт. 2019. 18 с.
- [54] Dwork C. Differential Privacy // Proceedings of ICALP. Springer, 2006. С. 1–12.
- [55] Lyu L., He X., Li Y. Differentially Private Representation for NLP: Formal Guarantee and An Empirical Study on Privacy and Fairness // Findings of EMNLP. 2020. С. 2355–2365.
- [56] Gentry C. Fully Homomorphic Encryption Using Ideal Lattices // Proceedings of the 41st Annual ACM Symposium on Theory of Computing (STOC). ACM, 2009. С. 169–178.
- [57] Fan J., Vercauteren F. Somewhat Practical Fully Homomorphic Encryption // IACR Cryptology ePrint Archive, Report 2012/144. 2012.
- [58] TFHE: Fast Fully Homomorphic Encryption over the Torus / I. Chillotti, N. Gama, M. Georgieva [и др.] // Journal of Cryptology. 2020. Т. 33, № 1. С. 34–91.
- [59] Johnson W. B., Lindenstrauss J. Extensions of Lipschitz Mappings into a Hilbert Space // Contemporary Mathematics. 1984. Т. 26. С. 189–206.
- [60] Privacy via the Johnson-Lindenstrauss Transform / K. Kenthapadi, A. Korolova, I. Mironov [и др.] // Journal of Privacy and Confidentiality. 2013. Т. 5, № 1.
- [61] Eckart C., Young G. The Approximation of One Matrix by Another of Lower Rank // Psychometrika. 1936. Т. 1, № 3. С. 211–218.
- [62] EVA: An Encrypted Vector Arithmetic Language and Compiler for Efficient Homomorphic Computation / R. Dathathri, B. Kostova, O. Saarikivi [и др.] // Proceedings of the 41st ACM SIGPLAN Conference on Programming Language Design and Implementation (PLDI). 2020. С. 546–561.
- [63] CHET: An Optimizing Compiler for Fully-Homomorphic Neural-Network Inferencing / R. Dathathri, O. Saarikivi, H. Chen [и др.] // Proceedings of the 40th ACM SIGPLAN Conference on Programming Language Design and Implementation (PLDI). 2019. С. 142–156.

- [64] Optimizations of FHE parameter selection for ML-as-a-service workloads / J. Lee, E. Lee, J.-S. No [и др.] // arXiv preprint arXiv:2202.07763. 2022.
- [65] Liu K., Kargupta H., Ryan J. Random Projection-Based Multiplicative Data Perturbation for Privacy Preserving Distributed Data Mining // IEEE Transactions on Knowledge and Data Engineering. 2006. T. 18, № 1. C. 92–106.
- [66] SnapToGrid: From Statistical to Interpretable Models for Biomedical Data / G. Plumb, D. Pachauri, R. Kondor [и др.] // arXiv preprint arXiv:1804.10840. 2018.
- [67] Intel HEXL: Accelerating Homomorphic Encryption with Intel AVX512-IFMA52 / F. Boemer, S. Kim, G. Seifu [и др.] // Proceedings of the 9th on Workshop on Encrypted Computing & Applied Homomorphic Cryptography (WAHC '21). 2021. C. 57–62. arXiv:2103.16400.
- [68] Friedman J. H. Greedy Function Approximation: A Gradient Boosting Machine // Annals of Statistics. 2001. T. 29, № 5. C. 1189–1232.
- [69] Chen T., Guestrin C. XGBoost: A Scalable Tree Boosting System // Proceedings of KDD. ACM, 2016. C. 785–794.
- [70] Tishby N., Pereira F. C., Bialek W. The information bottleneck method // arXiv preprint physics/0004057. 2000.
- [71] PIR with Compressed Queries and Amortized Query Processing / S. Angel, H. Chen, K. Laine [и др.] // Proceedings of the IEEE Symposium on Security and Privacy (SealPIR). 2018. C. 962–979.
- [72] Mughees M. H., Chen H., Ren L. OnionPIR: Response Efficient Single-Server PIR // Proceedings of the 2021 ACM SIGSAC Conference on Computer and Communications Security (CCS). 2021. C. 2292–2306.
- [73] One Server for the Price of Two: Simple and Fast Single-Server Private Information Retrieval (SimplePIR) / A. Henzinger, M. M. Hong, H. Corrigan-Gibbs [и др.] // Proceedings of the 32nd USENIX Security Symposium. 2023.
- [74] Private Web Search with Tiptoe / A. Henzinger, E. Dauterman, H. Corrigan-Gibbs [и др.] // Proceedings of the 29th ACM Symposium on Operating Systems Principles (SOSP). 2023.

Приложение А

Эксперимент 1. ML-автонастройка параметров СККС (глава 3): улучшенная методика на 198 конфигурациях с вложенной CV и независимой holdout-выборкой

Исходный код Python-скрипта `_rerun_exp1_improved.py` для ML-автоподбора параметров СККС (глава 3) приведён ниже. Скрипт реализует расширенную методику микробенчмаркинга 198 валидных конфигураций СККС, обучение регрессионной модели латентности (GradientBoosting на 10 признаках, вложенная 5×3 кросс-валидация + 20% holdout) и итоговую оценку $R^2_{\text{holdout}} = 0,985$, $\text{MAE}_{\text{holdout}} = 146$ мс. Результаты сохраняются в `exp1_outputs/microbench_v2.csv` и `exp1_outputs/regression_results.json`.

Листинг 1 — Исходный код `_rerun_exp1_improved.py`

```
1 """Improved ML autotuning for CKKS parameters (Experiment No.1, chapter 3).
2
3 Improvements over the original 32-configuration prototype:
4
5 1. EXPANDED CONFIGURATION GRID
6   - 4 polynomial moduli: N in {2048, 4096, 8192, 16384}
7   - Multiple coefficient-modulus chains per N (different bit structures with the
8     same total bit budget): e.g. for N=4096, we test [40,20,40], [40,40],
9     [50,30,50]
10  - 5 scales: Delta in {2^15, 2^20, 2^25, 2^30, 2^40}
11  - 6 projection dims: {32, 64, 128, 192, 256, 320}
12  - 3 batch sizes (number of CKKS rerank candidates): {16, 64, 256}
13  Filter: keep only configurations where N/2 >= projection_dim and
14  total_coeff_bits <= 109 (for N=4096) / 218 (N=8192) / 438 (N=16384).
15  Result: ~150 valid configurations.
16
17 2. RICHER FEATURE SPACE (10 features, was 5)
18   - poly_modulus_degree (N), log_N
19   - num_coeff_moduli (chain length)
20   - total_coeff_bits, first_coeff_bit, last_coeff_bit (bit structure)
21   - scale_bits, scale_to_total_ratio
22   - projection_dim, batch_size, vector_packing_ratio (proj_dim / n_slots)
23
24 3. PROPER METHODOLOGY
25   - 80/20 train/HOLDOUT split (true held-out test never used during training)
26   - Nested CV on training set: outer 5-fold for model evaluation, inner 3-fold
27     for hyperparameter selection via grid search
28   - Multiple model families: Ridge, KNN, ExtraTrees, GradientBoosting, SVR
```

```

28     - Final model: best model class re-fitted on full training set, evaluated on
29     the holdout set (single, unambiguous performance number)
30
31 4. HONEST REPORTING
32     - Both nested-CV cross-validated estimates (with std across folds)
33     - AND holdout test set numbers (single number, no estimation bias)
34     - Both reported in the dissertation
35 """
36 import json
37 import os
38 import time
39 import warnings
40 from itertools import product
41 from pathlib import Path
42 from typing import List, Tuple
43
44 import numpy as np
45 import pandas as pd
46 import tenseal as ts
47 from sklearn.ensemble import ExtraTreesRegressor, GradientBoostingRegressor,
48     RandomForestRegressor
49 from sklearn.linear_model import Ridge
50 from sklearn.model_selection import GridSearchCV, KFold, cross_val_score,
51     train_test_split
52 from sklearn.neighbors import KNeighborsRegressor
53 from sklearn.pipeline import Pipeline
54 from sklearn.preprocessing import StandardScaler
55 from sklearn.svm import SVR
56
57 warnings.filterwarnings("ignore")
58
59 OUT_DIR = Path("D:/vkr/draft/notebooks/exp1_outputs")
60 OUT_DIR.mkdir(parents=True, exist_ok=True)
61
62 RNG_SEED = 42
63 np.random.seed(RNG_SEED)
64
65 # -- Configuration grid -----
66 # For each (N, chain), the chain must satisfy:
67 #   - first and last bits in [40, 60] for stable rescaling
68 #   - middle bits = scale_bits (the rescale step)
69 #   - sum of bits <= tc128 secure budget for that N

```

```

69 SECURITY_BUDGET = { # max log2(Q) for tc128 / 128-bit security
70     2048: 54,
71     4096: 109,
72     8192: 218,
73     16384: 438,
74 }
75
76 CHAIN_TEMPLATES = [
77     # (description, bit_structure)
78     ("3-mod_balanced", [40, 20, 40]), # chain length 3, total 100
79     ("3-mod_higher", [50, 30, 50]), # chain length 3, total 130
80     ("2-mod_basic", [40, 40]), # chain length 2, total 80
81     ("4-mod_m1", [40, 20, 20, 40]), # chain length 4, total 120
82     ("4-mod_balanced", [50, 30, 30, 50]), # chain length 4, total 160
83     ("3-mod_high", [60, 40, 60]), # chain length 3, total 160
84 ]
85
86
87 def build_grid():
88     configs = []
89     for N in [2048, 4096, 8192, 16384]:
90         n_slots = N // 2
91         budget = SECURITY_BUDGET[N]
92         for chain_name, chain in CHAIN_TEMPLATES:
93             total_bits = sum(chain)
94             if total_bits > budget:
95                 continue
96             # scale must equal the middle modulus
97             scale_bits = chain[1] if len(chain) >= 3 else chain[0]
98             for proj_dim in [32, 64, 128, 192, 256, 320]:
99                 if proj_dim > n_slots:
100                     continue
101                 for batch_size in [16, 64, 256]:
102                     configs.append({
103                         "N": N,
104                         "chain": tuple(chain),
105                         "chain_name": chain_name,
106                         "scale_bits": scale_bits,
107                         "proj_dim": proj_dim,
108                         "batch_size": batch_size,
109                     })
110     return configs
111

```

```

112
113 # -- Microbenchmark -----
114 def measure_config(cfg: dict, n_reps: int = 3) -> Tuple[float, float]:
115     """Measure batch ct-pt rerank time. Returns (mean_ms, std_ms)."""
116     ctx = ts.context(
117         ts.SCHEME_TYPE.CKKS,
118         poly_modulus_degree=cfg["N"],
119         coeff_mod_bit_sizes=list(cfg["chain"]),
120     )
121     ctx.global_scale = 2 ** cfg["scale_bits"]
122     ctx.generate_galois_keys()
123
124     rng = np.random.default_rng(0)
125     q_vec = rng.standard_normal(cfg["proj_dim"]).astype(np.float32) / np.sqrt(cfg[
126         "proj_dim"])
127     docs = [rng.standard_normal(cfg["proj_dim"]).astype(np.float32) / np.sqrt(cfg[
128         "proj_dim"])
129         for _ in range(cfg["batch_size"])]
130
131     times = []
132     for _ in range(n_reps):
133         enc_q = ts.ckks_vector(ctx, q_vec.tolist())
134         t0 = time.perf_counter()
135         for d in docs:
136             score = (enc_q * d.tolist()).sum()
137             _ = score.decrypt()
138             times.append((time.perf_counter() - t0) * 1e3)
139
140     return float(np.mean(times)), float(np.std(times))
141
142 # -- Run microbenchmark -----
143 def measure_all(configs):
144     rows = []
145     for i, cfg in enumerate(configs):
146         try:
147             t0 = time.time()
148             mean_ms, std_ms = measure_config(cfg, n_reps=3)
149             elapsed = time.time() - t0
150         except Exception as e:
151             print(f" [{i+1}/{len(configs)}] SKIP {cfg}: {e}")
152             continue
153     n_slots = cfg["N"] // 2

```

```

153     chain = list(cfg["chain"])
154     rows.append({
155         "N": cfg["N"], "log_N": int(np.log2(cfg["N"])),
156         "num_coeff_moduli": len(chain),
157         "total_coeff_bits": sum(chain),
158         "first_coeff_bit": chain[0],
159         "last_coeff_bit": chain[-1],
160         "scale_bits": cfg["scale_bits"],
161         "scale_to_total_ratio": cfg["scale_bits"] / sum(chain),
162         "proj_dim": cfg["proj_dim"],
163         "batch_size": cfg["batch_size"],
164         "vector_packing_ratio": cfg["proj_dim"] / n_slots,
165         "chain_name": cfg["chain_name"],
166         "batch_time_ms_mean": mean_ms,
167         "batch_time_ms_std": std_ms,
168     })
169     if (i + 1) % 10 == 0 or i == len(configs) - 1:
170         print(f" [{i+1}/{len(configs)}] {cfg['N']}-{cfg['chain_name']}-d{cfg
171             ['proj_dim']}-b{cfg['batch_size']}: "
172             f"{mean_ms:>7.1f}+/-{std_ms:.1f} ms ({elapsed:.1f}s)")
173     return pd.DataFrame(rows)
174
175 # -- ML training with nested CV -----
176 FEATURES = [
177     "log_N", "num_coeff_moduli", "total_coeff_bits",
178     "first_coeff_bit", "last_coeff_bit",
179     "scale_bits", "scale_to_total_ratio",
180     "proj_dim", "batch_size", "vector_packing_ratio",
181 ]
182 TARGET = "batch_time_ms_mean"
183
184
185 def build_models():
186     """Build (model_name, pipeline, hyperparameter_grid_for_inner_cv) tuples."""
187     return [
188         (
189             "Ridge",
190             Pipeline([("scaler", StandardScaler()), ("model", Ridge(random_state=
191                 RNG_SEED))]),
192             {"model__alpha": [0.1, 1.0, 10.0]},
193         )

```

```

194         "KNN",
195         Pipeline([("scaler", StandardScaler()), ("model", KNeighborsRegressor
196         ())),
197         {"model__n_neighbors": [3, 5, 7], "model__weights": ["uniform", "
198         distance"]},
199     ),
200     (
201         "ExtraTrees",
202         Pipeline([("scaler", StandardScaler()),
203         ("model", ExtraTreesRegressor(random_state=RNG_SEED, n_jobs
204         =-1))]),
205         {"model__n_estimators": [100, 200], "model__max_depth": [3, 5, 7, None
206         ]},
207     ),
208     (
209         "GradientBoosting",
210         Pipeline([("scaler", StandardScaler()),
211         ("model", GradientBoostingRegressor(random_state=RNG_SEED))
212         ]),
213         {"model__n_estimators": [50, 100, 200], "model__max_depth": [2, 3,
214         4]},
215     ),
216     (
217         "SVR",
218         Pipeline([("scaler", StandardScaler()), ("model", SVR())]),
219         {"model__C": [1.0, 10.0, 100.0], "model__gamma": ["scale", 0.1]},
220     ),
221 ]
222
223 def nested_cv_evaluate(X, y, model_name, pipeline, param_grid, n_outer=5, n_inner
224 =3):
225     """Run nested CV: outer loop estimates generalisation, inner loop tunes HP."""
226     outer_cv = KFold(n_splits=n_outer, shuffle=True, random_state=RNG_SEED)
227     inner_cv = KFold(n_splits=n_inner, shuffle=True, random_state=RNG_SEED + 1)
228     fold_r2 = []
229     fold_mae = []
230     best_params_per_fold = []
231
232     for fold_idx, (train_idx, test_idx) in enumerate(outer_cv.split(X)):
233         X_tr, X_te = X[train_idx], X[test_idx]
234         y_tr, y_te = y[train_idx], y[test_idx]

```

```

230     gs = GridSearchCV(
231         pipeline, param_grid, cv=inner_cv,
232         scoring="r2", n_jobs=-1, refit=True,
233     )
234     gs.fit(X_tr, y_tr)
235     y_pred = gs.predict(X_te)
236
237     r2 = 1.0 - np.sum((y_te - y_pred) ** 2) / np.sum((y_te - y_te.mean()) **
238 2)
239
240     mae = float(np.mean(np.abs(y_te - y_pred)))
241     fold_r2.append(r2)
242     fold_mae.append(mae)
243     best_params_per_fold.append(gs.best_params_)
244
245     return {
246         "model": model_name,
247         "r2_nested_mean": float(np.mean(fold_r2)),
248         "r2_nested_std": float(np.std(fold_r2)),
249         "mae_nested_mean_ms": float(np.mean(fold_mae)),
250         "mae_nested_std_ms": float(np.std(fold_mae)),
251         "best_params_per_fold": best_params_per_fold,
252     }
253
254 def main():
255     # -- Step 1: build & measure configurations -----
256     print("=== Step 1: building configuration grid ===")
257     configs = build_grid()
258     print(f"Total configurations: {len(configs)}")
259     print(f"Estimated total time: ~{len(configs) * 3 / 60:.0f} min")
260
261     print("\n=== Step 2: microbenchmark ===")
262     df = measure_all(configs)
263     df.to_csv(OUT_DIR / "microbench_v2.csv", index=False)
264     print(f"Saved {len(df)} rows to microbench_v2.csv")
265     print(f"Latency range: [{df[TARGET].min():.1f}, {df[TARGET].max():.1f}] ms, "
266           f"median {df[TARGET].median():.1f} ms")
267
268     # -- Step 3: train/HOLDOUT split -----
269     print("\n=== Step 3: train (80%) / HOLDOUT (20%) split ===")
270     X = df[FEATURES].values.astype(np.float64)
271     y = df[TARGET].values.astype(np.float64)
272     X_train, X_holdout, y_train, y_holdout = train_test_split(

```



```

272     X, y, test_size=0.20, random_state=RNG_SEED,
273 )
274 print(f"  Train: {len(y_train)}  HOLDOUT: {len(y_holdout)}")
275 print(f"  Train target range: [{y_train.min():.0f}, {y_train.max():.0f}] ms")
276 print(f"  Holdout target range: [{y_holdout.min():.0f}, {y_holdout.max():.0f}]
    ms")
277
278 # -- Step 4: nested CV on TRAIN only -----
279 print("\n=== Step 4: nested CV (5 outer x 3 inner) on TRAIN ===")
280 cv_results = []
281 for model_name, pipeline, param_grid in build_models():
282     print(f"  -- {model_name} --")
283     res = nested_cv_evaluate(X_train, y_train, model_name, pipeline,
    param_grid)
284     print(f"      R^2(nested)  = {res['r2_nested_mean']:.4f} +/- {res['
    r2_nested_std']:.4f}")
285     print(f"      MAE(nested) = {res['mae_nested_mean_ms']:.1f} +/- {res['
    mae_nested_std_ms']:.1f} ms")
286     cv_results.append(res)
287
288 cv_df = pd.DataFrame([k: v for k, v in r.items() if k != "
    best_params_per_fold"]
289                      for r in cv_results])
290 cv_df.to_csv(OUT_DIR / "regression_metrics_v2.csv", index=False)
291
292 # -- Step 5: select best, refit on full TRAIN, evaluate on HOLDOUT -----
293 best = max(cv_results, key=lambda r: r["r2_nested_mean"])
294 best_name = best["model"]
295 print(f"\n=== Step 5: best model = {best_name}; refit on full train, evaluate
    on HOLDOUT ===")
296
297 name_to_builder = {n: (p, g) for n, p, g in build_models()}
298 pipeline, param_grid = name_to_builder[best_name]
299 final_gs = GridSearchCV(pipeline, param_grid, cv=KFold(n_splits=5, shuffle=
    True, random_state=RNG_SEED),
300                        scoring="r2", n_jobs=-1, refit=True)
301 final_gs.fit(X_train, y_train)
302 y_holdout_pred = final_gs.predict(X_holdout)
303
304 holdout_r2  = 1.0 - np.sum((y_holdout - y_holdout_pred) ** 2) / np.sum(
305     (y_holdout - y_holdout.mean()) ** 2)
306 holdout_mae = float(np.mean(np.abs(y_holdout - y_holdout_pred)))
307

```

```

308 print(f"  HOLDOUT R^2  = {holdout_r2:.4f}")
309 print(f"  HOLDOUT MAE = {holdout_mae:.1f} ms")
310 print(f"  Best HP: {final_gs.best_params_}")
311
312 # -- Step 6: feature importance -----
313 print("\n=== Step 6: feature importance (final model on full train) ===")
314 if hasattr(final_gs.best_estimator_.named_steps["model"], "
feature_importances_"):
315     importances = final_gs.best_estimator_.named_steps["model"].
feature_importances_
316     feat_imp = sorted(zip(FEATURES, importances), key=lambda x: -x[1])
317     for f, imp in feat_imp:
318         print(f"      {f:<25}: {imp:.4f}")
319     pd.DataFrame(feat_imp, columns=["feature", "importance"]).to_csv(
320         OUT_DIR / "feature_importance_v2.csv", index=False
321     )
322
323 # -- Step 7: save final summary -----
324 summary = {
325     "n_configurations":    int(len(df)),
326     "n_train":            int(len(y_train)),
327     "n_holdout":          int(len(y_holdout)),
328     "latency_range_ms":   [float(y.min()), float(y.max())],
329     "best_model":         best_name,
330     "nested_cv_r2_mean":  best["r2_nested_mean"],
331     "nested_cv_r2_std":   best["r2_nested_std"],
332     "nested_cv_mae_mean_ms": best["mae_nested_mean_ms"],
333     "nested_cv_mae_std_ms": best["mae_nested_std_ms"],
334     "holdout_r2":         float(holdout_r2),
335     "holdout_mae_ms":     float(holdout_mae),
336     "best_hyperparameters": final_gs.best_params_,
337 }
338 with open(OUT_DIR / "regression_metrics_v2.json", "w", encoding="utf-8") as f:
339     json.dump(summary, f, ensure_ascii=False, indent=2)
340 print(f"\nFinal summary:\n{json.dumps(summary, ensure_ascii=False, indent=2)}"
)
341 print(f"\nSaved to {OUT_DIR}")
342
343
344 if __name__ == "__main__":
345     main()

```

Алгоритм FixedParameterOptimizer с ограничением по Acc@1.

Ниже приведён исходный код скрипта `_optimize_with_acc1.py`, реализующего FixedParameterOptimizer с функциональным ограничением Acc@1 (мягкий уровень ≤ 5 п.п. согласован с допуском R1; строгий уровень ≤ 1 п.п. используется в интегральном эксперименте раздела 4.4) в дополнение к ограничению на безопасность ($\lambda \geq 128$). Алгоритм: (1) для каждой (N, chain) конфигурации запускается CKKS noise smoke-тест на синтетических данных при целевой $\text{proj_dim} = 336$; (2) $\max \text{abs error}$ скоринга отображается в предсказанную деградацию Acc@1 через лог-линейную интерполяцию по двум якорным точкам интегрального эксперимента; (3) per-query латентность предсказывается обученной GradientBoosting-моделью; (4) из feasible-набора выбирается минимально-латентная конфигурация. Скрипт замыкает методологический пробел: ранее ограничение Acc@1 декларировалось в тексте, но для конфигураций сетки за пределами двух реальных интегральных прогонов численно не проверялось.

Листинг 2 — Исходный код `_optimize_with_acc1.py`

```
1 """FixedParameterOptimizer with Acc@1 quality constraint.
2
3 The dissertation uses Acc@1 as the primary retrieval quality metric (R1).
4 This script enforces a quality constraint on Acc@1 at two strictness
5 levels (loose 5 pp, strict 1 pp) on top of the security constraint
6 ( $\lambda \geq 128$  bit).
7
8 1. For each (N, chain) combination, run a CKKS noise smoke-test at
9    proj_dim = 192 (canonical  $k = d/2$  for multilingual-e5-small in section 4.4):
10    encrypt one synthetic query, score against 12 random docs in CKKS ct-pt,
11    compute max abs error vs plaintext truth. The chosen winner is robust to
12    higher proj_dim used by larger encoders ( $k$  in {384, 512}).
13 2. Predict Acc@1 degradation from max abs error via log-linear
14    interpolation between two anchor data points:
15    (N=4096, [40,20,40], Delta=220): max_err ~ 0.82 -> Acc@1 = 0.304
16    (N=8192, [60,40,60], Delta=240): max_err ~ 5e-6 -> Acc@1 = 0.784
17 3. Predict per-query rerank latency at proj_dim=336 using the GradientBoosting
18    model trained on microbench_v2.csv (R2_holdout = 0.985).
19 4. Run optimize() with two strictness levels of the Acc@1 constraint
20    (loose 5 pp / strict 1 pp).
21
22 Output: exp1_outputs/optimize_with_acc1_results.json
23 """
24 import json
25 import time
26 from pathlib import Path
```

```

27
28 import numpy as np
29 import pandas as pd
30 import tenseal as ts
31 from sklearn.ensemble import GradientBoostingRegressor
32 from sklearn.model_selection import KFold, GridSearchCV
33 from sklearn.pipeline import Pipeline
34 from sklearn.preprocessing import StandardScaler
35
36 OUT = Path("D:/vkr/draft/notebooks/exp1_outputs")
37 RNG_SEED      = 42
38 PROJ_K_TARGET = 192  # canonical k = d/2 for e5-small (section 4.4)
39 N_DOCS_TEST   = 12
40 N_QUERIES     = 5
41
42 SECURITY_BUDGET = {2048: 54, 4096: 109, 8192: 218, 16384: 438}
43 CHAIN_TEMPLATES = [
44     ("3-mod_balanced", [40, 20, 40]),
45     ("3-mod_higher",   [50, 30, 50]),
46     ("2-mod_basic",    [40, 40]),
47     ("4-mod_ml",       [40, 20, 20, 40]),
48     ("4-mod_balanced", [50, 30, 30, 50]),
49     ("3-mod_high",     [60, 40, 60]),
50 ]
51
52
53 def is_secure(N, chain):
54     return sum(chain) <= SECURITY_BUDGET[N]
55
56
57 def fits_proj_dim(N, proj_dim):
58     return proj_dim <= N // 2
59
60
61 def noise_smoke_test(N, chain, scale_bits, proj_dim,
62                     n_queries=N_QUERIES, n_docs=N_DOCS_TEST, seed=0):
63     """Measure max abs error of CKKS ct-pt scoring on synthetic data."""
64     ctx = ts.context(ts.SCHEME_TYPE.CKKS, poly_modulus_degree=N,
65                     coeff_mod_bit_sizes=list(chain))
66     ctx.global_scale = 2 ** scale_bits
67     ctx.generate_galois_keys()
68     rng = np.random.default_rng(seed)
69     max_errs = []

```

```

70     for _ in range(n_queries):
71         q = rng.standard_normal(proj_dim).astype(np.float32)
72         q /= np.linalg.norm(q)
73         docs = rng.standard_normal((n_docs, proj_dim)).astype(np.float32)
74         docs /= np.linalg.norm(docs, axis=1, keepdims=True)
75         true_scores = (q @ docs.T).astype(np.float64)
76         cq = ts.ckks_vector(ctx, q.tolist())
77         ckks_scores = np.array([
78             float((cq * d.tolist()).sum().decrypt()[0]) for d in docs])
79         max_errs.append(float(np.max(np.abs(true_scores - ckks_scores))))
80     return float(np.mean(max_errs)), float(np.std(max_errs))
81
82
83 def predict_acc1_degradation(max_err):
84     """Log-linear interpolation between (max_err=5e-6, deg=0) and
85     (max_err=0.82, deg=0.478) calibrated from integral-experiment endpoints.
86     Saturates at 0.5 for max_err >> 1."""
87     if max_err <= 1e-5:
88         return 0.0
89     log_err = np.log10(max(max_err, 1e-10))
90     log_lo, log_hi = np.log10(5e-6), np.log10(0.82)
91     deg_lo, deg_hi = 0.0, 0.478
92     if log_err <= log_lo:
93         return 0.0
94     if log_err >= log_hi:
95         return min(0.478 + 0.05 * (log_err - log_hi), 0.5)
96     frac = (log_err - log_lo) / (log_hi - log_lo)
97     return float(deg_lo + frac * (deg_hi - deg_lo))
98
99
100 def train_latency_predictor():
101     """GradientBoosting on microbench_v2.csv with grid-CV."""
102     df = pd.read_csv(OUT / "microbench_v2.csv")
103     features = ["log_N", "num_coeff_moduli", "total_coeff_bits",
104                "first_coeff_bit", "last_coeff_bit", "scale_bits",
105                "scale_to_total_ratio", "proj_dim", "batch_size",
106                "vector_packing_ratio"]
107     X = df[features].values.astype(np.float64)
108     y = df["batch_time_ms_mean"].values.astype(np.float64)
109     pipe = Pipeline([
110         ("scaler", StandardScaler()),
111         ("model", GradientBoostingRegressor(random_state=RNG_SEED)),
112     ])

```

```

113 grid = {"model__n_estimators": [50, 100, 200],
114         "model__max_depth": [2, 3, 4]}
115 cv = KFold(n_splits=5, shuffle=True, random_state=RNG_SEED)
116 gs = GridSearchCV(pipe, grid, cv=cv, scoring="r2", n_jobs=-1, refit=True)
117 gs.fit(X, y)
118 return gs, features
119
120
121 def predict_latency_at(predictor, features, N, chain, proj_dim, batch_size=12):
122     feat = {
123         "log_N": int(np.log2(N)),
124         "num_coeff_moduli": len(chain),
125         "total_coeff_bits": sum(chain),
126         "first_coeff_bit": chain[0],
127         "last_coeff_bit": chain[-1],
128         "scale_bits": chain[1] if len(chain) >= 3 else chain[0],
129         "scale_to_total_ratio": (chain[1] if len(chain) >= 3 else chain[0])
130                                 / sum(chain),
131         "proj_dim": proj_dim,
132         "batch_size": batch_size,
133         "vector_packing_ratio": proj_dim / (N // 2),
134     }
135     X = np.array([[feat[f] for f in features]])
136     return float(predictor.predict(X)[0])
137
138
139 def main():
140     predictor, features = train_latency_predictor()
141     candidates = []
142     for N in [2048, 4096, 8192, 16384]:
143         if not fits_proj_dim(N, PROJ_K_TARGET):
144             continue
145         for chain_name, chain in CHAIN_TEMPLATES:
146             if not is_secure(N, chain):
147                 continue
148             scale_bits = chain[1] if len(chain) >= 3 else chain[0]
149             try:
150                 max_err_mean, max_err_std = noise_smoke_test(
151                     N, chain, scale_bits, PROJ_K_TARGET)
152             except Exception:
153                 continue
154             lat_pred = predict_latency_at(
155                 predictor, features, N, chain, PROJ_K_TARGET, batch_size=12)

```

```

156         acc1_deg = predict_acc1_degradation(max_err_mean)
157         candidates.append({
158             "N": N, "chain": list(chain), "chain_name": chain_name,
159             "scale_bits": scale_bits, "log_Q": sum(chain),
160             "max_err_mean": max_err_mean, "max_err_std": max_err_std,
161             "predicted_acc1_deg": acc1_deg,
162             "predicted_latency_ms": lat_pred,
163         })
164     pd.DataFrame(candidates).to_csv(
165         OUT / "optimize_with_acc1_candidates.csv", index=False)
166
167     def optimize(name, predicate):
168         feasible = [c for c in candidates if predicate(c)]
169         if not feasible:
170             return None
171         best = min(feasible, key=lambda c: c["predicted_latency_ms"])
172         return {"constraint": name,
173               "n_feasible_configs": len(feasible),
174               "best_config": best}
175
176     res_loose = optimize("Loose: Acc@1_deg <= 0.05 (R1 tolerance)",
177                          lambda c: c["predicted_acc1_deg"] <= 0.05)
178     res_strict = optimize("Strict: Acc@1_deg <= 0.01",
179                           lambda c: c["predicted_acc1_deg"] <= 0.01)
180     out_path = OUT / "optimize_with_acc1_results.json"
181     with open(out_path, "w", encoding="utf-8") as f:
182         json.dump({
183             "all_candidates": candidates,
184             "constraints": {"loose_acc1": res_loose,
185                             "strict_acc1": res_strict},
186             "calibration": {
187                 "anchor_lo_res": {"max_err": 0.82, "Acc@1_deg": 0.478},
188                 "anchor_hi_res": {"max_err": 5e-6, "Acc@1_deg": 0.000},
189                 "threshold_strict_acc1_deg_max": 0.01,
190                 "threshold_loose_acc1_deg_max": 0.05,
191             },
192         }, f, indent=2, ensure_ascii=False)
193
194
195 if __name__ == "__main__":
196     main()

```

Приложение Б

Эксперимент 2. Сравнение режимов CKKS ct-pt vs ct-ct (Эксп №1, разд. 4.3)

Полный исходный код ноутбука `exp2_ckks_modes.ipynb` приведён ниже. Файлы результатов сохраняются в директорию `exp2_outputs/`.

Листинг 3 — Исходный код ноутбука `exp2_ckks_modes.ipynb`

```
1 # ----- Cell 1 -----
2 # Локальная установка/проверка зависимостей.
3 # Если зависимости уже установлены в текущем окружении, скрипт ничего не сделает.
4 # При необходимости раскомментируйте строки ниже или установите вручную через pip.
5
6 import importlib
7 import sys
8
9
10 REQUIRED = {
11     # package_name: import_name
12     "tenseal": "tenseal",
13     "numpy": "numpy",
14     "pandas": "pandas",
15     "scikit-learn": "sklearn",
16     "torch": "torch",
17     "transformers": "transformers",
18     "datasets": "datasets",
19     "faiss-cpu": "faiss",
20     "sentence-transformers": "sentence_transformers",
21     "vec2text": "vec2text",
22     "tqdm": "tqdm",
23     "scipy": "scipy",
24     "nltk": "nltk",
25 }
26
27
28 missing = []
29 for pkg, mod in REQUIRED.items():
30     try:
31         importlib.import_module(mod)
32     except ImportError:
33         missing.append(pkg)
34
35 if missing:
```



```

36     print("Отсутствующие пакеты:", missing)
37     print("Установите вручную, например:")
38     print(f"    {sys.executable} -m pip install " + " ".join(missing))
39     print()
40     print("Замечание для GPU NVIDIA RTX 50xx (Blackwell, sm_120):")
41     print("    требуется PyTorch с CUDA 12.8+, например:")
42     print("    pip install --pre torch --index-url https://download.pytorch.org/
    whl/nightly/cu128")
43 else:
44     print("Все зависимости установлены.")
45
46 # ----- Cell 2 -----
47 import json
48 import time
49 import gc
50 from pathlib import Path
51
52 import numpy as np
53 import pandas as pd
54 import tenseal as ts
55
56 OUT_DIR = Path("./exp2_outputs")
57 OUT_DIR.mkdir(parents=True, exist_ok=True)
58
59 # ----- Cell 3 -----
60 # Параметры CKKS (те же что использовались при оптимизации параметров)
61 POLY_DEGREE = 8192
62 COEFF_MOD_BITS = [60, 40, 40, 60]
63 SCALE_BITS = 40
64
65 # Параметры эксперимента
66 N_DOCS = 1_000          # было 10_000; уменьшено для ускорения
67 PROJ_DIM = 256
68
69 # Параметры измерений
70 N_RUNS = 5              # количество различных random seeds
71 N_WARMUP = 5            # прогревочные запуски до измерений
72 N_MEASURE = 10          # число измерительных запусков на один семид
73 SEEDS = list(range(N_RUNS))
74
75 # ----- Cell 4 -----
76 def make_context(poly_degree, coeff_bits, scale_bits):
77     ctx = ts.context(scheme=ts.SCHEME_TYPE.CKKS,

```

```

78         poly_modulus_degree=poly_degree,
79         coeff_mod_bit_sizes=coeff_bits)
80     ctx.global_scale = 2 ** scale_bits
81     ctx.generate_galois_keys()
82     ctx.generate_relin_keys()
83     return ctx
84
85
86 def run_ct_pt(ctx, query, docs):
87     """Режим ct-pt: запрос зашифрован, документы в открытом виде."""
88     t0 = time.perf_counter()
89     cq = ts.ckks_vector(ctx, query.tolist())
90     scores = []
91     for d in docs:
92         # Покомпонентное умножение, затем сумма (скалярное произведение)
93         scores.append(float((cq * d.tolist()).sum().decrypt()[0]))
94     t1 = time.perf_counter()
95     return (t1 - t0) * 1000.0, np.array(scores)
96
97
98 def run_ct_ct(ctx, query, docs):
99     """Режим ct-ct: и запрос, и документы зашифрованы."""
100     t0 = time.perf_counter()
101     cq = ts.ckks_vector(ctx, query.tolist())
102     cdocs = [ts.ckks_vector(ctx, d.tolist()) for d in docs]
103     scores = []
104     for cd in cdocs:
105         product = cq * cd # требует релинеаризации
106         scores.append(float(product.sum().decrypt()[0]))
107     t1 = time.perf_counter()
108     return (t1 - t0) * 1000.0, np.array(scores)
109
110
111 def measure_one(mode_fn, ctx, n_docs, proj_dim, seed, n_warmup, n_measure):
112     rng = np.random.default_rng(seed)
113     query = rng.standard_normal(proj_dim).astype(np.float64)
114     docs = rng.standard_normal((n_docs, proj_dim)).astype(np.float64)
115     # Warmup на маленьком подмножестве
116     for _ in range(n_warmup):
117         _ = mode_fn(ctx, query, docs[:8])
118     times = []
119     for _ in range(n_measure):
120         t, _ = mode_fn(ctx, query, docs)

```

```

121         times.append(t)
122     return times
123
124 # ----- Cell 5 -----
125 import pandas as pd
126 from pathlib import Path
127 from joblib import Parallel, delayed
128
129 OUT_DIR = Path("./exp2_outputs")
130 OUT_DIR.mkdir(parents=True, exist_ok=True)
131
132
133 def _worker_seed(seed, n_docs, proj_dim, poly_degree, coeff_mod_bits,
134                 scale_bits, n_warmup, n_measure):
135     """Self-contained worker: runs ct-pt and ct-ct for one seed."""
136     import numpy as np
137     import tenseal as ts
138     import time
139
140     def _make_ctx():
141         ctx = ts.context(scheme=ts.SCHEME_TYPE.CKKS,
142                         poly_modulus_degree=poly_degree,
143                         coeff_mod_bit_sizes=coeff_mod_bits)
144         ctx.global_scale = 2 ** scale_bits
145         ctx.generate_galois_keys()
146         ctx.generate_relin_keys()
147         return ctx
148
149     def _run_ct_pt(ctx, query, docs):
150         t0 = time.perf_counter()
151         cq = ts.ckks_vector(ctx, query.tolist())
152         scores = []
153         for d in docs:
154             scores.append(float((cq * d.tolist()).sum().decrypt()[0]))
155         return (time.perf_counter() - t0) * 1000.0, np.array(scores)
156
157     def _run_ct_ct(ctx, query, docs):
158         t0 = time.perf_counter()
159         cq = ts.ckks_vector(ctx, query.tolist())
160         cdocs = [ts.ckks_vector(ctx, d.tolist()) for d in docs]
161         scores = []
162         for cd in cdocs:
163             scores.append(float((cq * cd).sum().decrypt()[0]))

```

```

164         return (time.perf_counter() - t0) * 1000.0, np.array(scores)
165
166     def _measure(mode_fn, ctx, n_warmup, n_measure):
167         rng = np.random.default_rng(seed)
168         query = rng.standard_normal(proj_dim).astype(np.float64)
169         docs = rng.standard_normal((n_docs, proj_dim)).astype(np.float64)
170         for _ in range(n_warmup):
171             mode_fn(ctx, query, docs[:8])
172         times = []
173         for _ in range(n_measure):
174             t, _ = mode_fn(ctx, query, docs)
175             times.append(t)
176         return times
177
178     ctx = _make_ctx()
179     t_ctpt = _measure(_run_ct_pt, ctx, n_warmup, n_measure)
180     t_ctct = _measure(_run_ct_ct, ctx, n_warmup, n_measure)
181     rows = []
182     for t in t_ctpt:
183         rows.append({"seed": seed, "mode": "ct-pt", "time_ms": t})
184     for t in t_ctct:
185         rows.append({"seed": seed, "mode": "ct-ct", "time_ms": t})
186     return rows
187
188
189 results = Parallel(n_jobs=5, backend="loky", verbose=10)(
190     delayed(_worker_seed)(
191         seed, N_DOCS, PROJ_DIM, POLY_DEGREE, COEFF_MOD_BITS,
192         SCALE_BITS, N_WARMUP, N_MEASURE
193     )
194     for seed in SEEDS
195 )
196
197 records = [row for seed_rows in results for row in seed_rows]
198 df = pd.DataFrame(records)
199 df.to_csv(OUT_DIR / "exp2_ckks_timing.csv", index=False)
200 print(df.head())
201
202 # ----- Cell 6 -----
203 def ci95_mean(x, n_boot=2000, seed=0):
204     rng = np.random.default_rng(seed)
205     x = np.asarray(x, dtype=float)
206     boot = []

```

```

207     for _ in range(n_boot):
208         idx = rng.integers(0, len(x), size=len(x))
209         boot.append(x[idx].mean())
210     boot = np.array(boot)
211     return float(x.mean()), float(np.quantile(boot, 0.025)), float(np.quantile(
212         boot, 0.975))
213
214 summary = {}
215 for mode in ["ct-pt", "ct-ct"]:
216     sub = df[df["mode"] == mode]["time_ms"].values
217     mean, lo, hi = ci95_mean(sub)
218     summary[mode] = {
219         "n": int(len(sub)),
220         "mean_ms": mean,
221         "std_ms": float(sub.std()),
222         "median_ms": float(np.median(sub)),
223         "ci95_lo_ms": lo,
224         "ci95_hi_ms": hi,
225     }
226
227 # Speedup factor
228 def bootstrap_ratio(a, b, n_boot=2000, seed=1):
229     """Bootstrap-оценка отношения mean(a) / mean(b)."""
230     rng = np.random.default_rng(seed)
231     a = np.asarray(a, dtype=float); b = np.asarray(b, dtype=float)
232     boot = []
233     for _ in range(n_boot):
234         ai = rng.integers(0, len(a), size=len(a))
235         bi = rng.integers(0, len(b), size=len(b))
236         boot.append(a[ai].mean() / b[bi].mean())
237     boot = np.array(boot)
238     return float(a.mean() / b.mean()), float(np.quantile(boot, 0.025)), float(np.
239         quantile(boot, 0.975))
240
241 a = df[df["mode"] == "ct-ct"]["time_ms"].values
242 b = df[df["mode"] == "ct-pt"]["time_ms"].values
243 ratio, ratio_lo, ratio_hi = bootstrap_ratio(a, b)
244 summary["speedup_ratio_ctct_to_ctpt"] = {
245     "point_estimate": ratio,
246     "ci95_lo": ratio_lo,
247     "ci95_hi": ratio_hi,

```

```
248 }
249
250 with open(OUT_DIR / "exp2_ckks_summary.json", "w", encoding="utf-8") as f:
251     json.dump(summary, f, ensure_ascii=False, indent=2)
252
253 print(json.dumps(summary, ensure_ascii=False, indent=2))
```

Приложение В

Эксперимент 3. Прямая атака Vec2Text при known/unknown R (Эксп №3, разд. 4.3)

Полный исходный код ноутбука `exp3_vec2text_attack.ipynb` приведён ниже. Файлы результатов сохраняются в директорию `exp3_outputs/`.

Листинг 4 — Исходный код ноутбука `exp3_vec2text_attack.ipynb`

```
1 # ----- Cell 1 -----
2 # Локальная установка/проверка зависимостей.
3 # Если зависимости уже установлены в текущем окружении, скрипт ничего не сделает.
4 # При необходимости раскомментируйте строки ниже или установите вручную через pip.
5
6 import importlib
7 import sys
8
9
10 REQUIRED = {
11     # package_name: import_name
12     "tenseal": "tenseal",
13     "numpy": "numpy",
14     "pandas": "pandas",
15     "scikit-learn": "sklearn",
16     "torch": "torch",
17     "transformers": "transformers",
18     "datasets": "datasets",
19     "faiss-cpu": "faiss",
20     "sentence-transformers": "sentence_transformers",
21     "vec2text": "vec2text",
22     "tqdm": "tqdm",
23     "scipy": "scipy",
24     "nltk": "nltk",
25 }
26
27
28 missing = []
29 for pkg, mod in REQUIRED.items():
30     try:
31         importlib.import_module(mod)
32     except ImportError:
33         missing.append(pkg)
34
35 if missing:
```

```

36     print("Отсутствующие пакеты:", missing)
37     print("Установите вручную, например:")
38     print(f"    {sys.executable} -m pip install " + " ".join(missing))
39     print()
40     print("Замечание для GPU NVIDIA RTX 50xx (Blackwell, sm_120):")
41     print("    требуется PyTorch с CUDA 12.8+, например:")
42     print("    pip install --pre torch --index-url https://download.pytorch.org/
    whl/nightly/cu128")
43 else:
44     print("Все зависимости установлены.")
45
46 # ----- Cell 2 -----
47 import json
48 import gc
49 from pathlib import Path
50
51 import numpy as np
52 import pandas as pd
53 import torch
54
55 from datasets import load_dataset
56 from sklearn.utils.extmath import randomized_svd
57
58 OUT_DIR = Path("./exp3_outputs")
59 OUT_DIR.mkdir(parents=True, exist_ok=True)
60
61 DEVICE = "cuda" if torch.cuda.is_available() else "cpu"
62 print(f"Device: {DEVICE}")
63
64 # ----- Cell 3 -----
65 # Проверка совместимости PyTorch с GPU
66 import torch
67
68 print(f"torch.__version__ = {torch.__version__}")
69 print(f"CUDA available      = {torch.cuda.is_available()}")
70 if torch.cuda.is_available():
71     print(f"Device name          = {torch.cuda.get_device_name(0)}")
72     cap = torch.cuda.get_device_capability(0)
73     print(f"Compute capability = {cap[0]}.{cap[1]} (sm_{cap[0]}{cap[1]})")
74     if cap[0] >= 12:
75         print("Обнаружена архитектура Blackwell (sm_120+).")
76         print("Если ниже падают ядра, поставьте PyTorch nightly с CUDA 12.8:")

```



```

77     print(" pip install --pre torch --index-url https://download.pytorch.org/
    whl/nightly/cu128")
78     # Пробный тензор на GPU
79     x = torch.randn(4, 4, device="cuda")
80     print("Test tensor on GPU OK:", x.shape)
81 else:
82     print("CUDA недоступен; будет использоваться CPU (значительно медленнее для эн
    кодера).")
83
84 # ----- Cell 4 -----
85 K_FRACTIONS = [1.00, 0.75, 0.50, 0.25, 0.10]    # k/d
86 ROTATION_MODES = ["off", "on"]
87 # When rotation is on, the attacker can either know R (R^T undoes rotation)
88 # or NOT know R (cannot undo rotation, attack on rotated representation).
89 R_KNOWLEDGE_MODES = ["known", "unknown"]
90 N_TEST_TEXTS = 100
91 N_SVD_CORPUS = 5_000
92 EMBED_DIM = 768
93 N_ITER_VEC2TEXT = 10
94 BEAM_WIDTH = 0
95 N_BOOTSTRAP = 500
96 SEED = 42
97
98 torch.manual_seed(SEED)
99 np.random.seed(SEED)
100
101 # ----- Cell 5 -----
102 # Данные для обучения SVD-базиса
103 ag = load_dataset("ag_news", split="train")
104 svd_texts = ag["text"][:N_SVD_CORPUS]
105
106 # Тестовый набор: 50 ag_news + 50 синтетических с PII
107 random_state = np.random.default_rng(SEED)
108 ag_test_indices = random_state.choice(
109     np.arange(N_SVD_CORPUS, len(ag)), size=50, replace=False
110 )
111 ag_test = [ag["text"][int(i)] for i in ag_test_indices]
112
113 PII_TEMPLATES = [
114     "John Smith lives at {addr} in {city}. His phone number is {phone}.",
115     "Patient {name} (DOB {dob}) was diagnosed with {dx} on {date}.",
116     "Card holder {name}, card number {card}, expiration {exp}, CVV {cvv}.",
117     "Employee {name} (SSN {ssn}) works at {company}, salary ${salary} annually.",

```

```

118     "Dr. {name} prescribed {med} ({dose}) for patient {pname}.",
119     "Account holder: {name}. Account number: {acct}. Branch: {branch}.",
120     "{name} sent ${amount} to {recipient} via {bank} on {date}.",
121     "Passport holder {name}, passport no. {pn}, issued {date} in {country}.",
122 ]
123 NAMES = ["Alice Brown", "Bob Smith", "Carol Davis", "David Wilson",
124          "Eve Johnson", "Frank Miller", "Grace Lee", "Henry Martinez"]
125 CITIES = ["Boston", "Chicago", "Denver", "Eugene", "Fresno",
126           "Galveston", "Houston", "Indianapolis"]
127 ADDRS = ["123 Oak Street", "456 Maple Avenue", "789 Pine Road",
128           "321 Elm Boulevard", "654 Birch Lane", "987 Cedar Way"]
129
130
131 def synth_pii(seed: int) -> str:
132     rng = np.random.default_rng(seed)
133     tpl = PII_TEMPLATES[rng.integers(len(PII_TEMPLATES))]
134     fields = {
135         "name": NAMES[rng.integers(len(NAMES))],
136         "pname": NAMES[rng.integers(len(NAMES))],
137         "addr": ADDRS[rng.integers(len(ADDRS))],
138         "city": CITIES[rng.integers(len(CITIES))],
139         "phone": "(%03d) %03d-%04d" % tuple(rng.integers(100, 999, size=3)),
140         "dob": "19%02d-%02d-%02d" % (rng.integers(50, 99), rng.integers(1, 12),
rng.integers(1, 28)),
141         "dx": rng.choice(["hypertension", "diabetes type 2", "asthma", "migraine"
142 ]),
143         "date": "20%02d-%02d-%02d" % (rng.integers(20, 24), rng.integers(1, 12),
rng.integers(1, 28)),
144         "card": " ".join("%04d" % rng.integers(1000, 9999) for _ in range(4)),
145         "exp": "%02d/%02d" % (rng.integers(1, 12), rng.integers(25, 30)),
146         "cvv": "%03d" % rng.integers(100, 999),
147         "ssn": "%03d-%02d-%04d" % (rng.integers(100, 999), rng.integers(10, 99),
rng.integers(1000, 9999)),
148         "company": rng.choice(["Acme Corp", "Globex", "Initech", "Umbrella", "
Soylent", "Stark Inc"]),
149         "salary": "%d,000" % rng.integers(50, 200),
150         "med": rng.choice(["lisinopril", "metformin", "albuterol", "sumatriptan"])
151     },
152     "dose": "%dmg" % rng.choice([5, 10, 20, 50, 100]),
153     "acct": "%010d" % rng.integers(10**8, 10**10),
154     "branch": rng.choice(["Downtown", "Westside", "North Hills", "South End"])
155     ,
156     "amount": "%d.%02d" % (rng.integers(100, 5000), rng.integers(0, 99)),

```

```

154         "recipient": NAMES[rng.integers(len(NAMES))],
155         "bank": rng.choice(["WireBank", "FastTransfer", "PayLink"]),
156         "pn": "%c%c%07d" % (chr(rng.integers(65, 90)), chr(rng.integers(65, 90)),
rng.integers(0, 10**7)),
157         "country": rng.choice(["USA", "UK", "Germany", "Canada"]),
158     }
159     return tpl.format(**fields)
160
161
162 pii_test = [synth_pii(SEED + i) for i in range(50)]
163 test_texts = ag_test + pii_test
164 print(f"SVD corpus: {len(svd_texts)} texts, test corpus: {len(test_texts)} texts")
165 print("Sample PII text:", pii_test[0])
166
167 # ----- Cell 6 -----
168 import vec2text
169
170 # Загрузим официальный corrector для gtr-base + msmarco
171 corrector = vec2text.load_pretrained_corrector("gtr-base")
172 encoder = corrector.embedder # GTR-base encoder
173 tokenizer = corrector.embedder_tokenizer
174
175 encoder.eval()
176 encoder.to(DEVICE)
177
178
179 @torch.no_grad()
180 def encode(texts, batch_size=32, max_length=128):
181     """Возвращает np.ndarray (N, 768) с $L_2$-нормализацией."""
182     embs = []
183     for i in range(0, len(texts), batch_size):
184         chunk = texts[i:i + batch_size]
185         inputs = tokenizer(chunk, return_tensors="pt", truncation=True,
padding=True, max_length=max_length).to(DEVICE)
186         output = encoder(**inputs)
187         # Vec2Text использует mean pooling по non-padding tokens
188         mask = inputs["attention_mask"].unsqueeze(-1)
189         pooled = (output.last_hidden_state * mask).sum(dim=1) / mask.sum(dim=1).
clamp(min=1)
190         pooled = torch.nn.functional.normalize(pooled, dim=-1)
191         embs.append(pooled.cpu().numpy())
192     return np.concatenate(embs, axis=0)
193
194

```

```

195
196 print("Encoding SVD corpus...")
197 E_svd = encode(svd_texts, batch_size=32)
198 print("E_svd shape:", E_svd.shape)
199 print("Encoding test corpus...")
200 E_test = encode(test_texts, batch_size=32)
201 print("E_test shape:", E_test.shape)
202
203 # ----- Cell 7 -----
204 mu = E_svd.mean(axis=0, keepdims=True)
205 E_centered = E_svd - mu
206
207 print(f"Computing randomized SVD (n_components={EMBED_DIM}) ...")
208 _, _, Vt = randomized_svd(E_centered, n_components=EMBED_DIM, random_state=SEED)
209 V = Vt.T # (d, EMBED_DIM)
210
211
212 def make_rotation(k: int, seed: int = 7) -> np.ndarray:
213     rng = np.random.default_rng(seed)
214     A = rng.standard_normal((k, k))
215     Q, _ = np.linalg.qr(A)
216     return Q # orthogonal k x k
217
218
219 def project(E, k, rotation: bool, seed_rot: int = 7):
220     k_eff = min(k, V.shape[1])
221     Vk = V[:, :k_eff]
222     Ec = E - mu
223     Eproj = Ec @ Vk
224     if rotation:
225         R = make_rotation(k_eff, seed=seed_rot)
226         Eproj = Eproj @ R
227     return Eproj, Vk, (R if rotation else None)
228
229
230 def reconstruct(E_proj, Vk, R, R_known: bool = True):
231     """Back-project to original d-dim. If R is not None and R_known=False,
232     the attacker does NOT undo rotation (treats rotated proj as if no rotation).
233     """
234     if R is not None and R_known:
235         E_proj = E_proj @ R.T
236     Erec = E_proj @ Vk.T
237     Erec = Erec + mu

```

```

238     Erec = Erec / (np.linalg.norm(Erec, axis=1, keepdims=True) + 1e-9)
239     return Erec
240
241 # ----- Cell 8 -----
242 @torch.no_grad()
243 def attack(emb_recovered: np.ndarray, n_iter: int = N_ITER_VEC2TEXT,
244           beam_width: int = BEAM_WIDTH):
245     """Batched vec2text inversion with VRAM guard (mini-batches of 8)."""
246     import torch
247     mini_bs = 8
248     results = []
249     for start in range(0, len(emb_recovered), mini_bs):
250         batch = emb_recovered[start:start + mini_bs]
251         emb_t = torch.tensor(batch, dtype=torch.float32, device=DEVICE)
252         recovered = vec2text.invert_embeddings(
253             embeddings=emb_t,
254             corrector=corrector,
255             num_steps=n_iter,
256             sequence_beam_width=beam_width,
257         )
258         results.extend(recovered)
259         torch.cuda.empty_cache()
260     return results
261
262 # ----- Cell 9 -----
263 from nltk.translate.bleu_score import sentence_bleu, SmoothingFunction
264 from nltk.tokenize import word_tokenize
265
266 smoother = SmoothingFunction().method1
267
268
269 def bleu(reference: str, hypothesis: str) -> float:
270     ref = [word_tokenize(reference.lower())]
271     hyp = word_tokenize(hypothesis.lower())
272     if not hyp:
273         return 0.0
274     return sentence_bleu(ref, hyp, smoothing_function=smoother)
275
276
277 def token_overlap(reference: str, hypothesis: str) -> float:
278     a = set(word_tokenize(reference.lower()))
279     b = set(word_tokenize(hypothesis.lower()))
280     if not a and not b:

```

```

281         return 1.0
282     if not a or not b:
283         return 0.0
284     return len(a & b) / len(a | b)
285
286
287 def exact_match(reference: str, hypothesis: str) -> int:
288     return int(reference.strip() == hypothesis.strip())
289
290 # ----- Cell 10 -----
291 records = []
292 total = len(K_FRACTIONS) * (1 + len(R_KNOWLEDGE_MODES)) # off + 2 rotation
293     variants
294 done = 0
295 for kf in K_FRACTIONS:
296     k = max(1, int(round(kf * EMBED_DIM)))
297
298     # --- rotation=off (R is None; R_knowledge irrelevant) ---
299     Eproj, Vk, _ = project(E_test, k, rotation=False)
300     Erec = reconstruct(Eproj, Vk, None, R_known=True)
301     done += 1
302     print(f"[{done}/{total}] k={k} (k/d={kf:.2f}), rotation=off, R_knowledge=n/a")
303     recovered_texts = attack(Erec)
304     for i, (orig, rec) in enumerate(zip(test_texts, recovered_texts)):
305         records.append({
306             "k_fraction": kf, "k": k,
307             "rotation": "off", "R_knowledge": "n/a",
308             "sample_id": i, "is_pii": int(i >= 50),
309             "bleu": bleu(orig, rec),
310             "token_overlap": token_overlap(orig, rec),
311             "exact_match": exact_match(orig, rec),
312         })
313     pd.DataFrame(records).to_csv(OUT_DIR / "exp3_attack_results.csv", index=False)
314
315     # --- rotation=on, two variants of R-knowledge ---
316     Eproj_r, Vk_r, R_mat = project(E_test, k, rotation=True)
317     for r_known_label in R_KNOWLEDGE_MODES:
318         r_known = (r_known_label == "known")
319         Erec = reconstruct(Eproj_r, Vk_r, R_mat, R_known=r_known)
320         done += 1
321         print(f"[{done}/{total}] k={k} (k/d={kf:.2f}), rotation=on, R_knowledge={r_known_label}")
322     recovered_texts = attack(Erec)

```

```

322         for i, (orig, rec) in enumerate(zip(test_texts, recovered_texts)):
323             records.append({
324                 "k_fraction": kf, "k": k,
325                 "rotation": "on", "R_knowledge": r_known_label,
326                 "sample_id": i, "is_pii": int(i >= 50),
327                 "bleu": bleu(orig, rec),
328                 "token_overlap": token_overlap(orig, rec),
329                 "exact_match": exact_match(orig, rec),
330             })
331     pd.DataFrame(records).to_csv(OUT_DIR / "exp3_attack_results.csv", index=
False)
332
333 df = pd.DataFrame(records)
334 print(f"Saved {len(df)} rows to exp3_attack_results.csv")
335
336 # ----- Cell 11 -----
337 def bootstrap_ci(values, n_boot=N_BOOTSTRAP, seed=0):
338     rng = np.random.default_rng(seed)
339     values = np.asarray(values, dtype=float)
340     if len(values) == 0:
341         return 0.0, 0.0, 0.0
342     boot = []
343     for _ in range(n_boot):
344         idx = rng.integers(0, len(values), size=len(values))
345         boot.append(values[idx].mean())
346     boot = np.array(boot)
347     return float(values.mean()), float(np.quantile(boot, 0.025)), float(np.
quantile(boot, 0.975))
348
349
350 summary = {"by_kfraction_rotation_rknown": []}
351 for kf in K_FRACTIONS:
352     # rotation=off (R_knowledge=n/a)
353     sub = df[(df["k_fraction"] == kf) & (df["rotation"] == "off")]
354     if len(sub) > 0:
355         m_b, lo_b, hi_b = bootstrap_ci(sub["bleu"].values, seed=int(kf*1000))
356         m_t, lo_t, hi_t = bootstrap_ci(sub["token_overlap"].values, seed=int(kf
*1000)+100)
357         summary["by_kfraction_rotation_rknown"].append({
358             "k_fraction": kf, "rotation": "off", "R_knowledge": "n/a",
359             "n": int(len(sub)),
360             "bleu_mean": m_b, "bleu_ci95_lo": lo_b, "bleu_ci95_hi": hi_b,

```

```

361         "token_overlap_mean": m_t, "token_overlap_ci95_lo": lo_t, "
token_overlap_ci95_hi": hi_t,
362         "exact_match_rate": float(sub["exact_match"].mean()),
363     })
364     # rotation=on, both R_knowledge modes
365     for r_known_label in ["known", "unknown"]:
366         sub = df[(df["k_fraction"] == kf) & (df["rotation"] == "on") & (df["
R_knowledge"] == r_known_label)]
367         if len(sub) > 0:
368             sd = int(kf*1000) + (200 if r_known_label == "known" else 300)
369             m_b, lo_b, hi_b = bootstrap_ci(sub["bleu"].values, seed=sd)
370             m_t, lo_t, hi_t = bootstrap_ci(sub["token_overlap"].values, seed=sd
+50)
371             summary["by_kfraction_rotation_rknown"].append({
372                 "k_fraction": kf, "rotation": "on", "R_knowledge": r_known_label,
373                 "n": int(len(sub)),
374                 "bleu_mean": m_b, "bleu_ci95_lo": lo_b, "bleu_ci95_hi": hi_b,
375                 "token_overlap_mean": m_t, "token_overlap_ci95_lo": lo_t, "
token_overlap_ci95_hi": hi_t,
376                 "exact_match_rate": float(sub["exact_match"].mean()),
377             })
378
379 with open(OUT_DIR / "exp3_summary.json", "w", encoding="utf-8") as f:
380     json.dump(summary, f, ensure_ascii=False, indent=2)
381 print(json.dumps(summary, ensure_ascii=False, indent=2))

```


Приложение Г

Эксперимент 4. Покомпонентный анализ защиты (Эксп №2, разд. 4.3)

Полный исходный код ноутбука `exp4_ablation_components.ipynb` приведён ниже. Файлы результатов сохраняются в директорию `exp4_outputs/`.

Листинг 5 — Исходный код ноутбука `exp4_ablation_components.ipynb`

```
1 # ----- Cell 1 -----
2 # Локальная установка/проверка зависимостей.
3 # Если зависимости уже установлены в текущем окружении, скрипт ничего не сделает.
4 # При необходимости раскомментируйте строки ниже или установите вручную через pip.
5
6 import importlib
7 import sys
8
9
10 REQUIRED = {
11     # package_name: import_name
12     "tenseal": "tenseal",
13     "numpy": "numpy",
14     "pandas": "pandas",
15     "scikit-learn": "sklearn",
16     "torch": "torch",
17     "transformers": "transformers",
18     "datasets": "datasets",
19     "faiss-cpu": "faiss",
20     "sentence-transformers": "sentence_transformers",
21     "vec2text": "vec2text",
22     "tqdm": "tqdm",
23     "scipy": "scipy",
24     "nltk": "nltk",
25 }
26
27
28 missing = []
29 for pkg, mod in REQUIRED.items():
30     try:
31         importlib.import_module(mod)
32     except ImportError:
33         missing.append(pkg)
34
35 if missing:
```

```

36     print("Отсутствующие пакеты:", missing)
37     print("Установите вручную, например:")
38     print(f"    {sys.executable} -m pip install " + " ".join(missing))
39     print()
40     print("Замечание для GPU NVIDIA RTX 50xx (Blackwell, sm_120):")
41     print("    требуется PyTorch с CUDA 12.8+, например:")
42     print("    pip install --pre torch --index-url https://download.pytorch.org/
    whl/nightly/cu128")
43 else:
44     print("Все зависимости установлены.")
45
46 # ----- Cell 2 -----
47 import json, time, gc
48 from pathlib import Path
49
50 import numpy as np
51 import pandas as pd
52 import torch
53 import faiss
54 import tenseal as ts
55
56 from datasets import load_dataset
57 from transformers import AutoTokenizer, AutoModel
58 from sklearn.utils.extmath import randomized_svd
59 from tqdm.auto import tqdm
60
61 OUT_DIR = Path("./exp4_outputs")
62 OUT_DIR.mkdir(parents=True, exist_ok=True)
63 DEVICE = "cuda" if torch.cuda.is_available() else "cpu"
64 print(f"Device: {DEVICE}")
65
66 # ----- Cell 3 -----
67 # Проверка совместимости PyTorch с GPU
68 import torch
69
70 print(f"torch.__version__ = {torch.__version__}")
71 print(f"CUDA available      = {torch.cuda.is_available()}")
72 if torch.cuda.is_available():
73     print(f"Device name          = {torch.cuda.get_device_name(0)}")
74     cap = torch.cuda.get_device_capability(0)
75     print(f"Compute capability = {cap[0]}.{cap[1]} (sm_{cap[0]}{cap[1]})")
76     if cap[0] >= 12:
77         print("Обнаружена архитектура Blackwell (sm_120+).")

```

```

78         print("Если ниже падают ядра, поставьте PyTorch nightly с CUDA 12.8:")
79         print("  pip install --pre torch --index-url https://download.pytorch.org/
whl/nightly/cu128")
80     # Пробный тензор на GPU
81     x = torch.randn(4, 4, device="cuda")
82     print("Test tensor on GPU OK:", x.shape)
83 else:
84     print("CUDA недоступен; будет использоваться CPU (значительно медленнее для эн
кодера).")
85
86 # ----- Cell 4 -----
87 N_DOCS = 30_000          # было 100_000; уменьшено для ускорения
88 N_QUERIES = 200          # было 500
89 EMBED_DIM = 312         # rubert-tiny2
90 PROJ_K = 32
91 TOP_K_RETRIEVE = 64     # для кандидатного пула второго уровня
92 SEEDS = [11, 23, 47]    # было [11, 23, 47, 59, 73]
93 N_BOOTSTRAP = 500       # было 1000
94
95 # ----- Cell 5 -----
96 ds = load_dataset("wikimedia/wikipedia", "20231101.ru", split="train", streaming=
True)
97 texts = []
98 for row in tqdm(ds, total=N_DOCS + N_QUERIES):
99     if row.get("text") and len(row["text"]) > 200:
100         texts.append((row.get("title", ""), row["text"]))
101     if len(texts) >= N_DOCS + N_QUERIES:
102         break
103
104 def split_paragraphs(t):
105     parts = [p.strip() for p in t.splitlines() if 80 <= len(p.strip()) <= 1000]
106     return parts
107
108 query_doc_pairs = []
109 docs = []
110 doc_text_to_id = {}
111 for title, body in texts:
112     paragraphs = split_paragraphs(body)
113     if not paragraphs:
114         continue
115     for p in paragraphs[:3]:
116         if p not in doc_text_to_id:
117             doc_text_to_id[p] = len(docs)

```

```

118         docs.append(p)
119     if len(query_doc_pairs) < N_QUERIES and title:
120         target_doc_id = doc_text_to_id[paragraphs[0]]
121         query_doc_pairs.append((title, target_doc_id))
122     if len(docs) >= N_DOCS:
123         break
124
125 print(f"Corpus size: {len(docs)} documents")
126 print(f"Queries: {len(query_doc_pairs)}")
127 print("Sample query:", query_doc_pairs[0][0])
128 print("Sample target doc:", docs[query_doc_pairs[0][1][:200], "...")
129
130 # ----- Cell 6 -----
131 tokenizer = AutoTokenizer.from_pretrained("cointegrated/rubert-tiny2")
132 encoder = AutoModel.from_pretrained("cointegrated/rubert-tiny2").to(DEVICE).eval()
133
134
135 @torch.no_grad()
136 def encode(texts, batch_size=64, max_length=128):
137     embs = []
138     for i in tqdm(range(0, len(texts), batch_size)):
139         chunk = texts[i:i + batch_size]
140         inputs = tokenizer(chunk, return_tensors="pt", truncation=True,
141                             padding=True, max_length=max_length).to(DEVICE)
142         output = encoder(**inputs)
143         cls = output.last_hidden_state[:, 0, :]
144         cls = torch.nn.functional.normalize(cls, dim=-1)
145         embs.append(cls.cpu().numpy())
146     return np.concatenate(embs, axis=0)
147
148
149 print("Encoding documents...")
150 E_docs = encode(docs)
151 print("E_docs shape:", E_docs.shape)
152 print("Encoding queries...")
153 E_queries = encode([q for q, _ in query_doc_pairs])
154 print("E_queries shape:", E_queries.shape)
155
156 target_ids = np.array([t for _, t in query_doc_pairs])
157
158 # ----- Cell 7 -----
159 mu = E_docs.mean(axis=0, keepdims=True)
160 print("Computing SVD ...")

```

```

161 _, _, Vt = randomized_svd(E_docs - mu, n_components=PROJ_K, random_state=42)
162 V_k = Vt.T
163
164
165 def make_rotation(seed):
166     rng = np.random.default_rng(seed)
167     A = rng.standard_normal((PROJ_K, PROJ_K))
168     Q, _ = np.linalg.qr(A)
169     return Q
170
171
172 def project(E, rotation_R=None):
173     p = (E - mu) @ V_k
174     if rotation_R is not None:
175         p = p @ rotation_R
176     return p
177
178 # ----- Cell 8 -----
179 def make_ckks_context():
180     ctx = ts.context(scheme=ts.SCHEME_TYPE.CKKS,
181                      poly_modulus_degree=4096,
182                      coeff_mod_bit_sizes=[40, 20, 40])
183     ctx.global_scale = 2 ** 20
184     ctx.generate_galois_keys()
185     return ctx
186
187
188 def ckks_rerank(query_proj, candidate_projs, ctx):
189     cq = ts.ckks_vector(ctx, query_proj.tolist())
190     scores = []
191     for d in candidate_projs:
192         scores.append(float((cq * d.tolist()).sum().decrypt()[0]))
193     return np.array(scores)
194
195
196 def _ckks_score_query(i, query_vec, cand_vecs,
197                       poly_degree, coeff_bits, scale_bits):
198     """Loky worker: encrypt query, score candidates, return (i, scores)."""
199     import numpy as np
200     import tenseal as ts
201     ctx = ts.context(scheme=ts.SCHEME_TYPE.CKKS,
202                      poly_modulus_degree=poly_degree,
203                      coeff_mod_bit_sizes=coeff_bits)

```

```

204     ctx.global_scale = 2 ** scale_bits
205     ctx.generate_galois_keys()
206     cq = ts.ckks_vector(ctx, query_vec.tolist())
207     scores = []
208     for d in cand_vecs:
209         scores.append(float((cq * d.tolist()).sum().decrypt()[0]))
210     return i, np.array(scores)
211
212 # ----- Cell 9 -----
213 def accuracy_at_k(predicted_ids, target_ids, k):
214     return float(np.mean([t in p[:k] for p, t in zip(predicted_ids, target_ids)]))
215
216
217 def mrr(predicted_ids, target_ids):
218     rrs = []
219     for p, t in zip(predicted_ids, target_ids):
220         try:
221             rrs.append(1.0 / (list(p).index(t) + 1))
222         except ValueError:
223             rrs.append(0.0)
224     return float(np.mean(rrs))
225
226
227 def bootstrap_ci_metric(predicted_ids, target_ids, fn, n_boot=N_BOOTSTRAP, seed=0)
228     :
229     rng = np.random.default_rng(seed)
230     n = len(target_ids)
231     boot = []
232     for _ in range(n_boot):
233         idx = rng.integers(0, n, size=n)
234         boot.append(fn([predicted_ids[i] for i in idx], [target_ids[i] for i in
235             idx]))
236     boot = np.array(boot)
237     return float(fn(predicted_ids, target_ids)), float(np.quantile(boot, 0.025)),
238         float(np.quantile(boot, 0.975))
239
240 # ----- Cell 10 -----
241 import time
242 from joblib import Parallel, delayed
243
244 _CKKS_POLY = 4096
245 _CKKS_BITS = [40, 20, 40]
246 _CKKS_SCALE = 20

```

```

244
245
246 def evaluate(config_name, seed,
247               use_svd: bool, use_rotation: bool, use_ckks: bool):
248     R = make_rotation(seed) if use_rotation else None
249
250     if use_svd:
251         E_docs_eff = project(E_docs, rotation_R=R)
252         E_q_eff = project(E_queries, rotation_R=R)
253     else:
254         E_docs_eff = E_docs
255         E_q_eff = E_queries
256
257     # Уровень 1: быстрый поиск
258     index = faiss.IndexFlatIP(E_docs_eff.shape[1])
259     index.add(E_docs_eff.astype(np.float32))
260
261     t0 = time.perf_counter()
262     D_l1, I_l1 = index.search(E_q_eff.astype(np.float32), TOP_K_RETRIEVE)
263     t_l1 = (time.perf_counter() - t0) * 1000.0 / len(E_q_eff)
264
265     if use_ckks:
266         # Уровень 2: CKKS-переранжирование с параллельными запросами (loky)
267         per_query_lat = [0.0] * len(E_q_eff)
268         I_final = np.zeros_like(I_l1)
269
270         results = Parallel(n_jobs=8, backend="loky")(
271             delayed(_ckks_score_query)(
272                 i,
273                 E_q_eff[i],
274                 E_docs_eff[I_l1[i]],
275                 _CKKS_POLY, _CKKS_BITS, _CKKS_SCALE,
276             )
277             for i in range(len(E_q_eff))
278         )
279         for i, scores in results:
280             order = np.argsort(-scores)
281             I_final[i] = I_l1[i][order]
282         latency_p95 = float(np.percentile(per_query_lat, 95)) + t_l1
283     else:
284         I_final = I_l1
285         latency_p95 = t_l1
286

```

```

287     a1, a1_lo, a1_hi = bootstrap_ci_metric(I_final, target_ids,
288                                         lambda p, t: accuracy_at_k(p, t, 1),
289                                         seed=seed)
290     a10, a10_lo, a10_hi = bootstrap_ci_metric(I_final, target_ids,
291                                         lambda p, t: accuracy_at_k(p, t, 10)
292                                         ,
293                                         seed=seed + 1)
294     mrr_val, mrr_lo, mrr_hi = bootstrap_ci_metric(I_final, target_ids,
295                                         mrr, seed=seed + 2)
296     return {
297         "config": config_name,
298         "seed": seed,
299         "acc1": a1, "acc1_ci_lo": a1_lo, "acc1_ci_hi": a1_hi,
300         "acc10": a10, "acc10_ci_lo": a10_lo, "acc10_ci_hi": a10_hi,
301         "mrr": mrr_val, "mrr_ci_lo": mrr_lo, "mrr_ci_hi": mrr_hi,
302         "latency_p95_ms": latency_p95,
303     }
304
305 CONFIGS = [
306     ("baseline_dense", dict(use_svd=False, use_rotation=False, use_ckks=False)),
307     ("svd_only", dict(use_svd=True, use_rotation=False, use_ckks=False)),
308     ("svd_rotation", dict(use_svd=True, use_rotation=True, use_ckks=False)),
309     ("svd_ckks_no_rot", dict(use_svd=True, use_rotation=False, use_ckks=True)),
310     ("full", dict(use_svd=True, use_rotation=True, use_ckks=True)),
311 ]
312
313 records = []
314 for name, kw in CONFIGS:
315     for seed in SEEDS:
316         print(f"-- config={name}, seed={seed}")
317         rec = evaluate(name, seed, **kw)
318         records.append(rec)
319     pd.DataFrame(records).to_csv(OUT_DIR / "exp4_ablation_results.csv", index=
320     False)
321
322 print("Done. Saved to exp4_ablation_results.csv")
323 print(pd.DataFrame(records).groupby("config")[["acc1", "acc10", "mrr", "
324         latency_p95_ms"]].agg(["mean", "std"]))

```


Приложение Д

Эксперимент 5. Интегральный двухстадийный конвейер: client-side PQ + server-side CKKS-перанкинг (Эксп №4, разд. 4.4)

Исходный код Python-скрипта `_rerun_exp5_v3_multi.py` для интегрального эксперимента §4.4 приведён ниже. Скрипт реализует двухстадийный протокол client-side PQ-фильтрация в ротированном пространстве + server-side CKKS-перанкинг 40 кандидатов в режиме ct-pt и поддерживает кросс-энкодерный замер на пяти retrieval-моделях (multilingual-e5-small, multilingual-e5-base, paraphrase-multilingual-mpnet-base-v2, multilingual-e5-large, BAAI/bge-m3) с параметризованным выбором k . В основной операционной точке используется $k = d/2$ с $M = k/4$, что обеспечивает $\sigma_{rec} \geq 0,10$ для всех протестированных энкодеров. PQ-индекс строится на ротированных эмбедингах $E_{rot} = E_{proj}R^T$, что устраняет серверную атаку Прокруста на восстановление R . Per-seed PQ-индекс (так как при разных R кодбук различается). CKKS-конфигурация: $N = 8192$, $[60, 40, 60]$, $\Delta = 2^{40}$. На каждом из 5 сидов ротации ($R \in O(k)$, client-secret) прогоняются все 500 self-retrieval-запросов. Результаты сохраняются в `exp5_outputs/v3_multi_results.json` (основная точка $k = d/2$) и `v3_multi_results_highprot.json` / `v3_multi_results.json` в зависимости от выбора режима.

Листинг 6 — Исходный код `_rerun_exp5_v3_multi.py`

```
1 """Multi-encoder integral experiment with PQ + CKKS architecture (v3).
2
3 CHANGE FROM v2: PQ index is trained in the ROTATED space (E_rot), not in
4 the V_k-projected space. This closes the server-side Procrustes attack on R:
5 in v2, server had E_rot exactly + PQ-decoded approximations of E_proj,
6 allowing it to solve arg min_Q ||hat{E_proj} Q^T - E_rot||_F. In v3, both
7 PQ artifact and server's E_rot live in the same rotated space.
8
9 Per-seed PQ index (since R differs between seeds).
10
11 Runs the proposed scheme across 5 encoders in the canonical operating
12 point k = d/2:
13     multilingual-e5-small (d=384, k=192), multilingual-e5-base (d=768, k=384),
14     paraphrase-multilingual-mpnet-base-v2 (d=768, k=384),
15     multilingual-e5-large (d=1024, k=512), BAAI/bge-m3 (d=1024, k=512).
16 The auxiliary k = 7d/8 measurement uses the same script with the non_half
17 encoder tags.
18
19 Architecture:
20     1. SVD V_k (k = 7d/8) computed offline, client-secret along with mu.
21     2. Per-deployment rotation R, CLIENT-secret.
```

```

22 3. Public artifact: PQ codebook + per-doc PQ codes IN ROTATED SPACE
23    (faiss IndexPQ trained on  $E_{rot} = E_{proj} @ R^T$ ).
24 4. Per-query: client computes  $q_{proj}$ ,  $q_{tilde} = q_{proj} @ R^T$ ,
25    runs PQ search using  $q_{tilde}$  (in rotated space) for top-K_CANDS,
26    encrypts  $q_{tilde}$ , sends  $ct + cand\_ids$  to server.
27 5. Server: CKKS  $ct-pt$  rerank against  $E_{rot}[cand\_ids]$ , returns scores.
28 6. Client: decrypts, ranks, returns top-10.
29
30 Output: exp5_outputs/v3_multi_results.json
31 """
32 import json
33 import os
34 import time
35 from pathlib import Path
36
37 import numpy as np
38 import faiss
39 import tenseal as ts
40 from sklearn.utils.extmath import randomized_svd
41 from tqdm import tqdm
42
43 CACHE = Path("D:/vkr/draft/notebooks/_corpus_cache")
44 INDEX_CACHE = CACHE / "index_cache"
45 INDEX_CACHE.mkdir(exist_ok=True)
46 OUT = Path("D:/vkr/draft/notebooks/exp5_outputs")
47 OUT.mkdir(exist_ok=True)
48
49 SEED_BASE = 42
50 SEEDS = [11, 23, 47, 31, 53]
51 T_CRIT_95 = {3: 4.303, 4: 3.182, 5: 2.776}
52
53 # CKKS hi-res config (3-mod chain, one ct-pt mul per query)
54 CKKS_N = 8192
55 CKKS_COEFF = [60, 40, 60]
56 CKKS_SCALE_BITS = 40
57
58 TOP_K = 12
59 K_CANDS = int(os.environ.get("V2_K_CANDS", 40))
60 PQ_NBITS = 8
61
62
63 def acc_at_k(pred, rel, k):
64     return float(bool(set(pred[:k]) & rel))

```

```

65
66
67 def mrr_score(pred, rel):
68     for r, p in enumerate(pred, 1):
69         if p in rel:
70             return 1.0 / r
71     return 0.0
72
73
74 def ndcg_at_k(pred, rel, k=10):
75     return sum(1.0 / np.log2(i + 2) for i, p in enumerate(pred[:k]) if p in rel)
76
77
78 def aggregate(preds, qrels, fn):
79     return float(np.mean([fn(p, r) for p, r in zip(preds, qrels)]))
80
81
82 def metrics(preds, qrels):
83     return {
84         "acc1": aggregate(preds, qrels, lambda p, r: acc_at_k(p, r, 1)),
85         "acc10": aggregate(preds, qrels, lambda p, r: acc_at_k(p, r, 10)),
86         "mrr": aggregate(preds, qrels, mrr_score),
87         "ndcg10": aggregate(preds, qrels, ndcg_at_k),
88     }
89
90
91 def t_ci(values):
92     arr = np.asarray(values, dtype=np.float64)
93     n = len(arr)
94     if n < 2:
95         return float(arr.mean()), 0.0, 0.0
96     mean = float(arr.mean())
97     std = float(arr.std(ddof=1))
98     t = T_CRIT_95.get(n, 1.96)
99     return mean, std, float(t * std / np.sqrt(n))
100
101
102 def make_random_orthogonal(seed, dim):
103     rng = np.random.default_rng(seed)
104     A = rng.standard_normal((dim, dim))
105     Q, _ = np.linalg.qr(A)
106     return Q.astype(np.float32)
107

```

```

108
109 def compute_svd(E, k):
110     mu = E.mean(axis=0, keepdims=True).astype(np.float32)
111     rng = np.random.RandomState(0)
112     n_sample = min(200_000, len(E))
113     idx = rng.choice(len(E), size=n_sample, replace=False)
114     _, _, Vt = randomized_svd(E[idx] - mu, n_components=k, random_state=42)
115     return mu, Vt.T.astype(np.float32)
116
117
118 def build_or_load_pq_rotated(E_rot, m, nbits, encoder_tag, proj_k, seed):
119     """v3: PQ trained in the ROTATED space E_rot (per-seed, depends on R).
120
121     Closes the server-side Procrustes attack: in v2 (PQ in V_k space) server
122     had E_rot exactly + PQ-decoded approx of E_proj, allowing it to solve
123      $\arg \min_Q ||\hat{E\_proj} - Q^T E\_rot||_F$  to recover R. In v3 both PQ
124     artifact and server's E_rot live in the same rotated space.
125     """
126     path = INDEX_CACHE / (
127         f"v3_indexpq_{encoder_tag}_proj{proj_k}_M{m}_b{nbits}_seed{seed}.faiss"
128     )
129     if path.exists():
130         return faiss.read_index(str(path))
131     index_pq = faiss.IndexPQ(proj_k, m, nbits, faiss.METRIC_INNER_PRODUCT)
132     rng = np.random.RandomState(42)
133     train_idx = rng.choice(len(E_rot), size=min(200_000, len(E_rot)), replace=False)
134     index_pq.train(E_rot[train_idx].astype(np.float32))
135     index_pq.add(E_rot.astype(np.float32))
136     faiss.write_index(index_pq, str(path))
137     return index_pq
138
139
140 def make_ctx():
141     ctx = ts.context(ts.SCHEME_TYPE.CKKS, poly_modulus_degree=CKKS_N,
142                     coeff_mod_bit_sizes=CKKS_COEFF)
143     ctx.global_scale = 2 ** CKKS_SCALE_BITS
144     ctx.generate_galois_keys()
145     return ctx
146
147
148 def run_seed(seed, E_proj, qrels, E_q_proj, encoder_tag, proj_k, pq_m):
149     """v3: PQ trained per-seed in rotated space; q_tilde used for stage-1."""

```

```

150 R = make_random_orthogonal(seed, proj_k)
151 E_rot = (E_proj @ R.T).astype(np.float32)
152 # Per-seed PQ index, trained in the rotated space.
153 index_pq = build_or_load_pq_rotated(
154     E_rot, pq_m, PQ_NBITS, encoder_tag, proj_k, seed)
155 N_q = int(os.environ.get("V3_N_QUERIES", len(E_q_proj)))
156 N_q = min(N_q, len(E_q_proj))
157 preds = []
158 timings = {"encrypt_ms": [], "rerank_ms": [], "decrypt_ms": [],
159            "stage1_ms": [], "total_server_ms": []}
160 ctx = make_ctx()
161 for i in tqdm(range(N_q), desc=f" seed={seed} k={proj_k}", leave=False):
162     # Client: project, rotate (plaintext)
163     q_proj = E_q_proj[i]
164     q_tilde = (q_proj @ R.T).astype(np.float32)
165     # Stage-1: client-side PQ search in ROTATED space using q_tilde
166     t0 = time.time()
167     _, cand_ids_arr = index_pq.search(
168         q_tilde.reshape(1, -1).astype(np.float32), K_CANDS)
169     cand_ids = cand_ids_arr[0]
170     timings["stage1_ms"].append(1000 * (time.time() - t0))
171     # Encrypt q_tilde
172     t0 = time.time()
173     ct_q = ts.ckks_vector(ctx, q_tilde.tolist())
174     timings["encrypt_ms"].append(1000 * (time.time() - t0))
175     # Server CKKS rerank
176     t_server_start = time.time()
177     t0 = time.time()
178     scores = []
179     for j in cand_ids:
180         d_rot = E_rot[int(j)]
181         score = float((ct_q * d_rot.tolist()).sum().decrypt()[0])
182         scores.append(score)
183     timings["rerank_ms"].append(1000 * (time.time() - t0))
184     scores = np.array(scores)
185     order = np.argsort(-scores)[:TOP_K]
186     top_cand_ids = cand_ids[order]
187     top_scores = scores[order]
188     timings["total_server_ms"].append(1000 * (time.time() - t_server_start))
189     # Client: decrypt + final ranking (decrypt happened above)
190     t0 = time.time()
191     final_order = np.argsort(-top_scores)[:10]
192     timings["decrypt_ms"].append(1000 * (time.time() - t0))

```

```

193     preds.append([int(top_cand_ids[o]) for o in final_order])
194 m = metrics(preds, qrels)
195 m["timing_p50_ms"] = {k: float(np.percentile(v, 50)) for k, v in timings.items()}
196 m["timing_p95_ms"] = {k: float(np.percentile(v, 95)) for k, v in timings.items()}
197 m["timing_p99_ms"] = {k: float(np.percentile(v, 99)) for k, v in timings.items()}
198 m["k_cands"] = K_CANDS
199 return m
200
201
202 def baseline_dense(E_docs, E_queries, qrels):
203     """Exact FAISS-IP in raw d-dim space (no SVD, no rotation, no CKKS)."""
204     ix = faiss.IndexFlatIP(E_docs.shape[1])
205     ix.add(E_docs)
206     _, ids = ix.search(E_queries, 10)
207     preds = [list(int(j) for j in row) for row in ids]
208     return metrics(preds, qrels)
209
210
211 def baseline_proj(E_proj, E_q_proj, qrels):
212     """Exact FAISS-IP in V_k-projected k-dim space (no rotation, no CKKS).
213
214     Upper bound on the proposed scheme's quality, against which CKKS rerank
215     is essentially neutral (Pearson >= 0.999).
216     """
217     ix = faiss.IndexFlatIP(E_proj.shape[1])
218     ix.add(E_proj)
219     _, ids = ix.search(E_q_proj, 10)
220     preds = [list(int(j) for j in row) for row in ids]
221     return metrics(preds, qrels)
222
223
224 # Encoder configs
225 ENCODERS = [
226     {"tag": "e5small", "dim": 384, "k": 336, "pq_m": 84,
227      "docs": "E_docs_e5small_1000000.npy",
228      "queries": "E_queries_e5small_self_q500.npy"},
229     {"tag": "e5base", "dim": 768, "k": 672, "pq_m": 96,
230      "docs": "E_docs_e5base_1000000.npy",
231      "queries": "E_queries_e5base_self_q500.npy"},
232     {"tag": "mpnet", "dim": 768, "k": 672, "pq_m": 96,

```

```

233     "docs": "E_docs_mpnet_1000000.npy",
234     "queries": "E_queries_mpnet_self_q500.npy"},
235     # Extra high-Acc@1 encoders (d=1024, k = 7d/8 = 896, PQ M = 128)
236     {"tag": "e5large", "dim": 1024, "k": 896, "pq_m": 128,
237      "docs": "E_docs_e5large_1000000.npy",
238      "queries": "E_queries_e5large_self_q500.npy"},
239     {"tag": "bgem3", "dim": 1024, "k": 896, "pq_m": 128,
240      "docs": "E_docs_bgem3_1000000.npy",
241      "queries": "E_queries_bgem3_self_q500.npy"},
242 ]
243
244
245 def run_encoder(cfg):
246     E_docs = np.load(CACHE / cfg["docs"]).astype(np.float32)
247     E_queries = np.load(CACHE / cfg["queries"]).astype(np.float32)
248     rng_q = np.random.default_rng(SEED_BASE)
249     qrels = [{int(i)} for i in rng_q.choice(len(E_docs), size=500, replace=False)]
250     mu, Vk = compute_svd(E_docs, cfg["k"])
251     sample = E_docs[:5000]
252     e_proj_s = (sample - mu) @ Vk
253     e_recon = e_proj_s @ Vk.T + mu
254     sigma_rec = float(np.mean(np.linalg.norm(sample - e_recon, axis=1) /
255                               (np.linalg.norm(sample, axis=1) + 1e-9)))
256     bd = baseline_dense(E_docs, E_queries, qrels)
257     E_proj = ((E_docs - mu) @ Vk).astype(np.float32)
258     E_q_proj = ((E_queries - mu) @ Vk).astype(np.float32)
259     del E_docs
260     bp = baseline_proj(E_proj, E_q_proj, qrels)
261     # v3: per-seed PQ in rotated space, built inside run_seed.
262     seeds_results = []
263     for s in SEEDS:
264         m = run_seed(s, E_proj, qrels, E_q_proj, cfg["tag"], cfg["k"], cfg["pq_m"]
265 ]
266         seeds_results.append(m)
267     a1 = [m["acc1"] for m in seeds_results]
268     a10 = [m["acc10"] for m in seeds_results]
269     mrr = [m["mrr"] for m in seeds_results]
270     nd = [m["ndcg10"] for m in seeds_results]
271     server_p50 = [m["timing_p50_ms"]["total_server_ms"] for m in seeds_results]
272     server_p95 = [m["timing_p95_ms"]["total_server_ms"] for m in seeds_results]
273     return {
274         "encoder": cfg["tag"], "dim": cfg["dim"], "k": cfg["k"],
275         "pq_m": cfg["pq_m"], "sigma_rec": sigma_rec,

```

```

275     "baseline_dense": bd, "baseline_proj": bp,
276     "per_seed": seeds_results,
277     "aggregate": {
278         "acc1": {"mean": t_ci(a1)[0], "ci_half": t_ci(a1)[2]},
279         "acc10": {"mean": t_ci(a10)[0], "ci_half": t_ci(a10)[2]},
280         "mrr": {"mean": t_ci(mrr)[0], "ci_half": t_ci(mrr)[2]},
281         "ndcg10": {"mean": t_ci(nd)[0], "ci_half": t_ci(nd)[2]},
282         "server_p50_ms": {"mean": t_ci(server_p50)[0]},
283         "server_p95_ms": {"mean": t_ci(server_p95)[0]},
284     },
285 }
286
287
288 def main():
289     all_results = [run_encoder(cfg) for cfg in ENCODERS]
290     out_path = OUT / "v3_multi_results.json"
291     with open(out_path, "w", encoding="utf-8") as f:
292         json.dump({
293             "ckks_params": {"N": CKKS_N, "coeff": CKKS_COEFF,
294                             "scale_bits": CKKS_SCALE_BITS},
295             "common_params": {"k_cands": K_CANDS, "pq_nbits": PQ_NBITS,
296                               "rerank_top": TOP_K},
297             "encoders": all_results,
298         }, f, indent=2, ensure_ascii=False)
299
300
301 if __name__ == "__main__":
302     main()

```


Приложение Е

Промышленная клиент-серверная реализация защищённого поиска (Раздел 4.5)

Приведён исходный код клиент-серверной реализации защищённого семантического поиска для архитектуры раздела 4.5. Стек состоит из пяти Python-модулей: общий конфигурационный файл `config.py` (параметры протокола, CKKS, сети), модуль офлайнной подготовки индексов `indexer.py` (SVD-базис, ротация R , публичный PQ-артефакт), сервер на FastAPI + `uvicorn` `server.py` (хранит ротированную базу и реализует REST API), HTTP-клиент `client.py` (хранит секретные ключи и выполняет per-query протокол), а также скрипт бенчмарка `benchmark.py`. Запуск: `python server.py` в одном терминале, `python benchmark.py` в другом. Результаты замеров сохраняются в `exp_cs_v2_outputs/cs_v2_results.json`.

Листинг 7 — Файл `config.py` — общая конфигурация

```
1 """Shared configuration for the secure search client-server stack (v2 architecture
   ).
2
3 Architecture: client-side PQ stage-1 + server-side CKKS ct-pt rerank.
4
5 The client:
6   - holds the secret CKKS key, V_k, mu, R (all client-secrets),
7   - downloads the public PQ artifact (codebook + per-doc PQ codes) once at
8     session init via /index endpoint,
9   - per query: V_k-projects, rotates locally, runs PQ search to pick K_CANDS
10     candidates, encrypts the rotated query, sends ct + cand_ids to server.
11
12 The server:
13   - holds E_rot = (E - mu) @ V_k @ R^T (rotated, no plaintext leakage to client),
14   - per query: receives ct_q + cand_ids, performs K_CANDS ct-pt scalar-product
15     operations, returns K_CANDS encrypted scores.
16 """
17 from pathlib import Path
18
19 # -- Paths -----
20 NOTEBOOKS_DIR = Path(__file__).parent.parent
21 CACHE_DIR     = NOTEBOOKS_DIR / "_corpus_cache"
22
23 E_DOCS_PATH   = CACHE_DIR / "E_docs_e5small_1000000.npy"
24 E_QUERIES_PATH = CACHE_DIR / "E_queries_e5small_self_q500.npy"
25
26 INDEX_CACHE_DIR = CACHE_DIR / "index_cache"
```

```

27 INDEX_CACHE_DIR.mkdir(parents=True, exist_ok=True)
28
29 OUTPUT_DIR = NOTEBOOKS_DIR / "exp_cs_v2_outputs"
30 OUTPUT_DIR.mkdir(parents=True, exist_ok=True)
31
32 # -- Encoder / dimensions -----
33 MODEL_NAME      = "intfloat/multilingual-e5-small"
34 QUERY_PREFIX    = "query: "
35 EMB_DIM         = 384
36 PROJ_K          = 336
37
38 # -- Two-stage protocol parameters -----
39 PQ_M            = 84                # number of sub-quantizers (must divide PROJ_K)
40 PQ_NBITS        = 8                # bits per sub-quantizer code
41 K_CANDS         = 40                # stage-1 shortlist size handed to CKKS rerank
42 TOP_K_FINAL     = 10                # final top-K returned to user
43
44 # -- CKKS hi-res config (3-mod chain, single ct-pt mul per query) -----
45 CKKS_POLY_DEGREE = 8192
46 CKKS_COEFF_BITS  = [60, 40, 60]
47 CKKS_SCALE_BITS  = 40
48
49 # -- Network -----
50 SERVER_HOST      = "127.0.0.1"
51 SERVER_PORT      = 8770
52 SERVER_URL       = f"http://{SERVER_HOST}:{SERVER_PORT}"
53
54 # -- Rotation seed for offline server-side preparation -----
55 # WARNING: default value for benchmark/demo only. In production, ROTATION_SEED
56 # MUST be a cryptographically-strong secret (HSM / secret-manager); it is the
57 # principal secret of the protection scheme. Hardcoding the same seed across
58 # deployments means all share the same R, destroying per-deployment protection.
59 ROTATION_SEED    = 11

```

Листинг 8 — Файл indexer.py — офлайновая подготовка индексов

```

1  """Offline index preparation for v2 architecture: SVD, rotation, PQ artifact.
2
3  Roles:
4      - load_or_compute_svd / make_rotation: client-side secret artifacts.
5      - load_or_build_pq_index: PUBLIC PQ artifact (codebook + codes), trained
6        on V_k-projected docs (without rotation). Distributed to clients via
7        /index endpoint.
8      - build_E_rot: SERVER-side rotated document base used for CKKS rerank;

```

```

9     never leaves the server in plaintext form.
10 """
11 from pathlib import Path
12 import time
13
14 import faiss
15 import numpy as np
16 from sklearn.utils.extmath import randomized_svd
17
18 from config import (
19     E_DOCS_PATH, INDEX_CACHE_DIR, PROJ_K,
20     PQ_M, PQ_NBITS, ROTATION_SEED,
21 )
22
23
24 def load_or_compute_svd(E_docs):
25     cache = INDEX_CACHE_DIR / f"svd_e5small_proj{PROJ_K}.npz"
26     if cache.exists():
27         d = np.load(cache)
28         return d["mu"], d["Vk"]
29     print(f"[indexer] computing SVD k={PROJ_K} on {len(E_docs)} docs ...")
30     mu = E_docs.mean(axis=0, keepdims=True).astype(np.float32)
31     rng = np.random.RandomState(0)
32     n_sample = min(200_000, len(E_docs))
33     idx = rng.choice(len(E_docs), size=n_sample, replace=False)
34     _, _, Vt = randomized_svd(np.asarray(E_docs[idx]) - mu,
35                               n_components=PROJ_K, random_state=42)
36     Vk = Vt.T.astype(np.float32)
37     np.savez(cache, mu=mu, Vk=Vk)
38     return mu, Vk
39
40
41 def make_rotation(seed: int = ROTATION_SEED) -> np.ndarray:
42     """Full random orthogonal R in O(PROJ_K), CLIENT-secret."""
43     rng = np.random.default_rng(seed)
44     A = rng.standard_normal((PROJ_K, PROJ_K))
45     Q, _ = np.linalg.qr(A)
46     return Q.astype(np.float32)
47
48
49 def project(E, mu, Vk):
50     return ((E - mu) @ Vk).astype(np.float32)
51

```

```

52
53 def project_with_rotation(E, mu, Vk, R):
54     return ((E - mu) @ Vk) @ R.T).astype(np.float32)
55
56
57 def load_or_build_pq_index(E_rot, seed: int = ROTATION_SEED):
58     """v3: PQ artifact lives in the rotated space (E_rot = E_proj @ R^T).
59
60     Closes the server-side Procrustes attack on R: in v2 (PQ in V_k space)
61     server had E_rot exactly + PQ-decoded approx of E_proj, allowing it to
62     solve  $\arg \min_Q ||\hat{E\_proj} Q^T - E\_rot||_F$  to recover R. In v3 both
63     PQ artifact and server's E_rot live in the same rotated space.
64
65     Per-seed (depends on R); cached path includes seed.
66     """
67     path = INDEX_CACHE_DIR / (
68         f"v3cs_indexpq_e5small_proj{PROJ_K}_M{PQ_M}_b{PQ_NBITS}_seed{seed}.faiss"
69     )
70     if path.exists():
71         return faiss.read_index(str(path))
72     print(f"[indexer] training PQ-rot M={PQ_M} nbits={PQ_NBITS} "
73           f"seed={seed} on {len(E_rot)} docs ...")
74     index_pq = faiss.IndexPQ(PROJ_K, PQ_M, PQ_NBITS, faiss.METRIC_INNER_PRODUCT)
75     rng = np.random.RandomState(42)
76     train_idx = rng.choice(len(E_rot),
77                             size=min(200_000, len(E_rot)),
78                             replace=False)
79     index_pq.train(E_rot[train_idx].astype(np.float32))
80     index_pq.add(E_rot.astype(np.float32))
81     faiss.write_index(index_pq, str(path))
82     return index_pq
83
84
85 def build_E_rot(E_docs, mu, Vk, R):
86     """SERVER-side rotated document base. Never sent to client in plaintext."""
87     E_proj = ((E_docs - mu) @ Vk).astype(np.float32)
88     E_rot = (E_proj @ R.T).astype(np.float32)
89     return E_rot

```

Листинг 9 — Файл server.py — FastAPI-сервер

```

1 """Secure search server (FastAPI + uvicorn), v2 architecture.
2
3 Server holds:

```

```

4 - the rotated document base  $E_{\text{rot}} (= (E - \mu) @ V_k @ R^T)$ , in memory,
5 - per-session public CKKS contexts (no secret keys).
6
7 Endpoints:
8 POST /session -- client registers public CKKS context, gets session_id.
9 GET /index -- streams the public PQ artifact (codebook + codes) once at
10 session creation. Note: PQ codes are computed in
11  $V_k$ -projected space WITHOUT rotation, so they do not
12 reveal  $R$ .
13 POST /rerank -- per-query:  $ct_q + cand\_ids \rightarrow ct\_scores$ .
14 GET /health -- liveness check.
15
16 Per-query CKKS work:  $K\_CANDS$   $ct$ - $pt$  scalar-product operations, each at hi-res
17 config ( $N=8192$ ,  $[60,40,60]$ ,  $scale=2^{40}$ ).
18 """
19 import base64
20 import logging
21 import time
22 import uuid
23 from contextlib import asynccontextmanager
24 from typing import Dict, List
25
26 import faiss
27 import numpy as np
28 import tenseal as ts
29 import uvicorn
30 from fastapi import FastAPI, HTTPException
31 from fastapi.responses import StreamingResponse
32 from pydantic import BaseModel
33
34 from config import (
35     E_DOCS_PATH, PROJ_K, K_CANDS, SERVER_HOST, SERVER_PORT,
36 )
37 from indexer import (
38     load_or_compute_svd, make_rotation,
39     load_or_build_pq_index, build_E_rot,
40 )
41
42 logging.basicConfig(
43     level=logging.INFO,
44     format="%(asctime)s %(levelname)-8s %(message)s",
45     datefmt="%H:%M:%S",
46 )

```

```

47 log = logging.getLogger("cs_v2.server")
48
49
50 class _State:
51     E_rot: np.ndarray = None          # (N, k) plaintext rotated docs (server-only)
52     pq_index_bytes: bytes = None      # serialized public PQ artifact
53     sessions: Dict[str, ts.Context] = {}
54
55
56 _state = _State()
57
58
59 @asynccontextmanager
60 async def _lifespan(app: FastAPI):
61     log.info("Loading E_docs (%s) ...", E_DOCS_PATH)
62     E_docs = np.load(E_DOCS_PATH).astype(np.float32)
63     log.info("E_docs loaded: %s dtype=%s", E_docs.shape, E_docs.dtype)
64     mu, Vk = load_or_compute_svd(E_docs)
65     R = make_rotation()
66     _state.E_rot = build_E_rot(E_docs, mu, Vk, R)
67     # v3: PQ trained in the ROTATED space (closes server-side Procrustes attack).
68     # Both _state.E_rot and the PQ artifact now live in the same rotated space.
69     pq_index = load_or_build_pq_index(_state.E_rot, seed=11)
70     # Serialize PQ index to bytes once (for /index endpoint).
71     # Faiss requires a file path; tempfile with try/finally for safe cleanup.
72     import tempfile
73     import os as _os
74     with tempfile.NamedTemporaryFile(suffix=".faiss", delete=False) as tmpf:
75         tmp_path = tmpf.name
76     try:
77         faiss.write_index(pq_index, tmp_path)
78         with open(tmp_path, "rb") as f:
79             _state.pq_index_bytes = f.read()
80     finally:
81         try:
82             _os.unlink(tmp_path)
83         except OSError:
84             pass
85     del E_docs
86     log.info("Server ready -- %d docs @ k=%d, PQ artifact: %.1f MB",
87             _state.E_rot.shape[0], PROJ_K,
88             len(_state.pq_index_bytes) / (1024*1024))
89     yield

```

```

90     log.info("Server shutdown.")
91
92
93 app = FastAPI(title="Secure Search Server v2", lifespan=_lifespan)
94
95
96 class SessionRequest(BaseModel):
97     ckks_ctx_b64: str
98
99
100 class SessionResponse(BaseModel):
101     session_id: str
102     n_docs: int
103     proj_k: int
104     k_cands: int
105     pq_index_size: int
106
107
108 class RerankRequest(BaseModel):
109     session_id: str
110     candidate_ids: List[int]
111     ckks_query_b64: str
112
113
114 class RerankResponse(BaseModel):
115     ckks_scores_b64: List[str]
116     server_ckks_ms: float
117
118
119 @app.get("/health")
120 def health():
121     return {"status": "ok",
122           "n_docs": int(_state.E_rot.shape[0]) if _state.E_rot is not None else
123           0,
124           "proj_k": PROJ_K,
125           "k_cands": K_CANDS}
126
127 @app.post("/session", response_model=SessionResponse)
128 def create_session(req: SessionRequest):
129     ctx_pub = ts.context_from(base64.b64decode(req.ckks_ctx_b64))
130     sid = str(uuid.uuid4())
131     _state.sessions[sid] = ctx_pub

```

```

132     log.info("New session: %s (total: %d)", sid[:8], len(_state.sessions))
133     return SessionResponse(
134         session_id=sid,
135         n_docs=int(_state.E_rot.shape[0]),
136         proj_k=PROJ_K,
137         k_cands=K_CANDS,
138         pq_index_size=len(_state.pq_index_bytes),
139     )
140
141
142 @app.get("/index")
143 def download_pq_index():
144     """Stream the public PQ artifact (codebook + per-doc codes) to client."""
145     if _state.pq_index_bytes is None:
146         raise HTTPException(503, "PQ index not ready")
147     return StreamingResponse(
148         iter([_state.pq_index_bytes]),
149         media_type="application/octet-stream",
150         headers={
151             "X-Index-Type": "faiss-IndexPQ",
152             "X-Index-Bytes": str(len(_state.pq_index_bytes)),
153         },
154     )
155
156
157 @app.post("/rerank", response_model=RerankResponse)
158 def rerank(req: RerankRequest):
159     """CKKS ct-pt rerank of K_CANDS candidates against the encrypted query."""
160     if _state.E_rot is None:
161         raise HTTPException(503, "Server not ready")
162     ctx_pub = _state.sessions.get(req.session_id)
163     if ctx_pub is None:
164         raise HTTPException(404, f"Unknown session: {req.session_id}")
165     if not (1 <= len(req.candidate_ids) <= 1024):
166         raise HTTPException(400, f"candidate_ids size {len(req.candidate_ids)} out
167         of bounds")
168
169     cand_vecs = _state.E_rot[req.candidate_ids]
170     enc_q = ts.ckks_vector_from(ctx_pub, base64.b64decode(req.ckks_query_b64))
171
172     t0 = time.perf_counter()
173     scores_b64 = []
174     for d in cand_vecs:

```



```

174         enc_score = (enc_q * d.tolist()).sum()
175         scores_b64.append(base64.b64encode(enc_score.serialize()).decode())
176         server_ckks_ms = (time.perf_counter() - t0) * 1e3
177
178         return RerankResponse(ckks_scores_b64=scores_b64,
179                               server_ckks_ms=server_ckks_ms)
180
181
182 if __name__ == "__main__":
183     uvicorn.run("server:app", host=SERVER_HOST, port=SERVER_PORT, log_level="info"
184                )

```

Листинг 10 — Файл client.py — защищённый клиент

```

1  """Secure search client (v3 architecture: PQ in rotated space).
2
3  Client holds:
4      - the secret CKKS key (never transmitted),
5      - V_k, mu, R (client-secrets, derived from owner's seed at onboarding),
6      - the public PQ artifact (codebook + per-doc codes), downloaded once at
7        session creation via /index. PQ is trained in the ROTATED space, so the
8        search key is q_tilde (not q_proj).
9
10 Per query:
11     1. compute q_proj = V_k^T (q - mu)  (project)
12     2. compute q_tilde = R q_proj      (rotate)  -- both plaintext, on client
13     3. PQ-search top-K_CANDS candidates locally in the ROTATED space using q_tilde
14     4. encrypt q_tilde under CKKS
15     5. POST /rerank with candidate_ids + ct_q -> get ct_scores
16     6. decrypt scores, take top-K_FINAL
17 """
18 import base64
19 import io
20 import time
21 from typing import Dict, List, Optional, Tuple
22
23 import faiss
24 import numpy as np
25 import requests
26 import tenseal as ts
27
28 from config import (
29     CKKS_POLY_DEGREE, CKKS_COEFF_BITS, CKKS_SCALE_BITS,
30     E_DOCS_PATH, K_CANDS, TOP_K_FINAL, SERVER_URL, ROTATION_SEED, PROJ_K,

```

```

31 )
32 from indexer import (
33     load_or_compute_svd, make_rotation, project_with_rotation, project,
34 )
35
36
37 class SecureSearchClientV2:
38     """v2 client: client-side PQ stage-1 + server-side CKKS rerank via HTTP."""
39
40     def __init__(self, server_url: str = SERVER_URL,
41                 rotation_seed: int = ROTATION_SEED):
42         self.server_url = server_url
43
44         # Build CKKS context with secret key
45         self._ctx = ts.context(
46             ts.SCHEME_TYPE.CKKS,
47             poly_modulus_degree=CKKS_POLY_DEGREE,
48             coeff_mod_bit_sizes=CKKS_COEFF_BITS,
49         )
50         self._ctx.global_scale = 2 ** CKKS_SCALE_BITS
51         self._ctx.generate_galois_keys()
52
53         # Register session
54         ctx_pub_b64 = base64.b64encode(
55             self._ctx.serialize(save_secret_key=False)
56         ).decode()
57         resp = requests.post(
58             f"{server_url}/session",
59             json={"ckks_ctx_b64": ctx_pub_b64},
60             timeout=60,
61         )
62         resp.raise_for_status()
63         info = resp.json()
64         self._session_id: str = info["session_id"]
65         self._n_docs: int = info["n_docs"]
66         self._k_cands: int = info["k_cands"]
67
68         # Local SVD + rotation (client-secrets derived from cached E_docs)
69         E_docs_mmap = np.load(E_DOCS_PATH, mmap_mode="r")
70         self._mu, self._Vk = load_or_compute_svd(E_docs_mmap)
71         self._R = make_rotation(rotation_seed)
72
73         # Download public PQ artifact once at session creation

```

```

74         self._pq_index = self._download_pq_index()
75
76     def _download_pq_index(self):
77         """GET /index -> deserialize faiss IndexPQ from bytes."""
78         resp = requests.get(f"{self.server_url}/index", timeout=120)
79         resp.raise_for_status()
80         # Tempfile with try/finally so the path is always unlinked,
81         # even if faiss.read_index raises.
82         import tempfile, os
83         tmpf = tempfile.NamedTemporaryFile(suffix=".faiss", delete=False)
84         tmp_path = tmpf.name
85         try:
86             tmpf.write(resp.content); tmpf.close()
87             idx = faiss.read_index(tmp_path)
88         finally:
89             try:
90                 os.unlink(tmp_path)
91             except OSError:
92                 pass
93         return idx
94
95     def _encrypt(self, vec: np.ndarray) -> str:
96         return base64.b64encode(
97             ts.ckks_vector(self._ctx, vec.tolist()).serialize()
98         ).decode()
99
100     def _decrypt_scores(self, scores_b64: List[str]) -> np.ndarray:
101         return np.array([
102             ts.ckks_vector_from(self._ctx, base64.b64decode(s)).decrypt()[0]
103             for s in scores_b64
104         ])
105
106     def _local_pq_search(self, q_tilde: np.ndarray, top_k: int) -> List[int]:
107         """v3: asymmetric PQ-distance search in the ROTATED space using q_tilde.
108
109         PQ index is trained on  $E_{rot} = E_{proj} @ R^T$ ; the search key is the
110         rotated query  $q_{tilde}$  ( $= R q_{proj}$ ), not  $q_{proj}$ .
111         """
112         _, I = self._pq_index.search(q_tilde.reshape(1, -1).astype(np.float32),
113 top_k)
114         return I[0].tolist()
115
116     def _post_rerank(self, candidate_ids: List[int], ckks_b64: str) -> dict:

```

```

116     resp = requests.post(
117         f"{self.server_url}/rerank",
118         json={
119             "session_id": self._session_id,
120             "candidate_ids": candidate_ids,
121             "ckks_query_b64": ckks_b64,
122         },
123         timeout=60,
124     )
125     resp.raise_for_status()
126     return resp.json()
127
128 def search_emb(self, q_emb: np.ndarray,
129                 top_k: int = TOP_K_FINAL,
130                 ) -> Tuple[List[int], Dict[str, float]]:
131     """Search by pre-computed query embedding (skips encoder).
132
133     Returns (top-K final doc ids, timing dict).
134     """
135     t_start = time.perf_counter()
136
137     # 1. Project + rotate (plaintext, client-side)
138     t0 = time.perf_counter()
139     q_proj = project(q_emb[None], self._mu, self._Vk)[0]
140     q_tilde = (q_proj @ self._R.T).astype(np.float32)
141     t_project_ms = (time.perf_counter() - t0) * 1e3
142
143     # 2. Stage-1: PQ-search in the ROTATED space (v3) using q_tilde
144     t0 = time.perf_counter()
145     candidate_ids = self._local_pq_search(q_tilde, K_CANDS)
146     t_pq_ms = (time.perf_counter() - t0) * 1e3
147
148     # 3. Encrypt rotated query
149     t0 = time.perf_counter()
150     ckks_b64 = self._encrypt(q_tilde)
151     t_encrypt_ms = (time.perf_counter() - t0) * 1e3
152
153     # 4. POST /rerank (HTTP round-trip)
154     t0 = time.perf_counter()
155     data = self._post_rerank(candidate_ids, ckks_b64)
156     t_network_ms = (time.perf_counter() - t0) * 1e3
157
158     # 5. Decrypt + final ranking

```

```

159         t0 = time.perf_counter()
160         scores = self._decrypt_scores(data["ckks_scores_b64"])
161         order = np.argsort(-scores)[:top_k]
162         doc_ids = [candidate_ids[i] for i in order]
163         t_decrypt_ms = (time.perf_counter() - t0) * 1e3
164
165         return doc_ids, {
166             "total_ms": (time.perf_counter() - t_start) * 1e3,
167             "project_ms": t_project_ms,
168             "client_pq_ms": t_pq_ms,
169             "encrypt_ms": t_encrypt_ms,
170             "network_ms": t_network_ms,          # client-perceived RTT
171             "server_ckks_ms": data["server_ckks_ms"],
172             "decrypt_ms": t_decrypt_ms,
173         }

```

Листинг 11 — Файл benchmark.py — скрипт бенчмарка

```

1  """Loopback benchmark for v2 client-server stack.
2
3  Run AFTER server is up:
4      python -m uvicorn server:app --host 127.0.0.1 --port 8770 &
5
6  Then:
7      python benchmark.py
8
9  Protocol: standard industrial-bench measurement on a single R (fixed per
10 deployment); latency variance is reported over the 500-query distribution
11 (p50/p95/p99). Quality metrics on the CS stack are identical to the
12 multi-encoder in-process measurement (the PQ+CKKS protocol is the same; HTTP
13 is just transport).
14
15 Repeats the full 500-query loop NREPEATS times for stable timing percentiles.
16 """
17 import json
18 import os
19 import time
20 from pathlib import Path
21
22 import numpy as np
23 import requests
24 from tqdm import tqdm
25
26 from config import (

```

```

27     E_DOCS_PATH, E_QUERIES_PATH, OUTPUT_DIR, SERVER_URL,
28     TOP_K_FINAL, K_CANDS, PROJ_K, ROTATION_SEED,
29 )
30 from client import SecureSearchClientV2
31
32 SEEDS_QRELS = 42
33 NREPEATS     = int(os.environ.get("CS_REPEATS", 1))
34 T_CRIT_95    = {3: 4.303, 4: 3.182, 5: 2.776}
35
36
37 def acc_at_k(pred, rel, k):
38     return float(bool(set(pred[:k]) & rel))
39
40
41 def mrr_score(pred, rel):
42     for r, p in enumerate(pred, 1):
43         if p in rel:
44             return 1.0 / r
45     return 0.0
46
47
48 def ndcg_at_k(pred, rel, k=10):
49     return sum(1.0 / np.log2(i + 2) for i, p in enumerate(pred[:k]) if p in rel)
50
51
52 def t_ci(values):
53     arr = np.asarray(values, dtype=np.float64)
54     n = len(arr)
55     if n < 2:
56         return float(arr.mean()), 0.0, 0.0
57     mean = float(arr.mean())
58     std = float(arr.std(ddof=1))
59     t = T_CRIT_95.get(n, 1.96)
60     return mean, std, float(t * std / np.sqrt(n))
61
62
63 def percentiles(timings_dict):
64     out = {}
65     for k, v in timings_dict.items():
66         a = np.asarray(v, dtype=np.float64)
67         out[k] = {"p50": float(np.percentile(a, 50)),
68                  "p95": float(np.percentile(a, 95)),
69                  "p99": float(np.percentile(a, 99)),

```

```

70         "mean": float(a.mean())}
71     return out
72
73
74 def wait_for_server(url=SERVER_URL, timeout=120):
75     print(f"[bench] waiting for {url}/health (up to {timeout}s) ...")
76     t0 = time.time()
77     while time.time() - t0 < timeout:
78         try:
79             r = requests.get(f"{url}/health", timeout=2)
80             if r.status_code == 200:
81                 print(f"[bench] server up: {r.json()}")
82                 return
83             except Exception:
84                 pass
85             time.sleep(1)
86     raise RuntimeError(f"Server {url} did not become healthy in {timeout}s")
87
88
89 def run_one_pass(client, E_queries, qrels, n_q, label):
90     """Single sweep over 500 queries; collects per-query timings."""
91     preds = []
92     timings = {"total_ms": [], "project_ms": [], "client_pq_ms": [],
93               "encrypt_ms": [], "network_ms": [], "server_ckks_ms": [],
94               "decrypt_ms": []}
95     for i in tqdm(range(n_q), desc=label):
96         doc_ids, t = client.search_emb(E_queries[i], top_k=TOP_K_FINAL)
97         preds.append(doc_ids)
98         for k in timings:
99             timings[k].append(t[k])
100     a1 = float(np.mean([acc_at_k(p, r, 1) for p, r in zip(preds, qrels[:n_q])]))
101     a10 = float(np.mean([acc_at_k(p, r, 10) for p, r in zip(preds, qrels[:n_q])]))
102     mrr = float(np.mean([mrr_score(p, r) for p, r in zip(preds, qrels[:n_q])]))
103     ndcg = float(np.mean([ndcg_at_k(p, r) for p, r in zip(preds, qrels[:n_q])]))
104     return {"acc1": a1, "acc10": a10, "mrr": mrr, "ndcg10": ndcg,
105            "timings": timings}
106
107
108 def main():

```

```

109     wait_for_server()
110     E_queries = np.load(E_QUERIES_PATH).astype(np.float32)
111     rng = np.random.default_rng(SEEDS_QRELS)
112     n_docs = 1_000_000
113     qrels = [{int(i)} for i in rng.choice(n_docs, size=len(E_queries), replace=
False)]
114     n_q = int(os.environ.get("CS_N_QUERIES", len(E_queries)))
115
116     print(f"\n[bench] init client (rotation_seed={ROTATION_SEED}) ...")
117     client = SecureSearchClientV2(rotation_seed=ROTATION_SEED)
118     print("[bench] warmup (30 queries) ...")
119     for i in range(min(30, n_q)):
120         client.search_emb(E_queries[i], top_k=TOP_K_FINAL)
121
122     # Main passes (NREPEATS sweeps over the 500-query distribution).
123     # Latency variance comes from the 500-query distribution; metrics are
124     # stable across passes (deterministic protocol with fixed R).
125     pass_results = []
126     for rep in range(NREPEATS):
127         print(f"\n[bench] pass {rep+1}/{NREPEATS}")
128         pr = run_one_pass(client, E_queries, qrels, n_q, label=f" pass{rep+1}")
129         pct = percentiles(pr["timings"])
130         print(f"  Acc@1={pr['acc1']:.3f} Acc@10={pr['acc10']:.3f} "
131               f"MRR={pr['mrr']:.3f} NDCG@10={pr['ndcg10']:.3f}")
132         print(f"  end-to-end p50={pct['total_ms']['p50']:.0f} "
133               f"p95={pct['total_ms']['p95']:.0f} "
134               f"p99={pct['total_ms']['p99']:.0f} ms")
135         print(f"  client: project={pct['project_ms']['p50']:.1f} + "
136               f"pq={pct['client_pq_ms']['p50']:.1f} + "
137               f"enc={pct['encrypt_ms']['p50']:.1f} + "
138               f"dec={pct['decrypt_ms']['p50']:.1f} ms")
139         print(f"  network/HTTP p50={pct['network_ms']['p50']:.0f} ms; "
140               f"server CKKS p50={pct['server_ckks_ms']['p50']:.0f} ms")
141         pr["timings_pct"] = pct
142         pass_results.append(pr)
143
144     # Aggregate across passes (timing distributions are pooled)
145     pooled = {k: [] for k in pass_results[0]["timings"]}
146     for pr in pass_results:
147         for k in pooled:
148             pooled[k].extend(pr["timings"][k])
149     pooled_pct = percentiles(pooled)
150

```



```

151 summary = {
152     "config": {
153         "encoder": "multilingual-e5-small",
154         "n_docs": 1_000_000, "n_queries_per_pass": n_q,
155         "n_passes": NREPEATS,
156         "proj_k": PROJ_K, "k_cands": K_CANDS,
157         "ckks_N": 8192, "ckks_coeff": [60, 40, 60], "ckks_scale": 40,
158         "transport": "HTTP/JSON loopback (127.0.0.1)",
159         "rotation_seed": ROTATION_SEED,
160     },
161     "quality": {
162         "acc1": pass_results[0]["acc1"],
163         "acc10": pass_results[0]["acc10"],
164         "mrr": pass_results[0]["mrr"],
165         "ndcg10": pass_results[0]["ndcg10"],
166         "note": "deterministic on fixed R; CI from 5-seed run reported in
section 4.4 multi-encoder table",
167     },
168     "latency_pct_pooled": pooled_pct,
169     "per_pass": [{ "acc1": r["acc1"], "acc10": r["acc10"],
170                   "mrr": r["mrr"], "ndcg10": r["ndcg10"],
171                   "timings_pct": r["timings_pct"]}
172                 for r in pass_results],
173 }
174 out_path = OUTPUT_DIR / "cs_v2_results.json"
175 with open(out_path, "w", encoding="utf-8") as f:
176     json.dump(summary, f, indent=2, ensure_ascii=False)
177 print(f"\n[bench] saved -> {out_path}")
178 q = summary["quality"]
179 p = pooled_pct
180 print(f"\n=== AGGREGATE({NREPEATS} passes x {n_q} queries, "
181       f"pooled = {NREPEATS * n_q} queries) ===")
182 print(f"  Acc@1   = {q['acc1']:.3f}")
183 print(f"  Acc@10  = {q['acc10']:.3f}")
184 print(f"  MRR      = {q['mrr']:.3f}")
185 print(f"  NDCG@10 = {q['ndcg10']:.3f}")
186 print(f"  end-to-end (HTTP loopback): p50={p['total_ms']['p50']:.0f}  "
187       f"p95={p['total_ms']['p95']:.0f}  "
188       f"p99={p['total_ms']['p99']:.0f} ms")
189 print(f"  server CKKS rerank: p50={p['server_ckks_ms']['p50']:.0f}  "
190       f"p95={p['server_ckks_ms']['p95']:.0f} ms")
191 print(f"  network/HTTP/serialization: p50={p['network_ms']['p50']:.0f}  "
192       f"p95={p['network_ms']['p95']:.0f} ms")

```

```
193  
194  
195 if __name__ == "__main__":  
196     main()
```